```
import os
import json
import re
import time
import hmac
import hashlib
import requests as _requests_mod
import threading


class _SessionShim:
    """Same get/post/delete calls as `requests`, but through ONE shared Session s
    Binance/Telegram/etc. connections are reused instead of doing a fresh TCP+TLS
    handshake on every call. Only connection-setup failures are retried (nothing
    been sent at that point), so an order can never be sent twice by a retry."""
    def __init__(self):
        from requests.adapters import HTTPAdapter
        from urllib3.util.retry import Retry
        self._s = _requests_mod.Session()
        _retry = Retry(total=1, connect=1, read=0, status=0, redirect=0, other=0)
        _ad = HTTPAdapter(pool_connections=20, pool_maxsize=50, max_retries=_retr
        self._s.mount("https://", _ad)
        self._s.mount("http://", _ad)

    def get(self, *a, **k):    return self._s.get(*a, **k)
    def post(self, *a, **k):   return self._s.post(*a, **k)
    def delete(self, *a, **k): return self._s.delete(*a, **k)
    def put(self, *a, **k):    return self._s.put(*a, **k)
    def request(self, *a, **k): return self._s.request(*a, **k)
    def __getattr__(self, name):   # exceptions, codes, etc. come from the real m
        return getattr(_requests_mod, name)


requests = _SessionShim()
import websocket
from pywebpush import webpush, WebPushException
from datetime import datetime, timezone, time as dt_time, timedelta
from zoneinfo import ZoneInfo
from flask import Flask, request as flask_request, jsonify, send_file

try:
    from hyperliquid.exchange import Exchange as HLExchange
    from hyperliquid.info    import Info    as HLInfo
```

```
    from hyperliquid.utils.signing import OrderType as HLOrderType, TriggerOrderT
    from hyperliquid.utils.error import ClientError as HLClientError
    from eth_account         import Account as EthAccount
    HL_SDK_AVAILABLE = True
except ImportError:
    HL_SDK_AVAILABLE = False
    print("[HL] SDK not installed — mirror disabled")

try:
    import psycopg2
    import psycopg2.extras
except ImportError:
    psycopg2 = None
    print("[DB] psycopg2 not installed — trigger tracking disabled")

# ─────────────────────────────────────────────
# CONFIG — set these as environment variables
# ─────────────────────────────────────────────
BINANCE_API_KEY    = os.environ.get("BINANCE_API_KEY", "")
BINANCE_API_SECRET = os.environ.get("BINANCE_API_SECRET", "")
TELEGRAM_BOT_TOKEN = os.environ.get("TELEGRAM_BOT_TOKEN", "")
TELEGRAM_CHAT_ID   = os.environ.get("TELEGRAM_CHAT_ID", "")

SYMBOL                  = os.environ.get("SYMBOL", "BTCUSDT")
EARLY_THRESHOLD         = int(os.environ.get("EARLY_THRESHOLD", "6"))
WARN_THRESHOLD          = int(os.environ.get("WARN_THRESHOLD", "8"))

# ─────────────────────────────────────────────
# TESTING_MODE — temporary override, set by request while confirming the
# bot still executes correctly after the CRM/dashboard migration work.
# When true, this bypasses ONLY the same-side cooldown, the daily bias filter and
# the EMA entry guards at their enforcement points -- the daily trade
# cap, loss limits, curfew, Friday cutoff and US-open cooldown are always enforce
# underlying thresholds, counters, and alert messages are all left
# completely untouched, so turning this back off restores full original
# behavior with no values to remember or restore.
#
# Originally a fixed env-var constant read once at startup — now a live
# file-based flag (same pattern as the maintenance banner) so it can be
# toggled from the dashboard without a Railway env var edit + restart,
# which was the actual friction that caused this to get left on by
# accident before.
# >>> REMINDER: turn this back off once trading is confirmed fully live
# again. Nathan asked to be reminded of this. <<<
# ─────────────────────────────────────────────
# ── Durable settings store ──────────────────────────────────────────
```

```
# Railway wipes files next to the code on every redeploy, which kept resetting th
# client mirroring toggles. These small settings now live in Postgres (persistent
# the old JSON file is only read once to migrate the value, and used as the fallb
# there is no database. Reads are cached for a few seconds, writes update the cac
_KV_CACHE = {}
_KV_TTL   = 4.0
_kv_table_ready = False

def _kv_conn():
    import psycopg2
    return psycopg2.connect(os.environ["DATABASE_URL"])

def _kv_read(key: str, file_path: str, default):
    now = time.time()
    hit = _KV_CACHE.get(key)
    if hit and now - hit[0] < _KV_TTL:
        return hit[1]
    val, found = default, False
    if os.environ.get("DATABASE_URL"):
        try:
            conn = _kv_conn(); cur = conn.cursor()
            cur.execute("SELECT value FROM persistent_state WHERE key = %s", (key
            row = cur.fetchone()
            cur.close(); conn.close()
            if row:
                val, found = json.loads(row[0]), True
        except Exception as e:
            print(f"[Settings] DB read failed for {key}, using file fallback: {e}
    if not found:
        try:
            with open(file_path, "r") as f:
                val = json.load(f)
            if os.environ.get("DATABASE_URL"):
                _kv_write(key, file_path, val)   # one-time migration of the exis
        except Exception:
            val = default
    _KV_CACHE[key] = (now, val)
    return val

def _kv_write(key: str, file_path: str, value):
    global _kv_table_ready
    _KV_CACHE[key] = (time.time(), value)
    saved = False
    if os.environ.get("DATABASE_URL"):
        try:
            conn = _kv_conn(); cur = conn.cursor()
```

```
            if not _kv_table_ready:
                cur.execute("""CREATE TABLE IF NOT EXISTS persistent_state (
                                key TEXT PRIMARY KEY, value TEXT, updated_at TIM
                _kv_table_ready = True
            cur.execute("""INSERT INTO persistent_state (key, value, updated_at)
                           ON CONFLICT (key) DO UPDATE SET value = EXCLUDED.value
                        (key, json.dumps(value)))
            conn.commit(); cur.close(); conn.close()
            saved = True
        except Exception as e:
            print(f"[Settings] DB write failed for {key}, writing file instead: {
    if not saved:
        with open(file_path, "w") as f:
            json.dump(value, f)

TESTING_MODE_FILE    = os.path.join(os.path.dirname(os.path.abspath(__file__)), "
TESTING_MODE_ENV_DEFAULT = os.environ.get("TESTING_MODE", "false").lower() == "tr

def is_testing_mode() -> bool:
    """Live-checked on every use, not just at startup — reads the file if
    present, otherwise falls back to whatever TESTING_MODE was set to in
    Railway at boot. A missing/corrupt file fails to the env default rather
    than silently going one way or the other.
    """
    try:
        data = _kv_read("setting_testing_mode", TESTING_MODE_FILE, None)
        if not isinstance(data, dict):
            return TESTING_MODE_ENV_DEFAULT
        return bool(data.get("active", TESTING_MODE_ENV_DEFAULT))
    except Exception:
        return TESTING_MODE_ENV_DEFAULT

def set_testing_mode(active: bool):
    _kv_write("setting_testing_mode", TESTING_MODE_FILE,
              {"active": bool(active), "since": datetime.now(timezone.utc).isofor

if is_testing_mode():
    print("[TESTING_MODE] WARNING: same-side cooldown, bias filter and the EMA en

# ─────────────────────────────────────────────
# CLIENT MIRRORING TOGGLES — a deliberately temporary control, requested
# while some live clients don't have a funded balance yet, so the bot
# stops wasting attempts (and alert noise) trying to mirror onto an empty
# account. Meant to be removed from use once everyone live is properly
# funded — going forward, a client only gets pushed to live once every
# prior step is actually confirmed. File-backed so a disabled client
```

```
# doesn't silently re-enable itself on a restart.
# ──────────────────────────────────────────────
CLIENT_MIRRORING_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)),

def _load_client_mirroring_disabled() -> set:
    try:
        data = _kv_read("setting_client_mirroring", CLIENT_MIRRORING_FILE, {})
        return set((data or {}).get("disabled_client_ids", []))
    except Exception:
        return set()

def is_client_mirroring_enabled(client_id: str) -> bool:
    return client_id not in _load_client_mirroring_disabled()

def set_client_mirroring_enabled(client_id: str, enabled: bool):
    disabled = _load_client_mirroring_disabled()
    if enabled:
        disabled.discard(client_id)
    else:
        disabled.add(client_id)
    _kv_write("setting_client_mirroring", CLIENT_MIRRORING_FILE, {"disabled_clien


PROCESS_START_TIME = datetime.now(timezone.utc)  # for the dashboard's uptime dis

DAILY_TRADE_CAP_DEFAULT  = int(os.environ.get("DAILY_TRADE_CAP", "10"))
FIRST_LOSS_TRADE_CAP = int(os.environ.get("FIRST_LOSS_TRADE_CAP", "10"))  # tight
CONSECUTIVE_LOSS_ALERT       = int(os.environ.get("CONSECUTIVE_LOSS_ALERT", "2"))
BALANCE_ALERT_PCT       = float(os.environ.get("BALANCE_ALERT_PCT", "30"))
FEE_ALERT_THRESHOLD     = float(os.environ.get("FEE_ALERT_THRESHOLD", "60"))   #
REVENGE_WINDOW_MINS     = int(os.environ.get("REVENGE_WINDOW_MINS", "20"))      #
OVERTRADE_GAP_MINS      = int(os.environ.get("OVERTRADE_GAP_MINS", "120"))      #
SAME_SIDE_LIMIT         = int(os.environ.get("SAME_SIDE_LIMIT", "2"))
WEBHOOK_SECRET          = os.environ.get("WEBHOOK_SECRET", "")
WEBHOOK_PORT            = int(os.environ.get("PORT", os.environ.get("WEBHOOK_PORT
READ_API_SECRET         = os.environ.get("READ_API_SECRET", "")
MASTER_CLIENT_ID        = "1"

# ──────────────────────────────────────────────
# CRM CLIENT SYNC — pulls mirror clients' credentials from the Orion CRM
# instead of one Railway env var per field per client. Optional: if
# CRM_SYNC_URL/CRM_SYNC_SECRET aren't set, the bot falls back to the
# original CLIENT{n}_* env var scanning in _build_clients() below, so this
# is additive, not a breaking change.
# ──────────────────────────────────────────────
CRM_SYNC_URL            = os.environ.get("CRM_SYNC_URL", "").rstrip("/")   # e.g
```

```
CRM_SYNC_SECRET          = os.environ.get("CRM_SYNC_SECRET", "")           # must
CRM_SYNC_INTERVAL_SECONDS = int(os.environ.get("CRM_SYNC_INTERVAL_SECONDS", "300"
CRM_SYNC_STATUS = {
    "last_attempt": None, "last_success": None, "last_success_time": None,
    "last_count": None, "last_error": None,
}  # for the dashboard's "CRM sync status" display — updated by sync_clients_from
DAILY_LOSS_LIMIT        = float(os.environ.get("DAILY_LOSS_LIMIT", "160"))
DAILY_LOSS_COUNT_LIMIT = int(os.environ.get("DAILY_LOSS_COUNT_LIMIT", "3"))    #
DAILY_LOSS_WARN_PCT     = float(os.environ.get("DAILY_LOSS_WARN_PCT", "80"))
DAILY_PNL_TARGET        = float(os.environ.get("DAILY_PNL_TARGET", "100"))
PROFIT_LOCK_ENABLED     = os.environ.get("PROFIT_LOCK_ENABLED", "false").lower()
WEEKLY_PNL_TARGET       = float(os.environ.get("WEEKLY_PNL_TARGET", "500"))
ENTRY2_EXPIRY_HOURS     = float(os.environ.get("ENTRY2_EXPIRY_HOURS", "2"))
POLL_INTERVAL           = int(os.environ.get("POLL_INTERVAL", "45"))
SLOW_REFRESH_INTERVAL   = int(os.environ.get("SLOW_REFRESH_INTERVAL", "60"))
ORDER_FILL_POLL_INTERVAL = int(os.environ.get("ORDER_FILL_POLL_INTERVAL", "25"))

# ── DAYSCORE integration — auto-ticks the "$60 profit day" criterion when DAILY_
DAYSCORE_PB_URL            = os.environ.get("DAYSCORE_PB_URL", "https://pocketbas
DAYSCORE_PB_ADMIN_EMAIL    = os.environ.get("PB_ADMIN_EMAIL", "")
DAYSCORE_PB_ADMIN_PASSWORD = os.environ.get("PB_ADMIN_PASSWORD", "")
DAYSCORE_OWNER               = "nathan"
DAYSCORE_PROFIT_CRITERIA_ID  = 10   # must match the "$60 profit day" id in perfe

# ── Trade execution ──────────────────────────────────────────────────────
EXECUTION_ENABLED       = os.environ.get("EXECUTION_ENABLED", "false").lower() ==
TRADE_EXECUTION_PACING_SEC = float(os.environ.get("TRADE_EXECUTION_PACING_SEC", "
# A single /long or /short can chain 10+ Binance calls (guards, position
# checks, order placement) in the space of a second when fired directly.
# Signal-prompt trades used to have a human pause (a pre-trade question)
# between the light guard check and the actual heavy execution; that step
# is gone, so both paths now fire the same burst. This adds a small deliberate
# gap between each Binance-touching step during execution, spreading the
# same total call count over several seconds instead of firing it all in
# one burst. Applied twice, before the entry order only (see
# _execute_trade_inner). Temporary measure until the websocket migration
# removes the need for this kind of pacing entirely.
def _pace_execution():
    if TRADE_EXECUTION_PACING_SEC > 0:
        time.sleep(TRADE_EXECUTION_PACING_SEC)
# Paused for now to test reliability of the composite/HTF guards on their own.
# Set to "true" in Railway env vars to re-enable the 7am bias question + bias sup
BIAS_FILTER_ENABLED     = os.environ.get("BIAS_FILTER_ENABLED", "false").lower()
TRADE_SIZE_PCT          = float(os.environ.get("TRADE_SIZE_PCT", "2"))
STOP_PCT                = float(os.environ.get("STOP_PCT", "0.35"))
TP_PCT                  = float(os.environ.get("TP_PCT", "0.40"))
```

```
TP2_PCT                 = float(os.environ.get("TP2_PCT", "0.60"))
TP1_CLOSE_PCT           = float(os.environ.get("TP1_CLOSE_PCT", "50"))
TP1_SL_MOVE_PCT         = float(os.environ.get("TP1_SL_MOVE_PCT", "0.05"))
OVERNIGHT_BE_MIN_PROFIT_PCT = float(os.environ.get("OVERNIGHT_BE_MIN_PROFIT_PCT",
ENTRY2_OFFSET_PCT       = float(os.environ.get("ENTRY2_OFFSET_PCT", "0.12"))
ENTRY2_SIZE_PCT         = float(os.environ.get("ENTRY2_SIZE_PCT", "30"))
ENTRY2_SL_PCT           = float(os.environ.get("ENTRY2_SL_PCT", "0.3"))
LEVERAGE                = int(os.environ.get("LEVERAGE", "60"))
MAKER_ENTRY             = os.environ.get("MAKER_ENTRY", "true").lower() == "true"
MAKER_OFFSET_PCT        = float(os.environ.get("MAKER_OFFSET_PCT", "0.01"))
MAKER_TIMEOUT_SEC       = int(os.environ.get("MAKER_TIMEOUT_SEC", "120"))

# ── Hyperliquid Mirror — generalized multi-client registry ──────────────
# Client #3 onward: this was rebuilt from the earlier hl_/hl2_ duplicated
# pattern into a proper looped/config-list structure, since a real client #3
# arrived. New clients are added purely via CLIENT{n}_* env vars — no code
# changes needed. Client #1 and #2 keep working off their existing HL_/HL2_
# Railway vars (no need to rename anything already deployed); CLIENT1_*/
# CLIENT2_* env vars, if set, simply take precedence.
HL_BASE_URL             = "https://api.hyperliquid.xyz"
HL_COIN                 = "BTC"


class ClientConfig:
    def __init__(self, cid: str, private_key: str, address: str, leverage: int,
                 size_pct: float, dashboard_token: str = "", name: str = None):
        self.id          = cid            # e.g. "1", "2", "3" — used in logs/lab
        self.private_key = private_key
        self.address     = address.lower() if address else address
        self.leverage    = 40  # Hyperliquid max; always 40x
        self.size_pct    = size_pct
        self.dashboard_token = dashboard_token  # unique per-client link secret (
        self.name        = name           # from the CRM — used for the dashboard
        # per-client runtime state (was a separate set of globals per client befo
        self.trades_today        = 0
        self.wins_today          = 0
        self.losses_today        = 0

    def is_configured(self) -> bool:
        return HL_SDK_AVAILABLE and bool(self.private_key) and bool(self.address)

    def label(self) -> str:
        return f"Client #{self.id}"


def _env(key: str, default: str = "") -> str:
```

```
    return os.environ.get(key, default)


def _build_clients() -> list:
    clients = []

    # Client #1 — falls back to legacy HL_* vars already deployed on Railway
    c1_key  = _env("CLIENT1_PRIVATE_KEY")     or _env("HL_PRIVATE_KEY")
    c1_addr = _env("CLIENT1_ACCOUNT_ADDRESS") or _env("HL_ACCOUNT_ADDRESS")
    c1_lev  = int(_env("CLIENT1_LEVERAGE")    or _env("HL_LEVERAGE", "60"))
    c1_pct  = float(_env("CLIENT1_SIZE_PCT")  or _env("HL_SIZE_PCT", "70"))
    c1_dtok = _env("CLIENT1_DASHBOARD_TOKEN")
    clients.append(ClientConfig("1", c1_key, c1_addr, c1_lev, c1_pct, c1_dtok))

    # Client #2 — falls back to legacy HL2_* vars already deployed on Railway
    c2_key  = _env("CLIENT2_PRIVATE_KEY")     or _env("HL2_PRIVATE_KEY")
    c2_addr = _env("CLIENT2_ACCOUNT_ADDRESS") or _env("HL2_ACCOUNT_ADDRESS")
    c2_lev  = int(_env("CLIENT2_LEVERAGE")    or _env("HL2_LEVERAGE", "60"))
    c2_pct  = float(_env("CLIENT2_SIZE_PCT")  or _env("HL2_SIZE_PCT", "70"))
    c2_dtok = _env("CLIENT2_DASHBOARD_TOKEN")
    clients.append(ClientConfig("2", c2_key, c2_addr, c2_lev, c2_pct, c2_dtok))

    # Client #3 onward — pure CLIENT{n}_* vars, just keep adding without touching
    n = 3
    while True:
        key  = _env(f"CLIENT{n}_PRIVATE_KEY")
        addr = _env(f"CLIENT{n}_ACCOUNT_ADDRESS")
        dtok = _env(f"CLIENT{n}_DASHBOARD_TOKEN")
        # Stop scanning only once NONE of a client's fields are set.
        if not key and not addr and not dtok:
            break
        lev  = int(_env(f"CLIENT{n}_LEVERAGE", "60"))
        pct  = float(_env(f"CLIENT{n}_SIZE_PCT", "70"))
        clients.append(ClientConfig(str(n), key, addr, lev, pct, dtok))
        n += 1

    return clients


CLIENTS = _build_clients()  # populated at import time; HL_SDK_AVAILABLE must alr


def sync_clients_from_crm():
    """Pulls mirror clients' address/private_key/dashboard_token from the
    Orion CRM's /api/bot/sync endpoint and merges them into the global
    CLIENTS list, so onboarding a client is a CRM edit, not a Railway env
```

```
var edit + redeploy.

client "1" (Nathan's own Binance master account) is NEVER touched by
this — it always comes from env vars / legacy HL_* vars via
_build_clients(), since it isn't a Hyperliquid mirror client and the
CRM has no concept of it.

Existing ClientConfig objects are updated in place where the id already
exists, rather than replaced, so today's trade counters, win/loss
tallies, and Telegram polling offsets survive a resync — only the
identity/credential fields (address, private_key, dashboard_token) get
refreshed from the CRM. A client only appears at all once the CRM
confirms both a wallet address and a private key are set; that mirrors
ClientConfig.is_configured() so a resync can't introduce a half-set-up
client into the live list.

No-ops if CRM_SYNC_URL/CRM_SYNC_SECRET aren't set — env-var-only mode
keeps working exactly as before. Any failure (network, bad response,
CRM down) logs and leaves CLIENTS untouched rather than wiping the
existing list, since a transient CRM outage should never take down
mirroring for clients who were already configured.
"""
global CLIENTS
if not CRM_SYNC_URL or not CRM_SYNC_SECRET:
    return

try:
    resp = requests.get(
        f"{CRM_SYNC_URL}/api/bot/sync",
        headers={"x-bot-sync-key": CRM_SYNC_SECRET},
        timeout=10,
    )
    resp.raise_for_status()
    payload = resp.json()
except Exception as e:
    print(f"[CRM Sync] Failed to fetch client list — keeping existing clients
    CRM_SYNC_STATUS["last_attempt"] = datetime.now(timezone.utc).isoformat()
    CRM_SYNC_STATUS["last_success"] = False
    CRM_SYNC_STATUS["last_error"] = str(e)
    return

existing_by_id = {c.id: c for c in CLIENTS}
new_clients = []

master = existing_by_id.get(MASTER_CLIENT_ID)
if master:
```

```
            new_clients.append(master)

        seen_ids = set()
        for entry in payload.get("clients", []):
            cid = str(entry.get("id", "")).strip()
            if not cid or cid == MASTER_CLIENT_ID or cid in seen_ids:
                continue  # blank id, an attempt to override the master slot, or a du
            seen_ids.add(cid)

            address = entry.get("address")
            private_key = entry.get("private_key")
            dashboard_token = entry.get("dashboard_token")
            name = entry.get("name")
            if not address or not private_key:
                continue  # CRM's own sync endpoint already filters these, but don't

            # Leverage/size %/Telegram aren't CRM-managed fields (yet) — keep
            # reading them from this client's own env vars if Nathan has set
            # per-client overrides, else fall back to the same defaults
            # _build_clients() uses.
            lev  = int(os.environ.get(f"CLIENT{cid}_LEVERAGE", "60"))
            pct  = float(os.environ.get(f"CLIENT{cid}_SIZE_PCT", "70"))

            if cid in existing_by_id:
                c = existing_by_id[cid]
                c.address = address.lower() if address else address
                c.private_key = private_key
                c.dashboard_token = dashboard_token
                c.name = name
                c.leverage = 40
                c.size_pct = pct
            else:
                c = ClientConfig(cid, private_key, address, lev, pct, dashboard_token
            new_clients.append(c)

        CLIENTS = new_clients
        mirror_count = len(new_clients) - (1 if master else 0)
        print(f"[CRM Sync] Synced {mirror_count} mirror client(s) from CRM")
        CRM_SYNC_STATUS["last_attempt"] = datetime.now(timezone.utc).isoformat()
        CRM_SYNC_STATUS["last_success"] = True
        CRM_SYNC_STATUS["last_success_time"] = CRM_SYNC_STATUS["last_attempt"]
        CRM_SYNC_STATUS["last_count"] = mirror_count
        CRM_SYNC_STATUS["last_error"] = None


    def _crm_sync_loop():
```

```
    while True:
        time.sleep(CRM_SYNC_INTERVAL_SECONDS)
        sync_clients_from_crm()


if CRM_SYNC_URL and CRM_SYNC_SECRET:
    sync_clients_from_crm()  # initial pull at startup, before the periodic loop
    threading.Thread(target=_crm_sync_loop, daemon=True).start()
    print(f"[CRM Sync] Enabled — refreshing every {CRM_SYNC_INTERVAL_SECONDS}s fr
AUTO_TRADE_SIGNALS      = os.environ.get("AUTO_TRADE_SIGNALS", "").split(",")

# Cooldown window (UK time — auto-adjusts for GMT/BST)
COOLDOWN_START  = dt_time(14, 0)
COOLDOWN_END    = dt_time(15, 15)

def _parse_time(val: str, default: dt_time) -> dt_time:
    try:
        h, m = val.strip().split(":")
        return dt_time(int(h), int(m))
    except Exception:
        return default

_raw_cooldown_start = os.environ.get("COOLDOWN_START", "")
_raw_cooldown_end   = os.environ.get("COOLDOWN_END", "")
_raw_night_block    = os.environ.get("NIGHT_BLOCK_START", "")

if _raw_cooldown_start: COOLDOWN_START = _parse_time(_raw_cooldown_start, COOLDOW
if _raw_cooldown_end:   COOLDOWN_END   = _parse_time(_raw_cooldown_end,   COOLDOW
NIGHT_BLOCK_START = _parse_time(_raw_night_block, dt_time(22, 45))
UK_TZ           = ZoneInfo("Europe/London")

FUTURES_BASE    = "https://fapi.binance.com"
MAIN_BASE       = "https://api.binance.com"   # used for Universal Transfer (futu
DATABASE_URL    = os.environ.get("DATABASE_URL", "")
STATS_FILE      = "weekly_stats.json"

# ─────────────────────────────────────────────
# DAILY STATE
# ─────────────────────────────────────────────
early_warned_today          = False
warned_today                = False
stopped_today               = False
DAILY_TRADE_CAP              = DAILY_TRADE_CAP_DEFAULT   # can tighten to FIRST_L
first_loss_tightened_today  = False
loss_streak_alerted         = False
balance_up_alerted          = False
```

```
balance_down_alerted        = False
fee_alerted_today           = False
eod_summary_sent_today      = False
morning_recap_sent_today    = False
last_cooldown_warned_count  = 0
last_reset_day              = None
snapshot_balance            = None
yesterday_stats             = None
same_side_alerted           = False
same_side_block: dict       = {}
daily_losses_block          = False   # set True when 3 losses hit — resets midni
daily_losses_block_reset_at: float = 0.0  # timestamp of last midnight reset
last_loss_close_time: float = 0.0     # timestamp of last losing close — for reve
last_tp2_close_time: float = 0.0      # timestamp of last TP2 close — for post-wi
maker_entry_cancel_requested = False  # set True via /cancel_entry to abort pendi

htf_bias                    = None   # "bullish" or "bearish" — set by 8H structu
htf_bias_updated_at         = None   # timestamp of last update, for staleness ch
EMA_GUARD_ENABLED           = os.environ.get("EMA_GUARD_ENABLED", "false").lower(
SIMPLE_EMA_GUARD_ENABLED    = os.environ.get("SIMPLE_EMA_GUARD_ENABLED", "false")
signal_approved             = {"long": None, "short": None}
last_trigger                = None
last_trigger_time           = None
last_trigger_direction      = None
last_short_trigger_time     = None
last_long_trigger_time      = None
pending_trigger_id          = None
TRIGGER_ATTRIBUTION_SECS    = 3600
revenge_trade_ids_alerted   = set()
overtrade_alerted           = False
pending_bias_reset           = None   # holds signal_key awaiting a bias-reset co
elevated_silence_until       = 0.0    # timestamp until which prompts are silence

daily_target_alerted        = False
weekly_target_alerted       = False
daily_loss_warn_alerted     = False
daily_loss_stop_alerted     = False
dayscore_ticked_today       = False  # resets daily — guards against re-ticking a
profit_locked_today: float  = 0.0   # cumulative amount already moved to Spot tod
profit_lock_last_attempt_at: float = 0.0   # timestamp of last transfer attempt (
                                           # retrying/re-alerting every single p
                                           # failing (e.g. missing API permissio
                                           # failure message every ~15s until ma
PROFIT_LOCK_RETRY_COOLDOWN_SEC = 1800  # 30 min between retry attempts after a fa
overnight_alerted_1015      = False
overnight_alerted_1045      = False
```

```
overnight_auto_breakeven_done = False

entry2_placed_at: float     = 0.0
position_mins_profit: float = 0.0
position_mins_under: float  = 0.0
underwater_ratio_alerted    = False  # one-shot per position, resets when positio
UNDERWATER_RATIO_THRESHOLD  = 1.5    # alert when underwater time >= this multipl
UNDERWATER_ALERT_MIN_AGE_MINS = float(os.environ.get("UNDERWATER_ALERT_MIN_AGE_MI
position_first_seen_at       = 0.0   # when the poll loop first saw the CURRENT p
trade_mfe: float            = 0.0   # max favorable excursion (furthest in-profit
trade_mae: float            = 0.0   # max adverse excursion (furthest against-you
trade_entry_time: float     = 0.0   # timestamp when trade opened (for time_to_tp
retrace_protect_triggered   = False # whether auto-breakeven retracement protecti

tracked_orders: dict = {}
current_trade_entry: dict = {}
indicator_trade_ids: set  = set()

win_streak_alerted_at       = 0
daily_streaks_3plus         = 0

rules_broken_today          = {}


# ─────────────────────────────────────────────
# DATABASE
# ─────────────────────────────────────────────

def get_db():
    return psycopg2.connect(DATABASE_URL)


def init_db():
    if not DATABASE_URL:
        print("[DB] No DATABASE_URL set — trigger tracking disabled.")
        return
    try:
        conn = get_db()
        cur  = conn.cursor()
        cur.execute("""
            CREATE TABLE IF NOT EXISTS trigger_performance (
                id              SERIAL PRIMARY KEY,
                trade_date      DATE,
                trigger         TEXT,
                direction       TEXT,
                outcome         TEXT,
```

```
        pnl             FLOAT,
        entry_price     FLOAT,
        sl_price        FLOAT,
        tp1_price       FLOAT,
        tp2_price       FLOAT,
        mae             FLOAT,
        mfe             FLOAT,
        time_to_tp1_mins FLOAT,
        entry_time      TIMESTAMPTZ DEFAULT NOW(),
        close_time      TIMESTAMPTZ,
        created_at      TIMESTAMPTZ DEFAULT NOW()
    )
""")
cur.execute("""
    CREATE TABLE IF NOT EXISTS trigger_fires (
        id          SERIAL PRIMARY KEY,
        trigger     TEXT,
        fired_at    TIMESTAMPTZ DEFAULT NOW()
    )
""")
cur.execute("""
    CREATE TABLE IF NOT EXISTS invalidation_log (
        id          SERIAL PRIMARY KEY,
        logged_at   TIMESTAMPTZ DEFAULT NOW(),
        trigger     TEXT,
        direction   TEXT
    )
""")
cur.execute("""
    CREATE TABLE IF NOT EXISTS warnings_log (
        id          SERIAL PRIMARY KEY,
        logged_at   TIMESTAMPTZ DEFAULT NOW(),
        category    TEXT,
        message     TEXT
    )
""")
cur.execute("""
    CREATE TABLE IF NOT EXISTS compounding_targets (
        id                  SERIAL PRIMARY KEY,
        period_type         TEXT,          -- 'week' or 'cycle' (4-week)
        period_start        DATE,
        starting_balance    FLOAT,
        created_at          TIMESTAMPTZ DEFAULT NOW()
    )
""")
cur.execute("""
```

```
    CREATE TABLE IF NOT EXISTS persistent_state (
        key         TEXT PRIMARY KEY,
        value       TEXT,
        updated_at  TIMESTAMPTZ DEFAULT NOW()
    )
""")
cur.execute("""
    CREATE TABLE IF NOT EXISTS daily_balance_log (
        id          SERIAL PRIMARY KEY,
        log_date    DATE UNIQUE,
        balance     FLOAT,
        logged_at   TIMESTAMPTZ DEFAULT NOW()
    )
""")
cur.execute("""
    CREATE TABLE IF NOT EXISTS locked_targets (
        id                  SERIAL PRIMARY KEY,
        week_start          DATE UNIQUE,
        weekly_multiplier   FLOAT,
        daily_multiplier    FLOAT,
        locked_at           TIMESTAMPTZ DEFAULT NOW()
    )
""")
cur.execute("""
    CREATE TABLE IF NOT EXISTS virtual_trigger_trades (
        id              SERIAL PRIMARY KEY,
        trigger         TEXT,
        direction       TEXT,
        entry_price     FLOAT,
        target_price    FLOAT,
        stop_price      FLOAT,
        outcome         TEXT,          -- 'win', 'loss', or NULL while st
        exit_price      FLOAT,
        opened_at       TIMESTAMPTZ DEFAULT NOW(),
        closed_at       TIMESTAMPTZ
    )
""")
cur.execute("""
    CREATE TABLE IF NOT EXISTS htf_bias_changes (
        id              SERIAL PRIMARY KEY,
        logged_at       TIMESTAMPTZ DEFAULT NOW(),
        new_bias        TEXT,
        triggered_by    TEXT,
        action          TEXT
    )
""")
```

```
cur.execute("""
    CREATE TABLE IF NOT EXISTS track_record_daily (
        account_id  TEXT NOT NULL,
        day         DATE NOT NULL,
        trades      INTEGER DEFAULT 0,
        wins        INTEGER DEFAULT 0,
        losses      INTEGER DEFAULT 0,
        wins_usd    DOUBLE PRECISION DEFAULT 0,
        losses_usd  DOUBLE PRECISION DEFAULT 0,
        gross_pnl   DOUBLE PRECISION DEFAULT 0,
        fees        DOUBLE PRECISION DEFAULT 0,
        net_pnl     DOUBLE PRECISION DEFAULT 0,
        notional    DOUBLE PRECISION DEFAULT 0,
        updated_at  TIMESTAMPTZ DEFAULT NOW(),
        PRIMARY KEY (account_id, day)
    )
""")
cur.execute("""
    CREATE TABLE IF NOT EXISTS mirror_exec_log (
        id            SERIAL PRIMARY KEY,
        logged_at     TIMESTAMPTZ DEFAULT NOW(),
        account_id    TEXT,
        role          TEXT,
        direction     TEXT,
        ref_price     DOUBLE PRECISION,
        fill_price    DOUBLE PRECISION,
        slippage_bps  DOUBLE PRECISION
    )
""")
cur.execute("""
    CREATE TABLE IF NOT EXISTS trade_source_log (
        id          SERIAL PRIMARY KEY,
        order_id    TEXT,
        opened_ms   BIGINT,
        direction   TEXT,
        source      TEXT,
        label       TEXT
    )
""")
cur.execute("""
    CREATE TABLE IF NOT EXISTS track_record_source (
        account_id  TEXT NOT NULL,
        day         DATE NOT NULL,
        source      TEXT NOT NULL,
        trades      INTEGER DEFAULT 0,
        wins        INTEGER DEFAULT 0,
```

```
            losses      INTEGER DEFAULT 0,
            wins_usd    DOUBLE PRECISION DEFAULT 0,
            losses_usd  DOUBLE PRECISION DEFAULT 0,
            gross_pnl   DOUBLE PRECISION DEFAULT 0,
            fees        DOUBLE PRECISION DEFAULT 0,
            net_pnl     DOUBLE PRECISION DEFAULT 0,
            notional    DOUBLE PRECISION DEFAULT 0,
            PRIMARY KEY (account_id, day, source)
        )
    """)
    conn.commit()
    cur.close()
    conn.close()
    print("[DB] Tables ready.")

    # ── Migration: add entry quality columns if not yet present ──────
    try:
        conn = get_db()
        cur  = conn.cursor()
        for col, coltype in [
            ("entry_price",      "FLOAT"),
            ("sl_price",         "FLOAT"),
            ("tp1_price",        "FLOAT"),
            ("tp2_price",        "FLOAT"),
            ("mae",              "FLOAT"),
            ("mfe",              "FLOAT"),
            ("time_to_tp1_mins", "FLOAT"),
            ("time_to_tp2_mins", "FLOAT"),
            ("time_to_sl_mins",  "FLOAT"),
        ]:
            cur.execute(f"""
                ALTER TABLE trigger_performance
                ADD COLUMN IF NOT EXISTS {col} {coltype}
            """)
        conn.commit()
        cur.close()
        conn.close()
        print("[DB] Entry quality columns ready.")
    except Exception as e:
        print(f"[DB] Migration warning: {e}")


except Exception as e:
    print(f"[DB] Init error: {e}")
```

```
def save_persistent_state(key: str, value):
    """
    Saves a value to the database so it survives a restart — used for
    things that previously only lived in memory
    and got wiped back to None every time the bot restarted (which has
    happened repeatedly from rate-limit bans), even though the last real
    TradingView alert value was still perfectly valid.
    """
    if not DATABASE_URL:
        return
    try:
        conn = get_db()
        cur  = conn.cursor()
        cur.execute("""
            INSERT INTO persistent_state (key, value, updated_at)
            VALUES (%s, %s, NOW())
            ON CONFLICT (key) DO UPDATE SET value = EXCLUDED.value, updated_at =
        """, (key, str(value)))
        conn.commit()
        cur.close()
        conn.close()
    except Exception as e:
        print(f"[Persistent State] Save error for '{key}': {e}")


def load_persistent_state(key: str, max_age_sec: int = None):
    """
    Loads a previously saved value. If max_age_sec is given and the stored
    value is older than that, returns None instead — for values where a
    very stale restore would be worse than starting fresh (composite,
    for instance, shouldn't be trusted if it's a day old).
    Returns None if not found, too old, or on any error.
    """
    if not DATABASE_URL:
        return None
    try:
        conn = get_db()
        cur  = conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor)
        cur.execute("SELECT value, updated_at FROM persistent_state WHERE key = %
        row = cur.fetchone()
        cur.close()
        conn.close()
        if not row:
            return None
        if max_age_sec is not None:
            age = (datetime.now(timezone.utc) - row["updated_at"]).total_seconds(
```

```
            if age > max_age_sec:
                return None
        return row["value"]
    except Exception as e:
        print(f"[Persistent State] Load error for '{key}': {e}")
        return None
```

```
def db_log_htf_bias_change(new_bias: str, triggered_by: str, action: str):
    """Log when HTF bias changes are detected and handled."""
    if not DATABASE_URL:
        return
    try:
        conn = get_db()
        cur  = conn.cursor()
        cur.execute("""
            INSERT INTO htf_bias_changes (logged_at, new_bias, triggered_by, acti
            VALUES (NOW(), %s, %s, %s)
        """, (new_bias, triggered_by, action))
        conn.commit()
        cur.close()
        conn.close()
        print(f"[HTF] Bias change logged: {new_bias} via {triggered_by} ({action}
    except Exception as e:
        print(f"[HTF] Log error: {e}")


def db_log_trigger_fire(trigger: str):
    if not DATABASE_URL:
        return
    try:
        conn = get_db()
        cur  = conn.cursor()
        cur.execute("INSERT INTO trigger_fires (trigger) VALUES (%s)", (trigger,)
        conn.commit()
        cur.close()
        conn.close()
    except Exception as e:
        print(f"[DB] Log trigger fire error: {e}")
```

```
def db_log_invalidation(trigger: str, direction: str):
    if not DATABASE_URL:
        return
    try:
        conn = get_db()
        cur  = conn.cursor()
        cur.execute("INSERT INTO invalidation_log (trigger, direction) VALUES (%s
        conn.commit()
        cur.close()
        conn.close()
    except Exception as e:
        print(f"[DB] Log invalidation error: {e}")


def db_log_warning(category: str, message: str):
    """Log a warning-type alert for later review via /warnings."""
    if not DATABASE_URL:
        return
    try:
        conn = get_db()
        cur  = conn.cursor()
        cur.execute("INSERT INTO warnings_log (category, message) VALUES (%s, %s)
        conn.commit()
        cur.close()
        conn.close()
    except Exception as e:
        print(f"[DB] Log warning error: {e}")


def db_was_warning_logged_today(category: str) -> bool:
    """
    Checks the DATABASE (not an in-memory flag) for whether a warning of this
    category has already been logged today (UK calendar day). Used so
    once-per-day alerts (like the daily loss block) don't re-fire after a
    Railway restart wipes in-memory flags — the flag disappears on restart,
    but the DB record of "I already told you this today" survives.
    """
    if not DATABASE_URL:
        return False
    try:
        today_start_uk = datetime.now(UK_TZ).replace(hour=0, minute=0, second=0,
        today_start_utc = today_start_uk.astimezone(timezone.utc)
        conn = get_db()
        cur  = conn.cursor()
        cur.execute(
            "SELECT 1 FROM warnings_log WHERE category = %s AND logged_at >= %s L
```

```
            (category, today_start_utc)
        )
        row = cur.fetchone()
        cur.close()
        conn.close()
        return row is not None
    except Exception as e:
        print(f"[DB] Warning-today check error: {e}")
        return False  # fail open — if the DB check fails, don't block a real ale


def db_was_message_logged_today(category: str, must_contain: str) -> bool:
    """
    Like db_was_warning_logged_today, but for cases where the same category
    can legitimately fire multiple times per day (e.g. revenge_trade) and we
    need to dedup on a specific identifier (e.g. a Binance order ID) rather
    than "any alert of this category today".
    """
    if not DATABASE_URL:
        return False
    try:
        today_start_uk = datetime.now(UK_TZ).replace(hour=0, minute=0, second=0,
        today_start_utc = today_start_uk.astimezone(timezone.utc)
        conn = get_db()
        cur  = conn.cursor()
        cur.execute(
            "SELECT 1 FROM warnings_log WHERE category = %s AND message LIKE %s A
            (category, f"%{must_contain}%", today_start_utc)
        )
        row = cur.fetchone()
        cur.close()
        conn.close()
        return row is not None
    except Exception as e:
        print(f"[DB] Message-today check error: {e}")
        return False


def db_get_invalidation_stats() -> dict:
    """Returns {trigger: {invalidations, fires, rate_pct}} for every trigger that
    if not DATABASE_URL:
        return {}
    try:
        conn = get_db()
        cur  = conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor)
        cur.execute("SELECT trigger, COUNT(*) as cnt FROM invalidation_log GROUP
```

```
        inv_counts = {r["trigger"]: r["cnt"] for r in cur.fetchall()}
        cur.execute("SELECT trigger, COUNT(*) as cnt FROM trigger_fires GROUP BY
        fire_counts = {r["trigger"]: r["cnt"] for r in cur.fetchall()}
        cur.close()
        conn.close()

        stats = {}
        all_triggers = set(inv_counts.keys()) | set(fire_counts.keys())
        for t in all_triggers:
            fires = fire_counts.get(t, 0)
            invs  = inv_counts.get(t, 0)
            rate  = (invs / fires * 100) if fires > 0 else None
            stats[t] = {"invalidations": invs, "fires": fires, "rate_pct": rate}
        return stats
    except Exception as e:
        print(f"[DB] Invalidation stats error: {e}")
        return {}


def db_get_avg_trigger_gaps() -> dict:
    if not DATABASE_URL:
        return {}
    try:
        conn = get_db()
        cur  = conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor)
        cur.execute("""
            SELECT trigger, fired_at
            FROM trigger_fires
            ORDER BY trigger, fired_at
        """)
        rows = cur.fetchall()
        cur.close()
        conn.close()

        from collections import defaultdict
        fires = defaultdict(list)
        for r in rows:
            fires[r["trigger"]].append(r["fired_at"])

        avg_gaps = {}
        for trigger, times in fires.items():
            if len(times) < 2:
                avg_gaps[trigger] = None
            else:
                gaps = [(times[i] - times[i-1]).total_seconds() / 60 for i in ran
                avg_gaps[trigger] = sum(gaps) / len(gaps)
```

```
        return avg_gaps
    except Exception as e:
        print(f"[DB] Avg trigger gaps error: {e}")
        return {}


def db_save_trigger(trigger: str, direction: str,
                    entry_price: float = None, sl_price: float = None,
                    tp1_price: float = None, tp2_price: float = None) -> int | No
    if not DATABASE_URL:
        return None
    try:
        conn = get_db()
        cur  = conn.cursor()
        cur.execute("""
            INSERT INTO trigger_performance
                (trade_date, trigger, direction, outcome, pnl,
                 entry_price, sl_price, tp1_price, tp2_price)
            VALUES (%s, %s, %s, 'open', 0, %s, %s, %s, %s)
            RETURNING id
        """, (datetime.now(UK_TZ).date(), trigger, direction,
              entry_price, sl_price, tp1_price, tp2_price))
        row_id = cur.fetchone()[0]
        conn.commit()
        cur.close()
        conn.close()
        return row_id
    except Exception as e:
        print(f"[DB] Save trigger error: {e}")
        return None


def db_update_outcome(row_id: int, outcome: str, pnl: float,
                      mae: float = None, mfe: float = None,
                      time_to_tp1_mins: float = None,
                      time_to_tp2_mins: float = None,
                      time_to_sl_mins: float = None):
    if not DATABASE_URL or not row_id:
        return
    try:
        conn = get_db()
        cur  = conn.cursor()
        cur.execute("""
            UPDATE trigger_performance
            SET outcome = %s, pnl = %s, close_time = NOW(),
                mae = %s, mfe = %s,
```

```
                time_to_tp1_mins = %s,
                time_to_tp2_mins = %s,
                time_to_sl_mins  = %s
            WHERE id = %s
        """, (outcome, pnl, mae, mfe,
              time_to_tp1_mins, time_to_tp2_mins, time_to_sl_mins,
              row_id))
        conn.commit()
        cur.close()
        conn.close()
    except Exception as e:
        print(f"[DB] Update outcome error: {e}")


def db_get_trigger_performance() -> list:
    if not DATABASE_URL:
        return []
    try:
        conn = get_db()
        cur  = conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor)
        cur.execute("""
            SELECT trigger, direction, outcome, pnl
            FROM trigger_performance
            WHERE outcome IN ('win', 'loss')
            ORDER BY created_at DESC
        """)
        rows = cur.fetchall()
        cur.close()
        conn.close()
        return rows
    except Exception as e:
        print(f"[DB] Get trigger performance error: {e}")
        return []


def is_rate_limit_error(exc: Exception) -> bool:
    """Detects Binance 418 (IP auto-banned) or 429 (rate limit warning) responses
    return isinstance(exc, requests.exceptions.HTTPError) and exc.response is not


def sign_request(params: dict) -> dict:
    query = "&".join(f"{k}={v}" for k, v in params.items())
    signature = hmac.new(
        BINANCE_API_SECRET.encode(),
        query.encode(),
        hashlib.sha256
```

```
    ).hexdigest()
    params["signature"] = signature
    return params


def get_futures_trades(start_ms: int, end_ms: int = None) -> list:
    now_ms  = int(time.time() * 1000)
    params  = {
        "symbol":    SYMBOL,
        "startTime": start_ms,
        "endTime":   end_ms or now_ms,
        "limit":     1000,
        "timestamp": now_ms,
    }
    params = sign_request(params)
    headers = {"X-MBX-APIKEY": BINANCE_API_KEY}
    resp = requests.get(f"{FUTURES_BASE}/fapi/v1/userTrades",
                        params=params, headers=headers, timeout=10)
    resp.raise_for_status()
    return resp.json()


def get_futures_trades_today() -> list:
    # Use UK local midnight as the day boundary — matches check_and_reset()
    # and every other "today" calculation in the bot. Using UTC midnight here
    # created a ~1hr window each night (UTC midnight to UK midnight, or the
    # reverse depending on BST/GMT) where this function and the reset logic
    # disagreed on which day it was, causing yesterday's trades to still
    # count as "today" right after the flags had already reset.
    now_uk = datetime.now(UK_TZ)
    today_start_uk = now_uk.replace(hour=0, minute=0, second=0, microsecond=0)
    today_start_utc = today_start_uk.astimezone(timezone.utc)
    return get_futures_trades(int(today_start_utc.timestamp() * 1000))


_slow_refresh_cache = {"trades": None, "balance": None, "fetched_at": 0}
_last_slow_checks_run = 0.0  # gates WPR/HTF flip/aggressive volume/virtual trigg

def get_slow_refresh_data() -> tuple:
    """
    Cached wrapper around get_futures_trades_today() + get_usdt_balance(),
    used ONLY by the main polling loop. Trade history and balance genuinely
    don't change meaningfully within a SLOW_REFRESH_INTERVAL window — real
    fills only land occasionally, so hitting Binance for this every single
    fast-loop tick was pure waste. On-demand commands (/stats, /experiment
    etc.) still call the underlying functions directly for fresh data —
```

```
    this cache only affects the background loop's own bookkeeping.
    Returns (trades, balance).
    """
    now = time.time()
    if _slow_refresh_cache["trades"] is not None and (now - _slow_refresh_cache["
        return _slow_refresh_cache["trades"], _slow_refresh_cache["balance"]
    trades = get_futures_trades_today()
    try:
        balance = get_usdt_balance()
    except Exception as e:
        print(f"[Slow Refresh] Balance fetch error: {e}")
        balance = _slow_refresh_cache["balance"]  # reuse last known balance rath
    _slow_refresh_cache["trades"]     = trades
    _slow_refresh_cache["balance"]    = balance
    _slow_refresh_cache["fetched_at"] = now
    return trades, balance


def get_usdt_balance() -> float:
    now_ms = int(time.time() * 1000)
    params = {"timestamp": now_ms}
    params = sign_request(params)
    headers = {"X-MBX-APIKEY": BINANCE_API_KEY}
    resp = requests.get(f"{FUTURES_BASE}/fapi/v2/balance",
                        params=params, headers=headers, timeout=10)
    resp.raise_for_status()
    for asset in resp.json():
        if asset["asset"] == "USDT":
            return float(asset["balance"])
    raise ValueError("USDT balance not found.")


def get_usdt_available_balance() -> float:
    """Free USDT not locked in margin — the actual transferable amount."""
    now_ms = int(time.time() * 1000)
    params = {"timestamp": now_ms}
    params = sign_request(params)
    headers = {"X-MBX-APIKEY": BINANCE_API_KEY}
    resp = requests.get(f"{FUTURES_BASE}/fapi/v2/balance",
                        params=params, headers=headers, timeout=10)
    resp.raise_for_status()
    for asset in resp.json():
        if asset["asset"] == "USDT":
            return float(asset["availableBalance"])
    raise ValueError("USDT balance not found.")
```

```
def binance_transfer_futures_to_spot(amount: float) -> dict:
    """Universal Transfer: USDⓈ-M Futures -> Spot. Internal transfer, no withdra
    now_ms = int(time.time() * 1000)
    params = {
        "type":      "UMFUTURE_MAIN",
        "asset":     "USDT",
        "amount":    f"{amount:.2f}",
        "timestamp": now_ms,
    }
    params  = sign_request(params)
    headers = {"X-MBX-APIKEY": BINANCE_API_KEY}
    resp = requests.post(f"{MAIN_BASE}/sapi/v1/asset/transfer",
                         params=params, headers=headers, timeout=10)
    resp.raise_for_status()
    return resp.json()


# ─────────────────────────────────────────────
# HELPERS — ANALYSIS
# ─────────────────────────────────────────────

def group_by_order(trades: list) -> list:
    orders = {}
    for t in trades:
        oid = t.get("orderId", t.get("id"))
        if oid not in orders:
            orders[oid] = {
                "orderId":     oid,
                "side":        t.get("side", ""),
                "time":        int(t["time"]),
                "realizedPnl": 0.0,
                "commission":  0.0,
                "notional":    0.0,  # sum of price*qty across fills, for ROI cal
                "qty":         0.0,
            }
        price = float(t.get("price", 0))
        qty   = float(t.get("qty", 0))
        orders[oid]["realizedPnl"] += float(t.get("realizedPnl", 0))
        orders[oid]["commission"]  += float(t.get("commission", 0))
        orders[oid]["notional"]    += price * qty
        orders[oid]["qty"]         += qty
        orders[oid]["time"]         = min(orders[oid]["time"], int(t["time"]))

    return sorted(orders.values(), key=lambda o: o["time"])
```

```
def group_by_position(trades: list) -> list:
    orders = group_by_order(trades)
    positions = []
    current = None

    for o in orders:
        is_entry = o["realizedPnl"] == 0

        if is_entry:
            if current and current["realizedPnl"] != 0:
                positions.append(current)
            if current is None or current["realizedPnl"] != 0:
                current = {
                    "time":        o["time"],
                    "close_time":  o["time"],
                    "side":        o["side"],
                    "realizedPnl": 0.0,
                    "commission":  o["commission"],
                    "order_ids":   [o["orderId"]],
                    "entry_notional": o["notional"],  # $ value of entry fills, f
                }
            else:
                current["commission"]  += o["commission"]
                current["order_ids"].append(o["orderId"])
                current["entry_notional"] += o["notional"]  # scaling in (Entry2
        else:
            if current is None:
                current = {
                    "time":        o["time"],
                    "close_time":  o["time"],
                    "side":        o["side"],
                    "realizedPnl": 0.0,
                    "commission":  0.0,
                    "order_ids":   [],
                    "entry_notional": 0.0,
                }
            current["realizedPnl"] += o["realizedPnl"]
            current["commission"]  += o["commission"]
            current["close_time"]   = o["time"]
            current["order_ids"].append(o["orderId"])

    if current and current["realizedPnl"] != 0:
        positions.append(current)

    return positions
```

```
TRADING_WINDOW_START_HOUR = int(os.environ.get("TRADING_WINDOW_START_HOUR", "8"))
TRADING_WINDOW_END_HOUR   = int(os.environ.get("TRADING_WINDOW_END_HOUR", "23"))

# ─────────────────────────────────────────────────
# DASHBOARD STATUS BANNERS — maintenance + automatic market-hours note
# ─────────────────────────────────────────────────

MAINTENANCE_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), "main

# ─────────────────────────────────────────────────
# CLIENT DOPAMINE — balance high-water mark + milestone tracking, entirely
# server-side and file-backed so these are genuine achievements (survives
# restarts, can't be re-triggered by refreshing the page, can't be faked
# client-side). Two entry points:
#   hl_realised_high_event() — called on every dashboard poll, based on
#     realised closed-trade profit only, drives the confetti-on-new-high moment.
#   check_milestones() — called once per page load (alongside history,
#     which already computes real daily P&L), drives the one-time banners.
# Both share one small JSON file, one entry per client id.
# ─────────────────────────────────────────────────
CLIENT_MILESTONES_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)),

def _load_client_milestones() -> dict:
    try:
        return _kv_read("setting_client_milestones", CLIENT_MILESTONES_FILE, {})
    except Exception:
        return {}

def _save_client_milestones(data: dict):
    _kv_write("setting_client_milestones", CLIENT_MILESTONES_FILE, data)

def ensure_milestone_entry(client_id: str, balance: float):
    """Creates the client's tracking entry the first time their dashboard is
    seen (first_seen drives the 30-day milestone, baseline_balance is kept
    for reference). It deliberately no longer decides what counts as a "new
    high": comparing the live account balance meant open (unrealised) profit,
    deposits and wallet transfers all looked like achievements, and the
    stored peak reset on every redeploy -- so the confetti fired on page open
    for no real reason. See hl_realised_high_event() for the real check."""
    data = _load_client_milestones()
    if client_id not in data:
        data[client_id] = {
            "first_seen": datetime.now(timezone.utc).isoformat(),
            "baseline_balance": balance,
```

```
            "peak_balance": balance,
            "shown_milestones": [],
            "last_high_event_ms": 0,
        }
        _save_client_milestones(data)


# Per-client Hyperliquid fills, cached briefly so the Today box and the
# new-high check don't each hit Hyperliquid on every dashboard poll.
_hl_fills_cache = {}
HL_FILLS_CACHE_SECS = 60

def hl_client_fills(c, days: int = 60) -> list:
    now = time.time()
    hit = _hl_fills_cache.get(c.id)
    if hit and now - hit[0] < HL_FILLS_CACHE_SECS:
        return hit[1]
    info = hl_get_info()
    start_ms = int((now - days * 86400) * 1000)
    fills = []
    try:
        for _ in range(5):  # page forward in case there are more than 2000 fills
            page = info.user_fills_by_time(c.address, start_ms)
            if not page:
                break
            fills.extend(page)
            if len(page) < 2000:
                break
            start_ms = max(int(f.get("time", 0)) for f in page) + 1
    except HLClientError as e:
        if e.status_code == 422:
            fills = []   # never-funded wallet: nothing to show yet, not an error
        else:
            raise
    _hl_fills_cache[c.id] = (now, fills)
    return fills


def hl_today_stats(fills: list) -> dict:
    """Today's closed trades and wins from the client's real fills, grouped by
    order id exactly like get_hl_client_daily_history, so the Today box and
    the 7-day chart can never disagree. Replaces the old in-memory counters,
    which only moved when the bot itself closed a client position and were
    wiped on every redeploy -- trades that ended on the client's own stop or
    take-profit orders were never counted."""
    now_uk = datetime.now(UK_TZ)
```

```
    start_ms = int(now_uk.replace(hour=0, minute=0, second=0, microsecond=0)
                   .astimezone(timezone.utc).timestamp() * 1000)
    orders = {}
    for f in fills:
        o = orders.setdefault(f.get("oid"), {"pnl": 0.0, "time": 0})
        o["pnl"] += float(f.get("closedPnl", 0) or 0)
        o["time"] = max(o["time"], int(f.get("time", 0)))
    trades = wins = 0
    for o in orders.values():
        if o["pnl"] == 0 or o["time"] < start_ms:
            continue
        trades += 1
        if o["pnl"] > 0:
            wins += 1
    return {"trades": trades, "wins": wins, "win_rate": (wins / trades * 100) if


def hl_realised_high_event(c, fills: list, window_secs: int = 1800, min_closes: i
    """True only when a trade that closed in the last `window_secs` pushed the
    client's cumulative REALISED profit (closed P&L less fees) above every
    earlier point -- a genuine new high. Unrealised profit, deposits and
    transfers between wallets can't trigger it, it needs no stored peak to
    survive redeploys, and each event celebrates once."""
    pts, running = [], 0.0
    for f in sorted(fills, key=lambda x: int(x.get("time", 0))):
        closed = float(f.get("closedPnl", 0) or 0)
        running += closed - float(f.get("fee", 0) or 0)
        if closed != 0:
            pts.append((int(f.get("time", 0)), running))
    if len(pts) < min_closes:
        return False   # too little history for a "high" to mean anything
    last_t, last_v = pts[-1]
    if last_v <= max(v for _, v in pts[:-1]) + 0.01:
        return False
    if time.time() * 1000 - last_t > window_secs * 1000:
        return False
    data = _load_client_milestones()
    entry = data.get(c.id)
    if entry is None or int(entry.get("last_high_event_ms", 0)) >= last_t:
        return False
    entry["last_high_event_ms"] = last_t
    data[c.id] = entry
    _save_client_milestones(data)
    return True
```

```
def check_milestones(client_id: str, today_pnl: float) -> list:
    """Returns any milestones newly achieved as of this call, marking them
    shown so each one only ever fires once per client, ever. If this client
    has no tracking entry yet (dashboard hasn't loaded for them before),
    there's nothing to check yet — the dashboard call that runs alongside
    this always fires first and creates it.
    """
    data = _load_client_milestones()
    entry = data.get(client_id)
    if entry is None:
        return []

    shown = set(entry.get("shown_milestones", []))
    newly_achieved = []

    if today_pnl > 0 and "first_profitable_day" not in shown:
        newly_achieved.append("first_profitable_day")
        shown.add("first_profitable_day")

    first_seen = entry.get("first_seen")
    if first_seen:
        try:
            days = (datetime.now(timezone.utc) - datetime.fromisoformat(first_see
        except Exception:
            days = 0
        if days >= 30 and "30_days" not in shown:
            newly_achieved.append("30_days")
            shown.add("30_days")

    if newly_achieved:
        entry["shown_milestones"] = list(shown)
        data[client_id] = entry
        _save_client_milestones(data)

    return newly_achieved


# ──────────────────────────────────────────────
# CLIENT REMINDERS — a one-time message pushed to a specific client's own
# dashboard (not a global banner like maintenance). Set by the CRM when
# Nathan presses "Remind" on a client who's due for renewal; shown once,
# the next time that client's dashboard loads, then automatically cleared
# so it doesn't linger as a permanent banner.
# ──────────────────────────────────────────────
CLIENT_REMINDERS_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)),
```

```
def _load_client_reminders() -> dict:
    try:
        return _kv_read("setting_client_reminders", CLIENT_REMINDERS_FILE, {}) or
    except Exception:
        return {}

def _save_client_reminders(data: dict):
    _kv_write("setting_client_reminders", CLIENT_REMINDERS_FILE, data)

def set_client_reminder(client_id: str, message: str):
    data = _load_client_reminders()
    data[client_id] = {"message": message, "set_at": datetime.now(timezone.utc).i
    _save_client_reminders(data)

def pop_client_reminder(client_id: str) -> str:
    """Returns this client's pending reminder message, if any, and clears
    it — 'pop' rather than 'get', since a reminder is meant to be shown
    once, not sit as a permanent fixture on their dashboard."""
    data = _load_client_reminders()
    entry = data.pop(client_id, None)
    if entry is not None:
        _save_client_reminders(data)
        return entry.get("message")
    return None


# ──────────────────────────────────────────────
# PUSH NOTIFICATIONS — real OS-level alerts (via the Web Push API), for the
# master console ONLY. Nothing here touches or is visible to any client
# dashboard — the subscribe flow only ever runs on the page shown to
# MASTER_CLIENT_ID, so a client's browser never even sees the code that
# would prompt for notification permission.
#
# Requires two Railway env vars (generate once, keep forever — regenerating
# invalidates every existing subscription):
#   VAPID_PRIVATE_KEY, VAPID_PUBLIC_KEY
# and `pywebpush` added to requirements.txt.
#
# Scope for now, per Nathan: losses, and being stopped for the day. Not
# every trade close, not client-facing events — kept deliberately narrow.
# ──────────────────────────────────────────────
PUSH_SUBSCRIPTIONS_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__))
VAPID_PRIVATE_KEY = os.environ.get("VAPID_PRIVATE_KEY", "")
VAPID_PUBLIC_KEY  = os.environ.get("VAPID_PUBLIC_KEY", "")
VAPID_CLAIMS      = {"sub": "mailto:nathan@orion.local"}  # required by the Push
```

```
# Subscriptions live in Postgres (persistent_state), NOT a file next to the code:
# that file was wiped by every redeploy, which silently stopped all push alerts
# while the browser still showed "On". The file is only a fallback with no databa
_PUSH_SUBS_KEY = "push_subscriptions"

def _load_push_subscriptions() -> list:
    if DATABASE_URL:
        raw = load_persistent_state(_PUSH_SUBS_KEY)
        if raw:
            try:
                return json.loads(raw)
            except Exception:
                return []
        return []
    try:
        with open(PUSH_SUBSCRIPTIONS_FILE, "r") as f:
            return json.load(f)
    except Exception:
        return []

def _save_push_subscriptions(subs: list):
    if DATABASE_URL:
        save_persistent_state(_PUSH_SUBS_KEY, json.dumps(subs))
        return
    with open(PUSH_SUBSCRIPTIONS_FILE, "w") as f:
        json.dump(subs, f)

def add_push_subscription(subscription: dict):
    subs = _load_push_subscriptions()
    # Keyed by endpoint — the same browser/device re-subscribing (a token
    # refresh, say) replaces its old entry rather than piling up duplicates.
    subs = [s for s in subs if s.get("endpoint") != subscription.get("endpoint")]
    subs.append(subscription)
    _save_push_subscriptions(subs)

def remove_push_subscription(endpoint: str):
    subs = _load_push_subscriptions()
    subs = [s for s in subs if s.get("endpoint") != endpoint]
    _save_push_subscriptions(subs)

def send_push_notification(title: str, body: str):
    """Fire-and-forget to every registered device — one failing or expired
    subscription (e.g. the browser was uninstalled) is pruned quietly and
    doesn't stop delivery to any other device.
    """
    if not VAPID_PRIVATE_KEY or not VAPID_PUBLIC_KEY:
```

```
        return  # not configured yet — silently skip rather than error the caller
    subs = _load_push_subscriptions()
    if not subs:
        return
    payload = json.dumps({"title": title, "body": body})
    still_valid = []
    for sub in subs:
        try:
            webpush(
                subscription_info=sub,
                data=payload,
                vapid_private_key=VAPID_PRIVATE_KEY,
                vapid_claims=dict(VAPID_CLAIMS),
            )
            still_valid.append(sub)
        except WebPushException as e:
            status = getattr(e.response, "status_code", None)
            if status in (404, 410):
                print(f"[Push] Subscription expired/gone, removing: {sub.get('end
                continue  # drop this one, don't re-add it
            print(f"[Push] Send failed (keeping subscription): {e}")
            still_valid.append(sub)  # transient failure — keep it, don't give up
        except Exception as e:
            print(f"[Push] Unexpected error (keeping subscription): {e}")
            still_valid.append(sub)
    if len(still_valid) != len(subs):
        _save_push_subscriptions(still_valid)


def get_maintenance_status() -> dict:
    """Manually-set banner shown to every client above their balance —
    covers two distinct situations Nathan needs to flag: Orion being taken
    down for maintenance (security upgrade, logic change), and days he
    simply won't be taking trades (CPI/NFP/FOMC, US market holidays) even
    though the bot itself is fine. `type` picks the wording; `reason` is
    the specific detail. Stored in a small flat JSON file next to the
    script rather than env vars, so Nathan can toggle it via
    /api/admin/maintenance without a redeploy/restart — a restart would
    itself trigger the 'bot restarted mid-session' Telegram alert, which is
    exactly the noise this feature exists to avoid during planned work.
    Resets to inactive if this file doesn't exist or is unreadable, so a
    missing/corrupt file fails safe (no stuck banner) rather than fails
    loud.
    """
    try:
        data = _kv_read("setting_maintenance", MAINTENANCE_FILE, None)
```

```
        if not isinstance(data, dict):
            raise ValueError("no maintenance setting")
        return {
            "active": bool(data.get("active", False)),
            "reason": str(data.get("reason", "")),
            "type": str(data.get("type", "maintenance")),
        }
    except Exception:
        return {"active": False, "reason": "", "type": "maintenance"}


def set_maintenance_status(active: bool, reason: str, status_type: str = "mainten
    _kv_write("setting_maintenance", MAINTENANCE_FILE, {
        "active": bool(active),
        "reason": (reason or "").strip()[:200],
        "type": status_type if status_type in ("maintenance", "pause") else "main
        "since": datetime.now(timezone.utc).isoformat(),
    })


def get_market_status() -> dict:
    """Automatic (not manually set) note for when Orion simply isn't trading
    right now by design — weekends, or outside the TRADING_WINDOW_* hours —
    using the exact same weekday/hour checks the bot's own signal-suppression
    logic already uses, so this banner can never disagree with what the bot
    is actually doing.
    """
    now_uk = datetime.now(UK_TZ)
    if now_uk.weekday() >= 5:  # 5=Saturday, 6=Sunday
        return {"trading": False, "reason": "weekend"}
    if not (TRADING_WINDOW_START_HOUR <= now_uk.hour < TRADING_WINDOW_END_HOUR):
        return {"trading": False, "reason": "outside_hours"}
    return {"trading": True, "reason": None}


def build_stats(trades: list) -> dict:
    orders    = group_by_order(trades)
    positions = group_by_position(trades)
    wins      = [p for p in positions if p["realizedPnl"] > 0]
    losses    = [p for p in positions if p["realizedPnl"] < 0]
    total_pnl  = sum(p["realizedPnl"] for p in positions)
    total_fees = sum(o["commission"]  for o in orders)
    win_rate   = (len(wins) / len(positions) * 100) if positions else 0
    entries    = [o for o in orders if o["realizedPnl"] == 0]

    # ── Average ROI % — realizedPnl as a % of margin used (entry_notional / leve
```

```
    # This is what actually tells you whether wins or losses need work: a
    # 60% win rate with tiny average win % and huge average loss % is a
    # very different problem than a 40% win rate with the reverse.
    def position_roi_pct(p):
        margin = (p.get("entry_notional", 0) / LEVERAGE) if LEVERAGE > 0 else 0
        return (p["realizedPnl"] / margin * 100) if margin > 0 else None

    win_rois  = [r for r in (position_roi_pct(p) for p in wins)  if r is not None
    loss_rois = [r for r in (position_roi_pct(p) for p in losses) if r is not Non
    avg_win_roi_pct  = (sum(win_rois) / len(win_rois))   if win_rois  else 0
    avg_loss_roi_pct = (sum(loss_rois) / len(loss_rois)) if loss_rois else 0

    return {
        "total_trades":     len(entries),
        "closed_positions": len(positions),
        "wins":             len(wins),
        "losses":           len(losses),
        "win_rate":         win_rate,
        "total_pnl":        total_pnl,
        "total_fees":       total_fees,
        "net_pnl":          total_pnl - total_fees,
        "avg_win_roi_pct":  avg_win_roi_pct,
        "avg_loss_roi_pct": avg_loss_roi_pct,
    }


def get_futures_trades_range(start_ms: int, end_ms: int) -> list:
    """Binance's userTrades endpoint caps startTime/endTime to a 7-day span
    per request — anything wider (e.g. a 30-day history request) has to be
    split into consecutive <7-day chunks and concatenated. get_futures_trades()
    alone is fine for a single request under that limit; this wraps it for
    ranges that might exceed it.
    """
    CHUNK_MS = 7 * 24 * 60 * 60 * 1000 - 60_000  # just under 7 days, safety marg
    all_trades = []
    chunk_start = start_ms
    while chunk_start < end_ms:
        chunk_end = min(chunk_start + CHUNK_MS, end_ms)
        all_trades.extend(get_futures_trades(chunk_start, chunk_end))
        chunk_start = chunk_end + 1

    # Chunk boundaries can double-fetch a trade landing exactly on the edge —
    # de-dupe by trade id and restore chronological order before grouping.
    seen = set()
    deduped = []
    for t in all_trades:
```

```
        tid = t.get("id")
        if tid in seen:
            continue
        seen.add(tid)
        deduped.append(t)
    deduped.sort(key=lambda t: int(t["time"]))
    return deduped


def get_binance_daily_history(days: int = 7) -> list:
    """Buckets the master Binance account's closed positions into UK-calendar
    days for the last `days` days (today included). Reuses the same
    group_by_position() logic as build_stats() so a closed trade here means
    exactly the same thing it means everywhere else in the bot.
    """
    now_uk = datetime.now(UK_TZ)
    today_start_uk = now_uk.replace(hour=0, minute=0, second=0, microsecond=0)
    range_start_uk = today_start_uk - timedelta(days=days - 1)
    range_start_ms = int(range_start_uk.astimezone(timezone.utc).timestamp() * 10
    now_ms = int(time.time() * 1000)

    trades = get_futures_trades_range(range_start_ms, now_ms)
    positions = group_by_position(trades)

    buckets = {(range_start_uk + timedelta(days=i)).date(): {"trades": 0, "wins":
    for p in positions:
        close_date = datetime.fromtimestamp(p["close_time"] / 1000, tz=timezone.u
        if close_date in buckets:
            buckets[close_date]["trades"] += 1
            if p["realizedPnl"] > 0:
                buckets[close_date]["wins"] += 1
            buckets[close_date]["pnl"] += p["realizedPnl"]

    result = []
    for i in range(days):
        d = (range_start_uk + timedelta(days=i)).date()
        b = buckets[d]
        result.append({
            "date": d.isoformat(),
            "trades": b["trades"],
            "wins": b["wins"],
            "win_rate": (b["wins"] / b["trades"] * 100) if b["trades"] else 0,
            "pnl": b["pnl"],
        })
    result.reverse()  # most recent day first
    return result
```

```
def get_hl_client_daily_history(c, days: int = 7) -> list:
    """Same idea as get_binance_daily_history() but for a Hyperliquid mirror
    client, built from their raw fills. Fills are grouped by order id (same
    approach as Binance's group_by_order) since a single close can span
    multiple partial fills — an order is only counted as a closed trade if
    its combined closedPnl is non-zero, matching how c.trades_today is
    incremented elsewhere (a real close, not an entry fill).
    """
    now_uk = datetime.now(UK_TZ)
    today_start_uk = now_uk.replace(hour=0, minute=0, second=0, microsecond=0)
    range_start_uk = today_start_uk - timedelta(days=days - 1)
    range_start_ms = int(range_start_uk.astimezone(timezone.utc).timestamp() * 10

    info = hl_get_info()
    try:
        fills = info.user_fills_by_time(c.address, range_start_ms)
    except HLClientError as e:
        if e.status_code == 422:
            # Same never-funded-wallet case as the balance endpoint — no
            # history to show yet, not an actual error.
            fills = []
        else:
            raise

    orders = {}
    for f in fills:
        oid = f.get("oid")
        if oid not in orders:
            orders[oid] = {"closedPnl": 0.0, "time": int(f.get("time", 0))}
        orders[oid]["closedPnl"] += float(f.get("closedPnl", 0) or 0)
        orders[oid]["time"] = max(orders[oid]["time"], int(f.get("time", 0)))

    buckets = {(range_start_uk + timedelta(days=i)).date(): {"trades": 0, "wins":
    for o in orders.values():
        if o["closedPnl"] == 0:
            continue  # a pure entry fill, not a close
        close_date = datetime.fromtimestamp(o["time"] / 1000, tz=timezone.utc).as
        if close_date in buckets:
            buckets[close_date]["trades"] += 1
            if o["closedPnl"] > 0:
                buckets[close_date]["wins"] += 1
            buckets[close_date]["pnl"] += o["closedPnl"]

    result = []
```

```
    for i in range(days):
        d = (range_start_uk + timedelta(days=i)).date()
        b = buckets[d]
        result.append({
            "date": d.isoformat(),
            "trades": b["trades"],
            "wins": b["wins"],
            "win_rate": (b["wins"] / b["trades"] * 100) if b["trades"] else 0,
            "pnl": b["pnl"],
        })
    result.reverse()
    return result


# ============================================================================
# TRACK RECORD -- permanent, queryable performance history in Postgres
# ============================================================================
# Hyperliquid only keeps a limited window of fills, and JSON files next to the
# code are wiped on every Railway deploy -- so neither can be the long-term
# record. Postgres can. Every account (the Binance master and each Hyperliquid
# client) gets one row per day in track_record_daily, rebuilt idempotently from
# the exchange's own fills, so it can be re-run any time without double
# counting. Everything is measured NET of fees, per ROUND-TRIP trade (entry ->
# fully closed, including partial TP1 closes), by the UK date it closed.
# The record starts on TRACK_START_DATE; nothing before it is stored.
TRACK_START_DATE          = os.environ.get("TRACK_START_DATE", "2026-10-01")
TRACK_SNAPSHOT_EVERY_MINS = int(os.environ.get("TRACK_SNAPSHOT_EVERY_MINS", "60")
TRACK_MASTER_ID           = "master"


def _track_start():
    return datetime.strptime(TRACK_START_DATE, "%Y-%m-%d").date()


def log_exec_slippage(account_id: str, role: str, direction: str, ref_price, fill
    """Best-effort record of how far an entry fill landed from the market
    price at the moment the order was sent (positive = worse than the
    reference: paid more on a buy, received less on a sell). Can't be rebuilt
    afterwards, so it is recorded as it happens. Never raises -- logging must
    never get in the way of a trade."""
    try:
        if not DATABASE_URL or not ref_price or not fill_price:
            return
        ref, fill = float(ref_price), float(fill_price)
        if ref <= 0 or fill <= 0:
            return
```

```
        sign = 1.0 if direction == "long" else -1.0
        bps  = (fill - ref) / ref * 10000.0 * sign
        conn = get_db()
        cur  = conn.cursor()
        cur.execute(
            "INSERT INTO mirror_exec_log (account_id, role, direction, ref_price,
            "VALUES (%s, %s, %s, %s, %s, %s)",
            (str(account_id), role, direction, ref, fill, bps))
        conn.commit(); cur.close(); conn.close()
    except Exception as e:
        print(f"[Track] slippage log skipped: {e}")


def log_trade_source(order_id, direction: str, triggered_by) -> None:
    """Records, at entry, whether a trade came from a signal/indicator or from
    a manual /long or /short. Can't be rebuilt afterwards. Best-effort: never
    raises, never delays a trade."""
    try:
        if not DATABASE_URL:
            return
        tb  = str(triggered_by or "")
        src = "manual" if tb.lower().startswith("manual") else "signal"
        conn = get_db(); cur = conn.cursor()
        cur.execute("INSERT INTO trade_source_log (order_id, opened_ms, direction
                    "VALUES (%s,%s,%s,%s,%s)",
                    (str(order_id) if order_id else None, int(time.time() * 1000)
        conn.commit(); cur.close(); conn.close()
    except Exception as e:
        print(f"[Track] trade source log skipped: {e}")


def _load_source_log() -> list:
    try:
        conn = get_db(); cur = conn.cursor()
        cur.execute("SELECT order_id, opened_ms, source FROM trade_source_log ORD
        rows = [(o, int(ms), s_) for o, ms, s_ in cur.fetchall()]
        cur.close(); conn.close()
        return rows
    except Exception as e:
        print(f"[Track] source log read failed: {e}")
        return []


def _source_for_trip(trip: dict, log: list):
    """'signal' / 'manual' / None (no record: trade pre-dates source tracking)."""
    ids = {str(o) for o in (trip.get("order_ids") or [])}
```

```
    for o, _ms, src in log:
        if o and o in ids:
            return src
    # time match (client round trips): the latest logged entry shortly before thi
    open_ms = trip.get("open_ms")
    if open_ms:
        best = None
        for _o, ms, src in log:
            if open_ms - 180000 <= ms <= open_ms + 10000:
                best = src
        return best
    return None


def _empty_day():
    return {"trades": 0, "wins": 0, "losses": 0, "wins_usd": 0.0, "losses_usd": 0
            "gross_pnl": 0.0, "fees": 0.0, "net_pnl": 0.0, "notional": 0.0}


def _bucket_trips(trips: list, start_day, end_day) -> dict:
    """trips: [{close_ms, gross, fees, notional}] -> {date: daily totals}, only
    for days inside [start_day, end_day]. A trip is a win/loss on NET P&L."""
    days = {}
    for t in trips:
        d = datetime.fromtimestamp(t["close_ms"] / 1000, tz=timezone.utc).astimez
        if d < start_day or d > end_day:
            continue
        b = days.setdefault(d, _empty_day())
        net = t["gross"] - t["fees"]
        b["trades"] += 1
        b["gross_pnl"] += t["gross"]
        b["fees"] += t["fees"]
        b["net_pnl"] += net
        b["notional"] += t["notional"]
        if net > 0:
            b["wins"] += 1; b["wins_usd"] += net
        elif net < 0:
            b["losses"] += 1; b["losses_usd"] += -net
    return days


def master_round_trips(trades: list) -> list:
    """Binance fills -> round trips, using the bot's own position grouping."""
    out = []
    for p in group_by_position(trades):
        if p.get("realizedPnl", 0) == 0 and p.get("commission", 0) == 0:
```

```
            continue
        out.append({"open_ms": int(p["time"]), "order_ids": list(p.get("order_ids
                    "close_ms": int(p["close_time"]), "gross": float(p["realizedP
                    "fees": float(p["commission"]), "notional": float(p.get("entr
    return out


def hl_round_trips(fills: list) -> list:
    """Hyperliquid fills -> round trips: from the fill that takes the position
    off flat to the fill that returns it to flat (so partial TP1 closes and
    scaled-in entries belong to one trade). Uses startPosition when the
    exchange provides it, otherwise a running position."""
    trips, cur, running = [], None, 0.0
    for f in sorted((x for x in fills if x.get("coin") in (None, HL_COIN)), key=l
        sz  = float(f.get("sz", 0) or 0)
        sgn = 1.0 if str(f.get("side", "")).upper().startswith("B") else -1.0
        px  = float(f.get("px", 0) or 0)
        before = float(f["startPosition"]) if f.get("startPosition") not in (None
        after  = before + sgn * sz
        running = after
        if cur is None:
            cur = {"open_ms": int(f.get("time", 0)), "close_ms": 0, "gross": 0.0,
        cur["gross"] += float(f.get("closedPnl", 0) or 0)
        cur["fees"]  += float(f.get("fee", 0) or 0)
        if abs(before) < 1e-9 or (before > 0) == (sgn > 0):   # fill that opens /
            cur["notional"] += px * sz
        cur["close_ms"] = int(f.get("time", 0))
        if abs(after) < 1e-9:
            trips.append(cur); cur = None
    return trips


def _fetch_hl_fills(c, start_ms: int) -> list:
    info, fills = hl_get_info(), []
    try:
        for _ in range(8):   # page forward in case there are more than 2000 fill
            page = info.user_fills_by_time(c.address, start_ms)
            if not page:
                break
            fills.extend(page)
            if len(page) < 2000:
                break
            start_ms = max(int(f.get("time", 0)) for f in page) + 1
    except HLClientError as e:
        if e.status_code != 422:   # 422 = never-funded wallet: nothing to record
            raise
```

```
    return fills


def _upsert_track_days(account_id: str, days: dict, fill_gaps_from=None, end_day=
    """Writes one row per day. With fill_gaps_from, days with no trades are
    stored as zero rows so the equity curve has no holes."""
    if fill_gaps_from is not None and end_day is not None:
        d = fill_gaps_from
        while d <= end_day:
            days.setdefault(d, _empty_day())
            d += timedelta(days=1)
    if not days:
        return 0
    conn = get_db(); cur = conn.cursor()
    for d, b in days.items():
        cur.execute("""
            INSERT INTO track_record_daily
                (account_id, day, trades, wins, losses, wins_usd, losses_usd, gro
            VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,NOW())
            ON CONFLICT (account_id, day) DO UPDATE SET
                trades=EXCLUDED.trades, wins=EXCLUDED.wins, losses=EXCLUDED.losse
                wins_usd=EXCLUDED.wins_usd, losses_usd=EXCLUDED.losses_usd, gross
                fees=EXCLUDED.fees, net_pnl=EXCLUDED.net_pnl, notional=EXCLUDED.n
        """, (account_id, d, b["trades"], b["wins"], b["losses"], b["wins_usd"],
              b["gross_pnl"], b["fees"], b["net_pnl"], b["notional"]))
    conn.commit(); cur.close(); conn.close()
    return len(days)


def _upsert_source_days(account_id: str, trips: list, first, today) -> None:
    """Per-day totals split by trade source (signal / manual). Trips with no
    source record are left out here -- they show up as 'unlabelled'."""
    log = _load_source_log()
    buckets = {"signal": [], "manual": []}
    for t in trips:
        src = _source_for_trip(t, log)
        if src in buckets:
            buckets[src].append(t)
    conn = get_db(); cur = conn.cursor()
    # clear the window first so a re-labelled or re-bucketed trade can't linger
    cur.execute("DELETE FROM track_record_source WHERE account_id=%s AND day>=%s"
    for src, ts in buckets.items():
        for d, b in _bucket_trips(ts, first, today).items():
            cur.execute("""
                INSERT INTO track_record_source
                    (account_id, day, source, trades, wins, losses, wins_usd, los
```

```
                VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s)
                ON CONFLICT (account_id, day, source) DO UPDATE SET
                    trades=EXCLUDED.trades, wins=EXCLUDED.wins, losses=EXCLUDED.l
                    wins_usd=EXCLUDED.wins_usd, losses_usd=EXCLUDED.losses_usd, g
                    fees=EXCLUDED.fees, net_pnl=EXCLUDED.net_pnl, notional=EXCLUD
            """, (account_id, d, src, b["trades"], b["wins"], b["losses"], b["win
                  b["gross_pnl"], b["fees"], b["net_pnl"], b["notional"]))
    conn.commit(); cur.close(); conn.close()


def snapshot_track_record(lookback_days=None) -> str:
    """Rebuilds the daily rows for the master and every configured client.
    lookback_days=None rebuilds everything since TRACK_START_DATE (first run /
    backfill); a small number just refreshes the most recent days."""
    if not DATABASE_URL:
        return "no database"
    today = datetime.now(UK_TZ).date()
    start = _track_start()
    first = start if lookback_days is None else max(start, today - timedelta(days
    if first > today:
        return "track record not started yet"
    # fetch a day earlier so a trade opened before the window but closed inside i
    fetch_from = datetime(first.year, first.month, first.day, tzinfo=UK_TZ) - tim
    fetch_ms   = int(fetch_from.astimezone(timezone.utc).timestamp() * 1000)
    written = []
    try:
        trades = get_futures_trades_range(fetch_ms, int(time.time() * 1000))
        m_trips = master_round_trips(trades)
        n = _upsert_track_days(TRACK_MASTER_ID, _bucket_trips(m_trips, first, tod
                               fill_gaps_from=first, end_day=today)
        try:
            _upsert_source_days(TRACK_MASTER_ID, m_trips, first, today)
        except Exception as e:
            print(f"[Track] master source split failed: {e}")
        written.append(f"master {n}d")
    except Exception as e:
        print(f"[Track] master snapshot failed: {e}")
        written.append("master FAILED")
    for c in CLIENTS:
        if c.id == MASTER_CLIENT_ID or not c.is_configured():
            continue
        try:
            trips = hl_round_trips(_fetch_hl_fills(c, fetch_ms))
            days  = _bucket_trips(trips, first, today)
            if not days and not trips:
                continue   # nothing yet for this account
```

```
            c_first = min(days) if days else first
            n = _upsert_track_days(f"client_{c.id}", days, fill_gaps_from=c_first
            try:
                _upsert_source_days(f"client_{c.id}", trips, first, today)
            except Exception as e:
                print(f"[Track] client {c.id} source split failed: {e}")
            written.append(f"client {c.id} {n}d")
        except Exception as e:
            print(f"[Track] {c.label()} snapshot failed: {e}")
            written.append(f"client {c.id} FAILED")
    return ", ".join(written)


def track_record_loop():
    """First run backfills everything since TRACK_START_DATE; after that it
    refreshes the last 3 days every TRACK_SNAPSHOT_EVERY_MINS minutes (the
    window also re-settles yesterday after midnight)."""
    time.sleep(45)   # let startup settle first
    first_run = True
    while True:
        try:
            res = snapshot_track_record(None if first_run else 3)
            print(f"[Track] snapshot ({'backfill' if first_run else 'refresh'}):
            first_run = False
        except Exception as e:
            print(f"[Track] loop error: {e}")
        time.sleep(max(5, TRACK_SNAPSHOT_EVERY_MINS) * 60)


def compute_track_stats(rows: list) -> dict:
    """rows: daily dicts (day ascending) for one account or several summed.
    Everything is net of fees. Drawdown is measured on the cumulative DAILY
    net P&L curve starting from zero, so it is in dollars."""
    rows = sorted(rows, key=lambda r: r["day"])
    t  = sum(r["trades"] for r in rows);  w = sum(r["wins"] for r in rows);  l =
    wu = sum(r["wins_usd"] for r in rows); lu = sum(r["losses_usd"] for r in rows
    fees = sum(r["fees"] for r in rows); net = sum(r["net_pnl"] for r in rows); n
    equity, peak, peak_day, max_dd, dd_from, dd_to = 0.0, 0.0, None, 0.0, None, N
    for r in rows:
        equity += r["net_pnl"]
        if equity > peak:
            peak, peak_day = equity, r["day"]
        if peak - equity > max_dd:
            max_dd, dd_from, dd_to = peak - equity, peak_day, r["day"]
    best  = max(rows, key=lambda r: r["net_pnl"]) if rows else None
    worst = min(rows, key=lambda r: r["net_pnl"]) if rows else None
```

```
    return {
        "days": len(rows), "trades": t, "wins": w, "losses": l,
        "win_rate": (w / t * 100) if t else None,
        "avg_win": (wu / w) if w else None, "avg_loss": (lu / l) if l else None,
        "win_loss_ratio": ((wu / w) / (lu / l)) if (w and l and lu > 0) else None
        "profit_factor": (wu / lu) if lu > 0 else None,
        "expectancy": (net / t) if t else None,
        "net_pnl": net, "fees": fees,
        "fee_bps": (fees / notional * 10000) if notional > 0 else None,
        "net_bps": (net / notional * 10000) if notional > 0 else None,
        "max_drawdown": max_dd, "dd_from": dd_from, "dd_to": dd_to,
        "best_day": (best["day"], best["net_pnl"]) if best else None,
        "worst_day": (worst["day"], worst["net_pnl"]) if worst else None,
    }


def load_track_rows() -> dict:
    """{account_id: [daily rows]} straight from Postgres."""
    out = {}
    conn = get_db(); cur = conn.cursor()
    cur.execute("SELECT account_id, day, trades, wins, losses, wins_usd, losses_u
                "FROM track_record_daily WHERE day >= %s ORDER BY account_id, day
    for (acc, day, tr, w, l, wu, lu, gp, fe, npnl, nt) in cur.fetchall():
        out.setdefault(acc, []).append({"day": day, "trades": tr or 0, "wins": w
            "wins_usd": wu or 0.0, "losses_usd": lu or 0.0, "gross_pnl": gp or 0.
            "net_pnl": npnl or 0.0, "notional": nt or 0.0})
    cur.close(); conn.close()
    return out


def load_source_rows() -> dict:
    """{account_id: {source: [daily rows]}} for the signal/manual split."""
    out = {}
    conn = get_db(); cur = conn.cursor()
    cur.execute("SELECT account_id, source, day, trades, wins, losses, wins_usd,
                "FROM track_record_source WHERE day >= %s ORDER BY account_id, so
    for (acc, src, day, tr, w, l, wu, lu, gp, fe, npnl, nt) in cur.fetchall():
        out.setdefault(acc, {}).setdefault(src, []).append({"day": day, "trades":
            "losses": l or 0, "wins_usd": wu or 0.0, "losses_usd": lu or 0.0, "gr
            "fees": fe or 0.0, "net_pnl": npnl or 0.0, "notional": nt or 0.0})
    cur.close(); conn.close()
    return out


def load_slippage_stats() -> dict:
    """{role: (avg_bps, n)} for 'master' and 'client' entries since the start dat
```

```
    conn = get_db(); cur = conn.cursor()
    cur.execute("SELECT role, AVG(slippage_bps), COUNT(*) FROM mirror_exec_log "
                "WHERE logged_at >= %s GROUP BY role", (_track_start(),))
    res = {r: (a, n) for r, a, n in cur.fetchall()}
    cur.close(); conn.close()
    return res


def format_track_message() -> str:
    rows = load_track_rows()
    if not rows:
        return "📈 <b>Track record</b>\n\nNothing recorded yet. The first snapshc
    def m(v):  return "—" if v is None else f"{'+' if v >= 0 else '-'}${abs(v):,.
    def f(v, d=2, suf=""): return "—" if v is None else f"{v:.{d}f}{suf}"
    def block(title, st):
        lines = [f"<b>{title}</b>",
                 f"Trades {st['trades']} · win rate {f(st['win_rate'], 0, '%')} ·
                 f"Avg win {m(st['avg_win'])} · avg loss {('-$%.2f' % st['avg_los
                 f" · win/loss {f(st['win_loss_ratio'])}x",
                 f"Profit factor {f(st['profit_factor'])} · expectancy {m(st['exp
        if st["max_drawdown"] > 0:
            lines.append(f"Max drawdown -${st['max_drawdown']:,.2f} ({st['dd_from
        else:
            lines.append("Max drawdown none so far")
        lines.append(f"Fees ${st['fees']:,.2f}" + (f" · {st['fee_bps']:.1f} bps o
        if st["trades"] < 30:
            lines.append(f"⚠️ small sample (n={st['trades']}) — treat as indicati
        return "\n".join(lines)

    parts = [f"📈 <b>Track record</b> — since {_track_start():%d %b %Y}\nAll figu
    if TRACK_MASTER_ID in rows:
        st = compute_track_stats(rows[TRACK_MASTER_ID])
        parts.append(block(f"Master (Binance) — {st['days']} days", st))
    client_ids = sorted(a for a in rows if a.startswith("client_"))
    if client_ids:
        by_day = {}
        for a in client_ids:
            for r in rows[a]:
                b = by_day.setdefault(r["day"], dict(_empty_day(), day=r["day"]))
                for k in _empty_day():
                    b[k] += r[k]
        cst = compute_track_stats(list(by_day.values()))
        parts.append(block(f"Clients combined ({len(client_ids)} account{'s' if l
        per = []
        for a in client_ids:
            s_ = compute_track_stats(rows[a])
```

```
            cid = a.split("_", 1)[1]
            c_  = next((c for c in CLIENTS if c.id == cid), None)
            label = (c_.name if c_ and c_.name else f"Client #{cid}")
            per.append(f"{label}: {s_['trades']} trades · {f(s_['win_rate'], 0, '
        parts.append("<b>Per client</b>\n" + "\n".join(per))
    # bot-only (signal) vs manual split
    try:
        srows = load_source_rows()
    except Exception:
        srows = {}
    if srows:
        def merged(src, accts):
            by_day = {}
            for a in accts:
                for r in srows.get(a, {}).get(src, []):
                    b = by_day.setdefault(r["day"], dict(_empty_day(), day=r["day
                    for k in _empty_day():
                        b[k] += r[k]
            return list(by_day.values())
        def line(label, rs):
            if not rs:
                return f"{label}: no trades"
            s_ = compute_track_stats(rs)
            return (f"{label}: {s_['trades']} trades · win {f(s_['win_rate'], 0,
                    f" · PF {f(s_['profit_factor'])} · {m(s_['expectancy'])}/trad
        def split(title, accts, all_rows):
            sig, man = merged("signal", accts), merged("manual", accts)
            lab = sum(r["trades"] for r in sig + man)
            out = [f"<b>{title}</b>", line("🤖 Bot signals only", sig), line("✋
            tot = sum(r["trades"] for r in all_rows)
            if tot > lab:
                out.append(f"{tot - lab} trade(s) with no source record (before t
            if lab and len(sig) and sum(r["trades"] for r in sig) < 30:
                out.append("⚠️ small sample")
            return "\n".join(out)
        def all_of(accts):
            by_day = {}
            for a in accts:
                for r in rows.get(a, []):
                    b = by_day.setdefault(r["day"], dict(_empty_day(), day=r["day
                    for k in _empty_day():
                        b[k] += r[k]
            return list(by_day.values())
        if TRACK_MASTER_ID in rows:
            parts.append(split("Master: bot vs manual", [TRACK_MASTER_ID], all_of
        if client_ids:
```

```
            parts.append(split("Clients: bot vs manual", client_ids, all_of(clien
    # client vs master: size-neutral comparison
    if TRACK_MASTER_ID in rows and client_ids:
        mst = compute_track_stats(rows[TRACK_MASTER_ID])
        slip = {}
        try:
            slip = load_slippage_stats()
        except Exception:
            pass
        def sl(role):
            v = slip.get(role)
            return f"{v[0]:+.1f} bps (n={v[1]})" if v else "not enough data yet"
        parts.append("<b>Clients vs master</b> (size-neutral)\n"
                     f"Fees: master {f(mst['fee_bps'], 1)} bps · clients {f(cst['
                     f"Net return: master {f(mst['net_bps'], 1)} bps · clients {f
                     f"Entry slippage vs market at order time (+ = worse): master
                     "Slippage is recorded from now on and can't be rebuilt for e
                     "Master enters with maker limits, clients with market orders
    return "\n\n".join(parts)


def get_consecutive_losses(trades: list) -> int:
    positions = group_by_position(trades)
    streak    = 0
    for p in reversed(positions):
        if p["realizedPnl"] < 0:
            streak += 1
        else:
            break
    return streak


# ─────────────────────────────────────────────
# TRADE EXECUTION
# ─────────────────────────────────────────────

# ─────────────────────────────────────────────
# LIVE WEBSOCKET DATA — replaces repeated REST polling
# ─────────────────────────────────────────────
# The real, structural fix for tonight's repeated Binance 418/1003 rate-limit
# bans. Every REST-based fix so far (caching, pacing, wider windows) only
# ever reduced call VOLUME — it never eliminated the underlying problem,
# which is that this bot was fundamentally polling REST endpoints on a
# timer for data that Binance can just push to us continuously for free.
#
# This opens ONE persistent websocket connection to Binance's combined
```

```
# stream endpoint, subscribed to mark price + the kline intervals the bot
# actually uses (30m, 1H, 4H, 8H). Every incoming message updates an in-memory
# store — zero REST weight cost per update, no matter how often price
# moves or how many guards check it. get_mark_price(),
# and get_live_williams_r()'s underlying kline fetch all now
# read from this store FIRST, only falling back to a real REST call if the
# websocket hasn't populated that specific piece of data yet (e.g. in the
# first few seconds after the bot starts, before the first message arrives).

_ws_mark_price = {"value": None, "updated_at": 0}
_ws_klines = {}  # keyed by interval string (e.g. "30m", "1h", "4h", "8h") -> lis
_ws_lock = threading.Lock()
_ws_connected = False
_ws_last_message_at = 0

WS_KLINE_INTERVALS = ["5m", "30m", "1h", "4h", "8h"]  # every interval the bot tr
WS_KLINE_HISTORY_LIMIT = 850  # candles kept in memory per interval — EMA(240) ne
                              # to converge properly, this comfortably covers th
WS_STALE_THRESHOLD_SEC = 30  # if no message received in this long, treat the web

def _ws_seed_kline_history(interval: str):
    """
    One-time REST call per interval at startup to seed historical closed
    candles, since the websocket only pushes NEW updates from the moment it
    connects — it has no memory of candles before that. This is the only
    REST kline call this system makes under normal operation; everything
    after this point comes from the stream.
    """
    try:
        resp = requests.get(
            f"{FUTURES_BASE}/fapi/v1/klines",
            params={"symbol": SYMBOL, "interval": interval, "limit": WS_KLINE_HIS
            timeout=15
        )
        resp.raise_for_status()
        klines = resp.json()
        # Drop the last one — it's still-forming, the websocket will replace
        # it with real-time updates and eventually a genuine close event.
        closed = klines[:-1]
        with _ws_lock:
            _ws_klines[interval] = closed
        print(f"[WS] Seeded {len(closed)} closed candles for {interval}")
    except Exception as e:
        print(f"[WS] Seed error for {interval}: {e}")
```

```
def _ws_on_message(ws, message):
    global _ws_last_message_at
    _ws_last_message_at = time.time()
    try:
        data = json.loads(message)
        payload = data.get("data", data)  # combined stream wraps in {"stream":..
        event_type = payload.get("e")

        if event_type == "markPriceUpdate":
            with _ws_lock:
                _ws_mark_price["value"] = float(payload["p"])
                _ws_mark_price["updated_at"] = time.time()

        elif event_type == "kline":
            k = payload["k"]
            interval = k["i"]
            is_closed = k["x"]
            candle = [k["t"], k["o"], k["h"], k["l"], k["c"], k["v"]]
            with _ws_lock:
                existing = _ws_klines.get(interval, [])
                if existing and existing[-1][0] == k["t"]:
                    # Same still-forming candle, in-place update if it's now clos
                    if is_closed:
                        existing[-1] = candle
                elif is_closed:
                    # A genuinely new closed candle
                    existing.append(candle)
                    if len(existing) > WS_KLINE_HISTORY_LIMIT:
                        existing.pop(0)
                _ws_klines[interval] = existing
    except Exception as e:
        print(f"[WS] Message handling error: {e}")


def _ws_on_error(ws, error):
    print(f"[WS] Error: {error}")


def _ws_on_close(ws, close_status_code, close_msg):
    global _ws_connected
    _ws_connected = False
    print(f"[WS] Connection closed ({close_status_code}: {close_msg}) — will auto


def _ws_on_open(ws):
    global _ws_connected
```

```
    _ws_connected = True
    print("[WS] Connected")


def _ws_run_forever():
    """
    Runs the websocket connection with automatic reconnection. Called once
    in its own daemon thread at bot startup. If the connection drops for
    any reason (network blip, Binance restart, etc.), it reconnects after
    a short delay rather than leaving the bot silently degraded back to
    pure REST polling forever.
    """
    streams = [f"{SYMBOL.lower()}@markPrice@1s"] + [f"{SYMBOL.lower()}@kline_{iv}
    stream_path = "/".join(streams)
    url = f"wss://fstream.binance.com/stream?streams={stream_path}"

    while True:
        try:
            ws = websocket.WebSocketApp(
                url,
                on_open=_ws_on_open,
                on_message=_ws_on_message,
                on_error=_ws_on_error,
                on_close=_ws_on_close,
            )
            ws.run_forever(ping_interval=180, ping_timeout=10)  # Binance sends i
        except Exception as e:
            print(f"[WS] run_forever crashed: {e}")
        print("[WS] Reconnecting in 5s...")
        time.sleep(5)


def start_websocket_feed():
    """Call once at bot startup — seeds kline history via REST, then opens the pe
    for interval in WS_KLINE_INTERVALS:
        _ws_seed_kline_history(interval)
    ws_thread = threading.Thread(target=_ws_run_forever, daemon=True)
    ws_thread.start()
    print("[WS] Feed thread started")


def _ws_is_healthy() -> bool:
    """True if the websocket has delivered a message recently — used to decide wh
    return _ws_connected and (time.time() - _ws_last_message_at) < WS_STALE_THRES
```

```
def ws_get_mark_price():
    """Returns the live websocket mark price, or None if unavailable/stale — call
    if not _ws_is_healthy():
        return None
    with _ws_lock:
        if _ws_mark_price["value"] is not None and (time.time() - _ws_mark_price[
            return _ws_mark_price["value"]
    return None


def ws_get_closed_klines(interval: str, min_count: int = 1):
    """Returns the in-memory closed-candle list for this interval, or None if una
    if not _ws_is_healthy():
        return None
    with _ws_lock:
        candles = _ws_klines.get(interval)
        if candles and len(candles) >= min_count:
            return list(candles)  # copy, so caller can't mutate the shared store
    return None


_mark_price_cache = {"value": None, "fetched_at": 0}
MARK_PRICE_CACHE_SECONDS = 15  # only used as the REST fallback cache now that th
                               # this fires only when the websocket is down/stal
                               # safety net rather than the main data path.

def get_mark_price() -> float:
    # ── Primary: live websocket data, zero REST cost ──
    ws_price = ws_get_mark_price()
    if ws_price is not None:
        return ws_price

    # ── Fallback: REST, only reached if the websocket is down or hasn't
    # delivered a price yet (e.g. the first second after bot startup) ──
    now = time.time()
    if _mark_price_cache["value"] is not None and (now - _mark_price_cache["fetch
        return _mark_price_cache["value"]
    resp = requests.get(
        f"{FUTURES_BASE}/fapi/v1/premiumIndex",
        params={"symbol": SYMBOL}, timeout=10
    )
    resp.raise_for_status()
    price = float(resp.json()["markPrice"])
    _mark_price_cache["value"]      = price
    _mark_price_cache["fetched_at"] = now
    return price
```

```
def calculate_ema(closes: list, period: int) -> float:
    """Standard EMA calculation. closes must be oldest-first."""
    if len(closes) < period:
        raise ValueError(f"Need at least {period} candles, got {len(closes)}")
    multiplier = 2 / (period + 1)
    ema = sum(closes[:period]) / period  # seed with SMA of first `period` closes
    for close in closes[period:]:
        ema = (close - ema) * multiplier + ema
    return ema


def calculate_williams_r(highs: list, lows: list, closes: list, length: int) -> f
    """
    Modified Williams %R over the most recent `length` candles — uses the
    TYPICAL PRICE (High + Low + Close) / 3 in place of the raw close, per
    Nathan's spec. Standard Williams %R uses close only; this version
    smooths that out slightly by incorporating the candle's full range.
    %R = (Highest High - Typical Price) / (Highest High - Lowest Low) * -100
    Range: -100 (at recent low) to 0 (at recent high).
    highs/lows/closes must be oldest-first, same length, aligned by index.
    """
    if len(closes) < length:
        raise ValueError(f"Need at least {length} candles, got {len(closes)}")
    recent_highs = highs[-length:]
    recent_lows  = lows[-length:]
    highest_high = max(recent_highs)
    lowest_low   = min(recent_lows)
    typical_price = (highs[-1] + lows[-1] + closes[-1]) / 3
    if highest_high == lowest_low:
        return -50.0  # flat range — neutral fallback, avoids divide-by-zero
    return (highest_high - typical_price) / (highest_high - lowest_low) * -100


_wpr_cache = {}  # keyed by interval string -> {"value": ..., "fetched_at": ...}
WPR_CACHE_SECONDS = 600  # REST fallback cache duration, only used when the webso
_wpr_last_error = None  # diagnosable reason for the most recent WPR fetch failur

def get_live_williams_r(interval: str = "30m", length: int = 200) -> float:
    """
    Calculates Williams %R on the LAST CLOSED candle for the given interval.
    Primary data source is the live websocket kline stream (zero REST cost);
    falls back to a direct REST fetch only if the websocket is down/stale
    or doesn't have enough history yet for this interval. Cache is keyed
    per-interval — previously a single shared cache meant a 2H WPR request
```

```
could incorrectly be served a cached 15m value or vice versa, a real
bug introduced when the 2H HTF WPR gate was added.
"""
global _wpr_last_error

# ── Primary: websocket kline data, zero REST cost ──
ws_klines = ws_get_closed_klines(interval, min_count=length)
if ws_klines is not None:
    try:
        recent = ws_klines[-length:]
        highs  = [float(k[2]) for k in recent]
        lows   = [float(k[3]) for k in recent]
        closes = [float(k[4]) for k in recent]
        wpr = calculate_williams_r(highs, lows, closes, length)
        _wpr_last_error = None
        return wpr
    except Exception as e:
        print(f"[WPR] Websocket calc error, falling back to REST: {e}")

# ── Fallback: REST, cached per-interval ──
now = time.time()
cached = _wpr_cache.get(interval)
if cached and (now - cached["fetched_at"]) < WPR_CACHE_SECONDS:
    return cached["value"]
try:
    resp = requests.get(
        f"{FUTURES_BASE}/fapi/v1/klines",
        params={"symbol": SYMBOL, "interval": interval, "limit": length + 5},
        timeout=10
    )
    resp.raise_for_status()
    klines = resp.json()
    closed_klines = klines[:-1]  # drop the still-forming current candle
    if len(closed_klines) < length:
        _wpr_last_error = f"only {len(closed_klines)} closed candles returned
        return None
    highs  = [float(k[2]) for k in closed_klines]
    lows   = [float(k[3]) for k in closed_klines]
    closes = [float(k[4]) for k in closed_klines]
    wpr = calculate_williams_r(highs, lows, closes, length)
    _wpr_cache[interval] = {"value": wpr, "fetched_at": now}
    _wpr_last_error = None
    return wpr
except requests.exceptions.HTTPError as e:
    _wpr_last_error = f"Binance HTTP error: {e}"
    print(f"[WPR] Live calculation error: {e}")
```

```
        return None
    except requests.exceptions.RequestException as e:
        _wpr_last_error = f"network/timeout error: {e}"
        print(f"[WPR] Live calculation error: {e}")
        return None
    except Exception as e:
        _wpr_last_error = f"unexpected error: {e}"
        print(f"[WPR] Live calculation error: {e}")
        return None


# ── WPR reversal/exhaustion signals ──────────────────────────────────
# WPR crossing DOWN through -18 = price just rolled over off its recent
# high — a SHORT entry (the extreme high has exhausted). WPR crossing UP
# through -82 = price just bounced off its recent low — a LONG/BUY entry
# (the extreme low has exhausted). Uses closed-candle WPR only, same
# discipline as the HTF flip signals, to avoid live-candle whipsaw.
WPR_SHORT_CROSS_ENABLED = os.environ.get("WPR_SHORT_CROSS_ENABLED", "false").lowe
WPR_LONG_CROSS_ENABLED  = os.environ.get("WPR_LONG_CROSS_ENABLED", "false").lower
_wpr_last_below_18 = None  # tracks previous closed-candle state for cross detect
_wpr_last_above_82 = None

def check_wpr_short_cross():
    global _wpr_last_below_18
    if not WPR_SHORT_CROSS_ENABLED:
        return
    wpr = get_live_williams_r(interval="30m", length=200)
    if wpr is None:
        return
    currently_below_18 = wpr < -18
    just_crossed_down = (_wpr_last_below_18 is False and currently_below_18 is Tr
    _wpr_last_below_18 = currently_below_18
    if just_crossed_down:
        threading.Thread(
            target=process_webhook_signal,
            args=({"signal": "wpr_crosses_down_18"},),
            daemon=True
        ).start()


def check_wpr_long_cross():
    global _wpr_last_above_82
    if not WPR_LONG_CROSS_ENABLED:
        return
    wpr = get_live_williams_r(interval="30m", length=200)
    if wpr is None:
```

```
        return
    currently_above_82 = wpr > -82
    just_crossed_up = (_wpr_last_above_82 is False and currently_above_82 is True
    _wpr_last_above_82 = currently_above_82
    if just_crossed_up:
        threading.Thread(
            target=process_webhook_signal,
            args=({"signal": "wpr_crosses_up_82"},),
            daemon=True
        ).start()




_ema120_guard_cache = {"value": None, "fetched_at": 0}
EMA120_GUARD_CACHE_SECONDS = 1200

def get_live_ema120_guard(interval: str = "30m") -> float:
    """
    Calculates the EMA(120) on the given interval — used by the standalone
    EMA_GUARD_ENABLED/SIMPLE_EMA_GUARD_ENABLED guard.
    """
    ws_klines = ws_get_closed_klines(interval, min_count=120)
    if ws_klines is not None:
        try:
            closes = [float(k[4]) for k in ws_klines]
            return calculate_ema(closes, 120)
        except Exception as e:
            print(f"[EMA] Websocket calc error, falling back to REST: {e}")

    now = time.time()
    if _ema120_guard_cache["value"] is not None and (now - _ema120_guard_cache["f
        return _ema120_guard_cache["value"]
    try:
        resp = requests.get(
            f"{FUTURES_BASE}/fapi/v1/klines",
            params={"symbol": SYMBOL, "interval": interval, "limit": 300},
            timeout=10
        )
        resp.raise_for_status()
        klines = resp.json()
        closes = [float(k[4]) for k in klines]
        ema = calculate_ema(closes, 120)
```

```
        _ema120_guard_cache["value"]      = ema
        _ema120_guard_cache["fetched_at"] = now
        return ema
    except Exception as e:
        print(f"[EMA] Live calculation error: {e}")
        return None
```

```
_symbol_info_cache = {"value": None, "fetched_at": 0}
SYMBOL_INFO_CACHE_SECONDS = 3600  # tick size / qty step for BTCUSDT essentially
                                  # this endpoint (/fapi/v1/exchangeInfo) return
                                  # symbol on the exchange, one of Binance's hea
                                  # was being hit fresh on every single trade at
                                  # sites, zero caching) — a real contributor to
                                  # trips when opening a trade with all its foll

def get_symbol_info() -> dict:
    now = time.time()
    if _symbol_info_cache["value"] is not None and (now - _symbol_info_cache["fet
        return _symbol_info_cache["value"]
    resp = requests.get(f"{FUTURES_BASE}/fapi/v1/exchangeInfo", timeout=15)
    resp.raise_for_status()
    for s in resp.json()["symbols"]:
        if s["symbol"] == SYMBOL:
            info = {"qty_step": 0.001, "price_tick": 0.1}
            for f in s.get("filters", []):
                if f["filterType"] == "LOT_SIZE":
                    info["qty_step"] = float(f["stepSize"])
                elif f["filterType"] == "PRICE_FILTER":
                    info["price_tick"] = float(f["tickSize"])
            _symbol_info_cache["value"]      = info
            _symbol_info_cache["fetched_at"] = now
            return info
    fallback = {"qty_step": 0.001, "price_tick": 0.1}
    _symbol_info_cache["value"]      = fallback
    _symbol_info_cache["fetched_at"] = now
    return fallback


def round_step(value: float, step: float) -> float:
    import math
```

```
    precision = max(0, -int(math.floor(math.log10(step))))
    return round(math.floor(value / step) * step, precision)


_leverage_set_at = 0.0


def set_leverage_for_symbol():
    """Leverage doesn't change between trades, so don't spend a round trip settin
    global _leverage_set_at
    if time.time() - _leverage_set_at < 3600:
        return {}
    now_ms = int(time.time() * 1000)
    params = {"symbol": SYMBOL, "leverage": LEVERAGE, "timestamp": now_ms}
    params = sign_request(params)
    headers = {"X-MBX-APIKEY": BINANCE_API_KEY}
    resp = requests.post(f"{FUTURES_BASE}/fapi/v1/leverage",
                         params=params, headers=headers, timeout=10)
    resp.raise_for_status()
    _leverage_set_at = time.time()
    return resp.json()


_open_position_cache = {"value": "unset", "fetched_at": 0}
OPEN_POSITION_CACHE_SECONDS = 10  # emergency-widened from 5s, same reasoning as

def invalidate_position_cache():
    """Call this immediately after any action that closes/changes the position, s
    next get_open_position() call fetches fresh data instead of returning a stale
    _open_position_cache["value"]      = "unset"
    _open_position_cache["fetched_at"] = 0


def get_open_position() -> dict | None:
    now = time.time()
    if _open_position_cache["value"] != "unset" and (now - _open_position_cache["
        return _open_position_cache["value"]
    now_ms = int(time.time() * 1000)
    params = {"symbol": SYMBOL, "timestamp": now_ms}
    params = sign_request(params)
    headers = {"X-MBX-APIKEY": BINANCE_API_KEY}
    resp = requests.get(f"{FUTURES_BASE}/fapi/v2/positionRisk",
                        params=params, headers=headers, timeout=10)
    resp.raise_for_status()
    result = None
    for pos in resp.json():
```

```
        if pos["symbol"] == SYMBOL and float(pos["positionAmt"]) != 0:
            result = pos
            break
    _open_position_cache["value"]      = result
    _open_position_cache["fetched_at"] = now
    return result


def cancel_open_orders():
    """Clears regular and algo (conditional) orders. They live in separate order
    since Binance's 2025-12-09 migration (algo orders: /fapi/v1/algoOpenOrders),
    deletes are needed. They are independent, so they run at the same time."""
    headers = {"X-MBX-APIKEY": BINANCE_API_KEY}

    def _delete(path):
        try:
            params = sign_request({"symbol": SYMBOL, "timestamp": int(time.time()
            requests.delete(f"{FUTURES_BASE}{path}", params=params, headers=heade
        except Exception:
            pass

    t = threading.Thread(target=_delete, args=("/fapi/v1/algoOpenOrders",), daemo
    t.start()
    _delete("/fapi/v1/allOpenOrders")
    t.join(timeout=12)


def place_algo_order(side: str, order_type: str, stop_price: float,
                     quantity: float = None, close_position: bool = False,
                     reduce_only: bool = False) -> dict:
    """
    Places a conditional (algo) order — STOP_MARKET, TAKE_PROFIT_MARKET, etc.
    Binance made this endpoint MANDATORY for these order types effective
    2025-12-09 — the old POST /fapi/v1/order endpoint now rejects them
    outright with error -4120 ("Order type not supported for this endpoint.
    Please use the Algo Order API endpoints instead."). This was discovered
    after the earlier "real SL order" fix silently failed in production —
    every stop-loss placed via the old endpoint since that fix was added
    never actually existed on the exchange, despite the bot reporting
    success in some paths. This function uses the correct, current endpoint.
    Returns the parsed JSON response, with "algoId" in place of "orderId".
    """
    now_ms = int(time.time() * 1000)
    params: dict = {
        "algoType":    "CONDITIONAL",
        "symbol":      SYMBOL,
```

```
        "side":        side,
        "type":        order_type,
        "triggerPrice": stop_price,
        "workingType": "MARK_PRICE",
        "timestamp":   now_ms,
    }
    if close_position:
        params["closePosition"] = "true"
    else:
        if quantity is not None:
            params["quantity"] = quantity
        if reduce_only:
            params["reduceOnly"] = "true"
    params  = sign_request(params)
    headers = {"X-MBX-APIKEY": BINANCE_API_KEY}
    resp = requests.post(f"{FUTURES_BASE}/fapi/v1/algoOrder",
                         params=params, headers=headers, timeout=10)
    if not resp.ok:
        # requests' raise_for_status() only reports the HTTP status, not
        # Binance's actual reason (e.g. -2021 "Order would immediately
        # trigger") — that detail lives in the response body, so pull it
        # out here and put it in the exception message that Telegram
        # ultimately shows, instead of a bare "400 Bad Request".
        try:
            err = resp.json()
            detail = f"{err.get('code')}: {err.get('msg')}"
        except Exception:
            detail = resp.text[:200]
        raise requests.exceptions.HTTPError(
            f"{resp.status_code} Binance error placing {order_type} algo order —
            response=resp,
        )
    return resp.json()


def cancel_algo_order(algo_id) -> dict:
    """Cancels a single algo (conditional) order by its algoId."""
    now_ms  = int(time.time() * 1000)
    headers = {"X-MBX-APIKEY": BINANCE_API_KEY}
    params  = sign_request({"symbol": SYMBOL, "algoId": algo_id, "timestamp": now
    resp = requests.delete(f"{FUTURES_BASE}/fapi/v1/algoOrder",
                           params=params, headers=headers, timeout=10)
    if not resp.ok:
        try:
            err = resp.json()
            detail = f"{err.get('code')}: {err.get('msg')}"
```

```
        except Exception:
            detail = resp.text[:200]
        raise requests.exceptions.HTTPError(
            f"{resp.status_code} Binance error cancelling algo order {algo_id} —
            response=resp,
        )
    return resp.json()


def get_open_algo_orders() -> list:
    """Returns all open algo (conditional) orders for this symbol — the new equiv
    now_ms  = int(time.time() * 1000)
    headers = {"X-MBX-APIKEY": BINANCE_API_KEY}
    params  = sign_request({"symbol": SYMBOL, "timestamp": now_ms})
    resp = requests.get(f"{FUTURES_BASE}/fapi/v1/openAlgoOrders",
                        params=params, headers=headers, timeout=10)
    resp.raise_for_status()
    data = resp.json()
    return data.get("orders", data) if isinstance(data, dict) else data


def cancel_all_algo_orders(keep_latest: bool = False):
    """
    Cancels open algo (conditional) orders for this symbol.
    keep_latest=True cancels every algo order EXCEPT the most recently
    placed one — used by adjust_sl() so placing the new SL first, then
    cleaning up, doesn't also cancel the SL we just placed.
    """
    if not keep_latest:
        now_ms  = int(time.time() * 1000)
        headers = {"X-MBX-APIKEY": BINANCE_API_KEY}
        params  = sign_request({"symbol": SYMBOL, "timestamp": now_ms})
        try:
            requests.delete(f"{FUTURES_BASE}/fapi/v1/algoOpenOrders",
                            params=params, headers=headers, timeout=10)
        except Exception as e:
            print(f"[Cancel Algo Orders] Error: {e}")
        return

    try:
        orders = get_open_algo_orders()
        if not orders:
            return
        # Keep the order with the highest algoId (most recently placed), cancel t
        latest = max(orders, key=lambda o: o.get("algoId", 0))
        for o in orders:
```

```
            if o.get("algoId") != latest.get("algoId"):
                try:
                    cancel_algo_order(o["algoId"])
                except Exception as e:
                    print(f"[Cancel Algo Orders] Error cancelling {o.get('algoId'
    except Exception as e:
        print(f"[Cancel Algo Orders] keep_latest error: {e}")


def place_order(side: str, order_type: str, quantity: float,
                stop_price: float = None, reduce_only: bool = False,
                limit_price: float = None, close_position: bool = False) -> dict:
    now_ms = int(time.time() * 1000)
    params: dict = {
        "symbol":    SYMBOL,
        "side":      side,
        "type":      order_type,
        "timestamp": now_ms,
    }
    if close_position:
        params["closePosition"] = "true"
        params["stopPrice"]     = stop_price
    else:
        params["quantity"] = quantity
        if order_type == "LIMIT":
            params["price"]       = limit_price
            params["timeInForce"] = "GTC"
        if stop_price is not None:
            params["stopPrice"] = stop_price
        if reduce_only:
            params["reduceOnly"] = "true"
    params  = sign_request(params)
    headers = {"X-MBX-APIKEY": BINANCE_API_KEY}
    resp = requests.post(f"{FUTURES_BASE}/fapi/v1/order",
                         params=params, headers=headers, timeout=10)
    resp.raise_for_status()
    return resp.json()


def place_maker_entry(side: str, quantity: float, direction: str, price_tick: flo
    global maker_entry_cancel_requested
    maker_entry_cancel_requested = False  # reset at start of each attempt
    mark = get_mark_price()
    if direction == "long":
        limit_price = round_step(mark * (1 - MAKER_OFFSET_PCT / 100), price_tick)
    else:
```

```
    limit_price = round_step(mark * (1 + MAKER_OFFSET_PCT / 100), price_tick)

send_telegram(f"Placing maker limit entry at ${limit_price:,.2f} (mark: ${mar
resp = place_order(side, "LIMIT", quantity, limit_price=limit_price)
order_id = resp.get("orderId")

deadline = time.time() + MAKER_TIMEOUT_SEC
while time.time() < deadline:
    time.sleep(4)  # was 2s — halved polling frequency to ease Binance API lo

    # Check for manual cancel request
    if maker_entry_cancel_requested:
        maker_entry_cancel_requested = False
        try:
            now_ms = int(time.time() * 1000)
            params = sign_request({"symbol": SYMBOL, "orderId": order_id, "ti
            headers = {"X-MBX-APIKEY": BINANCE_API_KEY}
            requests.delete(f"{FUTURES_BASE}/fapi/v1/order",
                            params=params, headers=headers, timeout=10)
        except Exception:
            pass
        send_telegram("✅ <b>Maker entry cancelled</b> — order removed, no pc
        return None
    now_ms = int(time.time() * 1000)
    params = sign_request({"symbol": SYMBOL, "orderId": order_id, "timestamp"
    headers = {"X-MBX-APIKEY": BINANCE_API_KEY}
    check = requests.get(f"{FUTURES_BASE}/fapi/v1/order",
                         params=params, headers=headers, timeout=10)
    check.raise_for_status()
    status = check.json().get("status")
    if status == "FILLED":
        send_telegram(f"Maker entry filled at ${limit_price:,.2f}")
        return resp
    if status in ("CANCELED", "EXPIRED", "REJECTED"):
        break

try:
    now_ms = int(time.time() * 1000)
    params = sign_request({"symbol": SYMBOL, "orderId": order_id, "timestamp"
    headers = {"X-MBX-APIKEY": BINANCE_API_KEY}
    requests.delete(f"{FUTURES_BASE}/fapi/v1/order",
                    params=params, headers=headers, timeout=10)
except Exception:
    pass

send_telegram(f"Maker entry not filled in {MAKER_TIMEOUT_SEC}s — falling back
```

```
    return place_order(side, "MARKET", quantity)

_execute_trade_lock = threading.Lock()

_STAIR_GATE_TRADES = os.environ.get("STAIRCASE_GATE_TRADES", "true").lower() != "


def _stair_trade_gate(direction, triggered_by=None):
    """(blocked, reason). Re-checks every staircase rule against live data at the
    Fails closed: if the colours or readings can't be made, the trade is rejected
    try:
        if datetime.now(UK_TZ).weekday() in _STAIR_SKIP_DAYS:
            return True, "skipped day"
        k4,  _ = _stair_get_klines(_STAIR_TF_H, _STAIR_ZLEN)
        k30, _ = _stair_get_klines(_STAIR_TF_M, _STAIR_ZLEN)
        k5,  _ = _stair_get_klines(_STAIR_TF_L, _STAIR_ZLEN)
        mp = _stair_mark_price()
        if not k4 or not k30 or not k5 or mp is None:
            return True, "no data to read the colours"
        h = _stair_composite([float(x[4]) for x in k4] + [mp])
        m = _stair_composite([float(x[4]) for x in k30] + ([mp] if _STAIR_LIVE_MI
        l = _stair_composite([float(x[4]) for x in k5])
        l_prev = _stair_composite([float(x[4]) for x in k5][:-1])
        if h is None or m is None or l is None or l_prev is None:
            return True, "no data to read the colours"
        cols = (_stair_colour(h), _stair_colour(m), _stair_colour(l))
        tag = f"4H {cols[0]} · 30m {cols[1]} · 5m {cols[2]}"
        if _stair_is_blocked(direction, *cols):
            return True, f"blocked combo ({tag})"
        if _stair_is_blocked_from(direction, cols[0], cols[1], _stair_colour(l_pr
            return True, f"blocked combo ({tag}, 5m was {_stair_colour(l_prev)})"
        # Long F (All Green below every midline) blocks shorts while it is active
        opp = ("long", "F") if direction == "short" else ("short", "F")
        if cols[0] == cols[1] == cols[2] == _STAIR_EPISODE[opp]:
            want, ivs = _STAIR_SETUP_CONDS[opp]
            sides = [_stair_above_mid(iv, mp) for iv in ivs]
            if any(x is None for x in sides):
                return True, "no data to check whether Long F / Short F is active
            if all(x is (want == "above") for x in sides):
                return True, f"{opp[0].title()} F is active ({tag}): no {directio
        label = (triggered_by or "").lower()
        conds = next((c for (d, L), c in _STAIR_SETUP_CONDS.items() if label == f
        if conds:
            want, ivs = conds
            if not all(_stair_above_mid(iv, mp) is (want == "above") for iv in iv
                return True, f"price must be {want} the {' and '.join(ivs)} midli
```

```
        elif (direction, label.rsplit(" ",1)[-1].upper()) not in _STAIR_GATE_EXEM
            blk, note = _stair_short_gate(mp, direction)
            if blk:
                return True, f"{direction} midline/WPR filter: {note} ({tag})"
        if direction == "long" and (direction, label.rsplit(" ", 1)[-1].upper())
            blk, note = _stair_oversold_gate()
            if blk:
                return True, f"oversold filter: {note} ({tag})"
            blk, note = _stair_green30_gate((cols[0], cols[1], None, cols[2]), mp
            if blk:
                return True, f"Green-30m filter: {note} ({tag})"
        for colour, side, want_above in (("Green", "long", False), ("Orange", "sh
            if direction == side and cols == (colour, colour, colour):
                above30, w5 = _stair_above_mid("30m", mp), _stair_wpr("5m")
                if above30 is None or w5 is None:
                    return True, f"no data to check All {colour}"
                ok = (above30 and w5 > _STAIR_ALLORANGE_WPR) if want_above else (
                if not ok:
                    return True, f"All {colour} filter: {'above' if above30 else
        return False, tag
    except Exception as e:
        print(f"[Staircase] trade gate error, blocking: {e}")
        return True, "rule check error"


def execute_trade(direction: str, size_pct: float = None, size_usdt: float = None
                  stop_pct: float = None, tp_pct: float = None,
                  triggered_by: str = None,
                  z_score: float = None) -> None:
    if not EXECUTION_ENABLED:
        send_telegram("⚠️ Execution is disabled. Set EXECUTION_ENABLED=true to en
        return

    # ── Staircase rules, checked live at the moment of placing ──────────
    if _STAIR_GATE_TRADES:
        blk, why = _stair_trade_gate(direction, triggered_by)
        if blk:
            send_telegram(f"🚫 <b>{direction.upper()} rejected — staircase rule</
                          f"<i>Set STAIRCASE_GATE_TRADES=false to bypass.</i>")
            return

    # ── Prevent overlapping executions ──────────────────────────────────
    # A single pass through this function makes up to ~10 Binance calls
    # (guards, position checks, symbol info). If a second /long, /short, or
    # signal fires while one is still running, both threads' full guard
    # chains overlap and can stack enough requests to trip Binance's rate
```

```
    # limit — this is what actually caused repeated bans tonight, not any
    # single uncached call. non-blocking acquire: if already running, this
    # call is rejected immediately rather than queuing (queuing would just
    # delay the same problem, not fix it).
    if not _execute_trade_lock.acquire(blocking=False):
        send_telegram(
            f"⏳ <b>Already processing a trade request — try again in a moment.</
            f"Another /long, /short, or signal is still being evaluated. "
            f"Rapid repeated commands were the actual cause of tonight's rate-lim
        )
        return
    try:
        _execute_trade_inner(direction, size_pct, size_usdt, stop_pct, tp_pct,
                             triggered_by, z_score)
    finally:
        _execute_trade_lock.release()


def _execute_trade_inner(direction: str, size_pct: float = None, size_usdt: float
                         stop_pct: float = None, tp_pct: float = None,
                         triggered_by: str = None,
                         z_score: float = None) -> None:
    # ── Time-based blocks ──────────────────────────────────────────
    uk_tz   = ZoneInfo("Europe/London")
    now_uk_full = datetime.now(uk_tz)
    now_uk  = now_uk_full.time()

    if now_uk_full.weekday() == 4 and now_uk >= dt_time(19, 0):
        send_telegram(
            f"<b>Trade blocked — Friday cutoff (19:00)</b>\n\n"
            f"No new positions after 19:00 on Fridays. Week's done — see you Mond
        )
        return

    if now_uk >= NIGHT_BLOCK_START:
        send_telegram(
            f"<b>Trade blocked — after 22:45</b>\n\n"
            f"No new positions after 22:45. Come back tomorrow."
        )
        return

    if COOLDOWN_START <= now_uk < COOLDOWN_END:
        send_telegram(
            f"<b>Trade blocked — US open cooldown</b>\n\n"
            f"No new positions between {COOLDOWN_START.strftime('%H:%M')} and "
            f"{COOLDOWN_END.strftime('%H:%M')} UK time.\n"
```

```
        f"Wait for volatility to settle."
    )
    return

# ── Same-side loss block ──────────────────────────────────────
block_until = same_side_block.get(direction, 0)
if not is_testing_mode() and time.time() < block_until:
    remaining_mins = int((block_until - time.time()) / 60)
    send_telegram(
        f"<b>{direction.upper()} blocked — consecutive loss cooldown</b>\n\n"
        f"You've been blocked from {direction}s after consecutive losses.\n"
        f"Unblocks in {remaining_mins}m."
    )
    return

# ── Daily losses block — hard stop after DAILY_LOSS_COUNT_LIMIT losses ─
# Checks real trade data directly, not just the in-memory flag — a
# restart resets the flag to False, but if you've genuinely had
# DAILY_LOSS_COUNT_LIMIT losses today, this must still block instantly,
# not wait for the next poll cycle to re-derive and re-set the flag.
try:
    todays_trades_for_block_check = get_futures_trades_today()
    todays_stats_for_block_check  = build_stats(todays_trades_for_block_check
    real_losses_today = todays_stats_for_block_check["losses"]
except Exception as e:
    print(f"[Daily Loss Block] Live check error: {e}")
    real_losses_today = 0  # fail open — don't block on a data-fetch error

if (daily_losses_block or real_losses_today >= DAILY_LOSS_COUNT_LIMIT):
    send_telegram(
        f"🚫 <b>Blocked — {DAILY_LOSS_COUNT_LIMIT} losses today</b>\n\n"
        f"You've hit {DAILY_LOSS_COUNT_LIMIT} losing trades today. Trading is
        f"Resets at midnight. Walk away and come back tomorrow."
    )
    return

# ── Daily hard stop (trade count or loss limit) ──────────────
if stopped_today:
    send_telegram(
        f"🚫 <b>Trade blocked — daily trading halted</b>\n\n"
        f"Either your trade count cap or daily loss limit was hit today.\n"
        f"Check /warnings for the exact reason. Resets at midnight."
    )
    return

# ── Revenge trade block ──────────────────────────────────────
```

```
if last_loss_close_time > 0 and (time.time() - last_loss_close_time) < REVENG
    remaining_secs = int(REVENGE_WINDOW_MINS * 60 - (time.time() - last_loss_
    remaining_mins = remaining_secs // 60
    remaining_s    = remaining_secs % 60
    send_telegram(
        f"🚫 <b>Revenge trade blocked — you just tried it</b>\n\n"
        f"Your last trade was a loss {int(time.time() - last_loss_close_time)
        f"You are not allowed to trade for {REVENGE_WINDOW_MINS} minutes afte
        f"Unblocks in {remaining_mins}m {remaining_s}s. Step away."
    )
    return

# ── TP2 close blocker (20 mins after TP2 hit) ──────────────────────
if last_tp2_close_time > 0 and (time.time() - last_tp2_close_time) < REVENGE_
    remaining_secs = int(REVENGE_WINDOW_MINS * 60 - (time.time() - last_tp2_c
    remaining_mins = remaining_secs // 60
    remaining_s    = remaining_secs % 60
    send_telegram(
        f"✋ <b>TP2 cooldown — step away</b>\n\n"
        f"You just banked a clean win {int(time.time() - last_tp2_close_time)
        f"Let it settle. No trading for {REVENGE_WINDOW_MINS} minutes after T
        f"Unblocks in {remaining_mins}m {remaining_s}s."
    )
    return

# ── Daily bias check ──────────────────────────────────────────────
if not is_testing_mode() and BIAS_FILTER_ENABLED and daily_bias is None:
    send_telegram(
        f"🚫 <b>{direction.upper()} blocked — no daily bias set</b>\n\n"
        f"Reply /bullish or /bearish to set your HTF bias."
    )
    return

# ── EMA premium/discount guard — don't chase, only enter on pullback ──
# Block longs when price is above the 240 EMA (buying a premium, chasing stre
# Block shorts when price is below the 240 EMA (selling a discount, chasing w
# EMA calculated live from Binance klines — no dependency on any TradingView
# indicator or alert field.
if not is_testing_mode() and EMA_GUARD_ENABLED:
    live_ema120 = get_live_ema120_guard()
    if live_ema120 is not None:
        try:
            ema_check_price = get_mark_price()
        except Exception:
            ema_check_price = None
        if ema_check_price is not None:
```

```
            if direction == "long" and ema_check_price > live_ema120:
                db_log_warning("ema_guard_block", f"Long blocked — price ${em
                send_telegram(
                    f"🚫 <b>Long blocked — price above 120 EMA (premium)</b>\
                    f"Price: ${ema_check_price:,.2f}\n"
                    f"120 EMA: ${live_ema120:,.2f}\n\n"
                    f"This is chasing strength, not buying a pullback. Wait f
                    f"come back to or below the 120 EMA before longing."
                )
                return
            if direction == "short" and ema_check_price < live_ema120:
                db_log_warning("ema_guard_block", f"Short blocked — price ${e
                send_telegram(
                    f"🚫 <b>Short blocked — price below 120 EMA (discount)</b
                    f"Price: ${ema_check_price:,.2f}\n"
                    f"120 EMA: ${live_ema120:,.2f}\n\n"
                    f"This is chasing weakness, not selling a bounce. Wait fo
                    f"come back to or above the 120 EMA before shorting."
                )
                return

# ── Simple 120 EMA trade-flexibility guard ──────────────────────────
# Block SHORT if price is below the 30m 120 EMA — designed to reduce
# how often Nathan needs to check Binance manually, while still
# allowing some trade flexibility rather than a full lockout.
if not is_testing_mode() and SIMPLE_EMA_GUARD_ENABLED:
    if direction == "short":
        ema_120 = get_live_ema120_guard()  # reuses the same cached 120 EMA a
        if ema_120 is not None:
            try:
                check_price = get_mark_price()
            except Exception:
                check_price = None
            if check_price is not None and check_price < ema_120:
                db_log_warning("simple_ema_guard_block", f"Short blocked — pr
                send_telegram(
                    f"🚫 <b>Short blocked — price below 30m 120 EMA</b>\n\n"
                    f"Price: ${check_price:,.2f}\n"
                    f"120 EMA: ${ema_120:,.2f}"
                )
                return

global entry2_placed_at

# ── Attribute trade to trigger if within 1 hour ─────────────────────
global pending_trigger_id
```

```
# Manual trades are never attributed to a signal trigger — otherwise a
# /long pressed within the hour of a signal pollutes that trigger's stats.
if not str(triggered_by or "").startswith("manual") and last_trigger and last
    pending_trigger_id = db_save_trigger(last_trigger, direction)
else:
    pending_trigger_id = None

stop_pct = stop_pct or STOP_PCT
tp_pct   = tp_pct   or TP_PCT

side       = "BUY"  if direction == "long"  else "SELL"
close_side = "SELL" if direction == "long"  else "BUY"

try:
    # Guards are done; pause before the heavy execution burst. This
    # replaces the natural gap the old pre-trade question gave signal
    # trades. Deliberately placed BEFORE the entry order and never
    # between entry and SL/TP, so a fresh position is never left
    # unprotected while waiting.
    _pace_execution()
    existing = get_open_position()
    if existing:
        amt      = float(existing["positionAmt"])
        side_str = "LONG" if amt > 0 else "SHORT"
        send_telegram(
            f"⚠️ <b>Already in a {side_str} position ({amt:+.4f} BTC).</b>\n"
            f"Use /close to exit first."
        )
        return

    set_leverage_for_symbol()
    _pace_execution()

    # mark price is fetched right before the entry so it stays fresh —
    # no pacing between here and the order.
    price      = get_mark_price()
    sym_info   = get_symbol_info()
    qty_step   = sym_info["qty_step"]
    price_tick = sym_info["price_tick"]
    balance    = get_usdt_balance()

    if size_usdt is not None:
        notional = size_usdt * LEVERAGE
        size_label = f"${size_usdt:,.2f}"
    else:
        pct      = size_pct if size_pct is not None else TRADE_SIZE_PCT
```

```
    notional = balance * (pct / 100) * LEVERAGE
    size_label = f"{pct}% of balance"

raw_qty  = notional / price
quantity = round_step(raw_qty, qty_step)

if quantity <= 0:
    send_telegram("❌ Position size too small — check your amount or bala
    return

tp2_pct = TP2_PCT
if direction == "long":
    sl_price  = round_step(price * (1 - stop_pct / 100), price_tick)
    tp1_price = round_step(price * (1 + tp_pct   / 100), price_tick)
    tp2_price = round_step(price * (1 + tp2_pct  / 100), price_tick)
else:
    sl_price  = round_step(price * (1 + stop_pct / 100), price_tick)
    tp1_price = round_step(price * (1 - tp_pct   / 100), price_tick)
    tp2_price = round_step(price * (1 - tp2_pct  / 100), price_tick)

qty_tp1 = round_step(quantity * (TP1_CLOSE_PCT / 100), qty_step)
qty_tp2 = round_step(quantity - qty_tp1, qty_step)
if qty_tp1 <= 0 or qty_tp2 <= 0:
    # Position too small to split (e.g. 0.001 BTC with a 0.001 step floor
    # the TP1 half to 0). A zero-size order is rejected by Binance and us
    # to abort the whole TP placement, so put the entire size on TP1.
    qty_tp1 = quantity
    qty_tp2 = None

cancel_open_orders()
if MAKER_ENTRY:
    entry_resp = place_maker_entry(side, quantity, direction, price_tick)
    if entry_resp is None:
        return  # cancelled by user
else:
    entry_resp = place_order(side, "MARKET", quantity)
try:
    log_exec_slippage(TRACK_MASTER_ID, "master", direction, price,
                      (entry_resp or {}).get("avgPrice") or (entry_resp o
except Exception:
    pass

# Clients enter the moment the master's entry is in. (This used to wait u
# after the master's stop-loss order and a database write.) Each client r
# its own thread, so this returns immediately and the master carries on b
if (triggered_by or "").startswith("STAIR") and not STAIR_MIRROR:
```

```
    print("[Mirror] Staircase trade — client mirroring skipped (STAIRCASE
else:
    mirror_open_all(direction, stop_pct, tp_pct, tp2_pct)

invalidate_position_cache()  # a new position just opened — don't let a s
current_trade_entry["price"]     = price
current_trade_entry["direction"] = direction
current_trade_entry["sl_price"]  = sl_price
current_trade_entry["tp1_price"] = tp1_price
current_trade_entry["tp2_price"] = tp2_price
current_trade_entry["qty_tp1"]   = qty_tp1
current_trade_entry["qty_tp2"]   = qty_tp2 or 0
current_trade_entry["tp1_filled"] = False
# Manual /long and /short pass triggered_by="manual_*" — those are
# manual trades, so they must NOT be counted as indicator trades.
if triggered_by and not str(triggered_by).startswith("manual") and entry_
    indicator_trade_ids.add(str(entry_resp["orderId"]))
log_trade_source((entry_resp or {}).get("orderId"), direction, triggered_

# ── Place REAL exchange-backed SL order on Binance ──────────────
# Previously the SL only lived in current_trade_entry (a Python
# dict in memory) and was "enforced" purely by the polling loop
# comparing mark price to that stored value every cycle. If the bot
# restarted, crashed, or got rate-limited for even a few minutes —
# all of which have happened — that in-memory value vanished and
# NOTHING was protecting the position, even though the Telegram
# message said "SL updated." TP1/TP2 already have real LIMIT orders
# placed below (reduce_only) — SL was the one genuine gap. This
# places a real STOP_MARKET order via Binance's algo order endpoint
# (mandatory since 2025-12-09 for conditional order types — the old
# /fapi/v1/order endpoint rejects STOP_MARKET outright), which fires
# on Binance's own servers regardless of whether this bot is even
# running. The software check in the polling loop is now a
# secondary/backup layer, not the only protection.
close_side = "SELL" if direction == "long" else "BUY"
try:
    place_algo_order(close_side, "STOP_MARKET", sl_price, close_position=
except requests.exceptions.HTTPError as e:
    if "-4130" in str(e):
        # A stray closePosition order from a previous crashed/
        # incomplete cycle is blocking this one — clear it and
        # retry once, same handling as adjust_sl().
        try:
            cancel_all_algo_orders(keep_latest=False)
            place_algo_order(close_side, "STOP_MARKET", sl_price, close_p
        except Exception as retry_err:
```

```
            send_telegram(f"⚠️ <b>Real SL order failed to place</b>\n\n{r
            print(f"[Real SL] Retry after -4130 also failed: {retry_err}"
    else:
        send_telegram(f"⚠️ <b>Real SL order failed to place</b>\n\n{e}\n\
        print(f"[Real SL] Placement error: {e}")
except Exception as e:
    send_telegram(f"⚠️ <b>Real SL order failed to place</b>\n\n{e}\n\nSof
    print(f"[Real SL] Placement error: {e}")

# ── Update trigger_performance row with entry price data ──────
global trade_mfe, trade_mae, trade_entry_time, retrace_protect_triggered
trade_mfe        = 0.0
trade_mae        = 0.0
trade_entry_time = time.time()
retrace_protect_triggered = False
if pending_trigger_id:
    try:
        conn = get_db()
        cur  = conn.cursor()
        cur.execute("""
            UPDATE trigger_performance
            SET entry_price = %s, sl_price = %s, tp1_price = %s, tp2_pric
            WHERE id = %s
        """, (price, sl_price, tp1_price, tp2_price, pending_trigger_id))
        conn.commit()
        cur.close()
        conn.close()
    except Exception as e:
        print(f"[DB] Entry price update error: {e}")

entry2_qty = round_step(quantity * (ENTRY2_SIZE_PCT / 100), qty_step)
if entry2_qty > 0:
    if direction == "long":
        entry2_price = round_step(price * (1 - ENTRY2_OFFSET_PCT / 100),
    else:
        entry2_price = round_step(price * (1 + ENTRY2_OFFSET_PCT / 100),
    e2_resp = place_order(side, "LIMIT", entry2_qty, limit_price=entry2_p
    register_order(e2_resp, "Entry2")
    entry2_placed_at = time.time()
else:
    entry2_qty   = 0
    entry2_price = None

tp1_resp = place_order(close_side, "LIMIT", qty_tp1,
                       limit_price=tp1_price, reduce_only=True)
register_order(tp1_resp, "TP1")
```

```
        if qty_tp2:
            tp2_resp = place_order(close_side, "LIMIT", qty_tp2,
                                   limit_price=tp2_price, reduce_only=True)
            register_order(tp2_resp, "TP2")

        signal_line = f"📡 Signal:   {triggered_by}\n" if triggered_by else ""
        zscore_line = f"📊 Z-Score:  {z_score}\n"       if z_score is not None el
        pos_size_usd = quantity * price / LEVERAGE
        entry2_size_usd = (entry2_qty * entry2_price / LEVERAGE) if entry2_qty >
        entry2_line = (
            f"📩 Entry2:  ${entry2_price:,.2f}  (-{ENTRY2_OFFSET_PCT}%) — ${entry
            if entry2_qty > 0 else "📩 Entry2:  not placed (position too small tc
        )
        tp2_line = f"🎯 TP2:     ${tp2_price:,.2f}  ({tp2_pct}%)\n" if qty_tp2 el

        # Max loss/win on initial position size (before Entry2 fills)
        max_loss_usd = quantity * abs(price - sl_price)
        max_win_usd  = qty_tp1 * abs(tp1_price - price)
        if qty_tp2:
            max_win_usd += qty_tp2 * abs(tp2_price - price)

        send_telegram(
            f"{'🟢' if direction == 'long' else '🔴'} <b>{'LONG' if direction ==
            f"Symbol:     {SYMBOL}\n"
            f"Entry:      ${price:,.2f}\n"
            f"Size:       ${pos_size_usd:,.2f} (×{LEVERAGE} lev)\n"
            f"{signal_line}"
            f"{zscore_line}"
            f"{entry2_line}"
            f"\n🛑 SL:      ${sl_price:,.2f}  ({stop_pct}%) — max loss {format_pn
            f"🎯 TP1:     ${tp1_price:,.2f}  ({tp_pct}%) — {TP1_CLOSE_PCT:.0f}% c
            f"{tp2_line}"
            f"💰 Max win: {format_pnl(max_win_usd)} (TP1+TP2 full)\n"
            f"\nBalance:    ${balance:,.2f} USDT"
        )
        print(f"[Execution] {direction.upper()} — qty {quantity}, SL {sl_price},
        start_prompt_silence("Position open — stay focused on the trade.")

    except requests.exceptions.HTTPError as e:
        try:
            err_json = e.response.json()
            err_msg  = f"Code {err_json.get('code')}: {err_json.get('msg')}"
        except Exception:
            err_msg  = str(e)
        send_telegram(f"❌ <b>Order failed</b>\n{err_msg}")
```

```
        print(f"[Execution ERROR] {err_msg}")
    except Exception as e:
        send_telegram(f"❌ <b>Execution error</b>\n{e}")
        print(f"[Execution ERROR] {e}")


_close_position_lock = threading.Lock()

def close_position_now(reason: str = None) -> None:
    if not EXECUTION_ENABLED:
        send_telegram("⚠️ Execution is disabled. Set EXECUTION_ENABLED=true to en
        return
    # Non-blocking: if a close is already in progress, reject the duplicate
    # rather than let two /close presses (or a manual /close overlapping a
    # guard-triggered force-close) both hammer Binance for the same position.
    if not _close_position_lock.acquire(blocking=False):
        send_telegram("⏳ A close is already being processed — please wait a mome
        return
    try:
        _close_position_inner(reason)
    finally:
        _close_position_lock.release()


def _close_position_inner(reason: str = None) -> None:
    try:
        pos = get_open_position()
        if not pos:
            send_telegram("ℹ️ No open position to close.")
            return

        amt       = float(pos["positionAmt"])
        side      = "SELL" if amt > 0 else "BUY"
        quantity  = abs(amt)
        direction = "LONG" if amt > 0 else "SHORT"

        cancel_open_orders()
        place_order(side, "MARKET", quantity, reduce_only=True)
        invalidate_position_cache()  # position is now closed — don't let a stale
        unreal_pnl  = float(pos.get("unRealizedProfit", 0))
        db_update_outcome(pending_trigger_id, "win" if unreal_pnl >= 0 else "loss
                          mae=trade_mae, mfe=trade_mfe)
        current_trade_entry.clear()

        entry_price = float(pos.get("entryPrice", 0))
        unreal_pnl  = float(pos.get("unRealizedProfit", 0))
```

```
        mark_price  = get_mark_price()
        header = "🛡️ <b>FORCE CLOSED — {direction}</b>" if reason else "🏳️ <b>{di
        reason_line = f"\nReason:     {reason}" if reason else ""
        send_telegram(
            f"{header.format(direction=direction)}\n\n"
            f"Symbol:     {SYMBOL}\n"
            f"Size:       ${quantity * mark_price / LEVERAGE:,.2f}\n"
            f"Entry:      ${entry_price:,.2f}\n"
            f"Exit:       ${mark_price:,.2f}\n"
            f"Unrealised: {format_pnl(unreal_pnl)}"
            f"{reason_line}"
        )
        if unreal_pnl < 0:
            send_push_notification("Position closed at a loss", f"{direction} clo
        mirror_close_all()
        print(f"[Execution] Position closed — {direction} {quantity}" + (f" — {re

    except requests.exceptions.HTTPError as e:
        err = e.response.json() if e.response else str(e)
        send_telegram(f"❌ <b>Close failed</b>\n{err}")
    except Exception as e:
        send_telegram(f"❌ <b>Close error</b>\n{e}")


def status_message() -> None:
    try:
        pos = get_open_position()
        if not pos:
            send_telegram("📊 No open position.")
            return
        amt         = float(pos["positionAmt"])
        direction   = "LONG 🟢" if amt > 0 else "SHORT 🔴"
        entry_price = float(pos.get("entryPrice", 0))
        mark_price  = get_mark_price()
        unreal_pnl  = float(pos.get("unRealizedProfit", 0))
        liq_price   = float(pos.get("liquidationPrice", 0))

        margin     = abs(amt) * entry_price / LEVERAGE if entry_price > 0 else 0
        unreal_pct = (unreal_pnl / margin * 100) if margin > 0 else 0
        unreal_sign = "+" if unreal_pct >= 0 else ""

        sl_price = current_trade_entry.get("sl_price")
        if sl_price:
            sl_loss    = abs(sl_price - entry_price) * abs(amt) if entry_price >
            sl_pct     = abs(sl_price - entry_price) / entry_price * 100 if entry
            sl_line    = f"🛑 SL:         ${sl_price:,.2f}  ({sl_pct:.2f}% — loss
```

```
else:
    sl_line = "🛑 SL:         Not set (use /sl to set)\n"

tp1_price = None
tp2_price = None
tp1_qty   = None
tp2_qty   = None
for oid, meta in tracked_orders.items():
    if meta["label"] == "TP1":
        tp1_price = meta["price"]
        tp1_qty   = meta["qty"]
    elif meta["label"] == "TP2":
        tp2_price = meta["price"]
        tp2_qty   = meta["qty"]

if tp1_price is None:
    try:
        now_ms  = int(time.time() * 1000)
        params  = {"symbol": SYMBOL, "timestamp": now_ms}
        params  = sign_request(params)
        headers = {"X-MBX-APIKEY": BINANCE_API_KEY}
        resp    = requests.get(f"{FUTURES_BASE}/fapi/v1/openOrders",
                               params=params, headers=headers, timeout=10
        live_orders = resp.json() if resp.status_code == 200 else []
        close_side  = "SELL" if amt > 0 else "BUY"
        limit_closes = sorted(
            [o for o in live_orders
             if o.get("type") == "LIMIT"
             and o.get("side") == close_side
             and o.get("reduceOnly")],
            key=lambda o: float(o["price"]),
            reverse=(amt < 0)
        )
        valid = [o for o in limit_closes
                 if entry_price > 0
                 and abs(float(o["price"]) - entry_price) / entry_price *
        if len(valid) >= 1:
            tp1_price = float(valid[0]["price"])
            tp1_qty   = float(valid[0].get("origQty", 0))
        if len(valid) >= 2:
            tp2_price = float(valid[1]["price"])
            tp2_qty   = float(valid[1].get("origQty", 0))
    except Exception:
        pass

if tp1_price and entry_price > 0:
```

```
            tp1_profit = abs(tp1_price - entry_price) * (tp1_qty or abs(amt))
            tp1_pct    = abs(tp1_price - entry_price) / entry_price * 100
            tp1_line   = f"🎯 TP1:        ${tp1_price:,.2f}  ({tp1_pct:.2f}% — pr
        else:
            tp1_line = "🎯 TP1:        Not set\n"

        if tp2_price and entry_price > 0:
            tp2_profit = abs(tp2_price - entry_price) * (tp2_qty or abs(amt))
            tp2_pct    = abs(tp2_price - entry_price) / entry_price * 100
            tp2_line   = f"🎯 TP2:        ${tp2_price:,.2f}  ({tp2_pct:.2f}% — pr
        else:
            tp2_line = "🎯 TP2:        Not set\n"

        def fmt_position_mins(mins: float) -> str:
            if mins >= 60:
                return f"{int(mins // 60)}h {int(mins % 60)}m"
            return f"{int(mins)}m"

        time_line = (
            f"⏱️ In profit: {fmt_position_mins(position_mins_profit)}  |  "
            f"Underwater: {fmt_position_mins(position_mins_under)}\n"
        )

        send_telegram(
            f"📊 <b>Open position: {direction}</b>\n\n"
            f"Symbol:       {SYMBOL}\n"
            f"Size:         ${abs(amt) * mark_price / LEVERAGE:,.2f}  (×{LEVERAGE
            f"Entry:        ${entry_price:,.2f}\n"
            f"Mark:         ${mark_price:,.2f}\n"
            f"Unrealised:   {format_pnl(unreal_pnl)}  ({unreal_sign}{unreal_pct:.
            f"{time_line}"
            f"{sl_line}"
            f"{tp1_line}"
            f"{tp2_line}"
            f"Liquidation:  ${liq_price:,.2f}"
        )
    except Exception as e:
        send_telegram(f"❌ Status error: {e}")


def adjust_sl(new_sl_price: float) -> None:
    if not EXECUTION_ENABLED:
        send_telegram("⚠️ Execution is disabled.")
        return
    try:
        pos = get_open_position()
```

```
if not pos:
    send_telegram("ℹ️ No open position to adjust SL on.")
    return

amt       = float(pos["positionAmt"])
direction = "LONG" if amt > 0 else "SHORT"
entry     = float(pos.get("entryPrice", 0))
quantity  = abs(amt)
close_side = "SELL" if amt > 0 else "BUY"

# ── Replace the REAL exchange-backed stop order safely ──
# Binance allows only ONE closePosition=true conditional order per
# side at a time (rejects a second with error -4130). A real SL is
# placed automatically the moment a position opens (see
# _execute_trade_inner), so almost every call here is genuinely
# replacing an existing order, not creating a first one — meaning
# the -4130 conflict is the NORMAL case, not an edge case. Handle
# it by cancelling the old stop and retrying immediately, rather
# than treating it as a hard failure.
try:
    place_algo_order(close_side, "STOP_MARKET", new_sl_price, close_posit
except requests.exceptions.HTTPError as e:
    if "-4130" in str(e):
        try:
            cancel_all_algo_orders(keep_latest=False)
            place_algo_order(close_side, "STOP_MARKET", new_sl_price, clo
        except Exception as retry_err:
            send_telegram(
                f"❌ <b>Failed to move SL</b>\n\nThe old stop was cancell
                f"but placing the new one also failed:\n{retry_err}\n\n"
                f"🚨 <b>Position may be UNPROTECTED.</b> Set a stop manua
            )
            print(f"[Adjust SL] Retry after -4130 also failed: {retry_err
            return
    else:
        # A failure that ISN'T the known -4130 conflict means we
        # never touched whatever stop already existed — check what
        # the exchange actually shows before saying anything
        # reassuring (an earlier version of this message always
        # claimed "still protected by the previous stop" even when
        # this was the very first SL attempt and no previous stop
        # had ever existed).
        try:
            existing_stops = get_open_algo_orders()
        except Exception:
            existing_stops = None
```

```
        if existing_stops:
            safety_note = "Position is still protected by the previous st
        elif existing_stops == []:
            safety_note = (
                "🚨 <b>No stop-loss is currently active on this position
                "Set one manually on the exchange now, or retry /sl immed
            )
        else:
            safety_note = "⚠️ Could not verify whether a stop-loss is cur

        send_telegram(f"❌ <b>Failed to place new SL order</b>\n\n{e}\n\n
        print(f"[Adjust SL] Real order placement error: {e}")
        return
except Exception as e:
    send_telegram(
        f"❌ <b>Failed to place new SL order</b>\n\n{e}\n\n"
        f"⚠️ Could not verify whether a stop-loss is currently active — c
    )
    print(f"[Adjust SL] Real order placement error (non-HTTP): {e}")
    return
# New master stop is live: start the client syncs NOW, in parallel with t
# master's own clean-up below, instead of after it.
if entry > 0:
    _sl_pct_signed = ((entry - new_sl_price) if amt > 0 else (new_sl_pric
    mirror_adjust_sl_all(_sl_pct_signed)
cancel_all_algo_orders(keep_latest=True)

current_trade_entry["sl_price"] = new_sl_price

sl_pct   = abs(new_sl_price - entry) / entry * 100 if entry > 0 else 0
max_loss = abs(new_sl_price - entry) * quantity if entry > 0 else 0

# Sync the SAME percentage to every client, recalculated against
# each of their own entries — not the master's absolute price,
# which could sit at a genuinely different % distance from a
# client's actual Hyperliquid fill. The percentage is SIGNED:
# positive = on the loss side of entry, negative = on the profit
# side (break-even / locked-in stops). It used to be unsigned, so a
# break-even or lock-in stop reached clients as a loss-side stop
# (or, for exact break-even, not at all).
if entry > 0:
    pass   # client sync already started above, right after the new stop

send_telegram(
    f"✏️ <b>SL updated — {direction}</b>\n\n"
```

```
            f"New SL:   ${new_sl_price:,.2f}  ({sl_pct:.2f}% from entry)\n"
            f"Entry:    ${entry:,.2f}\n"
            f"Max loss: {format_pnl(-max_loss)}\n\n"
            f"✅ Real order placed on exchange."
        )
    except requests.exceptions.HTTPError as e:
        try:
            err = e.response.json()
            send_telegram(f"❌ SL adjust failed\nCode {err.get('code')}: {err.get
        except Exception:
            send_telegram(f"❌ SL adjust failed: {e}")
    except Exception as e:
        send_telegram(f"❌ SL adjust error: {e}")


def tp2_ordering_error(new_tp2_price: float, is_short: bool):
    """TP2 must sit further out than TP1. Returns a message if the requested
    TP2 would land at or before TP1, else None. Not enforced once TP1 has
    filled (it no longer constrains anything), or if TP1 isn't known."""
    tp1 = current_trade_entry.get("tp1_price")
    if not tp1 or current_trade_entry.get("tp1_filled"):
        return None
    beyond = (new_tp2_price < tp1) if is_short else (new_tp2_price > tp1)
    if beyond:
        return None
    return (f"TP2 must be further out than TP1 (${tp1:,.2f}). "
            f"${new_tp2_price:,.2f} is at or before it — change not made.")


def adjust_tp(new_tp_price: float, label: str = "TP1") -> None:
    if not EXECUTION_ENABLED:
        send_telegram("⚠️ Execution is disabled.")
        return
    try:
        pos = get_open_position()
        if not pos:
            send_telegram("ℹ️ No open position to adjust TP on.")
            return

        amt        = float(pos["positionAmt"])
        direction  = "LONG" if amt > 0 else "SHORT"
        close_side = "SELL" if amt > 0 else "BUY"
        entry      = float(pos.get("entryPrice", 0))

        if label == "TP2":
            _err = tp2_ordering_error(new_tp_price, is_short=(amt < 0))
```

```
    if _err:
        send_telegram(f"🚫 <b>TP2 not changed</b>\n\n{_err}")
        return

sym_info   = get_symbol_info()
price_tick = sym_info["price_tick"]
qty_step   = sym_info["qty_step"]

tp_order_id = None
tp_qty      = None
for oid, meta in list(tracked_orders.items()):
    if meta["label"] == label:
        tp_order_id = int(oid)
        tp_qty      = meta["qty"]
        break

if tp_order_id:
    now_ms  = int(time.time() * 1000)
    params  = {"symbol": SYMBOL, "orderId": tp_order_id, "timestamp": now
    params  = sign_request(params)
    headers = {"X-MBX-APIKEY": BINANCE_API_KEY}
    try:
        requests.delete(f"{FUTURES_BASE}/fapi/v1/order",
                        params=params, headers=headers, timeout=10)
    except Exception:
        pass
    tracked_orders.pop(str(tp_order_id), None)
else:
    try:
        now_ms      = int(time.time() * 1000)
        params      = {"symbol": SYMBOL, "timestamp": now_ms}
        params      = sign_request(params)
        headers     = {"X-MBX-APIKEY": BINANCE_API_KEY}
        resp        = requests.get(f"{FUTURES_BASE}/fapi/v1/openOrders",
                                   params=params, headers=headers, timeou
        live_orders = resp.json() if resp.status_code == 200 else []
        for o in live_orders:
            if o.get("type") == "LIMIT" and o.get("side") == close_side a
                cancel_params = {"symbol": SYMBOL, "orderId": o["orderId"
                cancel_params = sign_request(cancel_params)
                requests.delete(f"{FUTURES_BASE}/fapi/v1/order",
                                params=cancel_params, headers={"X-MBX-API
                if tp_qty is None:
                    tp_qty = float(o.get("origQty", 0))
                break
    except Exception:
```

```
        pass

quantity = round_step(tp_qty or abs(amt), qty_step)
resp = place_order(close_side, "LIMIT", quantity,
                   limit_price=new_tp_price, reduce_only=True)
register_order(resp, label)

# Remember what TP1 was BEFORE this change (the TP2 auto-adjust below
# needs the old value to keep TP2 the same distance away), then record
# the new target on the live trade record. Without this, the dashboard
# summary kept showing the original TP1/TP2 after /tp, /tp2 or the
# dashboard buttons had moved the real order.
prev_tp1_price = current_trade_entry.get("tp1_price", entry)
current_trade_entry["tp1_price" if label == "TP1" else "tp2_price"] = new

# ── Auto-adjust TP2 if new TP1 would conflict ──────────────
if label == "TP1":
    tp2_meta = next(((oid, m) for oid, m in tracked_orders.items() if m["
    if tp2_meta:
        tp2_oid, tp2_m = tp2_meta
        tp2_price = tp2_m["price"]
        tp2_qty   = tp2_m["qty"]
        conflict  = (amt > 0 and new_tp_price >= tp2_price) or \
                    (amt < 0 and new_tp_price <= tp2_price)
        if conflict:
            try:
                # Cancel old TP2
                now_ms2  = int(time.time() * 1000)
                params2  = {"symbol": SYMBOL, "orderId": int(tp2_oid), "t
                params2  = sign_request(params2)
                requests.delete(f"{FUTURES_BASE}/fapi/v1/order",
                                params=params2, headers={"X-MBX-APIKEY":
                tracked_orders.pop(str(tp2_oid), None)
                # Place new TP2 same distance from new TP1 as it was from
                old_tp1_price = prev_tp1_price
                tp2_offset    = abs(tp2_price - old_tp1_price)
                if amt > 0:  # long
                    new_tp2_price = round_step(new_tp_price + tp2_offset,
                else:        # short
                    new_tp2_price = round_step(new_tp_price - tp2_offset,
                tp2_qty_rounded = round_step(tp2_qty or abs(amt) * (1 - T
                resp2 = place_order(close_side, "LIMIT", tp2_qty_rounded,
                                    limit_price=new_tp2_price, reduce_onl
                register_order(resp2, "TP2")
                current_trade_entry["tp2_price"] = new_tp2_price
                new_tp2_pct = abs(new_tp2_price - entry) / entry * 100 if
```

```
                        if new_tp2_pct > 0:
                            mirror_adjust_tp_all(new_tp2_pct, label="TP2")
                        send_telegram(
                            f"🔄 <b>TP2 auto-adjusted to avoid conflict</b>\n\n"
                            f"New TP2: ${new_tp2_price:,.2f}  (same offset from n
                        )
                    except Exception as e:
                        send_telegram(f"⚠️ TP2 auto-adjust failed: {e}\nUse /tp2

        max_profit = abs(new_tp_price - entry) * quantity if entry > 0 else 0
        tp_pct     = abs(new_tp_price - entry) / entry * 100 if entry > 0 else 0

        # Sync the SAME percentage to every client, recalculated against
        # each of their own entries — mirrors what /sl already does.
        # This was previously missing entirely: /tp and /tp2 only ever
        # touched the master's own Binance orders.
        if tp_pct > 0:
            mirror_adjust_tp_all(tp_pct, label=label)

        send_telegram(
            f"✏️ <b>{label} updated — {direction}</b>\n\n"
            f"Entry:       ${entry:,.2f}\n"
            f"New {label}:   ${new_tp_price:,.2f}  ({tp_pct:.2f}% from entry)\n"
            f"Max profit:  {format_pnl(max_profit)}"
        )
    except requests.exceptions.HTTPError as e:
        try:
            err = e.response.json()
            send_telegram(f"❌ {label} adjust failed\nCode {err.get('code')}: {er
        except Exception:
            send_telegram(f"❌ {label} adjust failed: {e}")
    except Exception as e:
        send_telegram(f"❌ {label} adjust error: {e}")


def move_to_breakeven() -> None:
    if not EXECUTION_ENABLED:
        send_telegram("⚠️ Execution is disabled.")
        return
    try:
        pos = get_open_position()
        if not pos:
            send_telegram("ℹ️ No open position.")
            return

        entry     = float(pos.get("entryPrice", 0))
```

```
        unreal    = float(pos.get("unRealizedProfit", 0))
        amt       = float(pos["positionAmt"])
        direction = "LONG" if amt > 0 else "SHORT"

        if unreal <= 0:
            send_telegram(
                f"⚠️ <b>Not in profit yet.</b>\n"
                f"Unrealised P&L is {format_pnl(unreal)} — "
                f"moving SL to breakeven would lock in a loss."
            )
            return

        sym_info   = get_symbol_info()
        price_tick = sym_info["price_tick"]
        be_price   = round_step(entry, price_tick)
        mark       = get_mark_price()

        adjust_sl(be_price)
        current_trade_entry["sl_price"] = be_price
        send_telegram(
            f"⚖️ <b>SL moved to breakeven — {direction}</b>\n\n"
            f"Entry price:     ${entry:,.2f}\n"
            f"New SL:          ${be_price:,.2f}  (exact entry — zero risk from he
            f"Current mark:    ${mark:,.2f}\n"
            f"Unrealised P&L:  {format_pnl(unreal)} (not locked in — still floati
            f"Worst case from here is a scratch trade. Upside still fully open."
        )
    except Exception as e:
        send_telegram(f"❌ Breakeven error: {e}")


def partial_close(pct: float) -> None:
    if not EXECUTION_ENABLED:
        send_telegram("⚠️ Execution is disabled.")
        return
    try:
        pos = get_open_position()
        if not pos:
            send_telegram("ℹ️ No open position to partially close.")
            return

        amt        = float(pos["positionAmt"])
        direction  = "LONG" if amt > 0 else "SHORT"
        close_side = "SELL" if amt > 0 else "BUY"
        sym_info   = get_symbol_info()
        qty_step   = sym_info["qty_step"]
```

```
        close_qty = round_step(abs(amt) * (pct / 100), qty_step)
        if close_qty <= 0:
            send_telegram("❌ Quantity too small to close.")
            return

        place_order(close_side, "MARKET", close_qty, reduce_only=True)
        invalidate_position_cache()  # position size just changed — don't let a s

        remaining_usd = (abs(amt) - close_qty) * get_mark_price()
        send_telegram(
            f"✂️ <b>Partial close {pct:.0f}% — {direction}</b>\n\n"
            f"Closed:     ${close_qty * get_mark_price():,.2f}\n"
            f"Remaining:  ${remaining_usd:,.2f}"
        )
    except requests.exceptions.HTTPError as e:
        try:
            err = e.response.json()
            send_telegram(f"❌ Partial close failed\nCode {err.get('code')}: {err
        except Exception:
            send_telegram(f"❌ Partial close failed: {e}")
    except Exception as e:
        send_telegram(f"❌ Partial close error: {e}")


# ─────────────────────────────────────────────
# PROFIT ALERT STATE
# ─────────────────────────────────────────────
BREAKEVEN_SUGGEST_PCT   = float(os.environ.get("BREAKEVEN_SUGGEST_PCT", "0.20"))
BREAKEVEN_TIME_MINS     = float(os.environ.get("BREAKEVEN_TIME_MINS", "40"))   #
RETRACE_PROTECT_PCT     = float(os.environ.get("RETRACE_PROTECT_PCT", "0.35")) #
RETRACE_LOCK_IN_PCT     = float(os.environ.get("RETRACE_LOCK_IN_PCT", "0.05")) #
breakeven_suggested     = False
breakeven_last_suggested_at: float = 0.0
BREAKEVEN_REPEAT_MINS   = 30

def check_profit_alert(pos: dict) -> None:
    global breakeven_suggested, breakeven_last_suggested_at
    if not pos:
        return
    if breakeven_suggested and (time.time() - breakeven_last_suggested_at) < BREA
        return

    entry     = float(pos.get("entryPrice", 0))
    mark      = get_mark_price()
    amt       = float(pos["positionAmt"])
```

```
    direction = "LONG" if amt > 0 else "SHORT"

    if entry <= 0:
        return

    pct_move = ((mark - entry) / entry * 100) if amt > 0 else ((entry - mark) / e

    triggered_by_pct  = pct_move >= BREAKEVEN_SUGGEST_PCT
    triggered_by_time = position_mins_profit >= BREAKEVEN_TIME_MINS

    if triggered_by_pct or triggered_by_time:
        if triggered_by_time and not triggered_by_pct:
            reason_line = f"In profit for {position_mins_profit:.0f}m without hit
        else:
            reason_line = f"Position up {pct_move:.2f}%"
        send_telegram(
            f"💡 <b>{reason_line} — consider breakeven</b>\n\n"
            f"Entry:  ${entry:,.2f}\n"
            f"Mark:   ${mark:,.2f}\n\n"
            f"Send /breakeven to move SL to entry and protect the trade."
        )
        breakeven_suggested = True
        breakeven_last_suggested_at = time.time()


# ─────────────────────────────────────────────
# 8AM MORNING BRIEF
# ─────────────────────────────────────────────
morning_brief_sent_today = False
weekly_digest_sent_today = False
weekly_okr_sent_today    = False
monthly_review_sent_today = False
daily_bias               = None
bias_question_sent_today = False
pending_signal           = None
SIGNAL_EXPIRY_SECS       = 5400  # 90 minutes
elevated_silence_until: float = 0.0  # suppress all prompts until this timestamp
pending_bias_reset: dict     = None   # {direction, structure_break}

# Today's trigger activity — reset at midnight
trigger_counts_today: dict      = {}   # {sig_key: count}
long_trigger_times_today: list  = []   # timestamps of long triggers today
short_trigger_times_today: list = []   # timestamps of short triggers today


def get_signal_track_record(sig_key: str) -> str:
```

```
    """Returns a short stat line for this signal's historical win rate and invali
    if not DATABASE_URL or not sig_key:
        return ""
    try:
        conn = get_db()
        cur  = conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor)
        cur.execute("""
            SELECT
                COUNT(*) FILTER (WHERE outcome IN ('win','loss')) as total,
                COUNT(*) FILTER (WHERE outcome = 'win') as wins
            FROM trigger_performance
            WHERE trigger = %s
        """, (sig_key,))
        r = cur.fetchone()

        cur.execute("""
            SELECT
                (SELECT COUNT(*) FROM trigger_fires WHERE trigger = %s) as fires,
                (SELECT COUNT(*) FROM invalidation_log WHERE trigger = %s) as inv
        """, (sig_key, sig_key))
        r2 = cur.fetchone()
        cur.close()
        conn.close()

        parts = []
        total = r["total"] if r else 0
        if total and total >= 3:
            wr = r["wins"] / total * 100
            parts.append(f"{wr:.0f}% WR ({total} trades)")
        fires = r2["fires"] if r2 else 0
        invs  = r2["invalidations"] if r2 else 0
        if fires and fires >= 3:
            inv_rate = invs / fires * 100
            parts.append(f"{inv_rate:.0f}% invalidation rate")
        if not parts:
            return ""
        return "📊 " + "  |  ".join(parts)
    except Exception as e:
        print(f"[Signal Track Record] Error: {e}")
        return ""


def send_signal_prompt(direction: str, label: str, sig_key: str = None):
    global pending_signal
    if elevated_silence_until > time.time():
        print(f"[Signal] Suppressed {label} — elevated silence active")
```

```
        return
    if stopped_today:
        print(f"[Signal] Suppressed {label} — daily trade limit hit")
        return
    # No point prompting for a trade the guards would refuse anyway —
    # during the US-open cooldown or after the night curfew, stay silent.
    _now_full = datetime.now(UK_TZ)
    _now_t = _now_full.time()
    if _now_full.weekday() == 4 and _now_t >= dt_time(19, 0):
        print(f"[Signal] Suppressed {label} — Friday cutoff")
        return
    if daily_losses_block:
        print(f"[Signal] Suppressed {label} — loss-streak block active")
        return
    if COOLDOWN_START <= _now_t < COOLDOWN_END:
        print(f"[Signal] Suppressed {label} — US open cooldown")
        return
    if _now_t >= NIGHT_BLOCK_START:
        print(f"[Signal] Suppressed {label} — after {NIGHT_BLOCK_START.strftime('
        return
    emoji = "🟢" if direction == "long" else "🔴"
    track_record = get_signal_track_record(sig_key)
    track_line = f"\n{track_record}\n" if track_record else ""
    text = (
        f"⚡ <b>{label}</b>\n\n"
        f"Direction: {emoji} {direction.upper()}\n"
        f"{track_line}"
        f"\nDo you want to take this trade?\n"
        f"⏱️ Expires in 90 minutes."
    )
    payload = {
        "chat_id": TELEGRAM_CHAT_ID,
        "text": text,
        "parse_mode": "HTML",
        "reply_markup": {
            "inline_keyboard": [[
                {"text": "✅ Take Trade", "callback_data": f"take_{direction}"},
                {"text": "❌ Skip",       "callback_data": f"skip_{direction}"}
            ]]
        }
    }
    try:
        resp = requests.post(
            f"https://api.telegram.org/bot{TELEGRAM_BOT_TOKEN}/sendMessage",
            json=payload, timeout=10
        )
```

```
        msg_id = resp.json().get("result", {}).get("message_id")
        pending_signal = {
            "direction": direction,
            "label":     label,
            "timestamp": time.time(),
            "message_id": msg_id
        }
    except Exception as e:
        print(f"[Signal Prompt] Error: {e}")


# ═══════════════════════════════════════════════════════════════════
# STAIRCASE ALERTS — Telegram only, BTC only
# Colour-staircase setups (Short/Long A, B, C, A+, B+) from the composite on thre
# (default 4h / 30m / 5m, set with STAIRCASE_TF_HIGH / _MID / _LOW).
# Fires once per CLOSED candle of the fastest timeframe. The two slower timeframe
# (forming) candle, the fastest uses closed candles only. Tapping Take Trade runs
# execute path, including client mirroring (set STAIRCASE_MIRROR=false to stop th
# Disable with env STAIRCASE_ALERTS=false.
# ═══════════════════════════════════════════════════════════════════
STAIR_ENABLED     = os.environ.get("STAIRCASE_ALERTS", "true").lower() == "true"
STAIR_EXPIRY_SECS = 1800   # 30 minutes: repeat-alert suppression window (same se
STAIR_TAKE_SECS   = 3600   # how long a fired setup stays takeable (Telegram butt
STAIR_MIRROR      = os.environ.get("STAIRCASE_MIRROR", "true").lower() == "true"
_STAIR_LOOKBACK   = int(os.environ.get("STAIRCASE_LOOKBACK", "100"))       # your
_STAIR_ZLEN       = int(os.environ.get("STAIRCASE_ZLEN", "100"))           # z-sc
_STAIR_TF_H       = os.environ.get("STAIRCASE_TF_HIGH", "4h")    # slow timeframe
_STAIR_TF_M       = os.environ.get("STAIRCASE_TF_MID",  "30m")    # middle timefr
_STAIR_TF_L       = os.environ.get("STAIRCASE_TF_LOW",  "5m")   # fast timeframe
_STAIR_LIVE_MID    = os.environ.get("STAIRCASE_LIVE_MID", "false").lower() == "tr
_STAIR_TF_SECS    = {"1m": 60, "3m": 180, "5m": 300, "15m": 900, "30m": 1800, "1h
                     "2h": 7200, "4h": 14400, "6h": 21600, "8h": 28800, "12h": 43


def _stair_label(tf):
    return tf[:-1] + ("H" if tf.endswith("h") else "m")


_STAIR_LABELS = (_stair_label(_STAIR_TF_H), _stair_label(_STAIR_TF_M), _stair_lab
_STAIR_HIGH       = float(os.environ.get("STAIRCASE_EXTREME_HIGH", "90"))  # extr
_STAIR_LOW        = float(os.environ.get("STAIRCASE_EXTREME_LOW", "10"))   # extr
_STAIR_EMOJI      = {"Red": "🟥", "Orange": "🟧", "Grey": "⬜", "Green": "🟩", "

# (direction, letter, slow-TF colour, mid-TF colour, fast-TF colour BEFORE, fast-
# Palette matches the chart: Yellow counts as Orange, Blue counts as Green.
_STAIR_RULES = [
```

```
    ("short", "A++", "Red",    "Orange", "Red",    "Orange"),
    ("short", "B", "Red",    "Orange", "Orange", "Grey"),
    ("short", "A+", "Orange", "Red",   "Red",    "Orange"),
    ("short", "B+", "Orange", "Red",   "Orange", "Grey"),
    ("short", "D", "Green",  "Green",  "Orange", "Grey"),
    ("long",  "D", "Orange", "Orange", "Green",  "Grey"),
    ("long",  "A++", "Cyan",   "Green",  "Cyan",   "Green"),
    ("long",  "B", "Cyan",   "Green",  "Green",  "Grey"),
    ("long",  "A+", "Green",  "Cyan",  "Cyan",   "Green"),
    ("long",  "B+", "Green",  "Cyan",  "Green",  "Grey"),
    ("short", "G", "Cyan",   "Orange", "Red",    "Orange"),
    ("short", "H", "Cyan",   "Orange", "Grey",   "Green"),
    ("short", "E", "Green",  "Green", "Green",  "Grey"),
    ("short", "F", "Orange", "Orange", "Orange", "Orange"),
    ("short", "I", "Green",  "Orange", "Red",    "Orange"),
    ("long",  "I", "Orange", "Green",  "Cyan",   "Green"),
    ("long",  "F", "Green",  "Green", "Green",  "Green"),
    ("long",  "E", "Orange", "Orange", "Orange", "Grey"),
    ("long",  "G", "Red",    "Green", "Cyan",   "Green"),
    ("long",  "H", "Red",    "Green", "Grey",   "Orange"),
]
_STAIR_NOTES = {
    ("short", "A++"): "Early entry, best R",   ("long", "A++"): "Early entry, bes
    ("short", "B"): "Final confirmation",    ("long", "B"): "Final confirmation",
    ("short", "All Orange"): "Triple Orange short: only when price is above the 3
    ("long", "All Green"): "Triple Green long: only when price is below the 30m m
    ("long", "D"): "Last discount long in a heavy premium trend: price pushes up
    ("short", "D"): "Last premium short in a heavy downtrend: price pushes down a
    ("short", "A+"): "Best signal: 4H pulls back, 30m Red", ("long", "A+"): "Best
    ("short", "B+"): "A+ confirmation",    ("long", "B+"): "A+ confirmation",
    ("long", "G"): "Lowest discount in a strong higher timeframe",
    ("long", "H"): "2nd opportunity: last discount in a premium trend before the
    ("short", "G"): "Highest premium in a strong higher timeframe",
    ("short", "I"): "Green 4H, Orange 30m: the 5m turns down off Red",
    ("long", "I"): "Orange 4H, Green 30m: the 5m turns up off Cyan",
    ("long", "F"): "All Green long, only when price is below the midline on all t
    ("short", "F"): "All Orange short, only when price is above the midline on al
    ("short", "E"): "Fading the bounce: 5m climbs out of Green into Grey with 4H
    ("long", "E"): "Flip of the retired Short C: 5m drops from Orange into Grey w
    ("short", "H"): "2nd opportunity: last premium in a discount trend before the
}
_STAIR_SKIP_DAYS = {int(x) for x in os.environ.get("STAIRCASE_SKIP_DAYS", "5").sp
_stair_last_alert = {}   # (direction, letter) -> timestamp, stops repeat alerts
_stair_history    = []   # recent fires for the /staircase page (persisted to the
_stair_since      = {}   # settings signature -> when tracking under those settin
STAIR_TOKEN       = os.environ.get("STAIRCASE_TOKEN", "") or READ_API_SECRET
```

```
_stair_rest_cache = {}   # interval -> (fetched_at, closed candles) — keeps REST
_stair_last_why   = ""   # why the page has no data right now (shown on the page)


def _stair_get_klines(interval, n):
    """Closed candles, oldest first. Websocket first; cached REST fallback if the
    Returns (candles or None, source/reason)."""
    k = ws_get_closed_klines(interval, n)
    if k:
        return k, "ws"
    ttl = 20 if interval == _STAIR_TF_L else 60
    hit = _stair_rest_cache.get(interval)
    if hit and time.time() - hit[0] < ttl:
        return hit[1], "rest"
    try:
        resp = requests.get(f"{FUTURES_BASE}/fapi/v1/klines",
                            params={"symbol": SYMBOL, "interval": interval, "limi
        resp.raise_for_status()
        now_ms = int(time.time() * 1000)
        closed = [x for x in resp.json() if x[6] < now_ms]   # drop the forming c
        _stair_rest_cache[interval] = (time.time(), closed)
        return closed, "rest"
    except Exception as e:
        return None, f"{interval} REST error: {e}"


def _stair_mark_price():
    try:
        return get_mark_price()   # Orion's own helper: websocket first, REST fal
    except Exception as e:
        print(f"[Staircase] mark price error: {e}")
        return None


def _stair_cfg():
    base = f"{_STAIR_LOOKBACK}/{_STAIR_ZLEN}/{_STAIR_HIGH:g}/{_STAIR_LOW:g}"
    # timeframes only join the signature when they differ from the default, so de
    return base if (_STAIR_TF_H, _STAIR_TF_M, _STAIR_TF_L) == ("4h", "30m", "5m")


def _stair_save_since():
    try:
        save_persistent_state("stair_since", json.dumps(_stair_since))
    except Exception as e:
```

```
        print(f"[Staircase] since save error: {e}")


def _stair_composite(closes):
    """Same maths as the TradingView composite (STAIRCASE_* env vars, defaults 10
    if len(closes) < _STAIR_ZLEN:
        return None
    w = closes[-_STAIR_ZLEN:]
    mean = sum(w) / len(w)
    sd = (sum((x - mean) ** 2 for x in w) / len(w)) ** 0.5
    if sd == 0:
        return None
    z_norm = min(100.0, max(0.0, 50 + ((closes[-1] - mean) / sd) * 16.67))
    last = closes[-1]
    rank = sum(1 for i in range(_STAIR_LOOKBACK) if last > closes[-1 - i])
    return (z_norm + rank / _STAIR_LOOKBACK * 100) / 2


def _stair_colour(v):
    if v >= _STAIR_HIGH: return "Red"
    if v >= 55: return "Orange"
    if v >= 45: return "Grey"
    if v >= _STAIR_LOW: return "Green"
    return "Cyan"


# Blocked combos: (direction, slow-TF colour, mid-TF colour, fast-TF colour NOW).
# button) is ever sent for that direction in that state. Add rows as more states
_STAIR_BLOCKED = [
    ("short", "*", "Grey", "*", "30m is Grey: no trade in either direction until
    ("long",  "*", "Grey", "*", "30m is Grey: no trade in either direction until
    ("short", "*", "*", "Cyan", "5m already at the floor: shorting a move that's
    ("long", "*", "*", "Red", "5m already at the top: mirror of the short block",
    ("short", "*", "*", "Red", "5m at the top: you can't know where the top is, t
    ("long", "*", "*", "Cyan", "5m at the floor: you can't know where the bottom
    ("short", "Cyan", "Green", "Green", "4H Cyan / 30m Green: shorts only allowed
    ("short", "Cyan", "Green", "Cyan", "4H Cyan / 30m Green: shorts only allowed
    ("short", "Red", "Red", "Red", "All Red: no trade either way (logged as a cou
    ("long", "Red", "Red", "Red", "All Red: no trade either way (logged as a coun
    ("short", "Green", "Green", "Orange", "5m overshoot above two Green timeframe
    ("long", "Green", "Green", "Orange", "5m overshoot above two Green timeframes
    ("short", "Cyan", "Cyan", "Cyan", "All Cyan: no trade either way (logged as a
    ("long", "Cyan", "Cyan", "Cyan", "All Cyan: no trade either way (logged as a
    ("long", "Cyan", "Orange", "Orange", "4H Cyan / 30m Orange / 5m Orange: no lo
    ("long", "Cyan", "Orange", "Green", "4H Cyan / 30m Orange / 5m Green: no long
    ("short", "Red", "Green", "Green", "4H Red / 30m Green / 5m Green: no short,
```

```
    ("short", "Red", "Green", "Orange", "4H Red / 30m Green / 5m Orange: no short
    ("long", "Red", "Orange", "Orange", "Opposite side of the Short A++ state", "
    ("short", "Green", "Orange", "Grey", "4H Green / 30m Orange / 5m Grey: weak c
    ("long", "Green", "Orange", "Grey", "4H Green / 30m Orange / 5m Grey: weak co
]


# Conditional blocks judged on live data (side, condition, reason, date added). L
_STAIR_SHORT_GATE_WPR = float(os.environ.get("STAIRCASE_SHORT_GATE_WPR", "-25"))
_STAIR_GREEN_BAND = (float(os.environ.get("STAIRCASE_GREEN_BAND_LOW", "-85")), fl
_STAIR_ALLORANGE_WPR = float(os.environ.get("STAIRCASE_ALLORANGE_WPR", "-25"))
_stair_align_prev = {"Green": False, "Orange": False}   # was each alignment's co
_STAIR_OVERSOLD_WPR = float(os.environ.get("STAIRCASE_LONG_OVERSOLD_WPR", "-85"))
_STAIR_LONG_GATE_WPR = float(os.environ.get("STAIRCASE_LONG_GATE_WPR", "-75"))
_STAIR_BLOCKED_IF = [
    ("short", f"All Orange alignment, unless price is above the 30m midline AND t
     "Triple Orange is not a good short on its own: only alert when price is high
    ("long", f"30m is Green and price is below the 30m midline, unless the 5m WPR
     "A long off a Green 30m below its midline only alerts when the 5m WPR sits i
    ("long", f"All Green alignment, unless price is below the 30m midline AND the
     "Triple Green is not a good long on its own: only alert when price is low (b
    ("long", f"30m WPR below {_STAIR_OVERSOLD_WPR:g} unless the 5m WPR is above {
     "Don't buy a move that is already extremely oversold on the 30m, unless the
    ("long", f"price above the 30m midline AND 5m WPR above {_STAIR_LONG_GATE_WPR
     "Don't long inside an already heavy uptrend: only the very bottom of it (5m
    ("short", f"price below the 30m midline AND 5m WPR below {_STAIR_SHORT_GATE_W
     "Don't short inside an already heavy downtrend: only the very top of it (5m
]


def _stair_klines_n(interval, n):
    """At least n closed candles: websocket if it holds that many, otherwise a di
    k = ws_get_closed_klines(interval, n)
    if k:
        return k
    try:
        resp = requests.get(f"{FUTURES_BASE}/fapi/v1/klines",
                            params={"symbol": SYMBOL, "interval": interval, "limi
        resp.raise_for_status()
        now_ms = int(time.time() * 1000)
        return [x for x in resp.json() if x[6] < now_ms]
    except Exception as e:
        print(f"[Staircase] gate klines error ({interval}): {e}")
        return None
```

```
# Extra conditions a setup needs on top of its colours: price above these timefra
_STAIR_SETUP_CONDS = {("short", "D"): ("above", ("30m", "5m")), ("long", "D"): ("
                      ("short", "F"): ("above", ("4h", "30m", "5m")), ("long", "F
_STAIR_GATE_EXEMPT = {("short", "B"), ("short", "E"), ("long", "E"), ("short", "F


def _stair_oversold_gate():
    """(blocked, note). A long is blocked when the 30m WPR (length 200) is below
    Fails closed: if either reading can't be made, the long is blocked."""
    try:
        k30, k5 = _stair_klines_n("30m", 200), _stair_klines_n("5m", 200)
        if not k30 or len(k30) < 200 or not k5 or len(k5) < 200:
            return True, "no data to check the oversold filter"
        def w(k):
            r = k[-200:]
            return calculate_williams_r([float(x[2]) for x in r], [float(x[3]) fo
        w30, w5 = w(k30), w(k5)
        return (w30 < _STAIR_OVERSOLD_WPR and w5 <= _STAIR_OVERSOLD_WPR), f"30m W
    except Exception as e:
        print(f"[Staircase] oversold filter error, blocking: {e}")
        return True, "oversold filter error"


def _stair_green30_gate(cols, price):
    """(blocked, note). A long off a Green 30m is blocked when price is below the
    Fails closed: if either reading is missing, the long is blocked."""
    if cols[1] != "Green":
        return False, ""
    above, w5 = _stair_above_mid("30m", price), _stair_wpr("5m")
    if above is None or w5 is None:
        return True, "no data to check the Green-30m filter"
    if above:
        return False, ""
    return (not (_STAIR_GREEN_BAND[0] <= w5 <= _STAIR_GREEN_BAND[1])), f"price be


def _stair_wpr(interval):
    """WPR (length 200) on the last closed candle of `interval`, or None if there
    k = _stair_klines_n(interval, 200)
    if not k or len(k) < 200:
        return None
    r = k[-200:]
    return calculate_williams_r([float(x[2]) for x in r], [float(x[3]) for x in r


def _stair_above_mid(interval, price):
```

```
    """True/False: price vs the midline of the last 100 bars of `interval` (plus
    k = _stair_klines_n(interval, 100)
    if not k or len(k) < 100 or price is None:
        return None
    k = k[-100:]
    mid = (max(max(float(x[2]) for x in k), price) + min(min(float(x[3]) for x in
    return price > mid


def _stair_short_gate(price, direction="short"):
    """(blocked, note). A short is blocked when price is below the 30m midline AN
    Midline = (highest high + lowest low) / 2 of the last 100 30m bars plus the l
    Fails closed: if either reading can't be made, the short is blocked."""
    try:
        k30, k5 = _stair_klines_n("30m", 100), _stair_klines_n("5m", 200)
        if not k30 or len(k30) < 100 or not k5 or len(k5) < 200 or price is None:
            return True, "no data to check the short filter"
        k = k30[-100:]
        mid = (max(max(float(x[2]) for x in k), price) + min(min(float(x[3]) for
        r5 = k5[-200:]
        w = calculate_williams_r([float(x[2]) for x in r5], [float(x[3]) for x in
        below = price < mid
        hit = (below and w < _STAIR_SHORT_GATE_WPR) if direction == "short" else
        return hit, f"price {'below' if below else 'above'} 30m midline {mid:,.0f
    except Exception as e:
        print(f"[Staircase] short filter error, blocking: {e}")
        return True, "short filter error"


# Blocked only when the 5m got there from a particular colour: (side, 4H, 30m, 5m
_STAIR_BLOCKED_FROM = [
    ("long", "Green", "Orange", "Red", "Orange", "4H Green / 30m Orange / 5m Red→
]


def _stair_is_blocked_from(side, h, m, p, l):
    return any(b[0] == side and (b[1], b[2], b[3], b[4]) == (h, m, p, l) for b in


def _stair_is_blocked(d, h, m, l):
    return any(b[0] == d and all(x in ("*", y) for x, y in zip(b[1:4], (h, m, l))


# Long F (All Green) and Short F (All Orange) are states, not 5m turns: alerted o
_STAIR_EPISODE = {("long", "F"): "Green", ("short", "F"): "Orange"}
_stair_ep_active = None
```

```
def _stair_ep_load():
    global _stair_ep_active
    if _stair_ep_active is None:
        try:
            raw = load_persistent_state("stair_ep_active")
            _stair_ep_active = {tuple(x.split("_", 1)) for x in json.loads(raw)}
        except Exception:
            _stair_ep_active = set()
    return _stair_ep_active


def _stair_episode_hits(cols, mp):
    """Episode setups that should alert now (colours + midlines line up, and not
    active = _stair_ep_load(); hits = []; now_ok = set()
    for (d, L), c in _STAIR_EPISODE.items():
        if not (cols[0] == cols[1] == cols[3] == c) or _stair_is_blocked(d, cols[
            continue
        want, ivs = _STAIR_SETUP_CONDS[(d, L)]
        if all(_stair_above_mid(iv, mp) is (want == "above") for iv in ivs):
            now_ok.add((d, L))
            if (d, L) not in active:
                hits.append((d, L))
    if now_ok != active:
        _stair_ep_active.clear(); _stair_ep_active.update(now_ok)
        save_persistent_state("stair_ep_active", json.dumps(sorted(f"{a}_{b}" for
    return hits


def _stair_detect(h, m, p, l):
    return [(d, L) for (d, L, rh, rm, rp, rl) in _STAIR_RULES
            if (d, L) not in _STAIR_EPISODE and (h, m, p, l) == (rh, rm, rp, rl)


def _stair_suppressed():
    """Same quiet-hours / limit checks as send_signal_prompt. Returns a reason or
    if elevated_silence_until > time.time(): return "elevated silence"
    if stopped_today: return "daily trade limit hit"
    now_full = datetime.now(UK_TZ)
    t = now_full.time()
    if now_full.weekday() in _STAIR_SKIP_DAYS: return "skipped day"
    if now_full.weekday() == 4 and t >= dt_time(19, 0): return "Friday cutoff"
    if daily_losses_block: return "loss-streak block"
    if COOLDOWN_START <= t < COOLDOWN_END: return "US open cooldown"
    if t >= NIGHT_BLOCK_START: return "after curfew"
```

```
    return None


def _stair_send(direction, letter, vals, cols, price):
    global pending_signal
    key = (direction, letter)
    if time.time() - _stair_last_alert.get(key, 0) < STAIR_EXPIRY_SECS:
        return "repeat"
    why = _stair_suppressed()
    if why:
        print(f"[Staircase] Suppressed {direction} {letter} — {why}")
        return f"no alert: {why}"
    _stair_last_alert[key] = time.time()

    h, m, p, l = cols
    hv, mv, lv = vals
    dir_emoji = "🟢" if direction == "long" else "🔴"
    title = f"{direction.upper()} {letter}"
    E = _STAIR_EMOJI
    text = (
        f"{dir_emoji} <b>{title}</b> · BTC {price:,.0f}\n"
        f"{_STAIR_LABELS[0]} {E[h]} · {_STAIR_LABELS[1]} {E[m]} · {_STAIR_LABELS[
        f"<i>{_STAIR_NOTES[key]}</i>\n\n"
        f"<code>{_STAIR_LABELS[0]:<3} {hv:5.1f}</code> {E[h]} {h}\n"
        f"<code>{_STAIR_LABELS[1]:<3} {mv:5.1f}</code> {E[m]} {m}\n"
        f"<code>{_STAIR_LABELS[2]:<3} {lv:5.1f}</code> {E[l]} {l}  (was {E[p]} {p
        f"Price {price:,.1f} · {_STAIR_LABELS[2]} candle closed"
    )

    # Open position: show the alert but no trade buttons
    try:
        open_pos = get_open_position()
    except Exception:
        open_pos = None
    if open_pos:
        send_telegram(text + "\n\n🔒 <b>Orion has an open position — trade blocke
        return "alert sent, trade blocked (open position)"
    # An older staircase prompt still open: the newest setup replaces it (one pen
    ps_old = pending_signal
    if (ps_old and str(ps_old.get("label", "")).startswith("STAIR")
            and time.time() - ps_old["timestamp"] <= ps_old.get("expiry", SIGNAL_
        try:
            if ps_old.get("message_id"):
                edit_prompt_message(ps_old["message_id"], f"🔄 <b>{ps_old['label'
        except Exception:
            pass
```

```
        pending_signal = None
    # Another (non-staircase) signal prompt already waiting: its buttons act on p
    if pending_signal and time.time() - pending_signal["timestamp"] <= pending_si
        send_telegram(text + "\n\n⏸️ Another signal prompt is still open — no but
        return "alert sent, no buttons (another prompt open)"

    label = f"STAIR {direction.title()} {letter}"
    payload = {
        "chat_id": TELEGRAM_CHAT_ID,
        "text": text + f"\n\n⏱️ Expires in {STAIR_TAKE_SECS // 60} minutes.",
        "parse_mode": "HTML",
        "reply_markup": {"inline_keyboard": [[
            {"text": f"✅ Take {title}", "callback_data": f"take_{direction}"},
            {"text": "❌ Skip",          "callback_data": f"skip_{direction}"},
        ]]},
    }
    try:
        resp = requests.post(f"https://api.telegram.org/bot{TELEGRAM_BOT_TOKEN}/s
                             json=payload, timeout=10)
        msg_id = resp.json().get("result", {}).get("message_id")
        pending_signal = {
            "direction":  direction,
            "label":      label,
            "timestamp":  time.time(),
            "message_id": msg_id,
            "expiry":     STAIR_TAKE_SECS,
        }
        print(f"[Staircase] Alert sent: {label}")
        return "alert sent"
    except Exception as e:
        print(f"[Staircase] Telegram error: {e}")
        return "telegram error"


def _stair_check(k5):
    c5 = [float(x[4]) for x in k5]
    cur5, prev5 = _stair_composite(c5), _stair_composite(c5[:-1])
    k4,  _ = _stair_get_klines(_STAIR_TF_H, _STAIR_ZLEN)
    k30, _ = _stair_get_klines(_STAIR_TF_M, _STAIR_ZLEN)
    mp  = _stair_mark_price()
    if cur5 is None or prev5 is None or not k4 or not k30 or mp is None:
        return
    h = _stair_composite([float(x[4]) for x in k4]  + [mp])   # live slow-TF cand
    m = _stair_composite([float(x[4]) for x in k30] + ([mp] if _STAIR_LIVE_MID el
    if h is None or m is None:
        return
```

```
cols = (_stair_colour(h), _stair_colour(m), _stair_colour(prev5), _stair_colo
for direction, letter in _stair_detect(*cols) + _stair_episode_hits(cols, mp)
    conds = _STAIR_SETUP_CONDS.get((direction, letter))
    if conds:
        want, ivs = conds
        if not all(_stair_above_mid(iv, mp) is (want == "above") for iv in iv
            print(f"[Staircase] {direction} {letter} not fired: price must be
            continue
    elif (direction, letter) not in _STAIR_GATE_EXEMPT:
        blk, note = _stair_short_gate(mp, direction)
        if blk:
            print(f"[Staircase] {direction} {letter} blocked: {note}")
            continue
    if direction == "long" and (direction, letter) not in _STAIR_GATE_EXEMPT:
        blk, note = _stair_oversold_gate()
        if blk:
            print(f"[Staircase] long {letter} blocked (oversold): {note}")
            continue
        blk, note = _stair_green30_gate(cols, mp)
        if blk:
            print(f"[Staircase] long {letter} blocked (Green 30m): {note}")
            continue
    status = _stair_send(direction, letter, (h, m, cur5), cols, mp)
    if status != "repeat":
        _stair_history.append({"t": time.time(), "direction": direction, "let
                               "price": mp, "status": status, "cfg": _stair_c
        del _stair_history[:-500]
        try:
            save_persistent_state("stair_history", json.dumps(_stair_history)
        except Exception as e:
            print(f"[Staircase] history save error: {e}")

# All Green long / All Orange short: alert once, on the first closed 5m candl
# right side of the 30m midline AND the 5m WPR is past the line. Fails closed
for colour, side, want_above in (("Green", "long", False), ("Orange", "short"
    ok = False
    if cols[0] == colour and cols[1] == colour and cols[3] == colour:
        above30, w5 = _stair_above_mid("30m", mp), _stair_wpr("5m")
        if above30 is not None and w5 is not None:
            ok = (above30 and w5 > _STAIR_ALLORANGE_WPR) if want_above else (
        if not ok:
            print(f"[Staircase] {side} All {colour} not fired: above 30m midl
    if ok and not _stair_align_prev[colour]:
        blk, note = _stair_oversold_gate() if side == "long" else (False, "")
        if blk:
            print(f"[Staircase] long All {colour} blocked (oversold): {note}"
```

```
            else:
                status = _stair_send(side, f"All {colour}", (h, m, cur5), cols, m
                if status != "repeat":
                    _stair_history.append({"t": time.time(), "direction": side, "
                                           "price": mp, "status": status, "cfg":
                    del _stair_history[:-500]
                    try:
                        save_persistent_state("stair_history", json.dumps(_stair_
                    except Exception as e:
                        print(f"[Staircase] history save error: {e}")
        _stair_align_prev[colour] = ok


def _stair_closest(cols):
    """Setups that are exactly one step from firing (or firing now), with what's
    h, m, p, l = cols
    out = []
    for d, L, rh, rm, rp, rl in _STAIR_RULES:
        if (d, L) in _STAIR_EPISODE: continue
        if _stair_is_blocked(d, rh, rm, rl) or _stair_is_blocked_from(d, rh, rm,
        miss = []
        if h != rh: miss.append(f"{_STAIR_LABELS[0]} must be {rh} (now {h})")
        if m != rm: miss.append(f"{_STAIR_LABELS[1]} must be {rm} (now {m})")
        if (p, l) != (rp, rl): miss.append(f"{_STAIR_LABELS[2]} {rp} → {rl} (now
        if len(miss) == 1:
            out.append({"direction": d, "letter": L, "missing": miss[0]})
    return out


def _stair_snapshot():
    """Live BTC view for the page: slow timeframe includes the forming candle, mi
    global _stair_last_why
    k5,  s5  = _stair_get_klines(_STAIR_TF_L, _STAIR_ZLEN + 3)
    k4,  s4  = _stair_get_klines(_STAIR_TF_H, _STAIR_ZLEN)
    k30, s30 = _stair_get_klines(_STAIR_TF_M, _STAIR_ZLEN)
    mp = _stair_mark_price()
    missing = [name for name, v in ((f"{_STAIR_LABELS[2]} candles", k5), (f"{_STA
    if missing:
        reasons = [str(x) for x, v in ((s5, k5), (s4, k4), (s30, k30)) if not v]
        _stair_last_why = "no " + ", ".join(missing) + (f" ({'; '.join(reasons)})
        return None
    c5 = [float(x[4]) for x in k5]
    cur5, prev5 = _stair_composite(c5), _stair_composite(c5[:-1])
    h = _stair_composite([float(x[4]) for x in k4]  + [mp])
    m = _stair_composite([float(x[4]) for x in k30] + ([mp] if _STAIR_LIVE_MID el
    if None in (cur5, prev5, h, m):
```

```
        _stair_last_why = "not enough candle history yet"
        return None
    _stair_last_why = ""
    cols = (_stair_colour(h), _stair_colour(m), _stair_colour(prev5), _stair_colo
    live5 = _stair_composite(c5 + [mp])                      # the forming fast-T
    now_t = time.time()
    return {
        "price": mp, "h": h, "m": m, "l": cur5, "cols": list(cols),
        "live5": live5, "live5_col": _stair_colour(live5) if live5 is not None el
        "next_close": now_t - (now_t % _STAIR_TF_SECS.get(_STAIR_TF_L, 900)) + _S
        "source": "websocket" if all(x == "ws" for x in (s5, s4, s30)) else "REST
        "params": [_STAIR_LOOKBACK, _STAIR_ZLEN, _STAIR_HIGH, _STAIR_LOW],
        "tfs": list(_STAIR_LABELS),
        "firing": [{"direction": d, "letter": L} for d, L in _stair_detect(*cols)
        "close": _stair_closest(cols),
    }


_stair_console_cache = {"t": 0.0, "near": []}


def stair_console_status() -> dict:
    """For the master console. 'live' = setups the alert code actually accepted a
    that you can still press Take on right now (same gates, same expiry, same but
    'near' = setups exactly one step from firing. Never reads the raw colour matc
    now = time.time()
    live, seen = [], set()
    try:
        ps = pending_signal
        for e in reversed(_stair_history):
            if now - e.get("t", 0) > STAIR_TAKE_SECS:
                continue
            if str(e.get("status", "")) != "alert sent":      # suppressed / no b
                continue
            key = (e.get("direction"), e.get("letter"))
            if key in seen:
                continue
            seen.add(key)
            label = f"STAIR {str(key[0]).title()} {key[1]}"
            if ps and ps.get("label") == label and now - ps["timestamp"] <= ps.ge
                live.append({"direction": key[0], "letter": key[1],
                             "expires_at": ps["timestamp"] + ps.get("expiry", SIG
    except Exception as e:
        print(f"[Console] live setup read error: {e}")
    if now - _stair_console_cache["t"] > 8:
        try:
```

```
            snap = _stair_snapshot()
            _stair_console_cache["near"] = (snap or {}).get("close", []) if snap
        except Exception as e:
            print(f"[Console] near setup read error: {e}")
            _stair_console_cache["near"] = []
        _stair_console_cache["t"] = now
    live_keys = {(x["direction"], x["letter"]) for x in live}
    near = [n for n in _stair_console_cache["near"] if (n["direction"], n["letter
    return {"live": live, "near": near}


def _stair_loop():
    last_bar = None
    while True:
        try:
            time.sleep(5)
            k5, _src = _stair_get_klines(_STAIR_TF_L, _STAIR_ZLEN + 3)
            if not k5:
                if time.time() - _stair_loop.last_warn > 300:
                    print(f"[Staircase] waiting for {_STAIR_LABELS[2]} data: {_sr
                    _stair_loop.last_warn = time.time()
                continue
            bar = k5[-1][0]
            if last_bar is None:      # first read after startup: just remember t
                last_bar = bar
                continue
            if bar == last_bar:
                continue
            last_bar = bar
            _stair_check(k5)
        except Exception as e:
            print(f"[Staircase] loop error: {e}")
            time.sleep(30)


_stair_loop.last_warn = 0.0


def start_staircase_alerts():
    if not STAIR_ENABLED:
        print("[Staircase] Alerts disabled (STAIRCASE_ALERTS=false)")
        return
    try:
        saved = load_persistent_state("stair_history")
        if saved:
            _stair_history.extend(json.loads(saved)[-500:])
```

```
    except Exception as e:
        print(f"[Staircase] history load error: {e}")
    try:
        since = load_persistent_state("stair_since")
        if since:
            _stair_since.update(json.loads(since))
    except Exception as e:
        print(f"[Staircase] since load error: {e}")
    if _stair_cfg() not in _stair_since:
        _stair_since[_stair_cfg()] = time.time()
        _stair_save_since()
    threading.Thread(target=_stair_loop, daemon=True).start()
    print(f"[Staircase] Alert loop started ({'/'.join(_STAIR_LABELS)}, closes of


def answer_callback(callback_id: str, text: str = ""):
    try:
        requests.post(
            f"https://api.telegram.org/bot{TELEGRAM_BOT_TOKEN}/answerCallbackQuer
            json={"callback_query_id": callback_id, "text": text},
            timeout=5
        )
    except Exception as e:
        print(f"[Callback] Error: {e}")


def edit_prompt_message(message_id: int, text: str):
    try:
        requests.post(
            f"https://api.telegram.org/bot{TELEGRAM_BOT_TOKEN}/editMessageText",
            json={
                "chat_id":    TELEGRAM_CHAT_ID,
                "message_id": message_id,
                "text":       text,
                "parse_mode": "HTML"
            },
            timeout=5
        )
    except Exception as e:
        print(f"[Edit Message] Error: {e}")


def send_bias_invalidation_alert(label: str, direction: str, signal_key: str):
    global pending_bias_reset
    emoji     = "🟢" if direction == "long" else "🔴"
    new_bias  = "bullish" if direction == "long" else "bearish"
```

```
    text = (
        f"⚠️ <b>HTF regime change detected</b>\n\n"
        f"Structure break: {label}\n"
        f"Direction: {emoji} {direction.upper()}\n\n"
        f"Your bias is currently {daily_bias.upper()}.\n"
        f"This structure break suggests {new_bias.upper()} may be taking over.\n\
        f"Review and reset your bias?"
    )
    payload = {
        "chat_id": TELEGRAM_CHAT_ID,
        "text": text,
        "parse_mode": "HTML",
        "reply_markup": {
            "inline_keyboard": [[
                {"text": "🔄 Reset to Bullish",  "callback_data": f"reset_bias_bu
                {"text": "🔄 Reset to Bearish",  "callback_data": f"reset_bias_be
                {"text": "❌ Ignore",            "callback_data": f"ignore_reset_
            ]]
        }
    }
    try:
        resp   = requests.post(
            f"https://api.telegram.org/bot{TELEGRAM_BOT_TOKEN}/sendMessage",
            json=payload, timeout=10
        )
        msg_id = resp.json().get("result", {}).get("message_id")
        pending_bias_reset = {
            "signal_key": signal_key,
            "message_id": msg_id,
            "timestamp": time.time()
        }
    except Exception as e:
        print(f"[HTF Alert] Error: {e}")


def start_prompt_silence(reason: str = ""):
    """Silence all signal prompts for 10 minutes and notify when lifted."""
    global elevated_silence_until
    elevated_silence_until = time.time() + 600
    def _lift():
        time.sleep(600)
        send_telegram(f"🔔 <b>Prompts active again.</b>{(' ' + reason) if reason
    threading.Thread(target=_lift, daemon=True).start()


_hl_info_cache = None
```

```
_hl_info_lock = threading.Lock()


def hl_get_info():
    """One shared Info client. Building a new one makes the SDK re-download the
    exchange metadata (extra round trips) every single time, which added seconds
    to every client action."""
    global _hl_info_cache
    if _hl_info_cache is None:
        with _hl_info_lock:
            if _hl_info_cache is None:
                _hl_info_cache = HLInfo(HL_BASE_URL, skip_ws=True)
    return _hl_info_cache


def hl_get_mark_price() -> float:
    info = hl_get_info()
    all_mids = info.all_mids()
    price = float(all_mids.get(HL_COIN, 0))
    print(f"[HL Mirror] Mark price for {HL_COIN}: {price}")
    return price


_hl_exchange_cache = {}


def client_get_exchange(c):
    """Cached per client (same reason as hl_get_info: construction does extra net
    key = (c.id, c.address, c.private_key)
    ex = _hl_exchange_cache.get(key)
    if ex is None:
        account = EthAccount.from_key(c.private_key)
        ex = HLExchange(account, HL_BASE_URL, account_address=c.address)
        _hl_exchange_cache[key] = ex
    return ex


def _client_order_snapshot(c, info, position_dir: str, size: float, entry_px: flo
    """What this client has resting on Hyperliquid right now, split into the
    stop (losing side of the current price) and the take-profits (winning
    side, nearest to entry = TP1), plus any health warnings. Shared by the
    console rows and the background watcher so both always agree."""
    is_long = position_dir == "LONG"
    sl = None
    tps = []
    for o in info.open_orders(c.address):
```

```
            if o.get("coin") != HL_COIN:
                continue
            px = float(o.get("triggerPx") or o.get("limitPx") or 0)
            sz = float(o.get("sz") or 0)
            if px <= 0:
                continue
            is_sl = (px < mark) if is_long else (px > mark)
            if is_sl:
                sl = {"price": px, "size": sz} if sl is None else {"price": px, "size
            else:
                tps.append({"price": px, "size": sz})
        tps.sort(key=lambda t: abs(t["price"] - entry_px))
        snap = {"sl": sl, "tp1": None, "tp2": None, "tp1_hit": False, "warnings": []}
        if len(tps) >= 2:
            snap["tp1"], snap["tp2"] = tps[0], tps[-1]
        elif len(tps) == 1:
            snap["tp2"] = tps[0]          # the mirror always places two, so a lone T
            snap["tp1_hit"] = True
        w = snap["warnings"]
        if sl is None:
            w.append("No stop-loss")
        elif size > 0 and sl["size"] < size * 0.98:
            w.append("Stop covers only part of the position")
        if not tps:
            w.append("No take-profit")
        master_dir = (current_trade_entry.get("direction") or "").upper()
        if master_dir and master_dir != position_dir:
            w.append("Opposite to master")
        return snap


def _client_period_pnl(c, trips: list) -> dict:
    """Net P&L of closed trades for today / this week (UK time) from round trips.
    now = datetime.now(UK_TZ)
    day = now.date()
    wk  = now.isocalendar()[:2]
    today = week = 0.0
    for t in trips:
        if not t.get("close_ms"):
            continue
        d = datetime.fromtimestamp(t["close_ms"] / 1000, UK_TZ)
        net = t["gross"] - t["fees"]
        if d.date() == day:
            today += net
        if d.isocalendar()[:2] == wk:
            week += net
```

```
    return {"today_pnl": round(today, 2), "week_pnl": round(week, 2)}


def _fetch_one_client_position(c, results: dict):
    """Worker for get_all_client_positions() — one client's HL lookup,
    writing into a shared dict so every client can be checked in parallel
    rather than one at a time."""
    try:
        info  = hl_get_info()
        state = info.user_state(c.address)
        position = None
        for p in state.get("assetPositions", []):
            pos_data = p.get("position", {})
            szi = float(pos_data.get("szi", 0))
            if szi != 0:
                position = {
                    "direction": "LONG" if szi > 0 else "SHORT",
                    "size": abs(szi),
                    "unrealized_pnl": float(pos_data.get("unrealizedPnl", 0) or 0
                    "notional": abs(float(pos_data.get("positionValue", 0) or 0))
                    "margin": float(pos_data.get("marginUsed", 0) or 0),
                    "entry": float(pos_data.get("entryPx", 0) or 0),
                }
                break
        if position:
            try:
                snap = _client_order_snapshot(c, info, position["direction"], pos
                                              position["entry"], hl_get_mark_pric
                position.update({"sl": snap["sl"], "tp1": snap["tp1"], "tp2": sna
                                 "tp1_hit": snap["tp1_hit"], "warnings": snap["wa
            except Exception as e:
                print(f"[Client Positions] Order snapshot failed for {c.label()}:
        # What the NEXT mirrored trade will use, from the same inputs client_calc
        # margin = balance x size%, notional = margin x leverage.
        bal = float(state.get("marginSummary", {}).get("accountValue", 0) or 0)
        if bal == 0:
            bal = float(state.get("crossMarginSummary", {}).get("accountValue", 0
        plan_margin = bal * (c.size_pct / 100)
        # Last closed trade, net of fees. Never lets a fills failure break the ro
        last_trade = None
        period = {"today_pnl": 0.0, "week_pnl": 0.0}
        try:
            trips = hl_round_trips(hl_client_fills(c))
            period = _client_period_pnl(c, trips)
            if trips:
                t = trips[-1]
```

```
                last_trade = {"pnl": round(t["gross"] - t["fees"], 2), "close_ms"
        except Exception as e:
            print(f"[Client Positions] Last trade lookup failed for {c.label()}:
        results[c.id] = {"id": c.id, "name": c.name, "status": "ok", "position":
                         "mirroring_enabled": is_client_mirroring_enabled(c.id),
                         "balance": bal, "size_pct": c.size_pct, "leverage": c.le
                         "next_margin": plan_margin, "next_notional": plan_margin
                         "last_trade": last_trade, **period}
    except Exception as e:
        results[c.id] = {"id": c.id, "name": c.name, "status": "error", "mirrorin
        print(f"[Client Positions] Failed for {c.label()}: {e}")


# ── Client watcher: fill alerts + drift warnings ──────────────────────────
CLIENT_WATCH_EVERY_SECS = 30
_client_watch: dict = {}   # client id -> last-seen state


def _label_exit_fills(fills: list, last_orders: dict) -> list:
    """Groups closing fills by which resting order they matched (within 0.05%)."""
    refs = []
    if last_orders.get("sl"):  refs.append(("SL",  last_orders["sl"]["price"]))
    if last_orders.get("tp1"): refs.append(("TP1", last_orders["tp1"]["price"]))
    if last_orders.get("tp2"): refs.append(("TP2", last_orders["tp2"]["price"]))
    out = {}
    for f in fills:
        px, sz = float(f.get("px", 0) or 0), float(f.get("sz", 0) or 0)
        if px <= 0 or sz <= 0:
            continue
        kind = "Market/manual"
        for k, rp in refs:
            if rp and abs(px - rp) / rp <= 0.0005:
                kind = k
                break
        g = out.setdefault(kind, {"sz": 0.0, "notional": 0.0})
        g["sz"] += sz
        g["notional"] += px * sz
    return [(k, g["sz"], g["notional"] / g["sz"]) for k, g in out.items()]


def _client_closed_alert(c, st: dict):
    """Position went flat: say which order closed it, at what price, and the net
    try:
        info = hl_get_info()
        start = max(0, int(st.get("opened_ms", 0)) - 120000)
        fills = [f for f in info.user_fills_by_time(c.address, start) if f.get("c
```

```
            if not fills:
                send_telegram(f"🏁 <b>{c.label()}</b> position closed (no fills found
                return
            closing = [f for f in fills if "close" in str(f.get("dir", "")).lower() o
            gross = sum(float(f.get("closedPnl", 0) or 0) for f in fills)
            fees  = sum(float(f.get("fee", 0) or 0) for f in fills)
            net = gross - fees
            lines = []
            for kind, sz, px in _label_exit_fills(closing, st.get("orders") or {}):
                lines.append(f"  {kind}: {sz:.4f} BTC @ ${px:,.0f}")
            sign = "+" if net >= 0 else "-"
            send_telegram(
                f"🏁 <b>{c.label()} closed — {st.get('dir', '')} {sign}${abs(net):,.2
                + ("\n".join(lines) if lines else "  exit details unavailable")
            )
        except Exception as e:
            print(f"[Watch] closed-alert error for {c.label()}: {e}")


    def client_watch_once():
        clients = [c for c in CLIENTS if c.id != MASTER_CLIENT_ID and c.is_configured
        info = hl_get_info()
        for c in clients:
            try:
                state = info.user_state(c.address)
                pos = None
                for p in state.get("assetPositions", []):
                    pd = p.get("position", {})
                    if pd.get("coin") == HL_COIN and float(pd.get("szi", 0)) != 0:
                        pos = pd
                        break
                st = _client_watch.setdefault(c.id, {"open": False, "streaks": {}, "a
                if pos:
                    szi = float(pos["szi"])
                    size = abs(szi)
                    direction = "LONG" if szi > 0 else "SHORT"
                    entry = float(pos.get("entryPx", 0) or 0)
                    mark = hl_get_mark_price()
                    snap = _client_order_snapshot(c, info, direction, size, entry, ma
                    if not st["open"]:
                        st.update({"open": True, "opened_ms": int(time.time() * 1000)
                    elif size < st.get("size", size) * 0.95 and not st.get("partial_s
                        st["partial_sent"] = True
                        send_telegram(f"🎯 <b>{c.label()}</b> partial close — {st.get
                                      + (" (TP1 filled)" if snap["tp1_hit"] else ""))
                    st.update({"dir": direction, "size": size,
```

```
                        "orders": {"sl": snap["sl"], "tp1": snap["tp1"], "tp2"
                # Drift warnings: must persist for two checks before alerting, so
                # between the fill and the stop/TP orders being placed never fals
                current = set(snap["warnings"])
                for w in current:
                    st["streaks"][w] = st["streaks"].get(w, 0) + 1
                    if st["streaks"][w] >= 2 and w not in st["alerted"]:
                        st["alerted"].add(w)
                        send_telegram(f"⚠️ <b>{c.label()}</b> — {w} on an open {c
                for w in list(st["streaks"]):
                    if w not in current:
                        st["streaks"].pop(w, None)
                        st["alerted"].discard(w)
            else:
                if st["open"]:
                    st["open"] = False
                    st["partial_sent"] = False
                    _client_closed_alert(c, dict(st))
        except Exception as e:
            print(f"[Watch] {c.label()} check failed: {e}")


def client_watch_loop():
    time.sleep(60)   # let startup settle
    while True:
        try:
            client_watch_once()
        except Exception as e:
            print(f"[Watch] loop error: {e}")
        time.sleep(CLIENT_WATCH_EVERY_SECS)


def get_all_client_positions() -> list:
    """Checks every configured mirror client's current position on
    Hyperliquid, all in parallel, for the master dashboard's health-check
    panel — 'who's in a trade right now and who isn't,' at a glance.
    """
    clients = [c for c in CLIENTS if c.id != MASTER_CLIENT_ID and c.is_configured
    results = {}
    threads = [threading.Thread(target=_fetch_one_client_position, args=(c, resul
    for t in threads: t.start()
    for t in threads: t.join(timeout=5)
    # Preserve CLIENTS order rather than whatever order threads finished in
    return [results[c.id] for c in clients if c.id in results]
```

```
def client_get_balance(c) -> float:
    info = hl_get_info()
    try:
        state = info.user_state(c.address)
    except HLClientError as e:
        if e.status_code == 422:
            # Same never-funded-wallet case handled on the dashboard side —
            # a wallet with zero on-chain history errors instead of
            # returning an empty state. $0 balance, size calc below will
            # correctly skip mirroring rather than attempting a trade.
            print(f"[{c.label()}] No account state yet (never funded) — treating
            return 0.0
        raise
    print(f"[{c.label()}] Raw user state: {state}")
    margin_val = float(state.get("marginSummary", {}).get("accountValue", 0) or 0
    cross_val  = float(state.get("crossMarginSummary", {}).get("accountValue", 0)
    if margin_val == 0 and cross_val != 0:
        print(f"[{c.label()}] marginSummary was 0 but crossMarginSummary has {cro
        return cross_val
    return margin_val


_client_last_mark = {}      # client id -> mark price read during the size calcul
_client_lev_set_at = {}     # (client id, leverage) -> when leverage was last set


def client_calculate_size(c) -> float:
    try:
        from concurrent.futures import ThreadPoolExecutor
        with ThreadPoolExecutor(max_workers=2) as _ex:
            _fb = _ex.submit(client_get_balance, c)
            _fm = _ex.submit(hl_get_mark_price)
            balance    = _fb.result()
            mark_price = _fm.result()
        _client_last_mark[c.id] = mark_price
        print(f"[{c.label()}] Balance: {balance}, Mark: {mark_price}")
        if mark_price <= 0 or balance <= 0:
            print(f"[{c.label()}] Size calc failed — balance={balance} mark={mark
            return 0.0
        margin   = balance * (c.size_pct / 100)
        notional = margin * c.leverage
        size_btc = notional / mark_price
        print(f"[{c.label()}] Size: {size_btc} BTC (margin={margin}, notional={no
        return round(size_btc, 4)
    except Exception as e:
        print(f"[{c.label()}] Size calc error: {e}")
```

```
        send_telegram(f"⚠️ <b>{c.label()} size calc error</b>\n\n{e}")
        return 0.0


def client_mirror_open(c, direction: str, stop_pct: float = None, tp_pct: float =
    if not c.is_configured():
        print(f"[{c.label()}] Not configured — skipping")
        return
    try:
        exchange = client_get_exchange(c)
        size     = client_calculate_size(c)

        if size <= 0:
            send_telegram(f"⚠️ {c.label()}: could not calculate size — skipping")
            return

        is_buy = direction == "long"

        # Leverage doesn't change between trades: set it once an hour, not on eve
        _lk = (c.id, c.leverage)
        if time.time() - _client_lev_set_at.get(_lk, 0) > 3600:
            exchange.update_leverage(c.leverage, HL_COIN, is_cross=True)
            _client_lev_set_at[_lk] = time.time()

        # Reuse the price read a moment ago for the size calculation.
        _slip_ref = _client_last_mark.get(c.id)
        if not _slip_ref:
            try:
                _slip_ref = hl_get_mark_price()
            except Exception:
                _slip_ref = None
        result = exchange.market_open(HL_COIN, is_buy, size, None, 0.05)

        if result.get("status") == "ok":
            statuses = result.get("response", {}).get("data", {}).get("statuses",
            avg_px   = float(statuses[0].get("filled", {}).get("avgPx", 0)) if st
            log_exec_slippage(f"client_{c.id}", "client", direction, _slip_ref, a
            emoji    = "🟢" if is_buy else "🔴"
            send_telegram(
                f"✅ <b>{c.label()} Mirror opened</b>\n\n"
                f"Direction: {emoji} {direction.upper()}\n"
                f"Size:      {size} BTC\n"
                f"Entry:     ${avg_px:,.2f}"
            )

            # ── Compute this client's OWN SL/TP prices from THEIR OWN fill ──
```

```
# Binance and Hyperliquid prices genuinely diverge (a normal
# cross-exchange basis gap, sometimes tens of dollars during
# volatility). The master's SL/TP were originally calculated as
# a % distance from the MASTER's Binance entry — reusing those
# same absolute dollar figures here would mean a client's real
# resting stop fires based on how far HL's price is from
# BINANCE's entry level, not their own. Recalculating the same
# percentage against this client's actual avg_px is what makes
# "the same % gap stop" actually true for them specifically,
# regardless of any gap between the two exchanges' prices.
sl_price = tp1_price = tp2_price = None
if avg_px > 0:
    if direction == "long":
        if stop_pct: sl_price  = round(avg_px * (1 - stop_pct / 100))
        if tp_pct:   tp1_price = round(avg_px * (1 + tp_pct   / 100))
        if tp2_pct:  tp2_price = round(avg_px * (1 + tp2_pct  / 100))
    else:
        if stop_pct: sl_price  = round(avg_px * (1 + stop_pct / 100))
        if tp_pct:   tp1_price = round(avg_px * (1 - tp_pct   / 100))
        if tp2_pct:  tp2_price = round(avg_px * (1 - tp2_pct  / 100))

# ── Place REAL trigger orders on Hyperliquid for SL/TP1/TP2 ──
# Every client gets real resting orders, not just a Telegram
# message — otherwise the position is naked between polling
# cycles, protected by nothing but this bot noticing and
# manually closing it.
close_is_buy = not is_buy  # closing side is opposite of entry
real_sl_line, real_tp1_line, real_tp2_line = "", "", ""

# ── Split TP1/TP2 size the same way the master does ──
# Previously both TP1 and TP2 were placed reduce_only for the
# FULL position size, so whichever one filled first (almost
# always TP1) closed the client's entire position instead of
# TP1_CLOSE_PCT of it — the master runs the remainder to TP2,
# but clients had nothing left to run. TP1 now only closes its
# share; TP2 gets exactly what's left over, so the two always
# add back up to the full size regardless of rounding.
tp1_size = round(size * (TP1_CLOSE_PCT / 100), 4)
tp2_size = round(size - tp1_size, 4)
if tp2_full:          # TP1 already filled on the master: TP2 carries
    tp2_size = round(size, 4)
# SL stays at the full size — reduce_only caps to whatever
# position actually remains, so this is correct whether SL
# fires before TP1 (closes everything) or after (closes just
# the TP2 leg that's still open).
if sl_price:
```

```
        try:
            sl_order_type = HLOrderType(trigger=HLTriggerOrderType(
                triggerPx=round(sl_price), isMarket=True, tpsl="sl"
            ))
            sl_result = exchange.order(HL_COIN, close_is_buy, size, round
            if sl_result.get("status") == "ok":
                real_sl_line = f"🛑 SL:  ${sl_price:,.2f} (real order pla
            else:
                real_sl_line = f"🛑 SL:  ${sl_price:,.2f} ⚠️ FAILED TO PI
                print(f"[{c.label()}] SL order failed: {sl_result}")
        except Exception as e:
            real_sl_line = f"🛑 SL:  ${sl_price:,.2f} ⚠️ FAILED TO PLACE
            print(f"[{c.label()}] SL order error: {e}")

    if tp1_price:
        try:
            tp1_order_type = HLOrderType(trigger=HLTriggerOrderType(
                triggerPx=round(tp1_price), isMarket=True, tpsl="tp"
            ))
            tp1_result = exchange.order(HL_COIN, close_is_buy, tp1_size,
            if tp1_result.get("status") == "ok":
                real_tp1_line = f"🎯 TP1: ${tp1_price:,.2f} ({TP1_CLOSE_P
            else:
                real_tp1_line = f"🎯 TP1: ${tp1_price:,.2f} ⚠️ FAILED TO
                print(f"[{c.label()}] TP1 order failed: {tp1_result}")
        except Exception as e:
            real_tp1_line = f"🎯 TP1: ${tp1_price:,.2f} ⚠️ FAILED TO PLAC
            print(f"[{c.label()}] TP1 order error: {e}")

    if tp2_price:
        try:
            tp2_order_type = HLOrderType(trigger=HLTriggerOrderType(
                triggerPx=round(tp2_price), isMarket=True, tpsl="tp"
            ))
            tp2_result = exchange.order(HL_COIN, close_is_buy, tp2_size,
            if tp2_result.get("status") == "ok":
                real_tp2_line = f"🎯 TP2: ${tp2_price:,.2f} ({100 - TP1_C
            else:
                real_tp2_line = f"🎯 TP2: ${tp2_price:,.2f} ⚠️ FAILED TO
                print(f"[{c.label()}] TP2 order failed: {tp2_result}")
        except Exception as e:
            real_tp2_line = f"🎯 TP2: ${tp2_price:,.2f} ⚠️ FAILED TO PLAC
            print(f"[{c.label()}] TP2 order error: {e}")

    # Read back what is ACTUALLY resting on Hyperliquid so a wrong
    # TP split can be seen straight away instead of after the fact.
```

```
            try:
                time.sleep(1.5)
                _oo = [o for o in hl_get_info().open_orders(c.address) if o.get("
                _lines = []
                for o in sorted(_oo, key=lambda o: float(o.get("triggerPx") or o.
                    _px = float(o.get("triggerPx") or o.get("limitPx") or 0)
                    _lines.append(f"  ${_px:,.0f} x {o.get('sz')} BTC")
                send_telegram(f"🔎 <b>{c.label()} resting orders</b> (position {s
                              + ("\n".join(_lines) if _lines else "  none found")
            except Exception as _e:
                print(f"[{c.label()}] order read-back skipped: {_e}")

            # This was previously sent to the client's own Telegram via
            # notify_client() — clients no longer have Telegram at all, so
            # that channel silently went nowhere. This SL/TP1/TP2 placement
            # detail (including any "FAILED TO PLACE" warning) now goes to
            # the master instead, since you're the one who needs to know if
            # a client's real protective order didn't actually get placed.
            if "FAILED TO PLACE" in real_sl_line + real_tp1_line + real_tp2_line:
                send_telegram(
                    f"⚠️ <b>{c.label()} — one or more orders failed to place</b>\
                    f"{real_sl_line}"
                    f"{real_tp1_line}"
                    f"{real_tp2_line}"
                )
        else:
            send_telegram(f"⚠️ <b>{c.label()} Mirror failed to open</b>\n\n{resul
            print(f"[{c.label()}] Open failed: {result}")

    except Exception as e:
        send_telegram(f"⚠️ <b>{c.label()} Mirror error (open)</b>\n\n{e}")
        print(f"[{c.label()}] Open error: {e}")


def client_mirror_close(c, max_retries: int = 4, retry_delay: int = 15):
    if not c.is_configured():
        return
    try:
        info  = hl_get_info()
        state = info.user_state(c.address)

        # Find the open BTC position
        position  = None
        positions = state.get("assetPositions", [])
        for p in positions:
            pos_data = p.get("position", {})
```

```
    if pos_data.get("coin") == HL_COIN:
        szi = float(pos_data.get("szi", 0))
        if szi != 0:
            position = pos_data
            break

if not position:
    send_telegram(f"⚠️ <b>{c.label()} Mirror close</b> — no open position
    return

close_qty = abs(float(position["szi"]))
szi       = float(position["szi"])
is_buy    = szi < 0   # short position → close with buy, long → close wit
entry_px  = float(position.get("entryPx", 0))
print(f"[{c.label()}] Closing {close_qty} BTC — {'BUY' if is_buy else 'SE

exchange   = client_get_exchange(c)

# ── Cancel any lingering SL/TP trigger orders first ──────────────
# Otherwise a stale reduce-only trigger order from this position
# could unexpectedly interact with whatever opens next.
try:
    open_orders = info.open_orders(c.address)
    for o in open_orders:
        if o.get("coin") == HL_COIN:
            exchange.cancel(HL_COIN, o.get("oid"))
except Exception as e:
    print(f"[{c.label()}] Order cancel error (non-fatal): {e}")

avg_px     = 0
closed_pnl = 0
still_open = True

for attempt in range(1, max_retries + 1):
    # Use plain market_open in opposite direction — HL one-way netting cl
    # reduce_only removed as it may be rejected in agent setups
    result = exchange.market_open(HL_COIN, is_buy, close_qty, None, 0.05)
    print(f"[{c.label()}] Close attempt {attempt}/{max_retries} result: {

    if result.get("status") == "ok":
        statuses   = result.get("response", {}).get("data", {}).get("stat
        avg_px     = float(statuses[0].get("filled", {}).get("avgPx", 0))
        closed_pnl = float(statuses[0].get("filled", {}).get("closedPnl",

    # Verify with a fresh info call
    time.sleep(3)
```

```
        info2      = hl_get_info()
        state2     = info2.user_state(c.address)
        positions2 = state2.get("assetPositions", [])
        still_open = any(
            p.get("position", {}).get("coin") == HL_COIN
            and float(p.get("position", {}).get("szi", 0)) != 0
            for p in positions2
        )

        if not still_open:
            # Manual PnL fallback — HL's closedPnl field is unreliable/often
            if closed_pnl == 0 and avg_px > 0 and entry_px > 0:
                if is_buy:    # was short, profit if exit < entry
                    closed_pnl = (entry_px - avg_px) * close_qty
                else:         # was long, profit if exit > entry
                    closed_pnl = (avg_px - entry_px) * close_qty
                print(f"[{c.label()}] closedPnl was 0/missing — calculated ma
            break  # confirmed closed — stop retrying

        if attempt < max_retries:
            send_telegram(
                f"⚠️ <b>{c.label()} Mirror close attempt {attempt} failed — r
                f"{c.label()} position still open. Auto-retry {attempt}/{max_
            )
            # Recalculate qty in case it partially filled
            close_qty = abs(float(position_data.get("szi", close_qty))) if (p
                (p.get("position", {}) for p in positions2 if p.get("position
            )) else close_qty
            time.sleep(retry_delay)

    if still_open:
        send_telegram(
            f"🚨 <b>{c.label()} Mirror close FAILED after {max_retries} attem
            f"{c.label()}'s HL position is still open with no SL/TP watching
            f"Close it manually NOW at app.hyperliquid.xyz\n\n"
            f"{c.label()} address: <code>{c.address}</code>"
        )
        return

    emoji       = "✅" if closed_pnl >= 0 else "❌"
    close_label = "Target Hit" if closed_pnl > 0 else "Stop Hit" if closed_pn
    c.trades_today += 1
    if closed_pnl > 0:
        c.wins_today += 1
    elif closed_pnl < 0:
        c.losses_today += 1
```

```
        send_telegram(
            f"{emoji} <b>{c.label()} Mirror closed</b>\n\n"
            f"Exit:  ${avg_px:,.2f}\n"
            f"PnL:   ${closed_pnl:+.2f}"
        )
    except Exception as e:
        send_telegram(f"⚠️ <b>{c.label()} Mirror error (close)</b>\n\n{e}")
        print(f"[{c.label()}] Close error: {e}")


def mirror_open_all(direction: str, stop_pct: float = None, tp_pct: float = None,
    """Mirror a fresh entry to every configured client, each in its own thread.
    Takes the SL/TP as percentages, not absolute prices — see
    client_mirror_open() for why that distinction actually matters.
    Skips any client temporarily disabled via the dashboard's mirroring
    toggle — see is_client_mirroring_enabled().
    """
    for c in CLIENTS:
        if not c.is_configured():
            # Never silent: a client who's in the list but can't trade gets
            # flagged. Skips the master slot and fully-empty placeholder
            # slots (#1/#2 are always instantiated) to avoid alert noise.
            if c.id != MASTER_CLIENT_ID and (c.private_key or c.address):
                missing = []
                if not c.private_key:
                    missing.append("private key")
                if not c.address:
                    missing.append("wallet address")
                if not HL_SDK_AVAILABLE:
                    missing.append("HL SDK")
                reason = ", ".join(missing) or "unknown"
                print(f"[{c.label()}] NOT CONFIGURED — skipped this entry (missin
                send_telegram(
                    f"⚠️ <b>{c.label()} skipped — not configured</b>\n\n"
                    f"Missing: {reason}\n"
                    f"No mirror trade was opened for this client."
                )
            continue
        if not is_client_mirroring_enabled(c.id):
            print(f"[{c.label()}] Mirroring disabled — skipping this entry")
            continue
        threading.Thread(target=client_mirror_open, args=(c, direction, stop_pct,


def mirror_open_one(client_id: str):
    """/mirroropen <id>: bring ONE client into the master's already-open trade,
```

```
    matching the master's current SL/TP percentages (from the master's entry)."""
    c = next((x for x in CLIENTS if str(x.id) == str(client_id) and x.id != MASTE
    if not c:
        send_telegram(f"❌ No mirror client with id {client_id}.")
        return
    if not c.is_configured():
        send_telegram(f"❌ {c.label()} is not configured.")
        return
    direction = (current_trade_entry.get("direction") or "").lower()
    entry = float(current_trade_entry.get("price") or 0)
    if direction not in ("long", "short") or entry <= 0:
        send_telegram("❌ The master has no open trade to mirror.")
        return
    try:
        state = hl_get_info().user_state(c.address)
        for p in state.get("assetPositions", []):
            pd = p.get("position", {})
            if pd.get("coin") == HL_COIN and float(pd.get("szi", 0)) != 0:
                send_telegram(f"ℹ️ {c.label()} already has an open position — not
                return
    except Exception as e:
        send_telegram(f"❌ Could not check {c.label()}'s position: {e}")
        return

    def _tp_pct(level):
        return abs(level - entry) / entry * 100 if level else None

    sl = current_trade_entry.get("sl_price")
    if sl:
        # + = stop on the losing side of entry, - = locked-in/break-even side
        stop_pct = ((entry - sl) if direction == "long" else (sl - entry)) / entr
        if abs(stop_pct) < 1e-9:
            stop_pct = 0.0001
    else:
        stop_pct = None
    tp1_filled = bool(current_trade_entry.get("tp1_filled"))
    tp_pct  = None if tp1_filled else _tp_pct(current_trade_entry.get("tp1_price"
    tp2_pct = _tp_pct(current_trade_entry.get("tp2_price"))
    send_telegram(f"🔁 <b>/mirroropen</b> — bringing {c.label()} into the open {d
                  + (" (TP1 already hit — TP2 only)" if tp1_filled else ""))
    threading.Thread(target=client_mirror_open,
                     args=(c, direction, stop_pct, tp_pct, tp2_pct, tp1_filled),


def client_has_working_sl(c) -> bool:
    """Checks whether this client currently has a real resting stop order
```

```
on Hyperliquid — identified by which side of their entry price its
trigger sits on (below entry for a long, above for a short), rather
than assuming a specific field name in Hyperliquid's response, since
that directional relationship is the one thing guaranteed true
regardless of exactly how their API labels order types.

Used as the smarter alternative to force-closing every client the
instant the master's own Binance stop fires: only step in for a
client whose own protection genuinely isn't there, rather than
overriding everyone's independent Hyperliquid stop with a close timed
to Binance's price action.
"""
if not c.is_configured():
    return True  # not a real client — nothing to force-close either way
try:
    info  = hl_get_info()
    state = info.user_state(c.address)
    position = None
    for p in state.get("assetPositions", []):
        pos_data = p.get("position", {})
        if pos_data.get("coin") == HL_COIN and float(pos_data.get("szi", 0))
            position = pos_data
            break
    if not position:
        return True  # no open position at all — nothing to protect or force-

    szi      = float(position["szi"])
    entry_px = float(position.get("entryPx", 0))
    is_long  = szi > 0
    if entry_px <= 0:
        return True  # can't determine direction reliably — don't force-close

    mark = hl_get_mark_price()   # stop = losing side of current price (stays
    open_orders = info.open_orders(c.address)
    for o in open_orders:
        if o.get("coin") != HL_COIN:
            continue
        trigger_px = float(o.get("triggerPx") or o.get("limitPx") or 0)
        if trigger_px <= 0:
            continue
        is_sl = (trigger_px < mark) if is_long else (trigger_px > mark)
        if is_sl:
            return True
    return False
except Exception as e:
    print(f"[{c.label()}] Could not verify SL health — assuming protected to
```

```
        return True  # fail safe toward NOT force-closing on an uncertain check


def mirror_close_unhealthy(max_retries: int = 4, retry_delay: int = 15):
    """Safety-net replacement for blindly force-closing every client
    whenever the master's own SL/TP fires on Binance. Only closes a client
    if they genuinely have no working resting stop right now — anyone with
    a real order in place is left to close independently, at their own
    Hyperliquid price, on their own timing.
    """
    for c in CLIENTS:
        if c.is_configured() and not client_has_working_sl(c):
            print(f"[{c.label()}] No working SL detected on Hyperliquid — force-c
            threading.Thread(target=client_mirror_close, args=(c, max_retries, re


def client_adjust_sl(c, sl_pct: float):
    """Moves this ONE client's real resting SL order to the same percentage
    distance from THEIR OWN entry price on Hyperliquid — called for every
    configured client whenever /sl changes the master's stop, so a manual
    SL move stays in sync across every mirrored account instead of leaving
    clients sitting on their original opening-time stop forever.
    """
    if not c.is_configured():
        return
    try:
        info  = hl_get_info()
        # The three reads are independent, so run them together instead of one af
        from concurrent.futures import ThreadPoolExecutor
        with ThreadPoolExecutor(max_workers=3) as _ex:
            _f_state  = _ex.submit(info.user_state, c.address)
            _f_orders = _ex.submit(info.open_orders, c.address)
            _f_mids   = _ex.submit(info.all_mids)
            state       = _f_state.result()
            open_orders = _f_orders.result()
            mids        = _f_mids.result()
        position = None
        for p in state.get("assetPositions", []):
            pos_data = p.get("position", {})
            if pos_data.get("coin") == HL_COIN and float(pos_data.get("szi", 0))
                position = pos_data
                break
        if not position:
            print(f"[{c.label()}] SL sync skipped — no open position on Hyperliqu
            return
```

```
        szi      = float(position["szi"])
        entry_px = float(position.get("entryPx", 0))
        if entry_px <= 0:
            return
        is_long = szi > 0

        new_sl_price = round(entry_px * (1 - sl_pct / 100)) if is_long else round

        exchange = client_get_exchange(c)
        mark = float(mids.get(HL_COIN, 0) or 0)

        # The old stop is the order on the losing side of the CURRENT PRICE (belo
        # for a long, above it for a short); TPs sit on the other side. Judging b
        # rather than by entry keeps this right after a stop has moved to break-e
        # into profit. Collected now, cancelled only AFTER the new stop is in pla
        # the client is never without a stop (and keeps the old one if placing fa
        old_sl_oids = []
        for o in open_orders:
            if o.get("coin") != HL_COIN:
                continue
            trigger_px = float(o.get("triggerPx") or o.get("limitPx") or 0)
            if trigger_px <= 0 or mark <= 0:
                continue
            if (trigger_px < mark) if is_long else (trigger_px > mark):
                old_sl_oids.append(o.get("oid"))

        close_is_buy = not is_long
        size = abs(szi)
        sl_order_type = HLOrderType(trigger=HLTriggerOrderType(
            triggerPx=new_sl_price, isMarket=True, tpsl="sl"
        ))
        result = exchange.order(HL_COIN, close_is_buy, size, new_sl_price, sl_ord
        if result.get("status") == "ok":
            for oid in old_sl_oids:
                try:
                    exchange.cancel(HL_COIN, oid)
                except Exception as ce:
                    print(f"[{c.label()}] Old SL cancel error: {ce}")
            print(f"[{c.label()}] SL synced to ${new_sl_price:,.2f} ({sl_pct:.2f}
        else:
            send_telegram(f"⚠️ {c.label()}: failed to sync new SL — {result} (the
    except Exception as e:
        send_telegram(f"⚠️ {c.label()}: SL sync error — {e}")
        print(f"[{c.label()}] SL sync error: {e}")
```

```
def mirror_adjust_sl_all(sl_pct: float):
    """Syncs a new SL percentage to every configured client, each in its
    own thread, each recalculated against their own entry."""
    for c in CLIENTS:
        if c.is_configured():
            threading.Thread(target=client_adjust_sl, args=(c, sl_pct), daemon=Tr


def client_adjust_tp(c, tp_pct: float, label: str = "TP1"):
    """Moves this ONE client's real resting TP1 or TP2 order to the same
    percentage distance from THEIR OWN entry price on Hyperliquid —
    called for every configured client whenever /tp or /tp2 changes the
    master's take-profit, mirroring what client_adjust_sl already does
    for /sl. Previously /tp and /tp2 only touched the master's own
    Binance orders and never propagated to clients at all — every TP
    move made on the master was silently NOT applied to any client.

    TP1 and TP2 sit on the same side of entry (both above for a long,
    both below for a short), so they're told apart by distance from
    entry rather than side: TP1 is whichever qualifying order sits
    closer to entry, TP2 whichever sits farther. This matches how the
    two are actually opened (see client_mirror_open) and survives one
    of them not existing yet.
    """
    if not c.is_configured():
        return
    try:
        info  = hl_get_info()
        state = info.user_state(c.address)
        position = None
        for p in state.get("assetPositions", []):
            pos_data = p.get("position", {})
            if pos_data.get("coin") == HL_COIN and float(pos_data.get("szi", 0))
                position = pos_data
                break
        if not position:
            print(f"[{c.label()}] {label} sync skipped — no open position on Hype
            return

        szi      = float(position["szi"])
        entry_px = float(position.get("entryPx", 0))
        if entry_px <= 0:
            return
        is_long = szi > 0

        new_tp_price = round(entry_px * (1 + tp_pct / 100)) if is_long else round
```

```
exchange = client_get_exchange(c)

only_tp2_left = False
# Find every resting order on the TP side of entry, then pick
# TP1/TP2 by distance rather than an assumed field name — same
# approach client_adjust_sl uses for the SL side.
try:
    mark = hl_get_mark_price()   # TPs sit on the winning side of the cur
    open_orders = info.open_orders(c.address)
    tp_side_orders = []
    for o in open_orders:
        if o.get("coin") != HL_COIN:
            continue
        trigger_px = float(o.get("triggerPx") or o.get("limitPx") or 0)
        if trigger_px <= 0:
            continue
        is_tp_side = (trigger_px > mark) if is_long else (trigger_px < ma
        if is_tp_side:
            tp_side_orders.append((abs(trigger_px - entry_px), o))
    tp_side_orders.sort(key=lambda t: t[0])  # closest to entry first = T

    target_order = None
    skip_reason  = None
    master_tp1_filled = bool(current_trade_entry.get("tp1_filled"))
    if master_tp1_filled and label == "TP1":
        # TP1 has already filled — whatever TP order is left is TP2, neve
        print(f"[{c.label()}] TP1 sync skipped — TP1 already filled")
        return
    elif master_tp1_filled and label == "TP2" and len(tp_side_orders) ==
        # TP1 filled, so the single remaining TP-side order IS TP2: move
        # and give it the FULL remaining size (not a TP1/TP2 split).
        target_order = tp_side_orders[0][1]
        only_tp2_left = True
    elif label == "TP1" and tp_side_orders:
        target_order = tp_side_orders[0][1]
    elif label == "TP2" and len(tp_side_orders) >= 2:
        target_order = tp_side_orders[-1][1]
    elif label == "TP2" and len(tp_side_orders) == 1:
        # Only one TP-side order exists (TP1 already filled, or
        # TP2 was never placed) — nothing safe to identify as
        # TP2 specifically. Previously this fell through and
        # still placed a NEW order without cancelling anything,
        # leaving the client with two resting TP-side orders
        # instead of one correctly-priced one. Now it aborts the
        # whole sync for this client and says so on Telegram.
```

```
                skip_reason = "only one TP-side order found — can't safely tell T
            elif label == "TP2" and not tp_side_orders:
                skip_reason = "no TP-side orders found to sync against"
            elif label == "TP1" and not tp_side_orders:
                skip_reason = "no TP-side orders found to sync against"

            if skip_reason:
                send_telegram(f"⚠️ {c.label()}: {label} sync skipped — {skip_reas
                return

            if target_order:
                exchange.cancel(HL_COIN, target_order.get("oid"))
        except Exception as e:
            print(f"[{c.label()}] Existing {label} cancel error (continuing anywa

        close_is_buy = not is_long
        remaining    = abs(szi)
        tp1_size = round(remaining * (TP1_CLOSE_PCT / 100), 4)
        if only_tp2_left:
            size = round(remaining, 4)
        else:
            size = tp1_size if label == "TP1" else round(remaining - tp1_size, 4)
        if size <= 0:
            print(f"[{c.label()}] {label} sync skipped — computed size was zero")
            return

        tp_order_type = HLOrderType(trigger=HLTriggerOrderType(
            triggerPx=new_tp_price, isMarket=True, tpsl="tp"
        ))
        result = exchange.order(HL_COIN, close_is_buy, size, new_tp_price, tp_ord
        if result.get("status") == "ok":
            print(f"[{c.label()}] {label} synced to ${new_tp_price:,.2f} ({tp_pct
        else:
            send_telegram(f"⚠️ {c.label()}: failed to sync new {label} — {result}
    except Exception as e:
        send_telegram(f"⚠️ {c.label()}: {label} sync error — {e}")
        print(f"[{c.label()}] {label} sync error: {e}")


def mirror_adjust_tp_all(tp_pct: float, label: str = "TP1"):
    """Syncs a new TP1 or TP2 percentage to every configured client, each
    in its own thread, each recalculated against their own entry."""
    for c in CLIENTS:
        if c.is_configured():
            threading.Thread(target=client_adjust_tp, args=(c, tp_pct, label), da
```

```
def mirror_close_all(max_retries: int = 4, retry_delay: int = 15):
    """Close the mirrored position on every configured client, each in its own th
    for c in CLIENTS:
        if c.is_configured():
            threading.Thread(target=client_mirror_close, args=(c, max_retries, re


def execute_signal_trade(direction: str, label: str):
    """
    Execute a trade confirmed via Take Trade button.
    Checks time blocks, then executes.
    """
    now_uk_full = datetime.now(UK_TZ)
    uk_now = now_uk_full.time()

    # ── Time blocks ──────────────────────────────────────────────
    if now_uk_full.weekday() == 4 and uk_now >= dt_time(19, 0):
        send_telegram(
            f"🚫 <b>{direction.upper()} blocked — Friday cutoff (19:00)</b>\n\n"
            f"No new positions after 19:00 on Fridays. Week's done — see you Mond
        )
        return
    if COOLDOWN_START <= uk_now <= COOLDOWN_END:
        send_telegram(
            f"🚫 <b>{direction.upper()} blocked — US open cooldown</b>\n\n"
            f"No trades between {COOLDOWN_START.strftime('%H:%M')} and {COOLDOWN_
        )
        return
    if uk_now >= NIGHT_BLOCK_START:
        send_telegram(
            f"🚫 <b>{direction.upper()} blocked — night time</b>\n\n"
            f"No trades after 10:45pm UK time."
        )
        return

    # ── All clear — execute ──────────────────────────────────────
    execute_trade(direction=direction, triggered_by=label)


def send_bias_question():
    send_telegram(
        "🧭 <b>Good morning. What is the HTF 4H trend today?</b>\n\n"
        "Reply with:\n"
        "/bullish — bullish bias\n"
        "/bearish — bearish bias\n\n"
```

```
            "⚠️ Trades are blocked until you set your bias."
        )


def send_morning_brief():
    weekly = load_last_7_days_stats()
    if not weekly:
        send_telegram(
            "☀️ <b>Good morning.</b>\n\n"
            "No stats from yesterday yet. Get to work. 💪"
        )
        return

    yesterday = weekly[-1]
    week_pnl  = sum(d["net_pnl"] for d in weekly)
    gap       = WEEKLY_PNL_TARGET - week_pnl

    day_emoji = "✅" if yesterday["net_pnl"] > 0 else "❌"
    yest_line = (
        f"{day_emoji} Yesterday:  {format_pnl(yesterday['net_pnl'])} "
        f"({yesterday['wins']}W / {yesterday['losses']}L, "
        f"{yesterday['win_rate']:.0f}% WR)"
    )

    # ── 7-day rolling win rate vs 60% target ──────────────────────────
    winrate_line = ""
    if DATABASE_URL:
        try:
            conn = get_db()
            cur  = conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor)
            cur.execute("""
                SELECT
                    COUNT(*) FILTER (WHERE outcome = 'win')  as wins,
                    COUNT(*) FILTER (WHERE outcome = 'loss') as losses
                FROM trigger_performance
                WHERE entry_time >= NOW() - INTERVAL '7 days'
                  AND outcome IN ('win', 'loss')
            """)
            r = cur.fetchone()
            cur.close()
            conn.close()
            total_7d = (r["wins"] or 0) + (r["losses"] or 0) if r else 0
            if total_7d >= 3:
                wr_7d = r["wins"] / total_7d * 100
                wr_emoji = "✅" if wr_7d >= 60 else "⚠️"
                winrate_line = f"{wr_emoji} 7d win rate: {wr_7d:.0f}% (target 60%
```

```
    except Exception as e:
        print(f"[Morning Brief] Win rate calc error: {e}")

if gap <= 0:
    week_line = f"🎖️ Weekly target: HIT (+{format_pnl(abs(gap))} over)"
else:
    week_line = f"📊 Weekly target: {format_pnl(gap)} to go ({format_pnl(week

try:
    balance = get_usdt_balance()
    balance_yesterday = balance - yesterday["net_pnl"]
    bal_change = balance - balance_yesterday
    bal_emoji  = "🟢" if bal_change >= 0 else "🔴"
    bal_line = (
        f"\n💰 <b>BALANCE: ${balance:,.2f} USDT</b>\n"
        f"{bal_emoji} {format_pnl(bal_change)} vs yesterday (${balance_yester
    )
except Exception:
    bal_line = ""

# ── Daily game plan — week-so-far stats, with fallback to 30d if thin ──
plan_line = ""
if DATABASE_URL:
    try:
        uk_now = datetime.now(UK_TZ)
        week_start_uk = uk_now - timedelta(days=uk_now.weekday())  # Monday t
        week_start_uk = week_start_uk.replace(hour=0, minute=0, second=0, mic

        conn = get_db()
        cur  = conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor)

        # ── Try week-so-far first ──
        cur.execute("""
            SELECT
                AVG(pnl) FILTER (WHERE outcome = 'win')  as avg_win,
                AVG(pnl) FILTER (WHERE outcome = 'loss') as avg_loss,
                COUNT(*) FILTER (WHERE outcome = 'win')  as wins,
                COUNT(*) FILTER (WHERE outcome = 'loss') as losses
            FROM trigger_performance
            WHERE entry_time >= %s
              AND outcome IN ('win', 'loss')
        """, (week_start_uk.astimezone(timezone.utc),))
        r = cur.fetchone()
        total = (r["wins"] or 0) + (r["losses"] or 0) if r else 0
        data_source = "this week so far"
        confidence_note = ""
```

```
# ── Fall back to last 30 days if week-so-far is too thin ──
if total < 5:
    cur.execute("""
        SELECT
            AVG(pnl) FILTER (WHERE outcome = 'win')  as avg_win,
            AVG(pnl) FILTER (WHERE outcome = 'loss') as avg_loss,
            COUNT(*) FILTER (WHERE outcome = 'win')  as wins,
            COUNT(*) FILTER (WHERE outcome = 'loss') as losses
        FROM trigger_performance
        WHERE entry_time >= NOW() - INTERVAL '30 days'
          AND outcome IN ('win', 'loss')
    """)
    r = cur.fetchone()
    total = (r["wins"] or 0) + (r["losses"] or 0) if r else 0
    data_source = "last 30d (week-so-far too thin)"
    confidence_note = "\n  ⚠️ Early in the week — using 30d average a

# ── Avg fee per trade, from TODAY's live Binance data ──────
# (fees aren't stored in trigger_performance historically, only
# available live per-day from Binance directly)
avg_fee_today = 0.0
try:
    todays_trades_for_fee = get_futures_trades_today()
    todays_stats_for_fee  = build_stats(todays_trades_for_fee)
    trades_today_count = todays_stats_for_fee.get("closed_positions",
    if trades_today_count > 0:
        avg_fee_today = todays_stats_for_fee.get("total_fees", 0) / t
except Exception as fee_e:
    print(f"[Morning Brief] Fee calc error: {fee_e}")

# ── Worst 4-hour window, week-so-far, fallback to 30d ──
cur.execute("""
    SELECT
        (EXTRACT(HOUR FROM close_time AT TIME ZONE 'UTC' AT TIME ZONE
        COUNT(*) FILTER (WHERE outcome = 'win')  as wins,
        COUNT(*) FILTER (WHERE outcome = 'loss') as losses,
        COALESCE(SUM(pnl) FILTER (WHERE outcome IN ('win','loss')), 0
    FROM trigger_performance
    WHERE close_time IS NOT NULL
      AND close_time >= %s
      AND outcome IN ('win', 'loss')
    GROUP BY (EXTRACT(HOUR FROM close_time AT TIME ZONE 'UTC' AT TIME
    ORDER BY net_pnl ASC
""", (week_start_uk.astimezone(timezone.utc),))
window_rows = cur.fetchall()
```

```
window_source = "this week"
if not window_rows or sum((wr["wins"] or 0) + (wr["losses"] or 0) for
    cur.execute("""
        SELECT
            (EXTRACT(HOUR FROM close_time AT TIME ZONE 'UTC' AT TIME
            COUNT(*) FILTER (WHERE outcome = 'win')  as wins,
            COUNT(*) FILTER (WHERE outcome = 'loss') as losses,
            COALESCE(SUM(pnl) FILTER (WHERE outcome IN ('win','loss')
        FROM trigger_performance
        WHERE close_time IS NOT NULL
          AND close_time >= NOW() - INTERVAL '30 days'
          AND outcome IN ('win', 'loss')
        GROUP BY (EXTRACT(HOUR FROM close_time AT TIME ZONE 'UTC' AT
        ORDER BY net_pnl ASC
    """)
    window_rows = cur.fetchall()
    window_source = "last 30d"

cur.close()
conn.close()

plan_parts = []

if r and total >= 2 and r["avg_win"]:
    win_rate  = r["wins"] / total
    loss_rate = r["losses"] / total
    avg_win   = float(r["avg_win"])
    avg_loss  = abs(float(r["avg_loss"])) if r["avg_loss"] else 0
    avg_fee   = avg_fee_today
    expected_per_trade = (avg_win * win_rate) - (avg_loss * loss_rate

    if expected_per_trade > 0:
        trades_needed = DAILY_PNL_TARGET / expected_per_trade
        plan_parts.append(
            f"🎯 <b>To hit today's ${DAILY_PNL_TARGET:.0f} target:</b
            f"  ~{trades_needed:.1f} trades expected ({data_source}:
            f"{win_rate*100:.0f}% WR, avg win {format_pnl(avg_win)},
            f"{confidence_note}"
        )
        if avg_fee > 0:
            max_trades_before_fee_limit = int(FEE_ALERT_THRESHOLD / a
            if max_trades_before_fee_limit < trades_needed * 1.5:
                plan_parts.append(
                    f"\n💸 <b>Fee ceiling:</b> at your average fee "
                    f"({format_pnl(-avg_fee)}/trade), roughly {max_tr
                    f"trades before hitting the ${FEE_ALERT_THRESHOLD
```

```
                            f"stay disciplined, don't force extra trades."
                        )
                else:
                    plan_parts.append(
                        f"⚠️ <b>{data_source.capitalize()} expectancy is negative
                        f"(avg win {format_pnl(avg_win)}, avg loss {format_pnl(-a
                        f"  More trades won't fix this — the edge needs reviewing
                        f"{confidence_note}"
                    )

            if window_rows:
                worst_w = window_rows[0]
                worst_total = (worst_w["wins"] or 0) + (worst_w["losses"] or 0)
                if worst_total >= 2 and worst_w["net_pnl"] < 0:
                    worst_start = int(worst_w["chunk_start"])
                    sample_note = " (small sample — take as a lean, not a rule)"
                    plan_parts.append(
                        f"\n⏰ <b>Time to be careful:</b> {worst_start:02d}:00-{w
                        f"has been your worst window ({window_source}) — "
                        f"{format_pnl(worst_w['net_pnl'])} ({worst_w['wins']}W/{w
                    )

            if plan_parts:
                plan_line = "\n" + "\n".join(plan_parts)
        except Exception as e:
            print(f"[Morning Brief] Daily game plan error: {e}")

    compound_line = ""
    try:
        progress = get_compounding_progress()
        if progress and "week_multiplier" in progress:
            week_mult = progress["week_multiplier"]
            week_emoji = "✅" if week_mult >= WEEKLY_COMPOUND_TARGET else "📈"
            compound_line = (
                f"{week_emoji} Week compounding: {week_mult:.4f}x of {WEEKLY_COMP
                f"({progress['week_pct_of_target']:.0f}% there)"
            )
    except Exception as e:
        print(f"[Morning Brief] Compounding progress error: {e}")

    pace_line = ""
    try:
        pace = get_daily_compounding_pace()
        if pace:
            pace_emoji = "✅" if pace["on_pace"] else "⚠️"
            pace_line = (
```

```
                f"\n{pace_emoji} <b>To stay on pace for {WEEKLY_COMPOUND_TARGET:.
                f"  Need +{pace['required_pct_today']:.2f}% today (+{format_pnl(p
                f"  {pace['days_remaining']} trading day(s) left this week\n"
                f"  {'On pace so far.' if pace['on_pace'] else 'Behind pace — tod
            )
    except Exception as e:
        print(f"[Morning Brief] Daily pace error: {e}")

    lines = ["☀️ <b>Good morning. Here's where you stand:</b>\n", yest_line]
    if winrate_line:
        lines.append(winrate_line)
    if compound_line:
        lines.append(compound_line)
    if pace_line:
        lines.append(pace_line)
    lines.append(week_line)
    if bal_line:
        lines.append(bal_line)
    if plan_line:
        lines.append(plan_line)
    lines.append("\nStick to the plan. Trust your edge. 🎯")

    send_telegram("\n".join(lines))


def get_discipline_comparison(period_days: int) -> dict:
    """
    Compares the last `period_days` against the equivalent period before
    that (e.g. this week vs last week, this month vs last month) across
    the 4 core discipline metrics: fee threshold breaches, loss streak
    warnings, revenge trades, and loss-limit block days. Returns counts
    for both periods plus the delta, so callers can render an up/down
    arrow — the actual OKR the user asked to track over time.
    """
    result = {}
    if not DATABASE_URL:
        return result
    try:
        conn = get_db()
        cur  = conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor)

        metrics = [
            ("fee_threshold",  "fee_breaches"),
            ("loss_streak",    "loss_streak_warnings"),
            ("revenge_trade",  "revenge_trades"),
        ]
```

```
        for category, key in metrics:
            cur.execute("""
                SELECT
                    COUNT(*) FILTER (WHERE logged_at >= NOW() - INTERVAL '%s days
                    COUNT(*) FILTER (WHERE logged_at >= NOW() - INTERVAL '%s days
                                        AND logged_at <  NOW() - INTERVAL '%s day
                FROM warnings_log
                WHERE category = %s
            """, (period_days, period_days * 2, period_days, category))
            row = cur.fetchone()
            result[key] = {
                "current":  row["current_count"] or 0,
                "previous": row["previous_count"] or 0,
            }

        # Loss-limit block DAYS (distinct days, combining both categories) —
        # matches the exact definition already used in /analytics.
        cur.execute("""
            SELECT
                COUNT(DISTINCT logged_at::date) FILTER (WHERE logged_at >= NOW()
                COUNT(DISTINCT logged_at::date) FILTER (WHERE logged_at >= NOW()
                                                        AND logged_at <  NOW(
            FROM warnings_log
            WHERE category IN ('daily_loss_block', 'daily_loss_limit')
        """, (period_days, period_days * 2, period_days))
        row = cur.fetchone()
        result["loss_limit_days"] = {
            "current":  row["current_count"] or 0,
            "previous": row["previous_count"] or 0,
        }

        cur.close()
        conn.close()
    except Exception as e:
        print(f"[Discipline Comparison] Error: {e}")
    return result


def format_discipline_comparison_lines(comparison: dict) -> list:
    """
    Turns get_discipline_comparison()'s output into display lines with
    up/down arrows. Down (fewer incidents) is good — shown as ✅🔽.
    Up (more incidents) is bad — shown as ⚠️🔼. Flat is ➡️.
    """
    labels = [
        ("fee_breaches",         "💸 Fee threshold breaches"),
```

```
        ("loss_streak_warnings", "🔴 Loss streak warnings"),
        ("revenge_trades",       "😤 Revenge trades"),
        ("loss_limit_days",      "🚫 Loss-limit block days"),
    ]
    lines = []
    for key, label in labels:
        data = comparison.get(key)
        if not data:
            continue
        current, previous = data["current"], data["previous"]
        if current < previous:
            arrow = f"✅⬇️ down from {previous}"
        elif current > previous:
            arrow = f"⚠️🔺 up from {previous}"
        else:
            arrow = f"➡️ flat vs {previous}"
        lines.append(f"{label}: {current}  ({arrow})")
    return lines


def send_monthly_discipline_review():
    """
    1st of the month, 8:30am — compares the last 30 days against the 30
    days before that across the 4 core discipline metrics. Same idea as
    the weekly comparison, just zoomed out to catch slower trends the
    week-to-week view might miss.
    """
    try:
        comparison = get_discipline_comparison(30)
        comp_lines = format_discipline_comparison_lines(comparison)
        if not comp_lines:
            return
        msg = (
            "📆 <b>Monthly Discipline Review</b>\n\n"
            "<i>This month vs last month. Reduce these 4 numbers and you retain "
            "real money — independent of any single trade or signal.</i>\n\n"
            + "\n".join(comp_lines)
        )
        send_telegram(msg)
    except Exception as e:
        print(f"[Monthly Discipline Review] Error: {e}")


def send_weekly_okr():
    """
    Monday morning — reflects on the week just finished (last 7 days) and
```

```
gives concrete, data-backed things to do more of / less of to hit the
daily (1.1x) and weekly (1.4x) compounding targets. Every number here
is calculated from real trade data, not generic advice — if the data
doesn't support a claim, that section is skipped rather than filled
with something vague.
"""
if not DATABASE_URL:
    return
total = 0
win_rate = None
wr_row = {"wins": 0, "losses": 0}
try:
    conn = get_db()
    cur  = conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor)

    # ── Win rate over last 7 days ──
    cur.execute("""
        SELECT COUNT(*) FILTER (WHERE outcome = 'win')  as wins,
               COUNT(*) FILTER (WHERE outcome = 'loss') as losses
        FROM trigger_performance
        WHERE entry_time >= NOW() - INTERVAL '7 days'
          AND outcome IN ('win', 'loss')
    """)
    wr_row = cur.fetchone()
    total = (wr_row["wins"] or 0) + (wr_row["losses"] or 0) if wr_row else 0
    win_rate = (wr_row["wins"] / total * 100) if total > 0 else None

    # ── Worst trigger by win rate (virtual win-rate data), with its $ cost ─
    cur.execute("""
        SELECT trigger,
               COUNT(*) FILTER (WHERE outcome IN ('win','loss')) as total,
               COUNT(*) FILTER (WHERE outcome = 'win') as wins
        FROM virtual_trigger_trades
        WHERE opened_at >= NOW() - INTERVAL '7 days'
        GROUP BY trigger
        HAVING COUNT(*) FILTER (WHERE outcome IN ('win','loss')) >= 3
        ORDER BY (COUNT(*) FILTER (WHERE outcome = 'win')::float /
                  NULLIF(COUNT(*) FILTER (WHERE outcome IN ('win','loss')), 0
        LIMIT 1
    """)
    worst_trigger_row = cur.fetchone()

    # ── Real $ cost of that worst trigger, from actual trades taken on it ──
    worst_trigger_real_cost = None
    if worst_trigger_row:
        cur.execute("""
```

```
        SELECT COALESCE(SUM(pnl), 0) as total_pnl, COUNT(*) as n
        FROM trigger_performance
        WHERE trigger = %s
          AND entry_time >= NOW() - INTERVAL '7 days'
          AND outcome IN ('win', 'loss')
    """, (worst_trigger_row["trigger"],))
    wt_cost_row = cur.fetchone()
    if wt_cost_row and wt_cost_row["n"] > 0:
        worst_trigger_real_cost = float(wt_cost_row["total_pnl"])

# ── Worst 4-hour window, with its $ cost ──
cur.execute("""
    SELECT
        (EXTRACT(HOUR FROM close_time AT TIME ZONE 'UTC' AT TIME ZONE 'Eu
        COALESCE(SUM(pnl) FILTER (WHERE outcome IN ('win','loss')), 0) as
        COUNT(*) FILTER (WHERE outcome IN ('win','loss')) as total
    FROM trigger_performance
    WHERE close_time IS NOT NULL
      AND close_time >= NOW() - INTERVAL '7 days'
      AND outcome IN ('win', 'loss')
    GROUP BY (EXTRACT(HOUR FROM close_time AT TIME ZONE 'UTC' AT TIME ZON
    ORDER BY net_pnl ASC
""")
window_rows = cur.fetchall()
worst_window = window_rows[0] if window_rows and window_rows[0]["total"]

# ── Avg win/loss $ for the ratio math ──
cur.execute("""
    SELECT
        AVG(pnl) FILTER (WHERE outcome = 'win')  as avg_win,
        AVG(pnl) FILTER (WHERE outcome = 'loss') as avg_loss,
        COUNT(*) FILTER (WHERE outcome = 'loss') as loss_count
    FROM trigger_performance
    WHERE entry_time >= NOW() - INTERVAL '7 days'
      AND outcome IN ('win', 'loss')
""")
roi_row = cur.fetchone()

# ── Revenge trades and their real cost, last 7 days ──
cur.execute("""
    SELECT COUNT(*) as n FROM warnings_log
    WHERE category = 'revenge_trade' AND logged_at >= NOW() - INTERVAL '7
""")
revenge_row = cur.fetchone()

cur.close()
```

```
    conn.close()
except Exception as e:
    print(f"[Weekly OKR] Data fetch error: {e}")
    return

try:
    current_balance = get_usdt_balance()
except Exception:
    current_balance = None

sections = ["📋 <b>Weekly OKR — Monday Reset</b>\n"]
sections.append(
    "<i>Targets: 1.1x daily, 1.4x weekly. This isn't about trying harder — "
    "it's about doing 2 things more and 2 things less, based on what actually
    "cost or made you money last week.</i>\n"
)

if total > 0:
    wr_emoji = "✅" if win_rate >= 60 else "⚠️"
    sections.append(f"{wr_emoji} Last week: {win_rate:.0f}% win rate ({wr_row
else:
    sections.append("— No closed trades last week to review.")

# ── What a 1.1x day / 1.4x week actually requires, in plain $ terms ──
if current_balance and current_balance > 0:
    daily_gain_needed  = current_balance * 0.10
    weekly_gain_needed = current_balance * 0.40
    sections.append(
        f"\n📐 <b>What the targets actually mean right now:</b>\n"
        f"  1.1x a day = +{format_pnl(daily_gain_needed)} on today's balance\
        f"  1.4x a week = +{format_pnl(weekly_gain_needed)} by Friday"
    )

do_more = []
do_less = []

avg_win  = float(roi_row["avg_win"])  if roi_row and roi_row["avg_win"]  else
avg_loss = abs(float(roi_row["avg_loss"])) if roi_row and roi_row["avg_loss"]
loss_count = roi_row["loss_count"] if roi_row else 0

if avg_win > 0 and avg_loss > 0:
    ratio = avg_loss / avg_win
    if ratio >= 1.3:
        # If losses were even 30% smaller, here's what that would have saved
        potential_save = avg_loss * 0.30 * loss_count
        do_less.append(
```

```
            f"<b>Cut losses faster.</b> Avg loss ({format_pnl(-avg_loss)}) is
            f"avg win ({format_pnl(avg_win)}). Cutting losses just 30% smalle
            f"retained ~{format_pnl(potential_save)} last week alone."
        )
    elif ratio <= 0.6:
        do_more.append(
            f"<b>Let winners run further.</b> Avg win ({format_pnl(avg_win)})
            f"relative to avg loss ({format_pnl(-avg_loss)}) — give winning t
            f"room before taking profit."
        )

if worst_trigger_row and worst_trigger_real_cost is not None:
    wt_total = worst_trigger_row["total"]
    wt_wr = (worst_trigger_row["wins"] / wt_total * 100) if wt_total > 0 else
    if wt_wr < 40:
        trigger_label = worst_trigger_row["trigger"].replace("_", " ").title(
        cost_line = (
            f" — this cost you {format_pnl(worst_trigger_real_cost)} in real
            if worst_trigger_real_cost < 0 else ""
        )
        do_less.append(
            f"<b>Stop taking {trigger_label}.</b> {wt_wr:.0f}% win rate over
            f"virtual trades this week{cost_line} Avoiding this signal alone
            f"directly improved last week's result."
        )

if worst_window:
    w_start = int(worst_window["chunk_start"])
    do_less.append(
        f"<b>Avoid trading {w_start:02d}:00–{w_start+4:02d}:00.</b> This wind
        f"{format_pnl(worst_window['net_pnl'])} over {worst_window['total']}
        f"skipping it would have kept that {format_pnl(abs(worst_window['net_
    )

if revenge_row and revenge_row["n"] and revenge_row["n"] >= 3:
    do_less.append(
        f"<b>Cut the revenge-trade reflex.</b> {revenge_row['n']} revenge tra
        f"last week — each one is a trade taken from emotion, not the setup."
    )

if win_rate is not None and win_rate >= 60 and total >= 5:
    do_more.append(
        f"<b>Keep doing what you're doing.</b> {win_rate:.0f}% win rate over
        f"is above the 60% target — don't change the process, just do more of
    )
```

```
    sections.append("")
    if do_more:
        sections.append("📈 <b>Do more of:</b>\n" + "\n".join(f"  • {s}" for s in
    if do_less:
        sections.append("📉 <b>Do less of:</b>\n" + "\n".join(f"  • {s}" for s in
    if not do_more and not do_less:
        sections.append("No strong signal in the data this week — steady as you g

    # ── Week-over-week discipline comparison — the real OKR ──────────
    # Reducing these 4 numbers, independent of any signal or setup working,
    # directly retains money. This is compared against the equivalent
    # 7-day period before last week, so it's a genuine trend, not a
    # one-off snapshot.
    try:
        comparison = get_discipline_comparison(7)
        comp_lines = format_discipline_comparison_lines(comparison)
        if comp_lines:
            sections.append(
                "\n📊 <b>This week vs last week — the real OKR:</b>\n"
                "<i>Reduce these 4 numbers and you keep money, regardless of what
                + "\n".join(comp_lines)
            )
    except Exception as e:
        print(f"[Weekly OKR] Discipline comparison error: {e}")

    send_telegram("\n".join(sections))


def send_weekly_digest():
    """Sunday evening rollup pulling highlights from /patterns, /quality, /invali
    weekly = load_last_7_days_stats()
    week_pnl = sum(d["net_pnl"] for d in weekly) if weekly else 0
    week_wins   = sum(d["wins"] for d in weekly) if weekly else 0
    week_losses = sum(d["losses"] for d in weekly) if weekly else 0
    week_total  = week_wins + week_losses
    week_wr     = (week_wins / week_total * 100) if week_total > 0 else 0
    pnl_emoji   = "✅" if week_pnl >= 0 else "❌"

    sections = [
        f"📈 <b>Weekly Digest</b>\n",
        f"{pnl_emoji} <b>P&L:</b> {format_pnl(week_pnl)}  ({week_wins}W / {week_l
    ]

    if not DATABASE_URL:
        sections.append("\n⚠️ No database connected — deeper stats unavailable.")
        send_telegram("\n".join(sections))
```

```
    return

try:
    conn = get_db()
    cur  = conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor)

    # ── Most active / most reliable trigger this week ──
    cur.execute("""
        SELECT trigger,
               COUNT(*) FILTER (WHERE outcome IN ('win','loss')) as total,
               COUNT(*) FILTER (WHERE outcome = 'win') as wins,
               COALESCE(SUM(pnl) FILTER (WHERE outcome IN ('win','loss')), 0)
        FROM trigger_performance
        WHERE entry_time >= NOW() - INTERVAL '7 days'
          AND trigger NOT IN ('long_setup_10m', 'short_setup_10m', 'fork', '3
        GROUP BY trigger
        HAVING COUNT(*) FILTER (WHERE outcome IN ('win','loss')) >= 1
        ORDER BY net_pnl DESC
        LIMIT 1
    """)
    best_trigger = cur.fetchone()
    cur.execute("""
        SELECT trigger,
               COUNT(*) FILTER (WHERE outcome IN ('win','loss')) as total,
               COUNT(*) FILTER (WHERE outcome = 'win') as wins,
               COALESCE(SUM(pnl) FILTER (WHERE outcome IN ('win','loss')), 0)
        FROM trigger_performance
        WHERE entry_time >= NOW() - INTERVAL '7 days'
          AND trigger NOT IN ('long_setup_10m', 'short_setup_10m', 'fork', '3
        GROUP BY trigger
        HAVING COUNT(*) FILTER (WHERE outcome IN ('win','loss')) >= 1
        ORDER BY net_pnl ASC
        LIMIT 1
    """)
    worst_trigger = cur.fetchone()

    TRIGGER_LABELS = {
        "long": "📈 Long", "3_bulls": "🐂 3 Bulls",
        "strength": "💪 Strength", "3_bar_buy": "📈 3 Bar Buy",
        "3_bears": "🐻 3 Bears", "weakness": "📉 Weakness",
        "bearish": "🔴 Bearish", "short": "📉 Short", "30m_composite_crosses_
        "green_structure_break": "🟢 Green Structure Break",
        "orange_structure_break": "🟠 Orange Structure Break",
    }

    if best_trigger and worst_trigger and best_trigger["trigger"] != worst_tr
        best_label  = TRIGGER_LABELS.get(best_trigger["trigger"], best_trigge
```

```
        worst_label = TRIGGER_LABELS.get(worst_trigger["trigger"], worst_trig
        sections.append(
            f"\n🏆 <b>Best signal:</b> {best_label} ({format_pnl(best_trigger
            f"💔 <b>Worst signal:</b> {worst_label} ({format_pnl(worst_trigge
        )
    elif best_trigger:
        label = TRIGGER_LABELS.get(best_trigger["trigger"], best_trigger["tri
        sections.append(f"\n🏆 <b>Only signal fired:</b> {label} ({format_pnl

    # ── Warning counts by category this week ──
    cur.execute("""
        SELECT category, COUNT(*) as cnt
        FROM warnings_log
        WHERE logged_at >= NOW() - INTERVAL '7 days'
        GROUP BY category
        ORDER BY cnt DESC
    """)
    warning_rows = cur.fetchall()
    if warning_rows:
        WARNING_LABELS = {
            "counter_signal":     "Counter signals",
            "loss_streak":        "Loss streaks",
            "daily_loss_block":   "3-loss blocks",
            "daily_loss_limit":   "Daily loss limit hits",
            "revenge_trade":      "Revenge trade flags",
            "retrace_protect":    "Retracement saves",
            "fee_threshold":      "Fee threshold breaches",
            "first_loss_tighten": "First-trade-loss tightenings",
            "signal_skipped":     "Signals skipped",
        }
        top3 = warning_rows[:3]
        warn_lines = [f"  {WARNING_LABELS.get(r['category'], r['category'])}:
        sections.append(f"\n⚠️ <b>Top warnings this week:</b>\n" + "\n".join(
    else:
        sections.append("\n✅ <b>No warnings fired this week.</b> Clean disci

    # ── Most-skipped signal this week ──
    cur.execute("""
        SELECT message, COUNT(*) as cnt
        FROM warnings_log
        WHERE category = 'signal_skipped'
          AND logged_at >= NOW() - INTERVAL '7 days'
        GROUP BY message
        ORDER BY cnt DESC
        LIMIT 1
    """)
```

```
        top_skip = cur.fetchone()
        if top_skip:
            sections.append(
                f"\n🚫 <b>Most skipped:</b> {top_skip['message'].replace('Skipped
            )

        # ── Invalidation rate check — worst offender this week ──
        cur.execute("""
            SELECT trigger, COUNT(*) as cnt
            FROM invalidation_log
            WHERE logged_at >= NOW() - INTERVAL '7 days'
            GROUP BY trigger
            ORDER BY cnt DESC
            LIMIT 1
        """)
        top_inv = cur.fetchone()
        if top_inv:
            inv_label = TRIGGER_LABELS.get(top_inv["trigger"], top_inv["trigger"]
            sections.append(f"\n🔻 <b>Most invalidated:</b> {inv_label} ({top_inv

        # ── Brute-force closes this week ──
        cur.execute("""
            SELECT COUNT(*) as cnt
            FROM warnings_log
            WHERE category IN ('brute_force_close', 'hard_structure_invalidate',
              AND logged_at >= NOW() - INTERVAL '7 days'
        """)
        r_brute = cur.fetchone()
        if r_brute and r_brute["cnt"]:
            sections.append(f"\n🛡️ <b>Brute-force closes:</b> {r_brute['cnt']}x t

        cur.close()
        conn.close()
    except Exception as e:
        sections.append(f"\n⚠️ Digest query error: {e}")
        print(f"[Weekly Digest] Error: {e}")

    sections.append("\n\nFull detail: /patterns  /quality  /invalidations  /warni
    send_telegram("\n".join(sections))


def poll_order_fills():
    global entry2_placed_at
    while True:
        try:
            if tracked_orders:
```

```
filled_ids = []

# ── Fetch all open orders ONCE per cycle, not once per tracked ─
# order. Previously this called /fapi/v1/order individually for
# EVERY tracked order (Entry2 + TP1 + TP2 = 3 separate Binance
# calls) every single cycle, even though most cycles nothing has
# changed. Now: one bulk call to see which order IDs are still
# genuinely open — only orders that have DISAPPEARED from that
# list (meaning they filled/cancelled/expired) get the more
# detailed individual lookup, since that's the only time we
# actually need the fill price/qty details.
still_open_ids = set()
try:
    now_ms_bulk = int(time.time() * 1000)
    bulk_params = sign_request({"symbol": SYMBOL, "timestamp": no
    bulk_resp = requests.get(f"{FUTURES_BASE}/fapi/v1/openOrders"
                             params=bulk_params, headers={"X-MBX-
                             timeout=10)
    if bulk_resp.status_code == 200:
        still_open_ids = {o["orderId"] for o in bulk_resp.json()}
    else:
        # If the bulk call itself fails, fall back to checking
        # every tracked order individually this cycle rather
        # than silently skipping fill detection entirely.
        still_open_ids = set(tracked_orders.keys())
except Exception as e:
    print(f"[Fill Poll] Bulk open orders check failed, falling ba
    still_open_ids = set(tracked_orders.keys())

for order_id, meta in list(tracked_orders.items()):
    if order_id in still_open_ids:
        continue  # still genuinely open — no need to check detai

    now_ms = int(time.time() * 1000)
    params = {"symbol": SYMBOL, "orderId": order_id, "timestamp":
    params = sign_request(params)
    headers = {"X-MBX-APIKEY": BINANCE_API_KEY}
    resp = requests.get(f"{FUTURES_BASE}/fapi/v1/order",
                        params=params, headers=headers, timeout=1
    if resp.status_code != 200:
        continue
    order  = resp.json()
    status = order.get("status", "")

    if status == "FILLED":
        filled_ids.append(order_id)
```

```
avg_price = float(order.get("avgPrice") or meta.get("pric
qty       = float(order.get("executedQty") or meta.get("q
label     = meta["label"]

if label == "Entry2":
    entry2_placed_at = 0.0
    sl_line = ""
    tp_grow_line = ""
    max_lines = ""
    try:
        pos       = get_open_position()
        total_usd = abs(float(pos["positionAmt"])) * avg_
        if pos and current_trade_entry:
            avg_entry  = float(pos.get("entryPrice", 0))
            direction  = current_trade_entry.get("directi
            sym_info   = get_symbol_info()
            tick       = sym_info["price_tick"]
            step       = sym_info["qty_step"]
            stop_pct   = STOP_PCT   # use original SL % f
            if avg_entry > 0 and direction:
                if direction == "long":
                    new_sl = round_step(avg_entry * (1 -
                else:
                    new_sl = round_step(avg_entry * (1 +
                current_trade_entry["sl_price"] = new_sl
                current_trade_entry["price"]    = avg_ent
                sl_line = f"\nSL recalculated: ${new_sl:,

                # ── Grow TP2 (or TP1) to absorb Entry2 q
                tp2_price_existing = current_trade_entry.
                tp1_price_existing = current_trade_entry.
                close_side = "SELL" if direction == "long
                grown = False
                for oid, meta in list(tracked_orders.item
                    if meta["label"] == "TP2" and tp2_pri
                        try:
                            now_ms  = int(time.time() * 1
                            params  = {"symbol": SYMBOL,
                            params  = sign_request(params
                            headers = {"X-MBX-APIKEY": BI
                            requests.delete(f"{FUTURES_BA
                                            params=params
                            tracked_orders.pop(oid, None)
                            new_tp2_qty = round_step(curr
                            new_resp = place_order(close_
                                                   limit
```

```
                        register_order(new_resp, "TP2
                        current_trade_entry["qty_tp2"
                        tp_grow_line = f"\nTP2 grown
                        grown = True
                    except Exception as e3:
                        tp_grow_line = f"\n⚠️ Could n
                    break
            if not grown and tp1_price_existing:
                for oid, meta in list(tracked_orders.
                    if meta["label"] == "TP1":
                        try:
                            now_ms  = int(time.time()
                            params  = {"symbol": SYMB
                            params  = sign_request(pa
                            headers = {"X-MBX-APIKEY"
                            requests.delete(f"{FUTURE
                                            params=pa
                            tracked_orders.pop(oid, N
                            new_tp1_qty = round_step(
                            new_resp = place_order(cl
                                                   l
                            register_order(new_resp,
                            current_trade_entry["qty_
                            tp_grow_line = f"\nTP1 gr
                        except Exception as e3:
                            tp_grow_line = f"\n⚠️ Cou
                        break

            # ── Recalculate max win/loss on full pos
            total_qty = abs(float(pos["positionAmt"])
            max_loss_usd = total_qty * abs(avg_entry
            max_win_usd  = current_trade_entry["qty_t
            if current_trade_entry.get("qty_tp2"):
                max_win_usd += current_trade_entry["q
            max_lines = (
                f"\n\n🛑 New max loss: {format_pnl(-m
                f"\n💰 New max win:  {format_pnl(max_
            )
        else:
            sl_line = ""
except Exception as e:
    total_usd = qty * avg_price / LEVERAGE
send_telegram(
    f"📩 <b>Entry2 filled</b>\n\n"
    f"Price:          ${avg_price:,.2f}\n"
    f"Total margin:   ${total_usd:,.2f} ✅"
```

```
                f"{sl_line}"
                f"{tp_grow_line}"
                f"{max_lines}"
            )
        elif label == "TP1":
            current_trade_entry["tp1_filled"] = True
            time_to_tp1 = (time.time() - trade_entry_time) / 60 i
            if pending_trigger_id and time_to_tp1 is not None:
                try:
                    conn = get_db()
                    cur  = conn.cursor()
                    cur.execute("""
                        UPDATE trigger_performance
                        SET time_to_tp1_mins = %s, mfe = %s, mae
                        WHERE id = %s
                    """, (time_to_tp1, trade_mfe, trade_mae, pend
                    conn.commit()
                    cur.close()
                    conn.close()
                except Exception as e:
                    print(f"[DB] time_to_tp1 update error: {e}")
            send_telegram(
                f"🎯 <b>TP1 hit — partial close filled</b>\n\n"
                f"Price:   ${avg_price:,.2f}\n"
                f"Closed:  ${qty * avg_price / LEVERAGE:,.2f}\n"
                f"Moving SL to +{TP1_SL_MOVE_PCT}% from entry...
                f"{after_trade_summary()}"
            )
            send_push_notification("TP1 hit", f"Partial close ${q
            for oid, meta in list(tracked_orders.items()):
                if meta["label"] == "Entry2":
                    try:
                        now_ms  = int(time.time() * 1000)
                        params  = {"symbol": SYMBOL, "orderId": i
                        params  = sign_request(params)
                        headers = {"X-MBX-APIKEY": BINANCE_API_KE
                        requests.delete(f"{FUTURES_BASE}/fapi/v1/
                                        params=params, headers=he
                        tracked_orders.pop(oid, None)
                        entry2_placed_at = 0.0
                        send_telegram("🚫 Entry2 auto-cancelled -
                    except Exception as e2:
                        send_telegram(f"⚠️ Could not auto-cancel
                    break
            try:
                entry_info = current_trade_entry
```

```
            if entry_info:
                ep        = entry_info["price"]
                direction = entry_info["direction"]
                sym_info  = get_symbol_info()
                tick      = sym_info["price_tick"]
                new_sl    = round_step(ep * (1 + TP1_SL_MOVE_
                              else round_step(ep * (1 - TP1_SL_
                # Actually move the stop: place the real Bina
                # client (adjust_sl does both and reports on
                # set the in-memory value and send a message
                # never moved.
                adjust_sl(new_sl)
                current_trade_entry["sl_price"] = new_sl
                send_telegram(f"🔒 TP1 hit — stop moved to ${
            else:
                send_telegram("⚠️ Could not move SL — entry p
        except Exception as sl_e:
            send_telegram(f"⚠️ TP1 hit but SL move failed: {s
    elif label == "TP2":
        global last_tp2_close_time
        current_trade_entry.clear()
        last_tp2_close_time = time.time()
        time_to_tp2 = (time.time() - trade_entry_time) / 60 i
        db_update_outcome(pending_trigger_id, "win", qty * av
                          mae=trade_mae, mfe=trade_mfe,
                          time_to_tp2_mins=time_to_tp2)
        send_telegram(
            f"🏆 <b>TP2 hit — position fully closed</b>\n\n"
            f"Price:   ${avg_price:,.2f}\n"
            f"Closed:  ${qty * avg_price / LEVERAGE:,.2f}\n"
            f"Clean exit. ✅\n\n"
            f"{after_trade_summary()}"
        )
        send_push_notification("TP2 hit", f"Position closed $
        # Each client has their own real TP orders on
        # Hyperliquid, priced from THEIR OWN entry — let
        # those fire independently rather than forcing
        # every account closed the instant Binance's
        # price hits. Only step in as a safety net for
        # anyone whose own order genuinely isn't there.
        mirror_close_unhealthy()
        start_prompt_silence("TP2 hit — let the cortisol sett
    elif label == "SL":
        current_trade_entry.clear()
        last_loss_close_time = time.time()
        time_to_sl = (time.time() - trade_entry_time) / 60 if
```

```
                            db_update_outcome(pending_trigger_id, "loss", -(qty *
                                              mae=trade_mae, mfe=trade_mfe,
                                              time_to_sl_mins=time_to_sl)
                            send_telegram(
                                f"🛑 <b>Stop loss hit</b>\n\n"
                                f"Price:   ${avg_price:,.2f}\n"
                                f"Closed:  ${qty * avg_price / LEVERAGE:,.2f}\n"
                                f"Loss taken. Assess before the next trade.\n\n"
                                f"{after_trade_summary()}"
                            )
                            send_push_notification("Stop loss hit", f"Closed ${qt
                            # Same reasoning as the TP2 case above — clients
                            # have their own real SL orders on Hyperliquid.
                            mirror_close_unhealthy()

                    elif status in ("CANCELED", "EXPIRED", "REJECTED"):
                        filled_ids.append(order_id)

                for oid in filled_ids:
                    tracked_orders.pop(oid, None)

        except Exception as e:
            print(f"[Fill Poll ERROR] {e}")

        time.sleep(ORDER_FILL_POLL_INTERVAL)


def register_order(order_resp: dict, label: str) -> None:
    order_id = order_resp.get("orderId")
    if order_id:
        tracked_orders[order_id] = {
            "label": label,
            "price": float(order_resp.get("price") or order_resp.get("stopPrice")
            "qty":   float(order_resp.get("origQty", 0)),
        }


def pnl_summary() -> None:
    try:
        trades    = get_futures_trades_today()
        stats     = build_stats(trades)
        positions = group_by_position(trades)

        last_line = ""
        if positions:
            last      = positions[-1]
```

```
    result    = "✅ Win" if last["realizedPnl"] > 0 else "❌ Loss"
    last_line = f"Last trade:  {result}  {format_pnl(last['realizedPnl'])

streak_line = ""
win_streak  = get_consecutive_wins(trades)
loss_streak = get_consecutive_losses(trades)
if win_streak >= 2:
    streak_line = f"Streak:      🔥 {win_streak} wins in a row\n"
elif loss_streak >= 2:
    streak_line = f"Streak:      🔴 {loss_streak} losses in a row\n"
elif win_streak == 1:
    streak_line = f"Streak:      ✅ Last trade was a win\n"
elif loss_streak == 1:
    streak_line = f"Streak:      ❌ Last trade was a loss\n"

freq_lines = ""
if len(positions) >= 2:
    def fmt_dur(mins):
        if mins >= 60:
            return f"{int(mins//60)}h {int(mins%60)}m"
        return f"{int(mins)}m"

    gaps = []
    for i in range(1, len(positions)):
        prev_close = positions[i-1].get("close_time")
        curr_open  = positions[i].get("open_time")
        if prev_close and curr_open:
            gaps.append((curr_open - prev_close) / 60)

    hold_times = []
    for p in positions:
        o = p.get("open_time")
        c = p.get("close_time")
        if o and c and c > o:
            hold_times.append((c - o) / 60)

    if gaps:
        avg_gap = sum(gaps) / len(gaps)
        freq_lines += f"\nAvg gap:     {fmt_dur(avg_gap)}"
        freq_lines += f"  (shortest: {fmt_dur(min(gaps))}, longest: {fmt_
    if hold_times:
        avg_hold = sum(hold_times) / len(hold_times)
        freq_lines += f"\nAvg hold:    {fmt_dur(avg_hold)}"

wins_pnl   = [p["realizedPnl"] for p in positions if p["realizedPnl"] > 0
losses_pnl = [p["realizedPnl"] for p in positions if p["realizedPnl"] < 0
```

```
        avg_win    = sum(wins_pnl)   / len(wins_pnl)   if wins_pnl   else None
        avg_loss   = sum(losses_pnl) / len(losses_pnl) if losses_pnl else None
        rr_line = ""
        if avg_win and avg_loss:
            rr = abs(avg_win / avg_loss)
            rr_line = f"Avg win:     {format_pnl(avg_win)}\nAvg loss:    {format_
        elif avg_win:
            rr_line = f"Avg win:     {format_pnl(avg_win)}\n"
        elif avg_loss:
            rr_line = f"Avg loss:    {format_pnl(avg_loss)}\n"

        target     = DAILY_PNL_TARGET
        net        = stats["net_pnl"]
        pct_done   = (net / target * 100) if target else 0
        target_line = f"\nTarget:      {format_pnl(net)} / ${target:.0f}  ({pct_d

        try:
            balance  = get_usdt_balance()
            bal_line = f"\nBalance:     ${balance:,.2f} USDT"
        except Exception:
            bal_line = ""

        send_telegram(
            f"📊 <b>Today's stats</b>\n\n"
            f"Positions:   {stats['closed_positions']}  "
            f"({stats['wins']}W / {stats['losses']}L  —  {stats['win_rate']:.1f}%
            f"{last_line}"
            f"{streak_line}"
            f"\n{rr_line}"
            f"Gross P&L:   {format_pnl(stats['total_pnl'])}\n"
            f"Fees:        -${stats['total_fees']:.2f}\n"
            f"Net P&L:     {format_pnl(stats['net_pnl'])}"
            f"{target_line}"
            f"{bal_line}"
            f"{freq_lines}"
        )
    except Exception as e:
        if is_rate_limit_error(e):
            send_telegram(
                f"🚫 <b>Binance rate-limit hit — not a bug, back off needed.</b>\
                f"Your IP has been temporarily restricted by Binance ({e.response
                f"This usually clears within minutes to a couple hours on its own
                f"Avoid rapid repeated commands until it clears."
            )
        else:
            send_telegram(f"❌ Stats error: {e}")
```

```
def compare_summary() -> None:
    try:
        trades    = get_futures_trades_today()
        positions = group_by_position(trades)

        if not positions:
            send_telegram("📊 No closed positions today yet.")
            return

        ind_trades = [p for p in positions if any(str(oid) in indicator_trade_ids
        man_trades = [p for p in positions if not any(str(oid) in indicator_trade

        def fmt_block(label, emoji, t_list):
            if not t_list:
                return f"{emoji} <b>{label}</b>\nNo trades yet."
            wins     = [p for p in t_list if p["realizedPnl"] > 0]
            losses   = [p for p in t_list if p["realizedPnl"] < 0]
            wr       = len(wins) / len(t_list)
            avg_win  = sum(p["realizedPnl"] for p in wins)   / len(wins)   if win
            avg_loss = sum(p["realizedPnl"] for p in losses) / len(losses) if los
            pnl      = sum(p["realizedPnl"] for p in t_list)
            expectancy = (wr * avg_win) + ((1 - wr) * avg_loss)

            def _roi_pct(p):
                margin = (p.get("entry_notional", 0) / LEVERAGE) if LEVERAGE > 0
                return (p["realizedPnl"] / margin * 100) if margin > 0 else None
            win_rois_l  = [r for r in (_roi_pct(p) for p in wins)  if r is not No
            loss_rois_l = [r for r in (_roi_pct(p) for p in losses) if r is not N
            avg_win_roi  = (sum(win_rois_l) / len(win_rois_l))   if win_rois_l  e
            avg_loss_roi = (sum(loss_rois_l) / len(loss_rois_l)) if loss_rois_l e

            return (
                f"{emoji} <b>{label}</b>\n"
                f"Positions:   {len(t_list)}  ({len(wins)}W / {len(losses)}L)\n"
                f"Win rate:    {wr*100:.1f}%\n"
                f"Avg win:     {format_pnl(avg_win)}  (+{avg_win_roi:.2f}% ROI)\n
                f"Avg loss:    {format_pnl(avg_loss)}  ({avg_loss_roi:.2f}% ROI)\
                f"Expectancy:  {format_pnl(expectancy)} per trade\n"
                f"P&L:         {format_pnl(pnl)}"
            )

        ind_block = fmt_block("Indicator", "📡", ind_trades)
        man_block = fmt_block("Manual",    "🖐", man_trades)
```

```
        verdict = ""
        if ind_trades and man_trades:
            ind_wr  = len([p for p in ind_trades if p["realizedPnl"] > 0]) / len(
            man_wr  = len([p for p in man_trades if p["realizedPnl"] > 0]) / len(
            ind_wins_list  = [p["realizedPnl"] for p in ind_trades if p["realized
            ind_loss_list  = [p["realizedPnl"] for p in ind_trades if p["realized
            man_wins_list  = [p["realizedPnl"] for p in man_trades if p["realized
            man_loss_list  = [p["realizedPnl"] for p in man_trades if p["realized
            ind_exp = (ind_wr/100 * (sum(ind_wins_list)  / max(len(ind_wins_list)
                      ((1 - ind_wr/100) * (sum(ind_loss_list) / max(len(ind_loss_
            man_exp = (man_wr/100 * (sum(man_wins_list)  / max(len(man_wins_list)
                      ((1 - man_wr/100) * (sum(man_loss_list) / max(len(man_loss_
            wr_winner  = "📡 Indicator" if ind_wr  > man_wr  else ("🖐 Manual" if
            exp_winner = "📡 Indicator" if ind_exp > man_exp else ("🖐 Manual" if
            verdict = (
                f"\n🏆 WR:         {wr_winner} ({ind_wr:.1f}% vs {man_wr:.1f}%)\n
                f"🏆 Expectancy: {exp_winner} ({format_pnl(ind_exp)} vs {format_p
            )

        send_telegram(
            f"⚖️ <b>Manual vs Indicator — today</b>\n\n"
            f"{ind_block}\n\n"
            f"{man_block}"
            f"{verdict}"
        )
    except Exception as e:
        send_telegram(f"❌ Compare error: {e}")


def fill_entry2_at_market() -> None:
    global entry2_placed_at
    if not EXECUTION_ENABLED:
        send_telegram("⚠️ Execution is disabled.")
        return
    try:
        entry2_id  = None
        entry2_qty = None
        for oid, meta in list(tracked_orders.items()):
            if meta["label"] == "Entry2":
                entry2_id  = int(oid)
                entry2_qty = meta["qty"]
                break

        if not entry2_id:
            send_telegram("ℹ️ No Entry2 limit order found to fill.")
            return
```

```
now_ms = int(time.time() * 1000)
params = {"symbol": SYMBOL, "orderId": entry2_id, "timestamp": now_ms}
params = sign_request(params)
headers = {"X-MBX-APIKEY": BINANCE_API_KEY}
resp = requests.delete(f"{FUTURES_BASE}/fapi/v1/order",
                       params=params, headers=headers, timeout=10)
resp.raise_for_status()
tracked_orders.pop(str(entry2_id), None)
entry2_placed_at = 0.0

pos = get_open_position()
if not pos:
    send_telegram("⚠️ Entry2 cancelled but no open position found — not f
    return

amt       = float(pos["positionAmt"])
side      = "BUY" if amt > 0 else "SELL"
sym_info  = get_symbol_info()
qty_step  = sym_info["qty_step"]
fill_qty  = round_step(entry2_qty, qty_step)
mark      = get_mark_price()

place_order(side, "MARKET", fill_qty)
time.sleep(2)  # let position update

sl_line      = ""
tp_grow_line = ""
max_lines    = ""
try:
    pos2 = get_open_position()
    if pos2 and current_trade_entry:
        avg_entry  = float(pos2.get("entryPrice", 0))
        direction  = current_trade_entry.get("direction")
        tick       = sym_info["price_tick"]
        step       = qty_step
        stop_pct   = STOP_PCT   # use original SL % from avg entry, not t
        if avg_entry > 0 and direction:
            if direction == "long":
                new_sl = round_step(avg_entry * (1 - stop_pct / 100), tic
            else:
                new_sl = round_step(avg_entry * (1 + stop_pct / 100), tic
            current_trade_entry["sl_price"] = new_sl
            current_trade_entry["price"]    = avg_entry
            sl_line = f"\nSL recalculated: ${new_sl:,.2f} (avg entry ${av
```

```
tp2_price_existing = current_trade_entry.get("tp2_price")
tp1_price_existing = current_trade_entry.get("tp1_price")
close_side = "SELL" if direction == "long" else "BUY"
grown = False
for oid, meta in list(tracked_orders.items()):
    if meta["label"] == "TP2" and tp2_price_existing:
        try:
            now_ms2  = int(time.time() * 1000)
            params2  = {"symbol": SYMBOL, "orderId": int(oid)
            params2  = sign_request(params2)
            requests.delete(f"{FUTURES_BASE}/fapi/v1/order",
                            params=params2, headers=headers,
            tracked_orders.pop(oid, None)
            new_tp2_qty = round_step(current_trade_entry["qty
            new_resp = place_order(close_side, "LIMIT", new_t
                                   limit_price=tp2_price_exi
            register_order(new_resp, "TP2")
            current_trade_entry["qty_tp2"] = new_tp2_qty
            tp_grow_line = f"\nTP2 grown to cover Entry2: {ne
            grown = True
        except Exception as e3:
            tp_grow_line = f"\n⚠️ Could not grow TP2: {e3}"
        break
if not grown and tp1_price_existing:
    for oid, meta in list(tracked_orders.items()):
        if meta["label"] == "TP1":
            try:
                now_ms2  = int(time.time() * 1000)
                params2  = {"symbol": SYMBOL, "orderId": int(
                params2  = sign_request(params2)
                requests.delete(f"{FUTURES_BASE}/fapi/v1/orde
                                params=params2, headers=heade
                tracked_orders.pop(oid, None)
                new_tp1_qty = round_step(current_trade_entry[
                new_resp = place_order(close_side, "LIMIT", n
                                       limit_price=tp1_price
                register_order(new_resp, "TP1")
                current_trade_entry["qty_tp1"] = new_tp1_qty
                tp_grow_line = f"\nTP1 grown to cover Entry2:
            except Exception as e3:
                tp_grow_line = f"\n⚠️ Could not grow TP1: {e3
            break

total_qty_final = abs(float(pos2["positionAmt"]))
max_loss_usd = total_qty_final * abs(avg_entry - new_sl)
max_win_usd  = current_trade_entry["qty_tp1"] * abs(tp1_price
```

```
                    if current_trade_entry.get("qty_tp2"):
                        max_win_usd += current_trade_entry["qty_tp2"] * abs(tp2_p
                    max_lines = (
                        f"\n\n🛑 New max loss: {format_pnl(-max_loss_usd)}"
                        f"\n💰 New max win:  {format_pnl(max_win_usd)}"
                    )
            except Exception as e:
                print(f"[Fill2] SL recalc error: {e}")

            total_usd = (abs(amt) + fill_qty) * mark / LEVERAGE
            send_telegram(
                f"✅ <b>Entry2 filled at market</b>\n\n"
                f"Price:          ~${mark:,.2f}\n"
                f"Total margin:   ${total_usd:,.2f}"
                f"{sl_line}"
                f"{tp_grow_line}"
                f"{max_lines}"
            )
        except requests.exceptions.HTTPError as e:
            try:
                err = e.response.json()
                send_telegram(f"❌ Fill2 failed\nCode {err.get('code')}: {err.get('ms
            except Exception:
                send_telegram(f"❌ Fill2 failed: {e}")
        except Exception as e:
            send_telegram(f"❌ Fill2 error: {e}")


def cancel_entry2() -> None:
    global entry2_placed_at
    if not EXECUTION_ENABLED:
        send_telegram("⚠️ Execution is disabled.")
        return
    try:
        entry2_id = None
        for oid, meta in list(tracked_orders.items()):
            if meta["label"] == "Entry2":
                entry2_id = int(oid)
                break

        if not entry2_id:
            send_telegram("ℹ️ No Entry2 limit order found.")
            return

        now_ms  = int(time.time() * 1000)
        params  = {"symbol": SYMBOL, "orderId": entry2_id, "timestamp": now_ms}
```

```
        params  = sign_request(params)
        headers = {"X-MBX-APIKEY": BINANCE_API_KEY}
        resp    = requests.delete(f"{FUTURES_BASE}/fapi/v1/order",
                                  params=params, headers=headers, timeout=10)
        resp.raise_for_status()
        tracked_orders.pop(str(entry2_id), None)
        entry2_placed_at = 0.0
        send_telegram("✅ Entry2 limit order cancelled.")
    except requests.exceptions.HTTPError as e:
        try:
            err = e.response.json()
            send_telegram(f"❌ Cancel2 failed\nCode {err.get('code')}: {err.get('
        except Exception:
            send_telegram(f"❌ Cancel2 failed: {e}")
    except Exception as e:
        send_telegram(f"❌ Cancel2 error: {e}")


tg_last_update_id = 0


def poll_telegram_commands():
    global tg_last_update_id, daily_bias, pending_trigger_id, pending_signal, mak
    print("[Telegram] Command polling started.")

    while True:
        try:
            url    = f"https://api.telegram.org/bot{TELEGRAM_BOT_TOKEN}/getUpdate
            params = {"timeout": 30, "offset": tg_last_update_id + 1, "allowed_up
            resp   = requests.get(url, params=params, timeout=40)
            resp.raise_for_status()
            updates = resp.json().get("result", [])

            for update in updates:
                tg_last_update_id = update["update_id"]

                # ── Signal prompt expiry check ──────────────────────────
                if pending_signal and time.time() - pending_signal["timestamp"] >
                    msg_id = pending_signal.get("message_id")
                    label  = pending_signal["label"]
                    if msg_id:
                        edit_prompt_message(msg_id, f"⏱️ <b>{label}</b>\n\nSignal
                    pending_signal = None

                # ── Handle inline button callbacks ──────────────────────
                callback = update.get("callback_query", {})
```

```
            if callback:
                cb_id   = callback["id"]
                cb_data = callback.get("data", "")
                cb_chat = str(callback.get("message", {}).get("chat", {}).get
                msg_id  = callback.get("message", {}).get("message_id")

                if cb_chat != str(TELEGRAM_CHAT_ID):
                    answer_callback(cb_id)
                    continue

                if cb_data.startswith("take_") and pending_signal:
                    direction = pending_signal["direction"]
                    label     = pending_signal["label"]
                    answer_callback(cb_id, "✅ Taking trade...")
                    edit_prompt_message(msg_id, f"✅ <b>{label}</b>\n\nTrade
                    pending_signal = None
                    threading.Thread(
                        target=execute_signal_trade,
                        args=(direction, label),
                        daemon=True
                    ).start()
                elif cb_data.startswith("skip_") and pending_signal:
                    label = pending_signal["label"]
                    db_log_warning("signal_skipped", f"Skipped: {label}")
                    answer_callback(cb_id, "❌ Skipped.")
                    edit_prompt_message(msg_id, f"❌ <b>{label}</b>\n\nSkippe
                    pending_signal = None

                elif cb_data.startswith("reset_bias_") and pending_bias_reset
                    parts       = cb_data.split("_")
                    new_bias    = parts[2]   # bullish or bearish
                    signal_key  = "_".join(parts[3:])
                    msg_id      = pending_bias_reset["message_id"]
                    pending_bias_reset = None

                    daily_bias = new_bias
                    answer_callback(cb_id, f"✅ Bias reset to {new_bias.upper
                    edit_prompt_message(msg_id,
                        f"✅ <b>Bias reset to {new_bias.upper()}</b>\n\n"
                        f"HTF change confirmed. Trading with new bias.\n\n"
                        f"Triggered by: {signal_key}"
                    )
                    send_telegram(f"📊 Daily bias reset to <b>{new_bias.upper
                                  f"Triggered by structure break: {signal_key
                    db_log_htf_bias_change(new_bias, signal_key, "manual_rese
```

```
        elif cb_data.startswith("ignore_reset_") and pending_bias_res
            signal_key  = cb_data.split("ignore_reset_")[1]
            msg_id      = pending_bias_reset["message_id"]
            pending_bias_reset = None

            answer_callback(cb_id, "Ignored.")
            edit_prompt_message(msg_id,
                f"❌ Bias change ignored.\n\n"
                f"Staying with current bias: {daily_bias.upper()}"
            )
            db_log_htf_bias_change(daily_bias, signal_key, "ignored")

        else:
            answer_callback(cb_id, "No active signal.")
        continue

    msg = update.get("message", {})

    chat_id = str(msg.get("chat", {}).get("id", ""))
    if chat_id != str(TELEGRAM_CHAT_ID):
        continue

    text = msg.get("text", "").strip()

    parts = text.lower().split()
    if not parts or not parts[0].startswith("/"):
        continue

    cmd   = parts[0].lstrip("/").split("@")[0]

    size_usdt = None
    size_pct  = None
    stop_pct  = None
    tp_pct    = None

    raw_args = parts[1:]
    if raw_args:
        size_raw = raw_args[0]
        if size_raw.endswith("%"):
            try: size_pct = float(size_raw[:-1])
            except ValueError: pass
        else:
            try: size_usdt = float(size_raw)
            except ValueError: pass
        if len(raw_args) >= 3:
            try: stop_pct = float(raw_args[1])
```

```
            except ValueError: pass
            try: tp_pct   = float(raw_args[2])
            except ValueError: pass

    print(f"[Telegram CMD] /{cmd} size_usdt={size_usdt} size_pct={siz

    if cmd == "close":
        if raw_args and raw_args[0].replace(".", "").isdigit():
            partial_close(float(raw_args[0]))
        else:
            close_position_now()
    elif cmd == "long":
        # Manual /long should execute the same way it historically di
        # The newer grade/pink-candle flow was intercepting the comma
        # before execute_trade(), so /long could appear to do nothing
        # (or wait for a confirmation flow that was never completed).
        send_telegram("📈 <b>/LONG received</b> — executing trade che
        threading.Thread(
            target=execute_trade,
            kwargs=dict(
                direction="long",
                size_pct=size_pct,
                size_usdt=size_usdt,
                stop_pct=stop_pct,
                tp_pct=tp_pct,
                triggered_by="manual_long",
            ),
            daemon=True,
        ).start()
    elif cmd == "short":
        # Same restoration for /short. All existing execution guards
        # inside execute_trade() remain active.
        send_telegram("📉 <b>/SHORT received</b> — executing trade ch
        threading.Thread(
            target=execute_trade,
            kwargs=dict(
                direction="short",
                size_pct=size_pct,
                size_usdt=size_usdt,
                stop_pct=stop_pct,
                tp_pct=tp_pct,
                triggered_by="manual_short",
            ),
            daemon=True,
        ).start()
    elif cmd == "cut":
```

```
            partial_close(50)
        elif cmd == "mirroropen":
            if raw_args:
                mirror_open_one(raw_args[0].strip())
            else:
                send_telegram("❌ Usage: /mirroropen 3  (client id)")
        elif cmd == "sl":
            if raw_args:
                try:
                    arg = raw_args[0].replace("%", "")
                    pct_val = float(arg)
                    pos = get_open_position()
                    if not pos:
                        send_telegram("❌ No open position.")
                    else:
                        entry  = float(pos.get("entryPrice", 0))
                        amt    = float(pos["positionAmt"])
                        tick   = get_symbol_info()["price_tick"]
                        new_sl = round_step(entry * (1 - pct_val / 100),
                                 else round_step(entry * (1 + pct_val / 1
                        adjust_sl(new_sl)
                except ValueError:
                    send_telegram("❌ Usage: /sl 0.36  (percentage from e
            else:
                send_telegram("❌ Usage: /sl 0.36  (percentage from entry
        elif cmd == "tp":
            if raw_args:
                try:
                    pct_val = float(raw_args[0].replace("%", ""))
                    pos = get_open_position()
                    if not pos:
                        send_telegram("❌ No open position.")
                    else:
                        entry  = float(pos.get("entryPrice", 0))
                        amt    = float(pos["positionAmt"])
                        tick   = get_symbol_info()["price_tick"]
                        new_tp = round_step(entry * (1 + pct_val / 100),
                                 else round_step(entry * (1 - pct_val / 1
                        adjust_tp(new_tp, label="TP1")
                except ValueError:
                    send_telegram("❌ Usage: /tp 0.36  (percentage from e
            else:
                send_telegram("❌ Usage: /tp 0.36  (percentage from entry
        elif cmd == "tp2":
            if raw_args:
                try:
```

```
                pct_val = float(raw_args[0].replace("%", ""))
                pos = get_open_position()
                if not pos:
                    send_telegram("❌ No open position.")
                else:
                    entry   = float(pos.get("entryPrice", 0))
                    amt     = float(pos["positionAmt"])
                    tick    = get_symbol_info()["price_tick"]
                    new_tp2 = round_step(entry * (1 + pct_val / 100),
                              else round_step(entry * (1 - pct_val /
                    adjust_tp(new_tp2, label="TP2")
            except ValueError:
                send_telegram("❌ Usage: /tp2 0.50  (percentage from
        else:
            send_telegram("❌ Usage: /tp2 0.50  (percentage from entr
    elif cmd == "breakeven":
        move_to_breakeven()
    elif cmd in ("hl_debug", "hl2_debug", "clientdebug"):
        # hl_debug / hl2_debug kept as muscle-memory aliases for
        # clients #1/#2; clientdebug <id> works for any client,
        # e.g. /clientdebug 3
        if cmd == "hl_debug":
            debug_id = "1"
        elif cmd == "hl2_debug":
            debug_id = "2"
        else:
            debug_id = parts[1] if len(parts) > 1 else None
        target = next((c for c in CLIENTS if c.id == debug_id), None)
        if not target:
            ids = ", ".join(c.id for c in CLIENTS) or "none configure
            send_telegram(f"Usage: /clientdebug <id>\nConfigured clie
        elif not target.is_configured():
            send_telegram(f"⚠️ {target.label()} not configured — priv
        else:
            try:
                info    = hl_get_info()
                state   = info.user_state(target.address)
                balance = client_get_balance(target)
                mark    = hl_get_mark_price()
                size    = client_calculate_size(target)
                send_telegram(
                    f"<b>{target.label()} Debug</b>\n\n"
                    f"Address: {target.address}\n"
                    f"Balance: ${balance:.4f}\n"
                    f"Mark: ${mark:,.2f}\n"
                    f"Calc size: {size} BTC\n\n"
```

```
                    f"Raw state: {state}"
                )
            except Exception as e:
                send_telegram(f"{target.label()} Debug error: {e}")
    elif cmd == "blocked":
        out = ["🚫 <b>Blocked staircase states</b>\n<i>4H / 30m / 5m
        for side in ("short", "long"):
            rows = [b for b in _STAIR_BLOCKED if b[0] == side]
            conds = [b for b in _STAIR_BLOCKED_IF if b[0] == side]
            out.append(f"\n<b>{side.upper()}S blocked ({len(rows) + l
            for b in conds:
                out.append(f"• If {b[1]} — {b[2]} <i>({b[3]})</i>")
            for b in rows:
                out.append(f"• {' / '.join('any' if x == '*' else x f
            for b in _STAIR_BLOCKED_FROM:
                if b[0] == side:
                    out.append(f"• {b[1]} / {b[2]} / {b[4]} (5m after
        send_telegram("\n".join(out))
    elif cmd == "stats":
        pnl_summary()
    elif cmd == "patterns":
        breakdown = compute_trigger_breakdown()
        if not breakdown:
            send_telegram("📊 No trigger data in the database yet.")
        else:
            def fmt_mins(m):
                if m is None: return "—"
                if m >= 60: return f"{int(m//60)}h {int(m%60)}m"
                return f"{int(m)}m"

            lines = ["📊 <b>Trigger performance</b>\n<i>Bot-executed
            for t in breakdown:
                gap_str = fmt_mins(t["avg_gap_minutes"])
                if t["total"] > 0:
                    pnl_str = f"+${t['pnl']:.2f}" if t["pnl"] >= 0 el
                    lines.append(f"<b>{t['label']}</b>  {t['wins']}W
                else:
                    lines.append(f"<b>{t['label']}</b>  No trades yet
            send_telegram("\n\n".join(lines))
    elif cmd == "wsstatus":
        healthy = _ws_is_healthy()
        seconds_since_msg = time.time() - _ws_last_message_at if _ws_
        lines = [
            f"🔌 <b>Websocket Status</b>\n",
            f"Connected: {'✅ Yes' if _ws_connected else '❌ No'}",
            f"Healthy: {'✅ Yes' if healthy else '❌ No (stale or dis
```

```
        ]
        if seconds_since_msg is not None:
            lines.append(f"Last message: {seconds_since_msg:.0f}s ago
        else:
            lines.append("Last message: never received")
        lines.append("")
        lines.append("<b>Kline history per interval:</b>")
        for interval in WS_KLINE_INTERVALS:
            count = len(_ws_klines.get(interval, []))
            lines.append(f"  {interval}: {count} candles")
        mark_age = time.time() - _ws_mark_price["updated_at"] if _ws_
        if mark_age is not None:
            lines.append(f"\nMark price: ${_ws_mark_price['value']:,.
        else:
            lines.append("\nMark price: not yet received")
        if not healthy:
            lines.append("\n⚠️ Falling back to REST for all live data
        send_telegram("\n".join(lines))
    elif cmd == "live":
        now_uk_live = datetime.now(UK_TZ)
        reasons = []

        if now_uk_live.weekday() >= 5:
            reasons.append(f"📅 Weekend ({now_uk_live.strftime('%A')}

        if now_uk_live.weekday() == 4 and now_uk_live.time() >= dt_ti
            reasons.append("🕖 Friday cutoff (19:00+) — no new trades

        if COOLDOWN_START <= now_uk_live.time() < COOLDOWN_END:
            reasons.append(f"🇺🇸 US open cooldown — no prompts {COOLDC

        if now_uk_live.time() >= NIGHT_BLOCK_START:
            reasons.append(f"🌙 After {NIGHT_BLOCK_START.strftime('%H

        if elevated_silence_until > time.time():
            remaining_s = int(elevated_silence_until - time.time())
            reasons.append(f"⏸️ Prompts paused — {remaining_s // 60}m

        if daily_losses_block:
            reasons.append(f"🚫 {DAILY_LOSS_COUNT_LIMIT} losses today

        if stopped_today:
            reasons.append("🚫 Daily trade cap or $ loss limit hit —

        if reasons:
            send_telegram(
```

```
                "🔕 <b>Prompts are currently OFF.</b>\n\n" + "\n".joi
            )
        else:
            send_telegram(
                "🟢 <b>Prompts are live.</b>\n\n"
                "No active blocks — signals will prompt you normally
            )
    elif cmd == "winrate":
        WIN_RATE_TARGET = 60.0
        if not DATABASE_URL:
            send_telegram("📊 No database connected.")
        else:
            try:
                conn = get_db()
                cur  = conn.cursor(cursor_factory=psycopg2.extras.Rea
                windows = [("Today", "1 day"), ("7d", "7 days"), ("30
                lines = [f"🎯 <b>Win Rate — target {WIN_RATE_TARGET:.
                for label, interval in windows:
                    cur.execute("""
                        SELECT
                            COUNT(*) FILTER (WHERE outcome = 'win')
                            COUNT(*) FILTER (WHERE outcome = 'loss')
                        FROM trigger_performance
                        WHERE entry_time >= NOW() - INTERVAL %s
                          AND outcome IN ('win', 'loss')
                    """, (interval,))
                    r = cur.fetchone()
                    total = (r["wins"] or 0) + (r["losses"] or 0)
                    if total > 0:
                        wr = r["wins"] / total * 100
                        emoji = "✅" if wr >= WIN_RATE_TARGET else "
                        gap = wr - WIN_RATE_TARGET
                        gap_line = f"(+{gap:.0f}pts above target)" if
                        lines.append(f"{emoji} <b>{label}:</b> {wr:.0
                    else:
                        lines.append(f"— <b>{label}:</b> no closed tr
                cur.close()
                conn.close()
                send_telegram("\n".join(lines))
            except Exception as e:
                send_telegram(f"❌ Win rate report error: {e}")
    elif cmd == "compound":
        progress = get_compounding_progress()
        if not progress or "week_multiplier" not in progress:
            send_telegram("📊 Could not calculate compounding progres
        else:
```

```
            week_mult   = progress["week_multiplier"]
            week_target = WEEKLY_COMPOUND_TARGET
            week_emoji  = "✅" if week_mult >= week_target else "📈"
            week_pct    = progress["week_pct_of_target"]

            lines = [
                f"📊 <b>Compounding Progress</b>\n",
                f"{week_emoji} <b>This week:</b> {week_mult:.4f}x  (t
                f"  Start: ${progress['week_start_balance']:,.2f}  →
                f"  Target balance: ${progress['week_target_balance']
                f"  Progress to target: {week_pct:.1f}%"
            ]

            if "cycle_multiplier" in progress:
                cycle_mult   = progress["cycle_multiplier"]
                cycle_target = CYCLE_COMPOUND_TARGET
                cycle_emoji  = "✅" if cycle_mult >= cycle_target els
                cycle_pct    = progress["cycle_pct_of_target"]
                lines.append(
                    f"\n{cycle_emoji} <b>This 4-week cycle:</b> {cycl
                    f"  Cycle start: ${progress['cycle_start_balance'
                    f"  Target balance: ${progress['cycle_target_bala
                    f"  Progress to target: {cycle_pct:.1f}%"
                )

            send_telegram("\n".join(lines))
        elif cmd == "vwinrate":
            if not DATABASE_URL:
                send_telegram("📊 No database connected.")
            else:
                try:
                    days = 30
                    if len(parts) > 1:
                        try:
                            days = int(parts[1])
                        except (ValueError, IndexError):
                            pass
                    conn = get_db()
                    cur  = conn.cursor(cursor_factory=psycopg2.extras.Rea
                    cur.execute("""
                        SELECT
                            trigger,
                            COUNT(*) FILTER (WHERE outcome IN ('win','los
                            COUNT(*) FILTER (WHERE outcome = 'win') as wi
                            COUNT(*) FILTER (WHERE outcome = 'loss') as l
                            COUNT(*) FILTER (WHERE outcome IS NULL) as st
```

```
                    FROM virtual_trigger_trades
                    WHERE opened_at >= NOW() - INTERVAL '%s days'
                    GROUP BY trigger
                    ORDER BY (COUNT(*) FILTER (WHERE outcome = 'win')
                              NULLIF(COUNT(*) FILTER (WHERE outcome I
                """, (days,))
                rows = cur.fetchall()
                cur.close()
                conn.close()

                if not rows:
                    send_telegram(f"📊 No virtual trigger trades in t
                else:
                    TRIGGER_LABELS = {
                        "long": "📈 Long", "3_bulls": "🐂 3 Bulls",
                        "strength": "💪 Strength", "3_bar_buy": "📈 3
                        "3_bears": "🐻 3 Bears", "weakness": "📉 Weak
                        "bearish": "🔴 Bearish", "short": "📉 Short",
                        "green_structure_break": "🟢 Green Structure
                        "orange_structure_break": "🟠 Orange Structur


                        "30m_composite_crosses_up_30": "📈 30m Compos
                        "30m_composite_crosses_down_70": "📉 30m Comp
                    }
                    lines = [
                        f"🎯 <b>Virtual trigger win rates — last {day
                        f"<i>Simulated, not real trades — fixed {VIRT
                        f"applied to every signal fire, regardless of
                        f"bot or manual</i>\n"
                    ]
                    for r in rows:
                        label = TRIGGER_LABELS.get(r["trigger"], r["t
                        total = r["total"] or 0
                        if total > 0:
                            wr = r["wins"] / total * 100
                            open_note = f"  ({r['still_open']} still
                            lines.append(
                                f"<b>{label}</b>: {wr:.0f}% WR  ({r['
                            )
                        elif r["still_open"]:
                            lines.append(f"<b>{label}</b>: {r['still_
                    send_telegram("\n\n".join(lines))
            except Exception as e:
                send_telegram(f"❌ Virtual win rate report error: {e}
    elif cmd == "discipline":
```

```
        week_comp  = get_discipline_comparison(7)
        month_comp = get_discipline_comparison(30)
        week_lines  = format_discipline_comparison_lines(week_comp)
        month_lines = format_discipline_comparison_lines(month_comp)
        lines = ["📊 <b>Discipline Trend</b>\n"]
        if week_lines:
            lines.append("<b>This week vs last week:</b>\n" + "\n".jo
        if month_lines:
            lines.append("\n<b>This month vs last month:</b>\n" + "\n
        if not week_lines and not month_lines:
            lines.append("Not enough data yet.")
        send_telegram("\n".join(lines))
    elif cmd == "guards":
        try:
            testing = is_testing_mode()
            now_g   = datetime.now(UK_TZ)
            t_g     = now_g.time()

            def _state(enabled=True, testing_gated=True):
                if not enabled:
                    return "⚪ disabled"
                if testing_gated and testing:
                    return "⚠️ bypassed (testing mode)"
                return "🟢 enforced"

            def _blocking(cond):
                return "  ← BLOCKING NOW" if cond else ""

            trades_g = get_futures_trades_today()
            st_g     = build_stats(trades_g)
            cap_used = st_g["closed_positions"]
            net_g    = st_g["net_pnl"]
            loss_g   = st_g["losses"]

            lines = [f"🛡️ <b>Guards</b> — testing mode {'ON ⚠️' if te

            lines.append("<b>Always enforced</b>")
            lines.append(f"Friday cutoff — after 19:00{_blocking(now_
            lines.append(f"Curfew — after {NIGHT_BLOCK_START.strftime
            lines.append(f"US-open cooldown — {COOLDOWN_START.strftim
            lines.append(f"Trade cap — {cap_used}/{DAILY_TRADE_CAP}{_
            lines.append(f"$ loss limit — net {format_pnl(net_g)} / s
            lines.append(f"Loss count — {loss_g}/{DAILY_LOSS_COUNT_LI
            lines.append(f"Revenge window — {REVENGE_WINDOW_MINS}m af
            if stopped_today or daily_losses_block:
                lines.append("🛑 <b>Stopped for the day</b> — flag is
```

```
            lines.append("\n<b>Signal-quality guards</b>")
            lines.append(f"Same-side cooldown — {SAME_SIDE_LIMIT} los
            lines.append(f"Daily bias filter: {_state(BIAS_FILTER_ENA

            ema_txt = ""
            if EMA_GUARD_ENABLED or SIMPLE_EMA_GUARD_ENABLED:
                try:
                    e120 = get_live_ema120_guard()
                    mk   = get_mark_price()
                    if e120 is not None:
                        ema_txt = f" (price ${mk:,.0f} vs 120 EMA ${e
                except Exception:
                    ema_txt = " (live value unavailable)"
            lines.append(f"EMA guard: {_state(EMA_GUARD_ENABLED)}{ema
            lines.append(f"Short-only EMA guard: {_state(SIMPLE_EMA_G

            send_telegram("\n".join(lines))
        except Exception as e:
            send_telegram(f"❌ Guards error: {e}")
    elif cmd == "track":
        try:
            send_telegram(format_track_message())
        except Exception as e:
            send_telegram(f"Track record error: {e}")
    elif cmd == "config":
        try:
            import re as _re
            try:
                _src = open(__file__, encoding="utf-8").read()
            except Exception:
                _src = ""

            def _code_default(env_name):
                m = _re.search(r'environ\.get\(\s*"%s"\s*,\s*"([^"]*)
                return m.group(1) if m else None

            def _fmt(v):
                if isinstance(v, bool):
                    return "on" if v else "off"
                if isinstance(v, float):
                    return f"{v:g}"
                if isinstance(v, dt_time):
                    return v.strftime("%H:%M")
                if isinstance(v, (list, tuple)):
                    items = [str(x) for x in v if str(x).strip()]
```

```
            return ", ".join(items) if items else "none"
        return str(v)

    def _norm(x):
        x = str(x).strip().lower()
        try:
            return f"{float(x):g}"
        except ValueError:
            return x

    def _line(name, pyname=None, label=None):
        pyname = pyname or name
        if pyname not in globals():
            return None
        val = globals()[pyname]
        raw = os.environ.get(name)
        mark = ""
        if raw is not None and raw != "":
            dflt = _code_default(name)
            if dflt is None or _norm(dflt) != _norm(raw):
                mark = "  ⚠️ Railway" + (f" (code default {df
        return f"{label or name}: {_fmt(val)}{mark}"

    groups = [
        ("Daily limits", [
            ("DAILY_TRADE_CAP",), ("FIRST_LOSS_TRADE_CAP",),
            ("DAILY_LOSS_LIMIT",), ("DAILY_LOSS_WARN_PCT",),
            ("REVENGE_WINDOW_MINS",), ("SAME_SIDE_LIMIT",), (
            ("FEE_ALERT_THRESHOLD",), ("OVERTRADE_GAP_MINS",)
            ("DAILY_PNL_TARGET",), ("WEEKLY_PNL_TARGET",)]),
        ("Time rules", [
            ("NIGHT_BLOCK_START",), ("COOLDOWN_START",), ("CO
            ("TRADING_WINDOW_START_HOUR",), ("TRADING_WINDOW_
        ("Trade defaults", [
            ("LEVERAGE",), ("TRADE_SIZE_PCT",), ("STOP_PCT",)
            ("TP1_CLOSE_PCT",), ("TP1_SL_MOVE_PCT",), ("ENTRY
            ("ENTRY2_SL_PCT",), ("ENTRY2_EXPIRY_HOURS",), ("M
            ("MAKER_TIMEOUT_SEC",), ("TRADE_EXECUTION_PACING_
        ("Protection", [
            ("BREAKEVEN_SUGGEST_PCT",), ("BREAKEVEN_TIME_MINS
            ("RETRACE_LOCK_IN_PCT",), ("OVERNIGHT_BE_MIN_PROF
            ("UNDERWATER_ALERT_MIN_AGE_MINS",), ("PROFIT_LOCK
        ("Guard switches and thresholds", [
            ("BIAS_FILTER_ENABLED",), ("EMA_GUARD_ENABLED",),
            ("WPR_SHORT_CROSS_ENABLED",), ("WPR_LONG_CROSS_EN
        ("System", [
```

```
                    ("EXECUTION_ENABLED",), ("POLL_INTERVAL",), ("SLO
                    ("ORDER_FILL_POLL_INTERVAL",), ("CRM_SYNC_INTERVA
                    ("AUTO_TRADE_SIGNALS",)]),
                ]

                chunks, cur = [], "⚙️ <b>Config — live values</b>\n⚠️ = F
                overridden = 0
                for title, items in groups:
                    body = []
                    if title == "System":
                        body.append(f"TESTING MODE: {'ON ⚠️' if is_testin
                    for (n,) in items:
                        ln = _line(n)
                        if ln:
                            body.append(ln)
                            if "⚠️ Railway" in ln:
                                overridden += 1
                    sect = f"\n<b>{title}</b>\n" + "\n".join(body) + "\n"
                    if len(cur) + len(sect) > 3500:
                        chunks.append(cur); cur = ""
                    cur += sect
                cur += f"\n{overridden} setting(s) overridden by Railway.
                chunks.append(cur)
                for ch in chunks:
                    send_telegram(ch)
            except Exception as e:
                send_telegram(f"❌ Config error: {e}")
        elif cmd == "unmatched":
            if not DATABASE_URL:
                send_telegram("📊 No database connected.")
            else:
                try:
                    hours = 168  # default 7 days — this is a "did I miss
                    if len(parts) > 1:
                        try:
                            hours = int(parts[1])
                        except (ValueError, IndexError):
                            pass
                    conn = get_db()
                    cur  = conn.cursor(cursor_factory=psycopg2.extras.Rea
                    cur.execute("""
                        SELECT message, logged_at
                        FROM warnings_log
                        WHERE category = 'unrecognized_signal'
                          AND logged_at >= NOW() - INTERVAL '%s hours'
                        ORDER BY logged_at DESC
```

```
                    LIMIT 20
                """, (hours,))
                rows = cur.fetchall()
                cur.close()
                conn.close()
                if not rows:
                    send_telegram(f"✅ No unrecognized signals in the
                else:
                    lines = [f"❓ <b>Unrecognized signals — last {hou
                    for r in rows:
                        ts = r["logged_at"].strftime("%d %b %H:%M")
                        lines.append(f"<b>{ts}</b> — {r['message']}")
                    lines.append("\n👉 These fired from TradingView b
                    send_telegram("\n\n".join(lines))
            except Exception as e:
                send_telegram(f"❌ Unmatched signals report error: {e
    elif cmd == "warnings":
        if not DATABASE_URL:
            send_telegram("📊 No database connected.")
        else:
            try:
                hours = 24
                if len(parts) > 1:
                    try:
                        hours = int(parts[1])
                    except (ValueError, IndexError):
                        pass
                conn = get_db()
                cur  = conn.cursor(cursor_factory=psycopg2.extras.Rea
                cur.execute("""
                    SELECT category, message, logged_at
                    FROM warnings_log
                    WHERE logged_at >= NOW() - INTERVAL '%s hours'
                    ORDER BY logged_at DESC
                    LIMIT 30
                """, (hours,))
                rows = cur.fetchall()
                cur.close()
                conn.close()
                if not rows:
                    send_telegram(f"✅ No warnings in the last {hours
                else:
                    CATEGORY_EMOJI = {
                        "counter_signal":    "⚠️",
                        "loss_streak":       "🔴",
                        "daily_loss_block":  "🚫",
```

```
                        "revenge_trade":     "😤",
                        "retrace_protect":   "🛡️",
                        "fee_threshold":      "💸",
                        "first_loss_tighten": "🎯",
                        "daily_loss_limit":   "🚫",
                        "brute_force_close":  "🛡️",
                        "hard_structure_invalidate": "🚨",
                        "counter_signal_close": "🔄",
                        "ema_guard_block": "📐",
                        "simple_ema_guard_block": "📏",
                        "overnight_auto_breakeven": "😴",
                        "aggressive_volume": "📊",
                        "underwater_ratio": "⏱️",
                        "wpr_guard_block": "📐",
                        "time_in_trade_warning": "⏱️",
                        "htf_extreme_block": "🌡️",
                        "unrecognized_signal": "❓",
                    }
                    lines = [f"📋 <b>Warnings — last {hours}h</b> ({l
                    for r in rows:
                        emoji = CATEGORY_EMOJI.get(r["category"], "•"
                        ts    = r["logged_at"].strftime("%H:%M")
                        lines.append(f"{emoji} <b>{ts}</b> — {r['mess
                    send_telegram("\n".join(lines))
            except Exception as e:
                send_telegram(f"❌ Warnings report error: {e}")
    elif cmd == "analytics":
        if not DATABASE_URL:
            send_telegram("📊 No database connected.")
        else:
            try:
                days = 30
                if len(parts) > 1:
                    try:
                        days = int(parts[1])
                    except (ValueError, IndexError):
                        pass
                conn = get_db()
                cur  = conn.cursor(cursor_factory=psycopg2.extras.Rea
                sections = []

                # ── Section 1: Fee threshold breaches vs NY Open tra
                cur.execute("""
                    WITH fee_days AS (
                        SELECT DISTINCT logged_at::date as day
                        FROM warnings_log
```

```
            WHERE category = 'fee_threshold'
              AND logged_at >= NOW() - INTERVAL '%s days'
        ),
        ny_open_trades AS (
            SELECT DISTINCT entry_time::date as day
            FROM trigger_performance
            WHERE entry_time IS NOT NULL
              AND entry_time::time >= %s AND entry_time::
              AND entry_time >= NOW() - INTERVAL '%s days
        )
        SELECT
            (SELECT COUNT(*) FROM fee_days) as fee_day_co
            (SELECT COUNT(*) FROM fee_days WHERE day IN (
""", (days, COOLDOWN_START, COOLDOWN_END, days))
    r1 = cur.fetchone()
    if r1 and r1["fee_day_count"] and r1["fee_day_count"]
        pct = (r1["fee_and_ny_count"] / r1["fee_day_count
        sections.append(
            f"💸 <b>Fee threshold breaches</b>\n"
            f"  {r1['fee_day_count']} day(s) hit fee thre
            f"  {r1['fee_and_ny_count']}/{r1['fee_day_cou
            f"{COOLDOWN_START.strftime('%H:%M')}–{COOLDOW
        )
    else:
        sections.append("💸 <b>Fee threshold breaches</b>

    # ── Section 2: First trade of day = loss — what time
    cur.execute("""
        SELECT entry_time
        FROM trigger_performance
        WHERE id IN (
            SELECT DISTINCT ON (entry_time::date) id
            FROM trigger_performance
            WHERE entry_time IS NOT NULL AND outcome IN (
              AND entry_time >= NOW() - INTERVAL '%s days
            ORDER BY entry_time::date, entry_time ASC
        )
        AND outcome = 'loss'
    """, (days,))
    first_loss_rows = cur.fetchall()
    if first_loss_rows:
        hours_list = [r["entry_time"].hour for r in first
        avg_hour   = sum(hours_list) / len(hours_list)
        from collections import Counter
        hour_counts = Counter(hours_list)
        top_hour, top_count = hour_counts.most_common(1)[
```

```
    sections.append(
        f"🎯 <b>First trade of day = loss</b>\n"
        f"  Happened {len(first_loss_rows)} time(s) i
        f"  Avg time: {int(avg_hour):02d}:xx UK.\n"
        f"  Most common hour: {top_hour:02d}:xx ({top
    )
else:
    sections.append("🎯 <b>First trade of day = loss<

# ── Section 3: Loss streak warnings — time of day cl
cur.execute("""
    SELECT logged_at
    FROM warnings_log
    WHERE category = 'loss_streak'
      AND logged_at >= NOW() - INTERVAL '%s days'
    ORDER BY logged_at
""", (days,))
streak_rows = cur.fetchall()
if streak_rows:
    from collections import Counter
    streak_hours = Counter(r["logged_at"].hour for r
    top_streak_hour, top_streak_count = streak_hours.
    sections.append(
        f"🔴 <b>Loss streak warnings</b>\n"
        f"  Fired {len(streak_rows)} time(s) in the l
        f"  Most common hour: {top_streak_hour:02d}:x
    )
else:
    sections.append("🔴 <b>Loss streak warnings</b>\n

# ── Section 4: Revenge trades — clustering after los
cur.execute("""
    SELECT
        (SELECT COUNT(*) FROM warnings_log WHERE cate
        (SELECT COUNT(*) FROM warnings_log w1
         WHERE w1.category = 'revenge_trade'
           AND w1.logged_at >= NOW() - INTERVAL '%s d
           AND EXISTS (
               SELECT 1 FROM warnings_log w2
               WHERE w2.category = 'loss_streak'
                 AND w2.logged_at < w1.logged_at
                 AND w2.logged_at >= w1.logged_at - I
           )
        ) as revenge_after_streak
""", (days, days))
r4 = cur.fetchone()
```

```
if r4 and r4["revenge_count"] and r4["revenge_count"]
    pct4 = (r4["revenge_after_streak"] / r4["revenge_
    sections.append(
        f"😤 <b>Revenge trades</b>\n"
        f"  {r4['revenge_count']} flagged in the last
        f"  {r4['revenge_after_streak']}/{r4['revenge
    )
else:
    sections.append("😤 <b>Revenge trades</b>\n  None

# ── Section 5: Daily loss limit / block days — NY Op
cur.execute("""
    WITH block_days AS (
        SELECT DISTINCT logged_at::date as day
        FROM warnings_log
        WHERE category IN ('daily_loss_block', 'daily
          AND logged_at >= NOW() - INTERVAL '%s days'
    ),
    ny_open_trades AS (
        SELECT DISTINCT entry_time::date as day
        FROM trigger_performance
        WHERE entry_time IS NOT NULL
          AND entry_time::time >= %s AND entry_time::
          AND entry_time >= NOW() - INTERVAL '%s days
    )
    SELECT
        (SELECT COUNT(*) FROM block_days) as block_da
        (SELECT COUNT(*) FROM block_days WHERE day IN
""", (days, COOLDOWN_START, COOLDOWN_END, days))
r5 = cur.fetchone()
if r5 and r5["block_day_count"] and r5["block_day_cou
    pct5 = (r5["block_and_ny_count"] / r5["block_day_
    sections.append(
        f"🚫 <b>Loss limit / block days</b>\n"
        f"  {r5['block_day_count']} day(s) hit a loss
        f"  {r5['block_and_ny_count']}/{r5['block_day
        f"{COOLDOWN_START.strftime('%H:%M')}–{COOLDOW
    )
else:
    sections.append("🚫 <b>Loss limit / block days</b

# ── Section 7: Day-of-week performance ──
cur.execute("""
    SELECT EXTRACT(DOW FROM entry_time) as dow,
           COUNT(*) FILTER (WHERE outcome = 'win')  a
           COUNT(*) FILTER (WHERE outcome = 'loss') a
```

```
            COALESCE(SUM(pnl) FILTER (WHERE outcome IN
    FROM trigger_performance
    WHERE entry_time IS NOT NULL
      AND entry_time >= NOW() - INTERVAL '%s days'
    GROUP BY EXTRACT(DOW FROM entry_time)
    ORDER BY net_pnl ASC
""", (days,))
dow_rows = cur.fetchall()
if dow_rows:
    DOW_NAMES = {0: "Sun", 1: "Mon", 2: "Tue", 3: "We
    worst = dow_rows[0]
    best  = dow_rows[-1]
    worst_total = (worst["wins"] or 0) + (worst["loss
    best_total  = (best["wins"] or 0) + (best["losses
    if worst_total > 0 and best_total > 0:
        sections.append(
            f"📅 <b>Day-of-week performance</b>\n"
            f"  Worst: {DOW_NAMES[int(worst['dow'])]}
            f"({worst['wins']}W/{worst['losses']}L)\n
            f"  Best: {DOW_NAMES[int(best['dow'])]} —
            f"({best['wins']}W/{best['losses']}L)"
        )
else:
    sections.append("📅 <b>Day-of-week performance</b

# ── Section 7b: Hour-of-day performance (UK local ti
cur.execute("""
    SELECT EXTRACT(HOUR FROM entry_time AT TIME ZONE
           COUNT(*) FILTER (WHERE outcome = 'win')  a
           COUNT(*) FILTER (WHERE outcome = 'loss') a
           COALESCE(SUM(pnl) FILTER (WHERE outcome IN
    FROM trigger_performance
    WHERE entry_time IS NOT NULL
      AND entry_time >= NOW() - INTERVAL '%s days'
      AND outcome IN ('win', 'loss')
    GROUP BY EXTRACT(HOUR FROM entry_time AT TIME ZON
    ORDER BY net_pnl ASC
""", (days,))
hour_rows = cur.fetchall()
if hour_rows:
    worst_h = hour_rows[0]
    best_h  = hour_rows[-1]
    worst_h_total = (worst_h["wins"] or 0) + (worst_h
    best_h_total  = (best_h["wins"] or 0) + (best_h["
    if worst_h_total > 0 and best_h_total > 0:
        sections.append(
```

```
                f"🕐 <b>Hour-of-day performance (UK time)
                f"  Worst: {int(worst_h['hour']):02d}:00
                f"({worst_h['wins']}W/{worst_h['losses']}
                f"  Best: {int(best_h['hour']):02d}:00 —
                f"({best_h['wins']}W/{best_h['losses']}L)
            )
    else:
        sections.append("🕐 <b>Hour-of-day performance (U

    # ── Section 7c: P&L by 4-hour window, rolling 7d and
    for window_days, window_label in [(7, "7d"), (30, "30
        cur.execute("""
            SELECT
                (EXTRACT(HOUR FROM close_time AT TIME ZON
                COUNT(*) FILTER (WHERE outcome = 'win')
                COUNT(*) FILTER (WHERE outcome = 'loss')
                COALESCE(SUM(pnl) FILTER (WHERE outcome I
            FROM trigger_performance
            WHERE close_time IS NOT NULL
              AND close_time >= NOW() - INTERVAL '%s days
              AND outcome IN ('win', 'loss')
            GROUP BY (EXTRACT(HOUR FROM close_time AT TIM
            ORDER BY net_pnl ASC
        """, (window_days,))
        chunk_rows = cur.fetchall()
        if chunk_rows:
            worst_c = chunk_rows[0]
            best_c  = chunk_rows[-1]
            worst_c_total = (worst_c["wins"] or 0) + (wor
            best_c_total  = (best_c["wins"] or 0) + (best
            if worst_c_total > 0 and best_c_total > 0:
                worst_start = int(worst_c["chunk_start"])
                best_start  = int(best_c["chunk_start"])
                sections.append(
                    f"🕐 <b>P&L by 4-hour window — rollin
                    f"  Worst: {worst_start:02d}:00–{wors
                    f"({worst_c['wins']}W/{worst_c['losse
                    f"  Best: {best_start:02d}:00–{best_s
                    f"({best_c['wins']}W/{best_c['losses'
                )
        else:
            sections.append(f"🕐 <b>P&L by 4-hour window

    # ── Section 7d: First-trade-of-day loss, by 4-hour w
    for window_days, window_label in [(7, "7d"), (30, "30
        cur.execute("""
```

```
            SELECT
                (EXTRACT(HOUR FROM logged_at AT TIME ZONE
                COUNT(*) as cnt
            FROM warnings_log
            WHERE category = 'first_loss_tighten'
              AND logged_at >= NOW() - INTERVAL '%s days'
            GROUP BY (EXTRACT(HOUR FROM logged_at AT TIME
            ORDER BY cnt DESC
        """, (window_days,))
        fl_rows = cur.fetchall()
        if fl_rows:
            total_fl = sum(r["cnt"] for r in fl_rows)
            top_fl   = fl_rows[0]
            top_start = int(top_fl["chunk_start"])
            pct = (top_fl["cnt"] / total_fl * 100) if tot
            sections.append(
                f"🌅 <b>First-trade-of-day loss — rolling
                f"  Happened {total_fl} time(s).\n"
                f"  Most common window: {top_start:02d}:0
                f"({top_fl['cnt']}x, {pct:.0f}% of occurr
            )
        else:
            sections.append(f"🌅 <b>First-trade-of-day lo

    # ── Section 8: Trade sequence position — does perfor
    cur.execute("""
        WITH numbered AS (
            SELECT outcome, pnl,
                   ROW_NUMBER() OVER (PARTITION BY entry_
            FROM trigger_performance
            WHERE entry_time IS NOT NULL AND outcome IN (
              AND entry_time >= NOW() - INTERVAL '%s days
        )
        SELECT
            CASE WHEN seq = 1 THEN 'Trade #1' WHEN seq <=
            COUNT(*) FILTER (WHERE outcome = 'win')  as w
            COUNT(*) FILTER (WHERE outcome = 'loss') as l
            COALESCE(SUM(pnl), 0) as net_pnl
        FROM numbered
        GROUP BY bucket
        ORDER BY MIN(seq)
    """, (days,))
    seq_rows = cur.fetchall()
    if seq_rows:
        seq_lines = []
        for r in seq_rows:
```

```
        total = (r["wins"] or 0) + (r["losses"] or 0)
        if total > 0:
            wr = r["wins"] / total * 100
            seq_lines.append(f"  {r['bucket']}: {wr:.
    if seq_lines:
        sections.append("🔢 <b>Performance by trade c
else:
    sections.append("🔢 <b>Performance by trade order

# ── Section 9: Retracement protection — did it save
cur.execute("""
    SELECT COUNT(*) as fire_count
    FROM warnings_log
    WHERE category = 'retrace_protect'
      AND logged_at >= NOW() - INTERVAL '%s days'
""", (days,))
r9 = cur.fetchone()
if r9 and r9["fire_count"]:
    sections.append(
        f"🛡️ <b>Retracement protection fires</b>\n"
        f"  Fired {r9['fire_count']} time(s) in the l
        f"  Check /warnings for individual outcomes —
    )
else:
    sections.append("🛡️ <b>Retracement protection fir

# ── Section 10: Consecutive losses — getting bigger
cur.execute("""
    WITH ordered AS (
        SELECT pnl, outcome, entry_time,
               LAG(outcome) OVER (ORDER BY entry_time
               LAG(pnl) OVER (ORDER BY entry_time) as
        FROM trigger_performance
        WHERE entry_time IS NOT NULL AND outcome IN (
          AND entry_time >= NOW() - INTERVAL '%s days
    )
    SELECT
        COUNT(*) FILTER (WHERE outcome = 'loss' AND p
        COUNT(*) FILTER (WHERE outcome = 'loss' AND p
        COUNT(*) FILTER (WHERE outcome = 'loss' AND p
        COUNT(*) FILTER (WHERE outcome = 'loss' AND p
    FROM ordered
""", (days,))
r10 = cur.fetchone()
if r10 and r10["total_pairs"] and r10["total_pairs"]
    bigger_pct = r10["bigger"] / r10["total_pairs"] *
```

```
        sections.append(
            f"📉 <b>Back-to-back losses — sizing</b>\n"
            f"  {r10['total_pairs']} loss-after-loss pair
            f"  Bigger: {r10['bigger']} ({bigger_pct:.0f}
            + ("\n  ⚠️ Losses tend to grow when they foll
               if bigger_pct >= 55 else "")
        )
    else:
        sections.append("📉 <b>Back-to-back losses — sizi

    # ── Section 11: Most-skipped signals ──
    cur.execute("""
        SELECT message, COUNT(*) as cnt
        FROM warnings_log
        WHERE category = 'signal_skipped'
          AND logged_at >= NOW() - INTERVAL '%s days'
        GROUP BY message
        ORDER BY cnt DESC
        LIMIT 5
    """, (days,))
    skip_rows = cur.fetchall()
    if skip_rows:
        skip_lines = [f"  {r['message'].replace('Skipped:
        sections.append("🚫 <b>Most-skipped signals</b>\n
    else:
        sections.append("🚫 <b>Most-skipped signals</b>\n

    # ── Section 12: Loss streak — how fast does it happe
    cur.execute("""
        WITH streak_days AS (
            SELECT DISTINCT logged_at::date as day
            FROM warnings_log
            WHERE category = 'loss_streak'
              AND logged_at >= NOW() - INTERVAL '%s days'
        ),
        day_first_trade AS (
            SELECT entry_time::date as day, MIN(entry_tim
            FROM trigger_performance
            WHERE entry_time IS NOT NULL
            GROUP BY entry_time::date
        ),
        streak_time AS (
            SELECT MIN(w.logged_at) as streak_hit_at, d.f
            FROM warnings_log w
            JOIN day_first_trade d ON d.day = w.logged_at
            WHERE w.category = 'loss_streak'
```

```
            AND w.logged_at >= NOW() - INTERVAL '%s day
        GROUP BY w.logged_at::date, d.first_entry
    )
    SELECT AVG(EXTRACT(EPOCH FROM (streak_hit_at - fi
    FROM streak_time
    WHERE streak_hit_at > first_entry
""", (days, days))
r12 = cur.fetchone()
if r12 and r12["avg_mins"] is not None:
    mins = r12["avg_mins"]
    sections.append(
        f"⏱️ <b>Speed to loss streak</b>\n"
        f"  Avg {mins:.0f} min from day's first trade
        + ("\n  ⚠️ Streaks are forming fast — signals
           if mins < 60 else "")
    )
else:
    sections.append("⏱️ <b>Speed to loss streak</b>\n

# ── Section 13: Daily trade cap — how fast is it hit
cur.execute("""
    WITH cap_days AS (
        SELECT logged_at::date as day, MIN(logged_at)
        FROM warnings_log
        WHERE category IN ('daily_loss_block', 'daily
          AND logged_at >= NOW() - INTERVAL '%s days'
        GROUP BY logged_at::date
    ),
    day_first_trade AS (
        SELECT entry_time::date as day, MIN(entry_tim
        FROM trigger_performance
        WHERE entry_time IS NOT NULL
        GROUP BY entry_time::date
    )
    SELECT AVG(EXTRACT(EPOCH FROM (c.cap_hit_at - d.f
           COUNT(*) as cap_hit_count
    FROM cap_days c
    JOIN day_first_trade d ON d.day = c.day
    WHERE c.cap_hit_at > d.first_entry
""", (days,))
r13 = cur.fetchone()
if r13 and r13["avg_mins"] is not None:
    mins13 = r13["avg_mins"]
    hrs13  = mins13 / 60
    sections.append(
        f"🚦 <b>Speed to daily loss cap</b>\n"
```

```
                        f"  Hit {r13['cap_hit_count']} time(s) in the
                        f"  Avg {hrs13:.1f}h from day's first trade t
                        + ("\n  ⚠️ Cap is being hit quickly — possibl
                           if hrs13 < 2 else "")
                    )
                else:
                    sections.append("🚦 <b>Speed to daily loss cap</b

                cur.close()
                conn.close()

                header = f"🔬 <b>Behavioral analytics — last {days}d<
                send_telegram(header + "\n\n" + "\n\n".join(sections)
            except Exception as e:
                send_telegram(f"❌ Analytics report error: {e}")
    elif cmd == "invalidations":
        stats = db_get_invalidation_stats()
        RETIRED_SIGNALS = {
            "long_setup_10m", "short_setup_10m", "short_setup_5m", "f
            "3_bulls", "weakness", "3_bar_buy",
            "composite_higher_low", "composite_lower_high",
            "composite_crossed_above_15", "composite_crossed_below_15
            "composite_crossed_above_82", "composite_crossed_below_82
            "composite_crossed_above_18", "composite_crossed_below_18
            "composite_crossed_above_85", "composite_crossed_below_85
            "htf_crossed_above_12", "htf_crossed_below_12",
            "htf_crossed_above_88", "htf_crossed_below_88",
            "30m_bullish_structure_break", "30m_bearish_structure_bre
            "30m_crosses_down_2h_composite", "30m_crosses_up_2h_compo
            "15m_crosses_down_2h_composite", "15m_crosses_up_2h_compo
            # 30m/4H cross signals — valid trade signals but cannot b
            "30m_crosses_down_4h_composite", "30m_crosses_up_4h_compo
            # Old-style MTF combo naming — replaced by long/short nam
            "green_ltf_red_htf_bullish", "green_ltf_green_htf_bullish
            "green_ltf_cyan_htf_bullish", "green_ltf_yellow_htf_bulli
            "orange_ltf_red_htf_bearish", "orange_ltf_orange_htf_bear
            "orange_ltf_green_htf_bearish",
        }
        stats = {k: v for k, v in stats.items() if k not in RETIRED_S
        if not stats:
            send_telegram("📊 No invalidation data yet.")
        else:
            TRIGGER_LABELS = {
                "long": "📈 Long", "3_bulls": "🐂 3 Bulls", "strengt
                "3_bears": "🐻 3 Bears", "weakness": "📉 Weakness",
                "bearish": "🔴 Bearish", "short": "📉 Short", "30m_cc
```

```
                "green_structure_break": "🟢 Green Structure Break",
                "orange_structure_break": "🟠 Orange Structure Break"
            }
            # Sort worst-first — highest invalidation rate at top.
            # Triggers with zero fires (rate_pct is None) sort to the
            # Mismatched data (invalidations > fires) is capped so it
            # artificially dominate the top of the list with a broken
            def _sort_val(k):
                s = stats[k]
                if s["rate_pct"] is None:
                    return -1
                return min(s["rate_pct"], 100)
            sorted_keys = sorted(stats.keys(), key=_sort_val, reverse
            lines = ["⚠️ <b>Invalidation rates — worst first</b>\n"]
            for key in sorted_keys:
                s     = stats[key]
                label = TRIGGER_LABELS.get(key, key.replace("_", " ")
                if s["fires"] > 0 and s["invalidations"] > s["fires"]
                    lines.append(
                        f"<b>{label}</b>\n"
                        f"  ⚠️ Data mismatch: {s['invalidations']} in
                        f"  This shouldn't be possible — check the Tr
                        f"signal (delivery failures or misconfigurati
                    )
                elif s["rate_pct"] is not None:
                    lines.append(
                        f"<b>{label}</b>\n"
                        f"  Invalidated {s['invalidations']}/{s['fire
                    )
                else:
                    lines.append(
                        f"<b>{label}</b>\n"
                        f"  Invalidated {s['invalidations']}x — no fi
                    )
            send_telegram("\n\n".join(lines))
    elif cmd == "quality":
        if not DATABASE_URL:
            send_telegram("📊 No database connected.")
        else:
            try:
                conn = get_db()
                cur  = conn.cursor(cursor_factory=psycopg2.extras.Rea
                cur.execute(r"""
                    SELECT
                        trigger,
                        COUNT(*) FILTER (WHERE outcome IN ('win','los
```

```
            AVG(mae) FILTER (WHERE outcome IN ('win','los
            AVG(mfe) FILTER (WHERE outcome IN ('win','los
            AVG(mfe) FILTER (WHERE outcome = 'win')
            AVG(mfe) FILTER (WHERE outcome = 'loss')
            AVG(time_to_tp1_mins) FILTER (WHERE time_to_t
            AVG(time_to_tp2_mins) FILTER (WHERE time_to_t
            AVG(time_to_sl_mins)  FILTER (WHERE time_to_s
            COUNT(*) FILTER (WHERE mfe > 0 AND outcome =
        FROM trigger_performance
        WHERE trigger IS NOT NULL
          AND trigger NOT IN ('long_setup_10m', 'short_se
          AND trigger NOT LIKE 'exp\_%%' ESCAPE '\'
        GROUP BY trigger
        HAVING COUNT(*) FILTER (WHERE outcome IN ('win','
        ORDER BY avg_mfe DESC NULLS LAST
    """)
    rows = cur.fetchall()
    cur.close()
    conn.close()
    if not rows:
        send_telegram("📊 Not enough closed trades to shc
    else:
        TRIGGER_LABELS = {
            "long": "📈 Long", "3_bulls": "🐂 3 Bulls",
            "strength": "💪 Strength", "3_bar_buy": "📈 3
            "3_bears": "🐻 3 Bears", "weakness": "📉 Weak
            "bearish": "🔴 Bearish", "short": "📉 Short",
            "green_structure_break": "🟢 Green Structure
            "orange_structure_break": "🟠 Orange Structur
        }
        lines = ["🔬 <b>Entry quality report</b>\n<i>Bot-
        for r in rows:
            label    = TRIGGER_LABELS.get(r["trigger"], r
            avg_mae  = f"{abs(r['avg_mae']):.3f}%" if r["
            avg_mfe  = f"{r['avg_mfe']:.3f}%"     if r["a
            tp1_mins = f"{r['avg_tp1_mins']:.0f}m" if r["
            tp2_mins = f"{r['avg_tp2_mins']:.0f}m" if r["
            sl_mins  = f"{r['avg_sl_mins']:.0f}m"  if r["
            near     = r["near_miss"] or 0
            mfe_win  = f"{r['avg_mfe_win']:.3f}%"  if r["
            mfe_loss = f"{r['avg_mfe_loss']:.3f}%" if r["
            lines.append(
                f"<b>{label}</b>  ({r['total']} trades)\n
                f"  Avg MFE: {avg_mfe}  |  Avg MAE: {avg_
                f"  MFE wins: {mfe_win}  |  MFE losses: {
                f"  → TP1: {tp1_mins}  |  → TP2: {tp2_min
```

```
                        f"  Near-misses: {near}"
                    )
                send_telegram("\n\n".join(lines))
            except Exception as e:
                send_telegram(f"❌ Quality report error: {e}")
    elif cmd == "compare":
        compare_summary()
    elif cmd == "bullish":
        if daily_bias is not None:
            send_telegram(
                f"🔒 <b>Bias already set: {daily_bias.upper()}</b>\n\
                f"You cannot change your bias once it's set for the d
                f"Stay disciplined — reassess tomorrow morning."
            )
        else:
            daily_bias = "bullish"
            send_telegram("✅ <b>Bias set: BULLISH</b>")
    elif cmd == "bearish":
        if daily_bias is not None:
            send_telegram(
                f"🔒 <b>Bias already set: {daily_bias.upper()}</b>\n\
                f"You cannot change your bias once it's set for the d
                f"Stay disciplined — reassess tomorrow morning."
            )
        else:
            daily_bias = "bearish"
            send_telegram("✅ <b>Bias set: BEARISH</b>")
    elif cmd == "cancel_entry":
        if maker_entry_cancel_requested:
            send_telegram("⏳ Cancel already requested — waiting for
        else:
            maker_entry_cancel_requested = True
            send_telegram("🚫 <b>Cancel requested</b> — aborting make
    elif cmd == "cancel2":
        cancel_entry2()
    elif cmd == "fill2":
        fill_entry2_at_market()
    elif cmd == "status":
        print(f"[Telegram CMD] /status handled by MAIN bot for chat {
        status_message()
    elif cmd == "signals":
        uk_tz = ZoneInfo("Europe/London")
        def fmt_ts(ts):
            if ts is None:
                return "never"
            return datetime.fromtimestamp(ts, tz=uk_tz).strftime("%H:
```

```
lines = ["📡 <b>Active signals — all configured prompts</b>\n

signal_map = [
    ("long",                          "long",  "📈 Long",
    ("strength",                      "long",  "💪 Strength",
    ("green_structure_break",         "long",  "🟢 Green Structu
    ("30m_crosses_up_4h_composite","long",  "🔼 30m Crosses U
    ("30m_composite_crosses_up_30", "long", "📈 30m Composite
    ("3_bears",                       "short", "🐻 3 Bears",
    ("bearish",                       "short", "🔴 Bearish",
    ("short",                   "short", "📉 Short",
    ("orange_structure_break",        "short", "🟠 Orange Struct
    ("30m_crosses_down_4h_composite","short","🔻 30m Crosses
    ("30m_composite_crosses_down_70", "short", "📉 30m Compos
    ("30m_composite_crosses_down_85", "short", "📉 30m Compos
    ("30m_composite_crosses_up_15", "long", "📈 30m Composite
    ("wpr_crosses_down_18", "short", "📉 WPR Crosses Down -18
    ("wpr_crosses_up_82", "long", "📈 WPR Crosses Up -82", Nc
    ("buy", "long", "📈 Buy", None),
    ("green_ltf_red_htf_long",        "long",  "🎯 Red HTF + Gre
    ("grey_ltf_green_htf_long",       "long",  "🎯 Green HTF + G
    ("grey_ltf_cyan_htf_long",        "long",  "🎯 Cyan HTF + Gr
    ("green_ltf_green_htf_long",      "long",  "🎯 Green HTF + G
    ("green_ltf_cyan_htf_long",       "long",  "🎯 Cyan HTF + Gr
    ("orange_ltf_red_htf_short",      "short", "🎯 Red HTF + Ora
    ("grey_ltf_orange_htf_short",     "short", "🎯 Orange HTF +
    ("grey_ltf_red_htf_short",        "short", "🎯 Red HTF + Gre
    ("orange_ltf_orange_htf_short","short", "🎯 Orange HTF +
    ("orange_ltf_cyan_htf_short",     "short", "🎯 Cyan HTF + Or
]

long_lines  = []
short_lines = []
for sig_key, direction, label, last_fired in signal_map:
    entry = f"{label}"
    if direction == "long":
        long_lines.append(f"  • {entry}")
    else:
        short_lines.append(f"  • {entry}")

lines.append("🟢 <b>Long signals</b>")
lines.extend(long_lines)
lines.append("")
lines.append("🔴 <b>Short signals</b>")
lines.extend(short_lines)
```

```
lines.append("")
lines.append(f"Last long trigger:   {fmt_ts(last_long_trigger
lines.append(f"Last short trigger:  {fmt_ts(last_short_trigge
lines.append(f"Last signal fired:   {fmt_ts(last_trigger_time

# Most active triggers today
long_keys  = {"long", "strength"}
short_keys = {"3_bears", "bearish", "short"}

long_counts  = {k: v for k, v in trigger_counts_today.items()
short_counts = {k: v for k, v in trigger_counts_today.items()

if long_counts:
    top_long     = max(long_counts, key=long_counts.get)
    top_long_lbl = next((s[2] for s in signal_map if s[0] ==
    lines.append(f"\n🔥 Most active long:   {top_long_lbl} ({
else:
    lines.append(f"\n🔥 Most active long:   none yet today")

if short_counts:
    top_short     = max(short_counts, key=short_counts.get)
    top_short_lbl = next((s[2] for s in signal_map if s[0] ==
    lines.append(f"🔥 Most active short:  {top_short_lbl} ({s
else:
    lines.append(f"🔥 Most active short:  none yet today")

# Most active real trigger (excludes composite, setup and str
real_triggers = {"long", "strength", "3_bears", "bearish", "s
real_counts   = {k: v for k, v in trigger_counts_today.items(
if real_counts:
    top_overall     = max(real_counts, key=real_counts.get)
    top_overall_lbl = next((s[2] for s in signal_map if s[0]
    lines.append(f"⚡ Most active trigger: {top_overall_lbl}

# Avg time between triggers today
def avg_gap_str(times):
    if len(times) < 2:
        return "not enough data"
    gaps = [(times[i] - times[i-1]) / 60 for i in range(1, le
    avg  = sum(gaps) / len(gaps)
    return f"{int(avg)}m" if avg < 60 else f"{int(avg//60)}h

lines.append(f"⏱️ Avg gap (longs):    {avg_gap_str(sorted(lon
lines.append(f"⏱️ Avg gap (shorts):   {avg_gap_str(sorted(shc

send_telegram("\n".join(lines))
```

```
elif cmd == "help":
    send_telegram(
        "ℹ️ <b>All commands</b>\n\n"
        "<b>Enter a trade:</b>\n"
        "Trades open via signal prompts only — wait for a signal
        "<b>Manage position:</b>\n"
        "/sl 0.5 — move stop loss to 0.5% from entry\n"
        "/tp 0.4 — move TP1 to 0.4% from entry\n"
        "/tp2 0.6 — move TP2 to 0.6% from entry\n"
        "/breakeven — move SL to entry price\n"
        "/long — manually open a long\n"
        "/short — manually open a short\n"
        "/close — close full position\n"
        "/close 50 — close 50% of position\n"
        "/cut — cut position size by 50%\n"
        "/fill2 — cancel Entry2 limit and fill at market instead\
        "/cancel_entry — cancel pending maker entry before it fil
        "/cancel2 — cancel Entry2 limit order\n"
        "/bullish — set HTF bias to bullish\n"
        "/bearish — set HTF bias to bearish\n"
        "/status — entry, mark price, unrealised P&amp;amp;L\n\n"
        "<b>Info:</b>\n"
        "/signals — all configured signal prompts and status\n"
        "/stats — today's full stats\n"
        "/patterns — best performing setups\n"
        "/live — are prompts on right now, and why not if not\n"
        "/guards — every guard: on/off, bypassed by testing mode,
        "/track — performance record since 1 Oct: win rate, profi
        "/config — every limit and setting as the bot sees it; fl
        "/vwinrate — per-trigger win rate at fixed 0.35% TP / 0.2
        "/compound — weekly (2.0114x) and 4-week (16.3679x) compo
        "/winrate — win rate today/7d/30d vs 60% target\n"
        "/long, /short — manual trade, same guards as a signal (o
        "/wsstatus — websocket connection health, kline history,
        "/discipline — week/month discipline metric trend (fees,
        "/unmatched — TradingView alerts that fired but matched n
        "/warnings — recent discipline warnings (default 24h)\n"
        "/analytics — behavioral patterns (default 30d)\n"
        "📈 Weekly digest auto-sends every Sunday at 20:00\n"
        "/invalidations — invalidation rates per trigger\n"
        "/quality — entry quality: MFE, MAE, time to TP1\n"
        "/compare — manual vs indicator win rate today\n"
        "/blocked — every staircase state where a side is blocked
        "/help — this message"
    )
```

```
                else:
                    send_telegram("❓ Unknown command. Send /help for the full li

        except requests.exceptions.Timeout:
            pass
        except Exception as e:
            print(f"[Telegram CMD ERROR] {e}")
            time.sleep(5)


# ─────────────────────────────────────────────
# WEBHOOK SERVER — receives TradingView signals
# ─────────────────────────────────────────────

webhook_app = Flask(__name__)

@webhook_app.after_request
def add_cors_headers(response):
    response.headers["Access-Control-Allow-Origin"]  = "*"
    response.headers["Access-Control-Allow-Methods"] = "POST, GET, OPTIONS"
    response.headers["Access-Control-Allow-Headers"] = "Content-Type, X-Secret"
    return response

@webhook_app.route("/webhook", methods=["POST", "OPTIONS"])
def receive_webhook():
    if flask_request.method == "OPTIONS":
        return jsonify({"status": "ok"}), 200
    if WEBHOOK_SECRET:
        token = flask_request.headers.get("X-Secret", "")
        if token != WEBHOOK_SECRET:
            return jsonify({"error": "Unauthorized"}), 401

    data        = flask_request.get_json(silent=True) or {}
    threading.Thread(target=process_webhook_signal, args=(data,), daemon=True).st
    return jsonify({"status": "received"}), 200


def process_webhook_signal(data):
    global last_trigger, last_trigger_time, last_trigger_direction
    global last_long_trigger_time, last_short_trigger_time
    signal_name = data.get("signal", "Unknown Signal")
    now_ms      = int(time.time() * 1000)

    if signal_name == "8h_bullish_structure_break":
        global htf_bias, htf_bias_updated_at
        old_bias = htf_bias
```

```
    htf_bias = "bullish"
    htf_bias_updated_at = time.time()
    print(f"[HTF Bias] Updated to: bullish (8H structure break)")
    if old_bias != "bullish":
        send_telegram(
            f"🎯 <b>HTF bias — 8H Bullish Structure Break</b>\n\n"
            f"HTF bias set to BULLISH. Longs only now — "
            f"all shorts blocked until this flips."
        )
    return

if signal_name == "8h_bearish_structure_break":
    old_bias = htf_bias
    htf_bias = "bearish"
    htf_bias_updated_at = time.time()
    print(f"[HTF Bias] Updated to: bearish (8H structure break)")
    if old_bias != "bearish":
        send_telegram(
            f"🎯 <b>HTF bias — 8H Bearish Structure Break</b>\n\n"
            f"HTF bias set to BEARISH. Shorts only now — "
            f"all longs blocked until this flips."
        )
    return

# ── Composite threshold crossing — update state, notify only, no trade promp
COMPOSITE_THRESHOLD_SIGNALS = {
    "composite_crossed_above_18": "🟢 LTF composite crossed ABOVE 18 — longs
    "composite_crossed_below_18": "🔴 LTF composite crossed BELOW 18 — longs
    "composite_crossed_above_82": "🔴 LTF composite crossed ABOVE 82 — longs
    "composite_crossed_below_82": "🟢 LTF composite crossed BELOW 82 — longs
    "composite_crossed_above_50": "📊 LTF composite crossed ABOVE 50 — midlin
    "composite_crossed_below_50": "📊 LTF composite crossed BELOW 50 — midlin
    "htf_crossed_above_12":       "🟢 HTF composite crossed ABOVE 12 — longs
    "htf_crossed_below_12":       "🔴 HTF composite crossed BELOW 12 — longs
    "htf_crossed_above_88":       "🔴 HTF composite crossed ABOVE 88 — longs
    "htf_crossed_below_88":       "🟢 HTF composite crossed BELOW 88 — longs
}
if signal_name in COMPOSITE_THRESHOLD_SIGNALS:
    comp_val = data.get("composite")
    val_str  = f" ({float(comp_val):.1f})" if comp_val is not None else ""
    send_telegram(f"📊 {COMPOSITE_THRESHOLD_SIGNALS[signal_name]}{val_str}")

    # ── Brute-force protection — auto-close if composite invalidates the ope
    # Above 82/85 = extreme premium (bad for shorts). Below 18 = extreme disc
    FORCE_CLOSE_SHORT_SIGNALS = {"composite_crossed_above_82", "composite_cro
    FORCE_CLOSE_LONG_SIGNALS  = {"composite_crossed_below_18"}
```

```
    open_direction = current_trade_entry.get("direction")
    should_force_close = (
        (open_direction == "short" and signal_name in FORCE_CLOSE_SHORT_SIGNA
        (open_direction == "long"  and signal_name in FORCE_CLOSE_LONG_SIGNAL
    )
    if should_force_close:
        reason = (
            f"Composite crossed against open {open_direction.upper()} — "
            f"{COMPOSITE_THRESHOLD_SIGNALS[signal_name].splitlines()[0]}{val_
        )
        db_log_warning("brute_force_close", reason)
        close_position_now(reason=reason)
    return

print(f"[Webhook] Signal received: {signal_name}")

sig_key = signal_name.lower().replace(" ", "_").replace("-", "_")

INDICATOR_SIGNALS = {
    "long":                         ("long",  "📈 Long"),
    "strength":                     ("long",  "💪 Strength"),
    "4h_bullish_structure_break":  None,
    "3_bears":                      ("short", "🐻 3 Bears"),
    "bearish":                      ("short", "🔴 Bearish"),
    "short":                   ("short", "📉 Short"),
    "4h_bearish_structure_break":  None,
    "green_structure_break":            ("long",  "🟢 Green Structure Break
    "orange_structure_break":           ("short", "🟠 Orange Structure Break
    # MTF setup signals (7 total from BTC Composite Multi-TF)
    "green_ltf_red_htf_long":     ("long",  "🎯 Red HTF + Green LTF"),
    "grey_ltf_green_htf_long":    ("long",  "🎯 Green HTF + Grey LTF"),
    "grey_ltf_cyan_htf_long":     ("long",  "🎯 Cyan HTF + Grey LTF — 40%"),
    "green_ltf_green_htf_long":   ("long",  "🎯 Green HTF + Green LTF — 80%")
    "green_ltf_cyan_htf_long":    ("long",  "🎯 Cyan HTF + Green LTF — 70%"),
    "orange_ltf_red_htf_short":   ("short", "🎯 Red HTF + Orange LTF — 70%"),
    "grey_ltf_orange_htf_short":  ("short", "🎯 Orange HTF + Grey LTF"),
    "grey_ltf_red_htf_short":     ("short", "🎯 Red HTF + Grey LTF — 40%"),
    "orange_ltf_orange_htf_short":("short", "🎯 Orange HTF + Orange LTF — 80%
    "orange_ltf_cyan_htf_short":  ("short", "🎯 Cyan HTF + Orange LTF"),
    "composite_crossed_above_15":       None,
    "composite_crossed_below_15":       None,
    "composite_crossed_above_82":       None,
    "composite_crossed_below_82":       None,
    "composite_crossed_above_18":       None,
    "composite_crossed_below_18":       None,
    "composite_crossed_above_85":       None,
```

```
    "composite_crossed_below_85":        None,
    "htf_crossed_above_12":               ("long",  "🟢 HTF Crossed ABOVE 12 –
    "htf_crossed_below_12":               ("short", "🔴 HTF Crossed BELOW 12 –
    "htf_crossed_above_88":               ("short", "🔴 HTF Crossed ABOVE 88 –
    "htf_crossed_below_88":               ("long",  "🟢 HTF Crossed BELOW 88 –
    "30m_crosses_down_4h_composite":      ("short", "🔻 30m Crosses DOWN 4H Cc
    "30m_crosses_up_4h_composite":        ("long",  "🔼 30m Crosses UP 4H Comp
    "30m_composite_crosses_up_30":        ("long",  "📈 30m Composite Crosses
    "30m_composite_crosses_down_70":      ("short", "📉 30m Composite Crosses
    "30m_composite_crosses_down_85":      ("short", "📉 30m Composite Crosses
    "30m_composite_crosses_up_15":        ("long",  "📈 30m Composite Crosses
    "wpr_crosses_down_18":                ("short", "📉 WPR Crosses Down -18 –
    "wpr_crosses_up_82":                  ("long",  "📈 WPR Crosses Up -82 — I
    "buy":                                ("long",  "📈 Buy"),
}

INVALIDATION_SIGNALS = {
    "long_invalidated":        ("long",  "📈 Long"),
    "strength_invalidated":    ("long",  "💪 Strength"),
    "3_bears_invalidated":     ("short", "🐻 3 Bears"),
    "bearish_invalidated":     ("short", "🔴 Bearish"),
    "short_invalidated":       ("short", "📉 Short"),
    "green_structure_break_invalidated":  ("long",  "🟢 Green Structure Break
    "orange_structure_break_invalidated": ("short", "🟠 Orange Structure Brea
    "buy_invalidated":         ("long",  "📈 Buy"),
}

if sig_key == "composite_red_orange":
    send_telegram(
        "🟠 <b>Composite: Red → Orange</b>\n\n"
        "Composite has crossed from Red to Orange.\n"
        "Potential short setup building."
    )
    return

if sig_key == "4h_bearish_structure_break":
    send_telegram(
        "🔴 <b>4H Bearish Structure Break</b>\n\n"
        "A bearish structure break has been confirmed on the 4H.\n"
        "Bias shift — look for short setups on lower timeframes."
    )
    return

if sig_key == "ltf_bullish_flip":
    send_telegram(
        "🟢 <b>LTF Structure Flipped Bullish</b>\n\n"
```

```
            "Lower timeframe structure has shifted bullish.\n"
            "Look for long entries on pullbacks."
        )
        return

    if sig_key == "composite_cyan_green":
        send_telegram(
            "🟢 <b>Composite: Cyan → Green</b>\n\n"
            "Composite has crossed from Cyan to Green.\n"
            "Bullish momentum building — watch for long setups."
        )
        return

    if sig_key in INVALIDATION_SIGNALS:
        inv_direction, label = INVALIDATION_SIGNALS[sig_key]
        base_trigger = sig_key.replace("_invalidated", "")
        inv_tf = data.get("tf", "")
        if inv_tf:
            label = f"{label} ({inv_tf})"

        # Skip logging this invalidation if it's a weekend — thin liquidity
        # wicks would silently skew invalidation-rate stats. The force-close
        # protection below still runs regardless, since that's real risk
        # management for any position still open from before the weekend.
        now_uk_inv_check = datetime.now(UK_TZ)
        is_weekend = now_uk_inv_check.weekday() >= 5
        if not is_weekend:
            db_log_invalidation(base_trigger, inv_direction)
        else:
            print(f"[Invalidation] {sig_key} fired on weekend — skipping stats lo

        # ── Structure break invalidations force-close if you're in the matching
        # These are strong, objective invalidations (close beyond the candle's hi
        # not soft pattern failures — treat them as a hard signal, not just a war
        FORCE_CLOSE_INVALIDATIONS = {"orange_structure_break_invalidated", "green
        if sig_key in FORCE_CLOSE_INVALIDATIONS:
            open_direction = current_trade_entry.get("direction")
            if open_direction == inv_direction:
                reason = (
                    f"{label} invalidated — price closed beyond the candle's "
                    f"{'high' if inv_direction == 'short' else 'low'}, setup unam
                )
                db_log_warning("hard_structure_invalidate", reason)
                close_position_now(reason=reason)
                if inv_direction == "long":
                    last_long_trigger_time = None
```

```
            else:
                last_short_trigger_time = None
            return

    if inv_direction == "long":
        last_long_trigger_time = None
        send_telegram(
            f"⚠️ <b>Long invalidated — {label}</b>\n\n"
            f"Price closed below the trigger level.\n"
            f"If you're in a long, review your exit.\n"
            f"Do not enter another long until you get a new trigger.\n"
            f"Long setup signals are now blocked until a new trigger fires."
        )
    else:
        last_short_trigger_time = None
        send_telegram(
            f"⚠️ <b>Short invalidated — {label}</b>\n\n"
            f"Price closed above the trigger level.\n"
            f"If you're in a short, review your exit.\n"
            f"Do not enter another short until you get a new trigger.\n"
            f"Short setup signals are now blocked until a new trigger fires."
        )
    return

if sig_key in INDICATOR_SIGNALS:
    signal_data = INDICATOR_SIGNALS[sig_key]
    if signal_data is None:
        # Informational signal — already handled above or silently ignored
        print(f"[Signal] {sig_key} is informational — no prompt generated")
        return
    direction, label = signal_data
    tf = data.get("tf", "")
    if tf:
        label = f"{label} ({tf})"

    # ── Standardized virtual trigger win-rate tracking ──────────────
    # Fires for EVERY signal, independent of guards or blocks (loss
    # limit, trade cap, etc.) or whether a real trade was taken — fixed
    # 0.35% TP / 0.25% SL per trigger, so win rate reflects the trigger
    # itself, nothing else. EXCLUDED: weekends (thin liquidity skews
    # results) and the 14:00-15:30 UK window (same reasoning — a known
    # noisy/thin period not representative of normal conditions).
    now_uk_vt_check = datetime.now(UK_TZ)
    is_weekend_vt = now_uk_vt_check.weekday() >= 5
    is_blocked_window_vt = dt_time(14, 0) <= now_uk_vt_check.time() < dt_time
    if not is_weekend_vt and not is_blocked_window_vt:
```

```
    try:
        vt_entry_price = get_mark_price()
        open_virtual_trigger_trade(sig_key, direction, vt_entry_price)
    except Exception as e:
        print(f"[Virtual Trigger] Open hook error: {e}")

# ── Weekend cutoff — no prompts, no triggers, nothing at all ─────
# Applies to every trade signal (manual prompts, structure breaks,
# MTF combos, the experiment account) — thin weekend liquidity,
# not worth trading. Informational signals
# (structure breaks etc.) are unaffected — those are handled earlier
# and never reach this point.
now_uk_weekend_check = datetime.now(UK_TZ)
if now_uk_weekend_check.weekday() >= 5:  # 5 = Saturday, 6 = Sunday
    print(f"[Signal] Suppressed {sig_key} — weekend ({now_uk_weekend_chec
    return

last_trigger           = sig_key
last_trigger_time      = time.time()
last_trigger_direction = direction
db_log_trigger_fire(sig_key)

# Track today's activity
trigger_counts_today[sig_key] = trigger_counts_today.get(sig_key, 0) + 1
if direction == "short":
    short_trigger_times_today.append(time.time())
elif direction == "long":
    long_trigger_times_today.append(time.time())

# Only real candle/indicator signals open the setup window — not composit
VALID_LONG_TRIGGERS  = {"long", "strength", "green_structure_break", "30m
VALID_SHORT_TRIGGERS = {"3_bears", "bearish", "short", "orange_structure_

if sig_key in VALID_LONG_TRIGGERS:
    last_long_trigger_time = time.time()
elif sig_key in VALID_SHORT_TRIGGERS:
    last_short_trigger_time = time.time()

if BIAS_FILTER_ENABLED and daily_bias == "bullish" and direction == "shor
    # Check if this is a structure break that contradicts the bias
    structure_break_signals = {
        "4h_bullish_structure_break", "4h_bearish_structure_break",
        "orange_structure_break"
    }
    if sig_key in structure_break_signals:
        send_bias_invalidation_alert(label, direction, sig_key)
```

```
        print(f"[Signal] Suppressed {label} — bias is bullish, shorts filtere
        return
    if BIAS_FILTER_ENABLED and daily_bias == "bearish" and direction == "long
        # Check if this is a structure break that contradicts the bias
        structure_break_signals = {
            "4h_bullish_structure_break", "4h_bearish_structure_break",
            "green_structure_break"
        }
        if sig_key in structure_break_signals:
            send_bias_invalidation_alert(label, direction, sig_key)
        print(f"[Signal] Suppressed {label} — bias is bearish, longs filtered
        return

    # ── Counter-position alert ──────────────────────────────────────
    # If in a long and a bearish signal fires (or vice versa) — prompt to con
    open_direction = current_trade_entry.get("direction")
    if open_direction and open_direction != direction:
        entry_price = current_trade_entry.get("price", 0)
        mark        = get_mark_price()
        unreal_pct  = ((mark - entry_price) / entry_price * 100) if entry_pri
        if open_direction == "long":
            unreal_pct = unreal_pct
        else:
            unreal_pct = -unreal_pct

        # Pull real unrealised $ P&L directly from Binance rather than
        # approximating it — more reliable than deriving it from % and
        # an assumed position size.
        unreal_usd = None
        try:
            live_pos = get_open_position()
            if live_pos:
                unreal_usd = float(live_pos.get("unRealizedProfit", 0))
        except Exception as e:
            print(f"[Counter Signal] Could not fetch live unrealised $: {e}")

        usd_line = f" ({format_pnl(unreal_usd)})" if unreal_usd is not None e
        pnl_line = f"{'🟢' if unreal_pct >= 0 else '🔴'} Unrealised: {unreal_
        db_log_warning("counter_signal", f"Counter signal while in a {open_di
        severity_line = (
            "🚨 <b>You are currently LOSING on this trade AND the market just
            if unreal_pct < 0 else
            "🚨 <b>The market just signaled directly against your open positi
        )
        send_telegram(
            f"🛑🛑 <b>COUNTER SIGNAL — {open_direction.upper()} POSITION UNDE
```

```
            f"{severity_line}"
            f"You're in a <b>{open_direction.upper()}</b> and a <b>{direction
            f"{pnl_line}\n\n"
            f"⚠️ <b>This is not a routine notification — the setup you entere
            f"Do not ignore this. Review your position now.\n\n"
            f"👉 Use /close to exit immediately if you're not confident in st
        )
        return

    send_signal_prompt(direction, label, sig_key=sig_key)
    return

if EXECUTION_ENABLED and AUTO_TRADE_SIGNALS and signal_name in AUTO_TRADE_SIG
    sig_lower = signal_name.lower()

    size_usdt = data.get("size_usdt")
    size_pct  = data.get("size_pct")
    stop_pct  = data.get("stop_pct")
    tp_pct    = data.get("tp_pct")

    explicit_dir = data.get("direction", "").strip().lower()
    if explicit_dir in ("long", "buy") or any(k in sig_lower for k in ("buy",
        direction = "long"
    elif explicit_dir in ("short", "sell") or any(k in sig_lower for k in ("s
        direction = "short"
    else:
        direction = None

    if direction is None:
        print(f"[Webhook] Could not determine direction for signal: {signal_n
    else:
        threading.Thread(
            target=execute_trade,
            kwargs=dict(direction=direction, size_pct=size_pct, size_usdt=siz
                        stop_pct=stop_pct, tp_pct=tp_pct, triggered_by=signal
            daemon=True
        ).start()
    return

# ── Catch-all — this signal matched NOTHING above: not a known trade
# signal, not an invalidation, not a threshold crossing, not an
# AUTO_TRADE_SIGNALS entry. Without this, an alert set up in TradingView
# but never wired into bot.py just silently vanishes with zero trace,
# which is exactly how a real signal can go quietly missing for weeks.
print(f"[Webhook] UNRECOGNIZED signal — no handler matched: '{signal_name}' (
db_log_warning("unrecognized_signal", f"'{signal_name}' fired but matched no
```

```
    return



@webhook_app.route("/", methods=["GET"])
def orion_dashboard():
    dashboard_path = os.path.join(os.path.dirname(os.path.abspath(__file__)), "in
    return send_file(dashboard_path)

@webhook_app.route("/sw.js", methods=["GET"])
def orion_service_worker():
    """The service worker is what actually lets the browser receive and
    display a push notification even when the tab/app isn't open — this
    is the piece a plain in-app toast can never do. Served from the root
    path specifically so its scope covers the whole origin. Master-only in
    practice: nothing on a client's dashboard ever registers this, so a
    client's browser never runs this code at all.
    """
    sw_code = """
self.addEventListener('push', function(event) {
  let data = {};
  try { data = event.data ? event.data.json() : {}; } catch (e) { data = { title:
  const title = data.title || 'Orion';
  const options = {
    body: data.body || '',
    icon: '/icon-192.png',
    badge: '/icon-192.png',
  };
  event.waitUntil(self.registration.showNotification(title, options));
});

self.addEventListener('notificationclick', function(event) {
  event.notification.close();
  event.waitUntil(
    clients.matchAll({ type: 'window', includeUncontrolled: true }).then(function
      for (const client of clientList) {
        if ('focus' in client) return client.focus();
      }
      if (clients.openWindow) return clients.openWindow('/');
    })
  );
});
"""
    return sw_code, 200, {"Content-Type": "application/javascript"}
```

```
@webhook_app.route("/manifest.json", methods=["GET"])
def orion_manifest():
    """The Web App Manifest — dynamically generated per request rather than
    a static file, because start_url is what iOS/Android actually launch
    when a client taps their home-screen icon. It completely ignores
    whatever page/query-string they were on when they added it — a static
    manifest with start_url "/" means EVERY client's icon reopens a bare,
    tokenless page, which looks exactly like "this link isn't valid."
    The token is read from ?t= on this same request (index.html adds it
    via JS once the token is known client-side, since index.html itself is
    served as a static file and can't template it in server-side).
    """
    token = (flask_request.args.get("t") or "").strip()
    start_url = f"/?t={token}" if token else "/"
    manifest = {
        "name": "Orion",
        "short_name": "Orion",
        "start_url": start_url,
        "display": "standalone",
        "background_color": "#071019",
        "theme_color": "#071019",
        "icons": [
            {"src": "/icon-192.png", "sizes": "192x192", "type": "image/png"},
            {"src": "/icon-512.png", "sizes": "512x512", "type": "image/png"},
        ],
    }
    return jsonify(manifest), 200, {"Content-Type": "application/manifest+json"}

@webhook_app.route("/icon-192.png", methods=["GET"])
def orion_icon_192():
    path = os.path.join(os.path.dirname(os.path.abspath(__file__)), "icon-192.png
    return send_file(path, mimetype="image/png")

@webhook_app.route("/icon-512.png", methods=["GET"])
def orion_icon_512():
    path = os.path.join(os.path.dirname(os.path.abspath(__file__)), "icon-512.png
    return send_file(path, mimetype="image/png")

@webhook_app.route("/apple-touch-icon.png", methods=["GET"])
def orion_apple_touch_icon():
    path = os.path.join(os.path.dirname(os.path.abspath(__file__)), "apple-touch-
    return send_file(path, mimetype="image/png")

_fx_cache = {"ts": 0, "rates": None, "approx": True}
_FX_FALLBACK = {"GBP": 0.75, "EUR": 0.86}
```

```
@webhook_app.route("/api/fx", methods=["GET"])
def api_fx():
    """USD -> GBP/EUR display rates for the dashboards. Cached for 6 hours; falls
    fixed approximate rates (approx=true) if the lookup fails. Display only -- ba
    trading are always in USD."""
    now = time.time()
    if not _fx_cache["rates"] or now - _fx_cache["ts"] > 6 * 3600:
        try:
            r = requests.get("https://api.frankfurter.dev/v1/latest",
                             params={"base": "USD", "symbols": "GBP,EUR"}, timeou
            r.raise_for_status()
            rates = r.json().get("rates") or {}
            if rates.get("GBP") and rates.get("EUR"):
                _fx_cache.update(ts=now, rates={"GBP": float(rates["GBP"]), "EUR"
        except Exception as e:
            print(f"[FX] rate lookup failed: {e}")
            if not _fx_cache["rates"]:
                _fx_cache.update(ts=now - 5 * 3600, rates=dict(_FX_FALLBACK), app
    return jsonify({"rates": _fx_cache["rates"], "approx": _fx_cache["approx"]}),


@webhook_app.route("/health", methods=["GET"])
def health():
    return jsonify({"status": "running"}), 200


# ═══════════════════════════════════════════════════════════════════════
# STAIRCASE PAGE — live BTC view for the Home Screen (read-only)
# /staircase       the page itself (static shell, contains no data)
# /api/staircase   the data, only returned with the password in the X-Secret HEAD
# The password is typed once on the page and kept in the device's local storage,
# so it never appears in a URL, browser history or server logs.
# Needs STAIRCASE_TOKEN (or READ_API_SECRET) set on Railway. Never touches tradin
# ═══════════════════════════════════════════════════════════════════════
_stair_auth_fails = []   # timestamps of recent wrong passwords


def _stair_auth_ok():
    """None = no token configured, True/False = header matches. Header only, neve
    if not STAIR_TOKEN:
        return None
    provided = flask_request.headers.get("X-Secret", "")
    return hmac.compare_digest(provided, STAIR_TOKEN)
```

```
_STAIR_PAGE = r"""<!doctype html><html lang="en"><head><meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1,viewport-fit=co
<meta name="apple-mobile-web-app-capable" content="yes">
<meta name="apple-mobile-web-app-status-bar-style" content="black-translucent">
<meta name="apple-mobile-web-app-title" content="Staircase">
<link rel="apple-touch-icon" href="/apple-touch-icon.png">
<title>Staircase</title>
<style>
*{box-sizing:border-box}
body{margin:0;background:#0f1117;color:#d8dbe2;font:15px -apple-system,Segoe UI,R
.top{display:flex;justify-content:space-between;align-items:baseline;margin-botto
.top b{font-size:17px}.mut{color:#8b90a0;font-size:12px}
.card{background:#171a23;border:1px solid #262b38;border-radius:12px;padding:14px
.row{display:grid;grid-template-columns:42px 1fr;gap:8px;align-items:center;margi
.row:last-child{margin-bottom:0}
.lbl{color:#8b90a0;font-size:12px}
.chip{display:flex;justify-content:space-between;align-items:center;border-radius
.chip small{font-weight:500;font-size:11px;opacity:.75}
.pill{font-weight:600;font-size:12px;border-radius:5px;padding:2px 7px;color:#1a1
h2{font-size:13px;color:#8b90a0;font-weight:500;margin:0 0 8px}
.b{display:inline-block;border-radius:5px;padding:2px 8px;font-size:12px;font-wei
.line{display:flex;align-items:center;margin-bottom:7px;font-size:13px}
.line:last-child{margin-bottom:0}
.fire{border-color:#3dd68c}
.tabs{display:flex;gap:8px;margin-bottom:12px}
.tabs button{flex:1;padding:9px;border-radius:8px;border:1px solid #262b38;backgr
.tabs button.on{background:#222836;color:#d8dbe2;border-color:#3a4256}
.g3{display:grid;grid-template-columns:1fr 1fr 2fr;gap:5px}
.lbl2{color:#8b90a0;font-size:11px;text-align:center}
.pc{display:block;text-align:center;padding:8px 0;border-radius:6px;color:#1a1a19
.five{display:grid;grid-template-columns:1fr 14px 1fr;align-items:center;gap:2px}
.five i{text-align:center;color:#8b90a0;font-style:normal}
.tg{font-size:11px;padding:2px 6px;border-radius:4px;color:#1a1a19;font-weight:60
.empty{color:#8b90a0;font-size:13px}
input{width:100%;padding:12px;border-radius:8px;border:1px solid #333a4a;backgrou
button{width:100%;padding:12px;border-radius:8px;border:0;background:#2a78d6;colo
a.out{color:#8b90a0;font-size:12px;text-decoration:underline;cursor:pointer}
</style></head><body>
<div id="login" class="card" style="display:none">
 <h2>Password</h2>
 <input id="pw" type="password" autocomplete="current-password" placeholder="Stai
 <button id="go">Unlock</button>
 <div class="mut" id="err" style="margin-top:8px"></div>
</div>
<div id="app" style="display:none">
```

```
<div class="top"><b>BTC</b><span class="mut" id="meta">connecting…</span></div>
<div class="tabs"><button class="on" data-t="live">Live</button><button data-t="s
<div id="tab_live">
<div class="card" id="chips"><div class="empty">Loading…</div></div>
<div class="card" id="state"></div>
<div class="card"><h2>Last 7 days</h2><div id="counts"></div><div class="mut" id=
<div class="card"><h2>Recent</h2><div id="hist"><div class="empty">Nothing yet</d
</div>
<div id="tab_setups" style="display:none"></div>
<div class="mut" id="params" style="text-align:center;margin-bottom:8px"></div>
<div style="text-align:center"><a class="out" id="out">Lock this device</a>  
</div>
<script>
const KEY='stair_secret';
const CH={Red:'#e24b4a',Orange:'#ef9f27',Grey:'#888780',Green:'#3dd68c',Cyan:'#22
const SC={'short_A++': '#e24b4a', 'short_A+': '#f0605f', 'short_B': '#ef9f27', 's
const badge=(d,L)=>`<span class="b" style="background:${SC[d+'_'+L]}">${d==='shor
const uk=t=>new Date(t*1000).toLocaleString('en-GB',{timeZone:'Europe/London',wee
const $=id=>document.getElementById(id);
let timer=null;
function tick(){const el=document.getElementById('cd');if(!el||!window._nc)return
setInterval(tick,1000);
function showLogin(msg){clearInterval(timer);$('app').style.display='none';$('log
function setTab(t){
 $('tab_live').style.display=t==='live'?'block':'none';
 $('tab_setups').style.display=t==='setups'?'block':'none';
 document.querySelectorAll('.tabs button').forEach(b=>b.classList.toggle('on',b.d
}
function renderSetups(d){
 const rules=d.rules||[],counts=d.counts||{},s=d.snapshot,T=d.tfs||['4H','30m','5
 const firing=new Set(s?s.firing.map(f=>f.direction+'_'+f.letter):[]);
 const near=new Set(s?s.close.map(c=>c.direction+'_'+c.letter):[]);
 const chip=c=>`<span class="pc" style="background:${CH[c]}">${c}</span>`;
 const sec=(dir,title)=>`<h2 style="margin:4px 0 8px">${title}</h2>`+rules.filter
  const k=r.direction+'_'+r.letter,n=counts[k]||0;
  const tag=firing.has(k)?'<span class="tg" style="background:#3dd68c">firing</sp
  return `<div class="card"><div class="line" style="justify-content:space-betwee
 }).join('');
 $('tab_setups').innerHTML=sec('short','Shorts')+sec('long','Longs');
}
function render(d){
 const s=d.snapshot;const T=d.tfs||['4H','30m','5m'];
 if(!s){$('chips').innerHTML='<div class="empty">Waiting for the data feed…'+(d.w
 else{
  $('params').textContent='Timeframes '+T.join(' / ')+' · lookback '+s.params[0]+
  $('meta').textContent=s.price.toLocaleString('en-GB',{maximumFractionDigits:1})
```

```
  const r=(l,v,c,extra,pill)=>`<div class="row"><span class="lbl">${l}</span><spa
  const live=(s.live5!==null&&s.live5!==undefined)?`<span class="pill" style="bac
  $('chips').innerHTML=r(T[0],s.h,s.cols[0])+r(T[1],s.m,s.cols[1])+r(T[2],s.l,s.c
  window._lbl=T[2];window._nc=s.next_close;window._off=d.now-Date.now()/1000;tick
  const st=$('state');
  if(s.firing.length){
   st.className='card fire';
   st.innerHTML='<h2>Firing on the last '+T[2]+' close</h2>'+s.firing.map(f=>`<di
  }else{
   st.className='card';
   st.innerHTML='<h2>No setup firing</h2>'+(s.close.length?s.close.map(c=>`<div c
  }
 }
 const LETS={short:['A++','A+','B','B+','D','E','F','G','H','I','All Orange'],lon
 $('counts').innerHTML=['short','long'].map(dir=>`<div style="display:grid;grid-t
 $('since').textContent=d.since?'Tracking since '+uk(d.since)+' on these settings
 $('hist').innerHTML=d.history.length?d.history.map(e=>`<div style="opacity:${e.o
 renderSetups(d);
}
async function load(){
 const s=localStorage.getItem(KEY);
 if(!s){showLogin();return;}
 try{
  const r=await fetch('/api/staircase',{headers:{'X-Secret':s},cache:'no-store'})
  if(r.status===401){localStorage.removeItem(KEY);showLogin('Wrong password');ret
  if(r.status===429){showLogin('Too many attempts, wait a few minutes');return;}
  if(r.status===503){showLogin('Not set up yet: add STAIRCASE_TOKEN on Railway');
  if(!r.ok)throw new Error(r.status);
  $('login').style.display='none';$('app').style.display='block';
  render(await r.json());
  if(!timer)timer=setInterval(load,10000);
 }catch(e){$('meta').textContent='connection lost';}
}
$('go').onclick=()=>{const v=$('pw').value.trim();if(!v)return;localStorage.setIt
$('pw').addEventListener('keydown',e=>{if(e.key==='Enter')$('go').click();});
$('clr').onclick=async()=>{if(!confirm('Clear the recorded history and restart th
$('out').onclick=()=>{localStorage.removeItem(KEY);timer=null;showLogin();};
document.querySelectorAll('.tabs button').forEach(b=>b.onclick=()=>setTab(b.datas
document.addEventListener('visibilitychange',()=>{if(!document.hidden&&localStora
load();
</script></body></html>"""


@webhook_app.route("/staircase", methods=["GET"])
def staircase_page():
    resp = webhook_app.response_class(_STAIR_PAGE, mimetype="text/html")
```

```
    resp.headers["Cache-Control"] = "no-store"
    return resp


@webhook_app.route("/api/staircase", methods=["GET"])
def api_staircase():
    now = time.time()
    _stair_auth_fails[:] = [t for t in _stair_auth_fails if now - t < 300]
    if len(_stair_auth_fails) >= 10:
        return jsonify({"error": "Too many attempts"}), 429
    ok = _stair_auth_ok()
    if ok is None:
        return jsonify({"error": "STAIRCASE_TOKEN not set"}), 503
    if not ok:
        _stair_auth_fails.append(now)
        return jsonify({"error": "Unauthorized"}), 401
    try:
        snap = _stair_snapshot()
    except Exception as e:
        print(f"[Staircase] snapshot error: {e}")
        snap = None
    cfg = _stair_cfg()
    week_ago = now - 7 * 86400
    counts = {}
    for e in _stair_history:
        if e.get("cfg") == cfg and e["t"] >= week_ago:
            k = f"{e['direction']}_{e['letter']}"
            counts[k] = counts.get(k, 0) + 1
    recent = [dict(e, old=(e.get("cfg") != cfg)) for e in reversed(_stair_history
    resp = jsonify({"snapshot": snap, "why": _stair_last_why, "history": recent,
                    "since": _stair_since.get(cfg), "now": now, "tfs": list(_STAI
                    "rules": [{"direction": d, "letter": L, "h": rh, "m": rm, "p"
                               "note": _STAIR_NOTES[(d, L)]} for (d, L, rh, rm, r
    resp.headers["Cache-Control"] = "no-store"
    return resp


@webhook_app.route("/api/staircase/clear", methods=["POST"])
def api_staircase_clear():
    now = time.time()
    _stair_auth_fails[:] = [t for t in _stair_auth_fails if now - t < 300]
    if len(_stair_auth_fails) >= 10:
        return jsonify({"error": "Too many attempts"}), 429
    ok = _stair_auth_ok()
    if ok is None:
        return jsonify({"error": "STAIRCASE_TOKEN not set"}), 503
```

```
    if not ok:
        _stair_auth_fails.append(now)
        return jsonify({"error": "Unauthorized"}), 401
    _stair_history.clear()
    _stair_since[_stair_cfg()] = now
    try:
        save_persistent_state("stair_history", json.dumps(_stair_history))
    except Exception as e:
        print(f"[Staircase] history save error: {e}")
    _stair_save_since()
    print("[Staircase] History cleared from the page")
    return jsonify({"ok": True})


@webhook_app.route("/api/client/dashboard", methods=["GET"])
def api_client_dashboard():
    """Read-only data endpoint used by the Orion client dashboard.

    Auth model: each client gets their own unguessable dashboard_token
    (CLIENT{n}_DASHBOARD_TOKEN in Railway), embedded in their personal
    link (?t=TOKEN). There is no shared password and no free-text client
    ID — a client can only ever see the account their own token maps to,
    so client A cannot page through client B/C/... by changing a field.

    The client whose id == MASTER_CLIENT_ID ("1") is Nathan's own Binance
    master account (the bot's own balance/position/stats), not a
    Hyperliquid mirror. Every other client pulls their OWN Hyperliquid
    account state via hl_get_info(), the same call /balance and /status
    already use, so the dashboard and Telegram numbers stay consistent.
    """
    token = (flask_request.args.get("t") or flask_request.headers.get("X-Dashboar
    if not token:
        return jsonify({"error": "Missing access link token"}), 401

    c = next((c for c in CLIENTS if c.dashboard_token and c.dashboard_token == to
    if c is None:
        return jsonify({"error": "Invalid or expired access link"}), 401

    client_id = c.id

    if client_id == MASTER_CLIENT_ID:
        try:
            balance = get_usdt_balance()
            trades = get_futures_trades_today()
            stats = build_stats(trades)
```

```
position = get_open_position()
position_data = None
if position:
    size = float(position.get("positionAmt", 0))
    entry_price = float(position.get("entryPrice", 0))
    mark_price = float(position.get("markPrice", 0))
    unrealized_pnl = float(position.get("unRealizedProfit", 0))
    margin = (abs(size) * entry_price / LEVERAGE) if LEVERAGE > 0 els
    unrealized_pnl_pct = (unrealized_pnl / margin * 100) if margin >

    # TP1/TP2/SL — $ and % from entry, master-only for now.
    # Pulled from current_trade_entry (the same in-memory record
    # /sl, /tp, /tp2 and breakeven all keep updated) rather than
    # re-deriving from raw orders, so this always matches exactly
    # what a Telegram command would report.
    def _level(price_key, qty_key=None):
        price = current_trade_entry.get(price_key)
        if not price or entry_price <= 0:
            return None
        qty = current_trade_entry.get(qty_key) if qty_key else abs(si
        qty = qty or abs(size)
        return {
            "price": price,
            "pct_from_entry": abs(price - entry_price) / entry_price
            "usd": abs(price - entry_price) * qty,
        }

    position_data = {
        "side": "LONG" if size > 0 else "SHORT",
        "size": abs(size),
        "margin": margin,
        "entry_price": entry_price,
        "mark_price": mark_price,
        "unrealized_pnl": unrealized_pnl,
        "unrealized_pnl_pct": unrealized_pnl_pct,
        "tp1": _level("tp1_price", "qty_tp1"),
        "tp1_hit": bool(current_trade_entry.get("tp1_filled")),
        "entry2": next(({"price": m["price"], "qty": m["qty"],
                         "pct_from_entry": abs(m["price"] - entry_pri
                         "usd": m["qty"] * m["price"] / LEVERAGE}
                        for m in list(tracked_orders.values())
                        if m.get("label") == "Entry2" and m.get("pric
        "tp2": _level("tp2_price", "qty_tp2"),
        "sl":  _level("sl_price"),
    }
```

```
# Same avg-gap calculation as after_trade_summary() (the
# Telegram daily summary) — reused here so the dashboard shows
# the identical number, not a second slightly-different one.
avg_gap_minutes = None
positions_today = group_by_position(trades)
if len(positions_today) >= 2:
    sorted_pos  = sorted(positions_today, key=lambda p: int(p["time"]
    entry_times = [int(p["time"]) for p in sorted_pos]
    gaps_m      = [(entry_times[i] - entry_times[i-1]) / 60000 for i
    avg_gap_minutes = sum(gaps_m) / len(gaps_m)

last_trade = None
if positions_today:
    sorted_by_close = sorted(positions_today, key=lambda p: int(p["cl
    lp = sorted_by_close[-1]
    net_pnl_trade = float(lp["realizedPnl"]) - float(lp.get("commissi
    minutes_ago = (datetime.now(timezone.utc).timestamp() * 1000 - in
    last_trade = {
        "direction": "LONG" if lp["side"] == "BUY" else "SHORT",
        "pnl": net_pnl_trade,
        "minutes_ago": minutes_ago,
    }

# Today's profit-vs-underwater split — same calc as the EOD
# summary's "Position time today" block, reused here so the
# dashboard and Telegram never show two different numbers.
pos_profit_mins = pos_under_mins = 0.0
for p in positions_today:
    hold_ms = int(p["close_time"]) - int(p["time"])
    if hold_ms > 0:
        if p["realizedPnl"] > 0:
            pos_profit_mins += hold_ms / 60000
        else:
            pos_under_mins  += hold_ms / 60000
underwater_ratio = None
total_pos_mins = pos_profit_mins + pos_under_mins
if total_pos_mins > 0:
    underwater_ratio = {
        "profit_mins": pos_profit_mins,
        "underwater_mins": pos_under_mins,
        "profit_pct": pos_profit_mins / total_pos_mins * 100,
        "underwater_pct": pos_under_mins / total_pos_mins * 100,
        "ratio": (pos_profit_mins / pos_under_mins) if pos_under_mins
    }
```

```
        return jsonify({
            "client_id": client_id,
            "balance": balance,
            "today": {
                "trades": stats.get("closed_positions", 0),
                "wins": stats.get("wins", 0),
                "win_rate": stats.get("win_rate", 0),
            },
            "position": position_data,
            "staircase": stair_console_status(),
            "maintenance": get_maintenance_status(),
            "market_status": get_market_status(),
            "controls": {
                "risk": {
                    "trades_used": stats.get("closed_positions", 0),
                    "trades_cap": DAILY_TRADE_CAP,
                    "losses_used": stats.get("losses", 0),
                    "losses_cap": DAILY_LOSS_COUNT_LIMIT,
                    "pnl_today": stats.get("net_pnl", 0),
                    "pnl_cap": DAILY_LOSS_LIMIT,
                    "stopped_today": stopped_today,
                    "unrealized_pnl": position_data.get("unrealized_pnl") if
                    "fees_today": stats.get("total_fees", 0),
                    "fee_alert_threshold": FEE_ALERT_THRESHOLD,
                    "avg_gap_minutes": avg_gap_minutes,
                    "last_trade": last_trade,
                    "underwater_ratio": underwater_ratio,
                },
                "testing_mode": is_testing_mode(),
                "crm_sync": CRM_SYNC_STATUS,
                "uptime_seconds": (datetime.now(timezone.utc) - PROCESS_START
                "client_positions": get_all_client_positions(),
            },
        }), 200
    except requests.exceptions.HTTPError as e:
        status = e.response.status_code if e.response is not None else 502
        print(f"[Client Dashboard] Binance HTTP error: {status} {e}")
        return jsonify({"error": "Binance authentication/API error", "status"
    except Exception as e:
        print(f"[Client Dashboard] Error: {e}")
        return jsonify({"error": "Internal error"}), 500

if not c.is_configured():
    return jsonify({"error": "Client not yet linked to an exchange account"})

try:
```

```
info = hl_get_info()
try:
    state = info.user_state(c.address)
except HLClientError as e:
    if e.status_code == 422:
        # Hyperliquid returns this specific error for a wallet that has
        # never deposited/interacted with the exchange at all, rather
        # than a clean empty state — treat it as "$0, no position, no
        # activity" instead of failing the whole dashboard. Any client
        # who opens their link before their first deposit lands hits
        # this exact case.
        state = {"marginSummary": {"accountValue": 0}, "assetPositions":
    else:
        raise

margin_val = float(state.get("marginSummary", {}).get("accountValue", 0)
cross_val  = float(state.get("crossMarginSummary", {}).get("accountValue"
balance = cross_val if (margin_val == 0 and cross_val != 0) else margin_v

positions = state.get("assetPositions", [])
pos = next(
    (p.get("position") for p in positions
     if p.get("position", {}).get("coin") == HL_COIN
     and float(p.get("position", {}).get("szi", 0)) != 0),
    None,
)

position_data = None
if pos:
    size = float(pos.get("szi", 0))
    entry_price = float(pos.get("entryPx", 0))
    unrealized_pnl = float(pos.get("unrealizedPnl", 0))
    mark_price = hl_get_mark_price()

    margin_used = pos.get("marginUsed")
    if margin_used is not None:
        margin = float(margin_used)
    else:
        margin = (abs(size) * entry_price / c.leverage) if c.leverage > 0
    unrealized_pnl_pct = (unrealized_pnl / margin * 100) if margin > 0 el

    position_data = {
        "side": "LONG" if size > 0 else "SHORT",
        "size": abs(size),
        "entry_price": entry_price,
        "mark_price": mark_price,
```

```
                "unrealized_pnl": unrealized_pnl,
                "unrealized_pnl_pct": unrealized_pnl_pct,
            }

        ensure_milestone_entry(c.id, balance)
        try:
            fills = hl_client_fills(c)
            today_stats = hl_today_stats(fills)
            is_new_high = hl_realised_high_event(c, fills)
        except Exception as e:
            print(f"[Client Dashboard] {c.label()} fills lookup failed: {e}")
            today_stats, is_new_high = {"trades": 0, "wins": 0, "win_rate": 0}, F

        return jsonify({
            "client_id": client_id,
            "client_name": c.name,
            "balance": balance,
            "is_new_high": is_new_high,
            "reminder": pop_client_reminder(c.id),
            "today": today_stats,
            "position": position_data,
            "maintenance": get_maintenance_status(),
            "market_status": get_market_status(),
        }), 200
    except requests.exceptions.HTTPError as e:
        status = e.response.status_code if e.response is not None else 502
        print(f"[Client Dashboard] Hyperliquid HTTP error: {status} {e}")
        return jsonify({"error": "Hyperliquid authentication/API error", "status"
    except Exception as e:
        print(f"[Client Dashboard] Error: {e}")
        return jsonify({"error": "Internal error"}), 500


@webhook_app.route("/api/client/history", methods=["GET"])
def api_client_history():
    """Day-by-day trade/PnL history for the requesting client (default last
    7 days, ?days=N up to 30), plus totals across that window. Same
    token-based auth as /api/client/dashboard — a client's token only ever
    unlocks their own account's history, never anyone else's.

    Note: "totals" here are a roll-up of the fetched window, NOT a true
    account-lifetime figure — Binance/Hyperliquid history queries are
    range-limited, so a genuine since-account-inception total would need a
    separate paginated backfill job. Worth knowing before this gets labelled
    "all-time" anywhere client-facing.
    """
```

```
token = (flask_request.args.get("t") or flask_request.headers.get("X-Dashboar
if not token:
    return jsonify({"error": "Missing access link token"}), 401

c = next((c for c in CLIENTS if c.dashboard_token and c.dashboard_token == to
if c is None:
    return jsonify({"error": "Invalid or expired access link"}), 401

try:
    days = int(flask_request.args.get("days", 7))
except ValueError:
    days = 7
days = min(max(days, 1), 30)

try:
    if c.id == MASTER_CLIENT_ID:
        daily = get_binance_daily_history(days)
    else:
        if not c.is_configured():
            return jsonify({"error": "Client not yet linked to an exchange ac
        daily = get_hl_client_daily_history(c, days)

    total_trades = sum(d["trades"] for d in daily)
    total_wins   = sum(d["wins"] for d in daily)
    total_pnl    = sum(d["pnl"] for d in daily)

    # Milestones are a client-side dopamine feature — master (Nathan's
    # own account) doesn't get them, this is for mirror clients only.
    # `daily` is ordered most-recent-first, so daily[0] is today.
    new_milestones = []
    if c.id != MASTER_CLIENT_ID and daily:
        new_milestones = check_milestones(c.id, daily[0]["pnl"])

    return jsonify({
        "client_id": c.id,
        "days": daily,
        "new_milestones": new_milestones,
        "period_totals": {
            "trades": total_trades,
            "wins": total_wins,
            "win_rate": (total_wins / total_trades * 100) if total_trades els
            "pnl": total_pnl,
        },
    }), 200
except requests.exceptions.HTTPError as e:
    status = e.response.status_code if e.response is not None else 502
```

```
        print(f"[Client History] HTTP error: {status} {e}")
        return jsonify({"error": "Exchange authentication/API error", "status": s
    except Exception as e:
        print(f"[Client History] Error: {e}")
        return jsonify({"error": "Internal error"}), 500


@webhook_app.route("/api/admin/maintenance", methods=["GET", "POST"])
def api_admin_maintenance():
    """Lets Nathan set/clear the maintenance banner shown above every
    client's balance, without a redeploy (a redeploy would itself fire the
    'bot restarted mid-session' Telegram alert). Gated to the master
    token, same pattern as the rest of the /api/admin/* endpoints.
    GET: read current status (also embedded in /api/client/dashboard, this
         is just a direct way to check it).
    POST body: {"active": true/false, "reason": "CPI release", "type": "pause"}
                type is "maintenance" (bot itself is down/being worked on)
                or "pause" (bot is fine, just not trading today — news day,
                US holiday). Defaults to "maintenance" if omitted/invalid.
    """
    token = (flask_request.args.get("t") or flask_request.headers.get("X-Dashboar
    if not token:
        return jsonify({"error": "Missing access link token"}), 401

    master = next((c for c in CLIENTS if c.id == MASTER_CLIENT_ID), None)
    if master is None or not master.dashboard_token or token != master.dashboard_
        return jsonify({"error": "Unauthorized"}), 401

    if flask_request.method == "GET":
        return jsonify(get_maintenance_status()), 200

    body = flask_request.get_json(silent=True) or {}
    active = bool(body.get("active", False))
    reason = str(body.get("reason", ""))
    status_type = str(body.get("type", "maintenance"))
    try:
        set_maintenance_status(active, reason, status_type)
        return jsonify(get_maintenance_status()), 200
    except Exception as e:
        print(f"[Admin Maintenance] Error: {e}")
        return jsonify({"error": "Failed to save maintenance status"}), 500


@webhook_app.route("/api/admin/testing-mode", methods=["GET", "POST"])
def api_admin_testing_mode():
    """Lets Nathan flip TESTING_MODE from the dashboard instead of a Railway
```

```
    env var edit + restart — the actual friction that caused it to get left
    on by accident before. Gated to the master token, same pattern as
    /api/admin/maintenance.
    GET: read current state (also embedded in /api/client/dashboard under
         controls.testing_mode — this is just a direct way to check it).
    POST body: {"active": true/false}
    """
    token = (flask_request.args.get("t") or flask_request.headers.get("X-Dashboar
    if not token:
        return jsonify({"error": "Missing access link token"}), 401

    master = next((c for c in CLIENTS if c.id == MASTER_CLIENT_ID), None)
    if master is None or not master.dashboard_token or token != master.dashboard_
        return jsonify({"error": "Unauthorized"}), 401

    if flask_request.method == "GET":
        return jsonify({"active": is_testing_mode()}), 200

    body = flask_request.get_json(silent=True) or {}
    active = bool(body.get("active", False))
    try:
        set_testing_mode(active)
        return jsonify({"active": is_testing_mode()}), 200
    except Exception as e:
        print(f"[Admin Testing Mode] Error: {e}")
        return jsonify({"error": "Failed to save testing mode"}), 500


@webhook_app.route("/api/admin/client-mirroring", methods=["GET", "POST"])
def api_admin_client_mirroring():
    """Temporary per-client on/off switch for mirroring, requested while
    some live clients don't have a funded balance yet — lets Nathan stop
    the bot from attempting (and alerting on) trades for accounts that
    aren't ready, without touching their CRM record or config. Gated to
    the master token, same pattern as the other admin toggles.
    GET: returns every configured mirror client with its current enabled
         state (also embedded in /api/client/dashboard under
         controls.client_positions[].mirroring_enabled).
    POST body: {"client_id": "2", "enabled": false}
    """
    token = (flask_request.args.get("t") or flask_request.headers.get("X-Dashboar
    if not token:
        return jsonify({"error": "Missing access link token"}), 401

    master = next((c for c in CLIENTS if c.id == MASTER_CLIENT_ID), None)
    if master is None or not master.dashboard_token or token != master.dashboard_
```

```
        return jsonify({"error": "Unauthorized"}), 401

    if flask_request.method == "GET":
        clients = [c for c in CLIENTS if c.id != MASTER_CLIENT_ID and c.is_config
        return jsonify({
            "clients": [
                {"id": c.id, "name": c.name, "enabled": is_client_mirroring_enabl
                for c in clients
            ]
        }), 200

    body = flask_request.get_json(silent=True) or {}
    client_id = str(body.get("client_id", "")).strip()
    enabled = bool(body.get("enabled", True))
    if not client_id:
        return jsonify({"error": "Missing client_id"}), 400
    if not any(c.id == client_id for c in CLIENTS):
        return jsonify({"error": f"No configured client with id {client_id}"}), 4
    try:
        set_client_mirroring_enabled(client_id, enabled)
        return jsonify({"client_id": client_id, "enabled": is_client_mirroring_en
    except Exception as e:
        print(f"[Admin Client Mirroring] Error: {e}")
        return jsonify({"error": "Failed to save mirroring toggle"}), 500


@webhook_app.route("/api/admin/client-reminder", methods=["POST"])
def api_admin_client_reminder():
    """Lets the CRM push a one-time message to a specific client's own
    dashboard — used for the 'remind them their renewal is coming up'
    button. Authenticated with the same shared secret the CRM already uses
    to sync clients (CRM_SYNC_SECRET / BOT_SYNC_SECRET), rather than a
    second secret to configure, since this is the same trusted CRM→bot
    relationship in the other direction.
    Body: {"client_id": "2", "message": "..."}
    """
    provided_key = flask_request.headers.get("x-bot-sync-key", "")
    if not CRM_SYNC_SECRET or provided_key != CRM_SYNC_SECRET:
        return jsonify({"error": "Unauthorized"}), 401

    body = flask_request.get_json(silent=True) or {}
    client_id = str(body.get("client_id", "")).strip()
    message = (body.get("message") or "").strip()
    if not client_id or not message:
        return jsonify({"error": "Missing client_id or message"}), 400
    if not any(c.id == client_id for c in CLIENTS):
```

```
        return jsonify({"error": f"No configured client with id {client_id}"}), 4

    try:
        set_client_reminder(client_id, message)
        return jsonify({"ok": True}), 200
    except Exception as e:
        print(f"[Admin Client Reminder] Error: {e}")
        return jsonify({"error": "Failed to save reminder"}), 500


# ── Console quick-trade (Long / Short / Close) ──────────────────────────
# Same code path as Telegram /long, /short and /close. The request returns
# at once; the work runs in a thread and the console polls this status, so
# it can show a pulsing "sending" banner and then the real outcome.
_quick_trade = {"id": 0, "state": "idle", "action": None, "message": "", "ts": 0}
_quick_trade_lock = threading.Lock()


def _strip_tags(t: str) -> str:
    return re.sub(r"<[^>]+>", "", t or "").strip()


def _quick_trade_worker(tid: int, action: str, take_direction: str = None, take_l
    _tg_capture.msgs = []
    state, msg = "failed", "No result."
    try:
        pre = get_open_position()
        if action == "close":
            if not pre:
                state, msg = "failed", "No open position to close."
            else:
                close_position_now()
                invalidate_position_cache()
                state, msg = ("done", "Position closed.") if not get_open_positio
                             ("failed", "Close sent but the position is still ope
        else:
            if take_label:
                send_telegram(f"✅ <b>{take_label}</b> taken from the console — p
                execute_signal_trade(take_direction, take_label)
            else:
                send_telegram(("📈 <b>/LONG received</b>" if action == "long" els
                               + " (console) — executing trade checks...")
                execute_trade(direction=action, triggered_by=f"manual_{action}")
            invalidate_position_cache()
            post = get_open_position()
            want_long = (take_direction or action) == "long"
```

```
            changed = bool(post) and (float(post["positionAmt"]) > 0) == want_lon
                not pre or float(post["positionAmt"]) != float(pre["positionAmt"]
                or float(post.get("entryPrice", 0)) != float(pre.get("entryPrice"
            if changed:
                state, msg = "done", f"{(take_label or action).upper()} opened at
            else:
                why = next((_strip_tags(m) for m in reversed(_tg_capture.msgs)
                            if "received" not in m and "taken from the console" n
                state, msg = "failed", "Not executed. " + (why[:240] if why else
    except Exception as e:
        state, msg = "failed", f"Error: {e}"
    finally:
        _tg_capture.msgs = None
        with _quick_trade_lock:
            if _quick_trade["id"] == tid:
                _quick_trade.update(state=state, message=msg, ts=time.time())


@webhook_app.route("/api/admin/quick-trade", methods=["GET", "POST"])
def api_admin_quick_trade():
    """GET: current status. POST {"action": "long"|"short"|"close"|"take"|"skip"}
    take / skip act on the live staircase prompt, exactly like the Telegram butto
    token = (flask_request.args.get("t") or flask_request.headers.get("X-Dashboar
    master = next((c for c in CLIENTS if c.id == MASTER_CLIENT_ID), None)
    if not token or master is None or not master.dashboard_token or token != mast
        return jsonify({"error": "Unauthorized"}), 401
    if flask_request.method == "GET":
        with _quick_trade_lock:
            return jsonify(dict(_quick_trade)), 200
    global pending_signal
    action = ((flask_request.get_json(silent=True) or {}).get("action") or "").st
    if action not in ("long", "short", "close", "take", "skip"):
        return jsonify({"error": f"Unknown action: {action}"}), 400
    take_direction = take_label = None
    with _quick_trade_lock:
        if _quick_trade["state"] == "pending" and time.time() - _quick_trade["ts"
            return jsonify({"error": "A trade request is already in progress."}),
        if action in ("take", "skip"):
            # Same pending staircase prompt the Telegram Take / Skip buttons act
            # under the lock so it can only ever be taken once, from either place
            ps = pending_signal
            if (not ps or not str(ps.get("label", "")).startswith("STAIR")
                    or time.time() - ps["timestamp"] > ps.get("expiry", SIGNAL_EX
                return jsonify({"error": "No live setup to act on — it expired or
            take_direction, take_label = ps["direction"], ps["label"]
            msg_id = ps.get("message_id")
```

```
            pending_signal = None
            if action == "skip":
                try:
                    db_log_warning("signal_skipped", f"Skipped: {take_label}")
                    if msg_id:
                        edit_prompt_message(msg_id, f"❌ <b>{take_label}</b>\n\nS
                except Exception as e:
                    print(f"[Console] skip edit error: {e}")
                return jsonify({"ok": True, "skipped": True}), 200
            try:
                if msg_id:
                    edit_prompt_message(msg_id, f"✅ <b>{take_label}</b>\n\nTaken
            except Exception as e:
                print(f"[Console] take edit error: {e}")
        _quick_trade["id"] += 1
        tid = _quick_trade["id"]
        _quick_trade.update(state="pending", action=action, message="", ts=time.t
    threading.Thread(target=_quick_trade_worker, args=(tid, action, take_directio
    return jsonify({"ok": True, "id": tid}), 202


@webhook_app.route("/api/admin/trade-action", methods=["POST"])
def api_admin_trade_action():
    """Lets Nathan trigger the same manual trade actions from the dashboard
    that /tp, /tp2, /sl, /fill2, and /cancel2 already do via Telegram —
    calls the exact same underlying functions (adjust_tp, adjust_sl,
    fill_entry2_at_market, cancel_entry2), so there's no second, slightly
    different implementation to drift out of sync with the Telegram one.
    Master-token gated, same pattern as every other admin endpoint.
    Body: {"action": "tp1"|"tp2"|"sl"|"fill2"|"cancel2", "value": <pct, for tp1/t
    """
    token = (flask_request.args.get("t") or flask_request.headers.get("X-Dashboar
    if not token:
        return jsonify({"error": "Missing access link token"}), 401
    master = next((c for c in CLIENTS if c.id == MASTER_CLIENT_ID), None)
    if master is None or not master.dashboard_token or token != master.dashboard_
        return jsonify({"error": "Unauthorized"}), 401

    body = flask_request.get_json(silent=True) or {}
    action = (body.get("action") or "").strip()

    try:
        if action in ("tp1", "tp2", "sl"):
            raw_value = body.get("value")
            if raw_value is None or str(raw_value).strip() == "":
                return jsonify({"error": "Missing value (percentage)"}), 400
```

```
            try:
                # Same parsing as the Telegram command, quirks included —
                # this must behave identically to /tp, /tp2, /sl or the two
                # interfaces could give different results for the same input.
                pct_val = float(str(raw_value).replace("%", ""))
            except ValueError:
                return jsonify({"error": "Value must be a number, e.g. 0.36"}), 4
            pos = get_open_position()
            if not pos:
                return jsonify({"error": "No open position"}), 400
            entry = float(pos.get("entryPrice", 0))
            amt   = float(pos["positionAmt"])
            tick  = get_symbol_info()["price_tick"]
            if action == "sl":
                new_price = round_step(entry * (1 - pct_val / 100), tick) if amt
                adjust_sl(new_price)
            elif action == "tp1":
                new_price = round_step(entry * (1 + pct_val / 100), tick) if amt
                adjust_tp(new_price, label="TP1")
            else:  # tp2
                new_price = round_step(entry * (1 + pct_val / 100), tick) if amt
                _err = tp2_ordering_error(new_price, is_short=(amt < 0))
                if _err:
                    return jsonify({"error": _err}), 400
                adjust_tp(new_price, label="TP2")
        elif action == "fill2":
            fill_entry2_at_market()
        elif action == "cancel2":
            cancel_entry2()
        else:
            return jsonify({"error": f"Unknown action: {action}"}), 400

        return jsonify({"ok": True}), 200
    except Exception as e:
        print(f"[Admin Trade Action] Error: {e}")
        return jsonify({"error": str(e)}), 500


TRIGGER_LABELS = {
    "long": "📈 Long", "3_bulls": "🐂 3 Bulls", "strength": "💪 Strength",
    "3_bears": "🐻 3 Bears", "weakness": "📉 Weakness",
    "bearish": "🔴 Bearish", "short": "📉 Short", "30m_composite_crosses_up_30":
}

RETIRED_SIGNALS = {
    "long_setup_10m", "short_setup_10m", "short_setup_5m", "fork",
```

```
    "3_bulls", "weakness", "3_bar_buy",
    "composite_higher_low", "composite_lower_high",
    # Composite threshold signals — informational only, never generate trades
    "composite_crossed_above_15", "composite_crossed_below_15",
    "composite_crossed_above_82", "composite_crossed_below_82",
    "composite_crossed_above_18", "composite_crossed_below_18",
    "composite_crossed_above_85", "composite_crossed_below_85",
    "htf_crossed_above_12", "htf_crossed_below_12",
    "htf_crossed_above_88", "htf_crossed_below_88",
    # Old 2H structure breaks — informational only
    "30m_bullish_structure_break", "30m_bearish_structure_break",
    # Old cross signal names — replaced by 30m/4H versions
    "30m_crosses_down_2h_composite", "30m_crosses_up_2h_composite",
    "15m_crosses_down_2h_composite", "15m_crosses_up_2h_composite",
    # Old-style MTF combo naming — replaced by long/short naming
    "green_ltf_red_htf_bullish", "green_ltf_green_htf_bullish",
    "green_ltf_cyan_htf_bullish", "green_ltf_yellow_htf_bullish",
    "orange_ltf_red_htf_bearish", "orange_ltf_orange_htf_bearish",
    "orange_ltf_green_htf_bearish",
}

def compute_trigger_breakdown() -> list:
    """Shared by /patterns (Telegram) and the dashboard's trigger breakdown
    page — one aggregation, so the two views can never quietly disagree
    with each other. Returns a list of dicts, sorted by win rate (triggers
    with no trades yet last), retired signals and 'exp_' test signals
    excluded.
    """
    rows     = db_get_trigger_performance()
    avg_gaps = db_get_avg_trigger_gaps()
    if not rows and not avg_gaps:
        return []

    from collections import defaultdict
    groups = defaultdict(lambda: {"wins": 0, "losses": 0, "pnl": 0.0})
    for r in rows:
        key = r["trigger"]
        if r["outcome"] == "win":
            groups[key]["wins"] += 1
        else:
            groups[key]["losses"] += 1
        groups[key]["pnl"] += r["pnl"] or 0

    all_keys = {k for k in (set(groups.keys()) | set(avg_gaps.keys())) - RETIRED_
    sorted_keys = sorted(all_keys, key=lambda k: groups[k]["wins"] / max(groups[k
```

```
    breakdown = []
    for key in sorted_keys:
        g = groups.get(key, {"wins": 0, "losses": 0, "pnl": 0.0})
        total = g["wins"] + g["losses"]
        breakdown.append({
            "key": key,
            "label": TRIGGER_LABELS.get(key, key.replace("_", " ").title()),
            "wins": g["wins"],
            "losses": g["losses"],
            "total": total,
            "win_rate": (g["wins"] / total * 100) if total > 0 else None,
            "pnl": g["pnl"],
            "avg_gap_minutes": avg_gaps.get(key),
        })
    return breakdown


@webhook_app.route("/api/admin/trigger-performance", methods=["GET"])
def api_admin_trigger_performance():
    """Dashboard version of /patterns — same underlying aggregation
    (compute_trigger_breakdown), just returned as JSON for a proper visual
    page instead of a wall of Telegram text.
    """
    token = (flask_request.args.get("t") or flask_request.headers.get("X-Dashboar
    if not token:
        return jsonify({"error": "Missing access link token"}), 401
    master = next((c for c in CLIENTS if c.id == MASTER_CLIENT_ID), None)
    if master is None or not master.dashboard_token or token != master.dashboard_
        return jsonify({"error": "Unauthorized"}), 401

    try:
        return jsonify({"triggers": compute_trigger_breakdown()[:7]}), 200
    except Exception as e:
        print(f"[Admin Trigger Performance] Error: {e}")
        return jsonify({"error": str(e)}), 500


def _check_master_token() -> bool:
    token = (flask_request.args.get("t") or flask_request.headers.get("X-Dashboar
    if not token:
        return False
    master = next((c for c in CLIENTS if c.id == MASTER_CLIENT_ID), None)
    return master is not None and master.dashboard_token and token == master.dash


@webhook_app.route("/api/admin/push-vapid-key", methods=["GET"])
```

```
def api_admin_push_vapid_key():
    """The browser needs the PUBLIC key to subscribe — this is not a
    secret, it's fine to hand out to master's own dashboard on request."""
    if not _check_master_token():
        return jsonify({"error": "Unauthorized"}), 401
    if not VAPID_PUBLIC_KEY:
        return jsonify({"error": "Push notifications aren't configured on this se
    return jsonify({"public_key": VAPID_PUBLIC_KEY}), 200


@webhook_app.route("/api/admin/push-subscribe", methods=["POST"])
def api_admin_push_subscribe():
    """Stores a browser's push subscription after it opts in. Master-only —
    there is no equivalent client-facing endpoint."""
    if not _check_master_token():
        return jsonify({"error": "Unauthorized"}), 401
    subscription = flask_request.get_json(silent=True)
    if not subscription or not subscription.get("endpoint"):
        return jsonify({"error": "Invalid subscription payload"}), 400
    try:
        add_push_subscription(subscription)
        return jsonify({"ok": True}), 200
    except Exception as e:
        print(f"[Push Subscribe] Error: {e}")
        return jsonify({"error": "Failed to save subscription"}), 500


@webhook_app.route("/api/admin/push-unsubscribe", methods=["POST"])
def api_admin_push_unsubscribe():
    if not _check_master_token():
        return jsonify({"error": "Unauthorized"}), 401
    body = flask_request.get_json(silent=True) or {}
    endpoint = body.get("endpoint")
    if not endpoint:
        return jsonify({"error": "Missing endpoint"}), 400
    try:
        remove_push_subscription(endpoint)
        return jsonify({"ok": True}), 200
    except Exception as e:
        print(f"[Push Unsubscribe] Error: {e}")
        return jsonify({"error": "Failed to remove subscription"}), 500


@webhook_app.route("/api/compounding-history", methods=["GET"])
def api_compounding_history():
    """
```

```
Read-only endpoint for the external compounding tracker (a separate
static frontend, no relation to trade execution). Returns:
- weeks: every stored weekly starting balance in order (chained, this
  IS the actual weekly ledger)
- daily_balances: every logged daily balance snapshot (for comparing
  against a locked-in daily target within the current week)
- locked_targets: every week's locked weekly/daily multiplier targets
- current_balance: live balance right now
- weekly_target_multiplier: fallback default if a week has no lock
Never touches trading state.
"""
if READ_API_SECRET:
    provided = flask_request.headers.get("X-Secret", "") or flask_request.arg
    if provided != READ_API_SECRET:
        return jsonify({"error": "Unauthorized"}), 401

if not DATABASE_URL:
    return jsonify({"error": "Database not configured"}), 500

try:
    # Ensures this week's starting-balance row exists before we read —
    # same auto-seed logic the /compound Telegram command already
    # relies on. Means the tracker never has to wait for a scheduled
    # check to run first; opening the page is enough to seed a fresh
    # week if one doesn't exist yet (e.g. right after a manual reset).
    get_compounding_progress()

    conn = get_db()
    cur  = conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor)

    cur.execute("""
        SELECT period_start, starting_balance
        FROM compounding_targets
        WHERE period_type = 'week'
        ORDER BY period_start ASC
    """)
    weeks = [
        {"period_start": r["period_start"].isoformat(), "starting_balance": f
        for r in cur.fetchall()
    ]

    cur.execute("""
        SELECT log_date, balance
        FROM daily_balance_log
        ORDER BY log_date ASC
    """)
```

```
        daily_balances = [
            {"date": r["log_date"].isoformat(), "balance": float(r["balance"])}
            for r in cur.fetchall()
        ]

        cur.execute("""
            SELECT week_start, weekly_multiplier, daily_multiplier
            FROM locked_targets
            ORDER BY week_start ASC
        """)
        locked_targets = [
            {
                "week_start": r["week_start"].isoformat(),
                "weekly_multiplier": float(r["weekly_multiplier"]),
                "daily_multiplier": float(r["daily_multiplier"]),
            }
            for r in cur.fetchall()
        ]

        cur.close()
        conn.close()

        try:
            current_balance = get_usdt_balance()
        except Exception as e:
            print(f"[Compounding API] Balance fetch error: {e}")
            current_balance = None

        return jsonify({
            "weeks": weeks,
            "daily_balances": daily_balances,
            "locked_targets": locked_targets,
            "current_balance": current_balance,
            "weekly_target_multiplier": WEEKLY_COMPOUND_TARGET,
        }), 200
    except Exception as e:
        print(f"[Compounding API] Error: {e}")
        return jsonify({"error": "Internal error"}), 500


@webhook_app.route("/api/lock-target", methods=["POST", "OPTIONS"])
def api_lock_target():
    """
    Locks in the weekly and daily target multipliers for a given week,
    set from the tracker frontend. Upserts — re-locking the same week
    overwrites the previous values. Read/write is scoped entirely to the
```

```
    locked_targets table; never touches trading state or execution.
    """
    if flask_request.method == "OPTIONS":
        return jsonify({"status": "ok"}), 200

    if READ_API_SECRET:
        provided = flask_request.headers.get("X-Secret", "") or flask_request.arg
        if provided != READ_API_SECRET:
            return jsonify({"error": "Unauthorized"}), 401

    body = flask_request.get_json(silent=True) or {}
    week_start = body.get("week_start")
    weekly_multiplier = body.get("weekly_multiplier")
    daily_multiplier = body.get("daily_multiplier")

    if not week_start or weekly_multiplier is None or daily_multiplier is None:
        return jsonify({"error": "week_start, weekly_multiplier, and daily_multip

    try:
        weekly_multiplier = float(weekly_multiplier)
        daily_multiplier = float(daily_multiplier)
    except (TypeError, ValueError):
        return jsonify({"error": "weekly_multiplier and daily_multiplier must be

    ok = db_lock_week_targets(week_start, weekly_multiplier, daily_multiplier)
    if not ok:
        return jsonify({"error": "Failed to save — check server logs"}), 500

    return jsonify({
        "status": "locked",
        "week_start": week_start,
        "weekly_multiplier": weekly_multiplier,
        "daily_multiplier": daily_multiplier,
    }), 200


def start_webhook_server():
    # threaded=True so a slow-to-respond request (a dashboard poll waiting
    # on Hyperliquid, say) can't block an incoming TradingView webhook from
    # being received at all. This server handles both — a genuine gap
    # that existed before any of tonight's dashboard work, not something
    # introduced by it.
    webhook_app.run(host="0.0.0.0", port=WEBHOOK_PORT, debug=False, use_reloader=


def get_same_side_consecutive_losses(trades: list) -> tuple:
```

```
    positions = group_by_position(trades)
    if not positions:
        return 0, None

    streak = 0
    side   = None
    for p in reversed(positions):
        if p["realizedPnl"] < 0:
            p_side = p.get("side", "")
            if side is None:
                side = p_side
            if p_side == side:
                streak += 1
            else:
                break
        else:
            break

    return streak, side


def in_cooldown() -> bool:
    now_uk = datetime.now(UK_TZ).time()
    return COOLDOWN_START <= now_uk < COOLDOWN_END


def format_pnl(value: float) -> str:
    sign = "+" if value >= 0 else ""
    return f"{sign}${value:,.2f}"


def after_trade_summary() -> str:
    try:
        trades    = get_futures_trades_today()
        stats     = build_stats(trades)
        fees      = stats["total_fees"]
        remaining = FEE_ALERT_THRESHOLD - fees
        fee_pct   = (fees / FEE_ALERT_THRESHOLD * 100) if FEE_ALERT_THRESHOLD els
        fee_line  = (
            f"Fees: ${fees:.2f} / ${FEE_ALERT_THRESHOLD:.0f}  ({fee_pct:.0f}% — $
            if remaining > 0 else
            f"Fees: ${fees:.2f} — limit hit"
        )
        wr_line = f"{stats['closed_positions']} positions  ({stats['wins']}W / {s

        positions = group_by_position(trades)
```

```
        gap_line  = ""
        hold_line = ""
        if len(positions) >= 2:
            sorted_pos  = sorted(positions, key=lambda p: int(p["time"]))
            entry_times = [int(p["time"]) for p in sorted_pos]
            gaps_m      = [(entry_times[i] - entry_times[i-1]) / 60000 for i in r
            avg_gap     = sum(gaps_m) / len(gaps_m)
            gap_fmt     = f"{int(avg_gap // 60)}h {int(avg_gap % 60)}m" if avg_ga
            gap_flag    = " ⚠️" if avg_gap < 120 else " ✅"
            gap_line    = f"Avg gap:   {gap_fmt} between trades{gap_flag}"
        if len(positions) >= 1:
            hold_mins = []
            for p in positions:
                h = (int(p["close_time"]) - int(p["time"])) / 60000
                if h >= 0:
                    hold_mins.append(h)
            if hold_mins:
                avg_h     = sum(hold_mins) / len(hold_mins)
                hold_fmt  = f"{int(avg_h // 60)}h {int(avg_h % 60)}m" if avg_h >=
                hold_flag = " ⚠️ holding too short" if avg_h < 120 else " ✅"
                hold_line = f"Avg hold:  {hold_fmt} per trade{hold_flag}"

        lines = [wr_line, fee_line]
        if gap_line:  lines.append(gap_line)
        if hold_line: lines.append(hold_line)
        return "\n".join(lines)
    except Exception:
        return ""


# ─────────────────────────────────────────────
# HELPERS — WEEKLY STATS PERSISTENCE
# ─────────────────────────────────────────────

def load_weekly_stats() -> list:
    try:
        return _kv_read("setting_weekly_stats", STATS_FILE, []) or []
    except Exception:
        return []


def save_daily_stats_to_file(date_str: str, stats: dict):
    weekly = load_weekly_stats()
    weekly = [d for d in weekly if d.get("date") != date_str]
    weekly.append({"date": date_str, **stats})
    weekly = sorted(weekly, key=lambda d: d["date"])[-7:]
```

```
    _kv_write("setting_weekly_stats", STATS_FILE, weekly)


WEEKLY_COMPOUND_TARGET = 1.5   # end-of-week balance multiplier target (~50% week
CYCLE_COMPOUND_TARGET  = 5.0625   # end-of-4-week balance multiplier target (1.5^
CYCLE_WEEKS            = 4

def db_get_or_create_period_start(period_type: str, period_start_date) -> float:
    """
    Returns the stored starting balance for this period (week or 4-week cycle),
    creating it fresh if one doesn't exist yet. Persisted in the DB (not just
    memory) so a Railway restart mid-week doesn't lose the anchor balance —
    the whole point of this tracker breaks if the starting point silently
    resets after a crash.
    """
    if not DATABASE_URL:
        return None
    try:
        conn = get_db()
        cur  = conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor)
        cur.execute("""
            SELECT starting_balance FROM compounding_targets
            WHERE period_type = %s AND period_start = %s
            LIMIT 1
        """, (period_type, period_start_date))
        row = cur.fetchone()
        if row:
            cur.close()
            conn.close()
            return float(row["starting_balance"])

        # No record for this period yet — create one using today's balance
        balance = get_usdt_balance()
        cur.execute("""
            INSERT INTO compounding_targets (period_type, period_start, starting_
            VALUES (%s, %s, %s)
        """, (period_type, period_start_date, balance))
        conn.commit()
        cur.close()
        conn.close()
        return balance
    except Exception as e:
        print(f"[Compounding Target] DB error: {e}")
        return None
```

```
def db_log_daily_balance(log_date, balance: float):
    """
    Records one balance snapshot per calendar day (immutable — ON CONFLICT
    DO NOTHING, so a restart or re-run on the same day can't overwrite an
    already-logged value). This is the only source of daily granularity for
    the daily-target comparison view — the weekly compounding tracker alone
    only has Monday-to-Monday snapshots.
    """
    if not DATABASE_URL:
        return
    try:
        conn = get_db()
        cur  = conn.cursor()
        cur.execute("""
            INSERT INTO daily_balance_log (log_date, balance)
            VALUES (%s, %s)
            ON CONFLICT (log_date) DO NOTHING
        """, (log_date, balance))
        conn.commit()
        cur.close()
        conn.close()
    except Exception as e:
        print(f"[Daily Balance Log] DB error: {e}")


def db_lock_week_targets(week_start_date, weekly_multiplier: float, daily_multipl
    """
    Locks in (or updates) the weekly and daily target multipliers for a
    given week. Upsert on week_start — calling this again for the same
    week overwrites the previous lock, which is intentional (lets you
    correct a target you set before the week's actual is compared).
    """
    if not DATABASE_URL:
        return False
    try:
        conn = get_db()
        cur  = conn.cursor()
        cur.execute("""
            INSERT INTO locked_targets (week_start, weekly_multiplier, daily_mult
            VALUES (%s, %s, %s)
            ON CONFLICT (week_start) DO UPDATE
            SET weekly_multiplier = EXCLUDED.weekly_multiplier,
                daily_multiplier  = EXCLUDED.daily_multiplier,
                locked_at         = NOW()
        """, (week_start_date, weekly_multiplier, daily_multiplier))
        conn.commit()
```

```
        cur.close()
        conn.close()
        return True
    except Exception as e:
        print(f"[Locked Targets] DB error: {e}")
        return False


VIRTUAL_TP_PCT = 0.35   # fixed target % for standardized per-trigger win rate tr
VIRTUAL_SL_PCT = 0.25   # fixed stop % for the same

def open_virtual_trigger_trade(trigger: str, direction: str, entry_price: float):
    """
    Opens a standardized 'virtual trade' the moment any signal fires —
    completely independent of whether a real trade was taken. Fixed
    0.35% TP / 0.25% SL every time, so the only variable across trades is
    the trigger itself, giving a clean per-trigger win rate uncontaminated
    by discretion, sizing, or guard blocks. Skips opening a new one if this
    exact trigger already has an unresolved virtual trade open.
    """
    if not DATABASE_URL or entry_price <= 0:
        return
    try:
        conn = get_db()
        cur  = conn.cursor()
        cur.execute("""
            SELECT 1 FROM virtual_trigger_trades
            WHERE trigger = %s AND outcome IS NULL
            LIMIT 1
        """, (trigger,))
        if cur.fetchone():
            cur.close()
            conn.close()
            return  # already an open virtual trade for this trigger — skip

        if direction == "long":
            target_price = entry_price * (1 + VIRTUAL_TP_PCT / 100)
            stop_price   = entry_price * (1 - VIRTUAL_SL_PCT / 100)
        else:
            target_price = entry_price * (1 - VIRTUAL_TP_PCT / 100)
            stop_price   = entry_price * (1 + VIRTUAL_SL_PCT / 100)

        cur.execute("""
            INSERT INTO virtual_trigger_trades (trigger, direction, entry_price,
            VALUES (%s, %s, %s, %s, %s)
        """, (trigger, direction, entry_price, target_price, stop_price))
```

```
        conn.commit()
        cur.close()
        conn.close()
    except Exception as e:
        print(f"[Virtual Trigger] Open error: {e}")


def check_virtual_trigger_trades():
    """
    Runs every poll cycle — checks all open virtual trigger trades against
    the current mark price, closing any that have hit their fixed TP or SL.
    Also force-resolves anything still open by Friday 22:00 UK — a virtual
    trade shouldn't drift unresolved into the weekend, since weekend price
    action isn't representative of the conditions the trigger fired in.
    """
    if not DATABASE_URL:
        return
    try:
        conn = get_db()
        cur  = conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor)
        cur.execute("SELECT * FROM virtual_trigger_trades WHERE outcome IS NULL")
        open_trades = cur.fetchall()

        if not open_trades:
            cur.close()
            conn.close()
            return  # nothing open — skip the mark price fetch entirely

        mark = get_mark_price()
        if mark <= 0:
            cur.close()
            conn.close()
            return

        now_uk_vt = datetime.now(UK_TZ)
        force_resolve_friday = now_uk_vt.weekday() == 4 and now_uk_vt.time() >= d

        for vt in open_trades:
            direction = vt["direction"]
            hit_target = (direction == "long" and mark >= vt["target_price"]) or
                         (direction == "short" and mark <= vt["target_price"])
            hit_stop   = (direction == "long" and mark <= vt["stop_price"]) or \
                         (direction == "short" and mark >= vt["stop_price"])

            if hit_target or hit_stop:
                outcome = "win" if hit_target else "loss"
```

```
                cur.execute("""
                    UPDATE virtual_trigger_trades
                    SET outcome = %s, exit_price = %s, closed_at = NOW()
                    WHERE id = %s
                """, (outcome, mark, vt["id"]))
            elif force_resolve_friday:
                # Neither TP nor SL hit, but it's Friday 22:00+ — force a
                # resolution based on which side of entry the price landed,
                # rather than let it drift unresolved into the weekend.
                entry = vt["entry_price"]
                if direction == "long":
                    outcome = "win" if mark >= entry else "loss"
                else:
                    outcome = "win" if mark <= entry else "loss"
                cur.execute("""
                    UPDATE virtual_trigger_trades
                    SET outcome = %s, exit_price = %s, closed_at = NOW()
                    WHERE id = %s
                """, (outcome, mark, vt["id"]))
                print(f"[Virtual Trigger] Force-resolved trigger {vt['trigger']}

        conn.commit()
        cur.close()
        conn.close()
    except Exception as e:
        print(f"[Virtual Trigger] Check error: {e}")


def get_daily_compounding_pace() -> dict:
    """
    Each morning: given today's actual starting balance and how many trading
    days remain this week (Mon-Fri, week ends Friday 8pm), calculates the %
    and $ gain needed TODAY to stay on pace for the 2.0114x weekly target —
    assuming equal daily compounding for the remaining days. Recalculated
    fresh each morning off the real current balance, so it re-paces itself
    daily rather than assuming a fixed schedule.
    """
    now_uk = datetime.now(UK_TZ)
    monday_this_week = (now_uk - timedelta(days=now_uk.weekday())).date()

    week_start_balance = db_get_or_create_period_start("week", monday_this_week)
    if not week_start_balance or week_start_balance <= 0:
        return {}

    try:
        current_balance = get_usdt_balance()
```

```
    except Exception as e:
        print(f"[Daily Pace] Balance fetch error: {e}")
        return {}

    # Trading days remaining INCLUDING today, Mon=0 .. Fri=4, week ends Fri 8pm
    weekday = now_uk.weekday()
    if weekday > 4:
        return {}  # weekend — no pacing needed
    days_remaining_incl_today = 5 - weekday  # Mon->5, Tue->4, Wed->3, Thu->2, Fr

    week_target_balance = week_start_balance * WEEKLY_COMPOUND_TARGET
    if current_balance <= 0:
        return {}

    # Required daily multiplier, compounded evenly across remaining days,
    # to go from current_balance to week_target_balance.
    required_daily_multiplier = (week_target_balance / current_balance) ** (1 / d
    required_pct_today = (required_daily_multiplier - 1) * 100
    required_usd_today = current_balance * (required_daily_multiplier - 1)

    week_multiplier_so_far = current_balance / week_start_balance
    on_pace = week_multiplier_so_far >= (WEEKLY_COMPOUND_TARGET ** (weekday / 5))

    return {
        "current_balance": current_balance,
        "week_start_balance": week_start_balance,
        "week_target_balance": week_target_balance,
        "days_remaining": days_remaining_incl_today,
        "required_pct_today": required_pct_today,
        "required_usd_today": required_usd_today,
        "week_multiplier_so_far": week_multiplier_so_far,
        "on_pace": on_pace,
    }




def get_compounding_progress() -> dict:
    """
    Returns current progress toward the weekly (2.0114x) and cycle/4-week
    (16.3679x) compounding targets, anchored to the Monday-of-this-week
    starting balance and the start-of-current-4-week-cycle balance.
```

```
"""
now_uk = datetime.now(UK_TZ)
monday_this_week = (now_uk - timedelta(days=now_uk.weekday())).date()

# 4-week cycle start = the most recent Monday that's a multiple of
# CYCLE_WEEKS weeks after the very first recorded cycle start. Simplest
# honest approach: track cycle start as "the Monday of the week
# containing the first ever balance snapshot", then roll forward in
# CYCLE_WEEKS-week blocks from there.
cycle_start_date = None
if DATABASE_URL:
    try:
        conn = get_db()
        cur  = conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor)
        cur.execute("""
            SELECT MIN(period_start) as first_week FROM compounding_targets
            WHERE period_type = 'week'
        """)
        row = cur.fetchone()
        cur.close()
        conn.close()
        if row and row["first_week"]:
            first_week = row["first_week"]
            weeks_elapsed = (monday_this_week - first_week).days // 7
            cycles_elapsed = weeks_elapsed // CYCLE_WEEKS
            cycle_start_date = first_week + timedelta(weeks=cycles_elapsed *
        else:
            cycle_start_date = monday_this_week  # first week ever — this wee
    except Exception as e:
        print(f"[Compounding Target] Cycle calc error: {e}")
        cycle_start_date = monday_this_week

week_start_balance  = db_get_or_create_period_start("week", monday_this_week)
cycle_start_balance = db_get_or_create_period_start("cycle", cycle_start_date

try:
    current_balance = get_usdt_balance()
except Exception as e:
    print(f"[Compounding Target] Balance fetch error: {e}")
    return {}

result = {"current_balance": current_balance}

if week_start_balance and week_start_balance > 0:
    week_multiplier = current_balance / week_start_balance
    result["week_start_balance"] = week_start_balance
```

```
        result["week_multiplier"] = week_multiplier
        result["week_target_balance"] = week_start_balance * WEEKLY_COMPOUND_TARG
        result["week_pct_of_target"] = (week_multiplier - 1) / (WEEKLY_COMPOUND_T

    if cycle_start_balance and cycle_start_balance > 0:
        cycle_multiplier = current_balance / cycle_start_balance
        result["cycle_start_balance"] = cycle_start_balance
        result["cycle_multiplier"] = cycle_multiplier
        result["cycle_target_balance"] = cycle_start_balance * CYCLE_COMPOUND_TAR
        result["cycle_pct_of_target"] = (cycle_multiplier - 1) / (CYCLE_COMPOUND_
        result["cycle_start_date"] = cycle_start_date

    return result


def load_last_7_days_stats() -> list:
    return load_weekly_stats()


# ──────────────────────────────────────────────
# HELPERS — TELEGRAM
# ──────────────────────────────────────────────

_tg_capture = threading.local()


def send_telegram(message: str):
    _cap = getattr(_tg_capture, "msgs", None)
    if _cap is not None:
        _cap.append(message)
    url = f"https://api.telegram.org/bot{TELEGRAM_BOT_TOKEN}/sendMessage"
    payload = {"chat_id": TELEGRAM_CHAT_ID, "text": message, "parse_mode": "HTML"
    resp = requests.post(url, json=payload, timeout=10)
    resp.raise_for_status()
    print(f"[Telegram] {message[:80]}...")


def get_consecutive_wins(trades: list) -> int:
    positions = group_by_position(trades)
    streak    = 0
    for p in reversed(positions):
        if p["realizedPnl"] > 0:
            streak += 1
        else:
            break
    return streak
```

```
# ──────────────────────────────────────────────
# ALERTS
# ──────────────────────────────────────────────

def send_eod_summary(trades: list, balance_start: float, balance_end: float, date
    stats = build_stats(trades)
    balance_change = balance_end - balance_start

    orders  = group_by_order(trades)
    closing = [o for o in orders if o["realizedPnl"] != 0]
    hold_times_win  = []
    hold_times_loss = []
    sorted_orders   = sorted(orders, key=lambda o: o["time"])
    for i, o in enumerate(sorted_orders):
        if o["realizedPnl"] == 0:
            continue
        for j in range(i - 1, -1, -1):
            if sorted_orders[j]["realizedPnl"] == 0:
                hold_ms = o["time"] - sorted_orders[j]["time"]
                hold_min = hold_ms / 60000
                if 0 < hold_min < 1440:
                    if o["realizedPnl"] > 0:
                        hold_times_win.append(hold_min)
                    else:
                        hold_times_loss.append(hold_min)
                break

    avg_hold_win  = sum(hold_times_win)  / len(hold_times_win)  if hold_times_win
    avg_hold_loss = sum(hold_times_loss) / len(hold_times_loss) if hold_times_los

    save_daily_stats_to_file(date_str, {**stats,
        "balance_start":    balance_start,
        "balance_end":      balance_end,
        "streaks_3plus":    daily_streaks_3plus,
        "avg_hold_win":     avg_hold_win,
        "avg_hold_loss":    avg_hold_loss,
        "worst_chunk_label": None,
        "worst_chunk_pnl":   None,
    })

    try:
        _positions = group_by_position(trades)
        _uk_tz     = ZoneInfo("Europe/London")
        _chunks    = {}
```

```
    for _p in _positions:
        _ts    = datetime.fromtimestamp(int(_p["close_time"]) / 1000, tz=_uk_
        _start = (_ts.hour // 2) * 2
        _label = f"{_start:02d}:00-{_start+2:02d}:00"
        _chunks[_label] = _chunks.get(_label, 0) + _p["realizedPnl"]
    if _chunks:
        _worst_label = min(_chunks, key=_chunks.get)
        _worst_pnl   = _chunks[_worst_label]
        save_daily_stats_to_file(date_str, {**stats,
            "balance_start":     balance_start,
            "balance_end":       balance_end,
            "streaks_3plus":     daily_streaks_3plus,
            "avg_hold_win":      avg_hold_win,
            "avg_hold_loss":     avg_hold_loss,
            "worst_chunk_label": _worst_label if _worst_pnl < 0 else None,
            "worst_chunk_pnl":   _worst_pnl   if _worst_pnl < 0 else None,
            "rules_broken":      dict(rules_broken_today),
        })
except Exception:
    pass

pos_profit_mins = 0.0
pos_under_mins  = 0.0
try:
    closed_positions = group_by_position(trades)
    for p in closed_positions:
        entry_ms = int(p["time"])
        close_ms = int(p["close_time"])
        hold_ms  = close_ms - entry_ms
        if hold_ms > 0:
            if p["realizedPnl"] > 0:
                pos_profit_mins += hold_ms / 60000
            else:
                pos_under_mins  += hold_ms / 60000
except Exception:
    pos_profit_mins = position_mins_profit
    pos_under_mins  = position_mins_under

if pos_profit_mins + pos_under_mins > 0:
    p, u = pos_profit_mins, pos_under_mins
    total = p + u
    def _fmt_hm(mins):
        h, m = divmod(int(round(mins)), 60)
        return f"{h}h {m}m" if h else f"{m}m"
    pos_time_block = (
        f"In profit:   {_fmt_hm(p)}  ({p/total*100:.0f}% of time in a trade)\
```

```
        f"Underwater:  {_fmt_hm(u)}  ({u/total*100:.0f}% of time in a trade)\
        + (
            f"Ratio:       {p/u:.2f}x  Above 1.5x benchmark"
            if u > 0 and p / u >= 1.5 else
            f"Ratio:       {p/u:.2f}x  Below 1.5x benchmark (need {int(u * 1.
            if u > 0 else
            "Ratio:       All time in profit"
        )
    )
else:
    pos_time_block = "No closed positions tracked today."

# ── Weekly P&L so far, for context alongside today's numbers ──────
week_line = ""
try:
    weekly = load_last_7_days_stats()
    if weekly:
        week_pnl = sum(d["net_pnl"] for d in weekly)
        gap      = WEEKLY_PNL_TARGET - week_pnl
        if gap <= 0:
            week_line = f"🏅 Weekly P&L: {format_pnl(week_pnl)} — target HIT
        else:
            week_line = f"📊 Weekly P&L: {format_pnl(week_pnl)} so far ({form
except Exception as e:
    print(f"[EOD Summary] Weekly P&L calc error: {e}")

msg = (
    f"📋 <b>End of Day Summary — {date_str}</b>\n\n"
    f"Positions closed:   {stats['closed_positions']}  ({stats['wins']}W / {s
    f"Win rate:          {stats['win_rate']:.1f}%\n"
    f"Avg win ROI:       +{stats['avg_win_roi_pct']:.2f}%\n"
    f"Avg loss ROI:      {stats['avg_loss_roi_pct']:.2f}%\n\n"
    f"Gross P&L:         {format_pnl(stats['total_pnl'])}\n"
    f"Fees paid:         -${stats['total_fees']:.2f}\n"
    f"Net P&L:           {format_pnl(stats['net_pnl'])}\n\n"
    f"Balance:           ${balance_start:,.2f} → ${balance_end:,.2f} "
    f"({format_pnl(balance_change)})\n\n"
    f"{week_line}"
    f"⏱️ <b>Position time today</b>\n"
    f"{pos_time_block}"
)
send_telegram(msg)

try:
    positions = group_by_position(trades)
    if len(positions) >= 2:
```

```
        uk_tz = ZoneInfo("Europe/London")
        entry_times = []
        hold_mins   = []
        for p in positions:
            entry_ts = datetime.fromtimestamp(int(p["time"]) / 1000, tz=uk_tz
            close_ts = datetime.fromtimestamp(int(p["close_time"]) / 1000, tz
            entry_times.append(entry_ts)
            hold = (close_ts - entry_ts).total_seconds() / 60
            if hold >= 0:
                hold_mins.append(hold)

        entry_times.sort()
        gaps = []
        for i in range(1, len(entry_times)):
            gap_mins = (entry_times[i] - entry_times[i-1]).total_seconds() /
            gaps.append(gap_mins)

        avg_gap   = sum(gaps) / len(gaps)
        min_gap   = min(gaps)
        max_gap   = max(gaps)
        avg_hold  = sum(hold_mins) / len(hold_mins) if hold_mins else 0

        revenge_count = 0
        sorted_pos = sorted(positions, key=lambda p: int(p["time"]))
        for i in range(1, len(sorted_pos)):
            prev = sorted_pos[i - 1]
            curr = sorted_pos[i]
            if prev["realizedPnl"] < 0:
                gap_ms = int(curr["time"]) - int(prev["close_time"])
                if gap_ms <= REVENGE_WINDOW_MINS * 60 * 1000:
                    revenge_count += 1
        revenge_line = f"Revenge trades: {revenge_count} ⚠️" if revenge_count

        def fmt_mins(m):
            return f"{int(m // 60)}h {int(m % 60)}m" if m >= 60 else f"{m:.0f

        freq_msg = (
            f"<b>Trade frequency today</b>\n\n"
            f"Avg gap:       {fmt_mins(avg_gap)} between trades\n"
            f"Shortest gap:  {fmt_mins(min_gap)}\n"
            f"Longest gap:   {fmt_mins(max_gap)}\n\n"
            f"Avg hold time: {fmt_mins(avg_hold)} per trade\n\n"
            f"{revenge_line}"
        )
        send_telegram(freq_msg)
except Exception as e:
```

```
    print(f"[EOD Frequency] Error: {e}")

try:
    positions = group_by_position(trades)
    if positions:
        uk_tz  = ZoneInfo("Europe/London")
        chunks = {}

        for p in positions:
            close_ts  = datetime.fromtimestamp(int(p["close_time"]) / 1000, t
            hour      = close_ts.hour
            chunk_start = (hour // 4) * 4
            chunk_end   = chunk_start + 4
            label       = f"{chunk_start:02d}:00–{chunk_end:02d}:00"
            chunks[label] = chunks.get(label, 0) + p["realizedPnl"]

        if chunks:
            worst_chunk = min(chunks, key=chunks.get)
            worst_pnl   = chunks[worst_chunk]

            chunk_lines = ""
            for label, pnl in sorted(chunks.items()):
                marker = " ◀ worst" if label == worst_chunk and worst_pnl < 0
                chunk_lines += f"  {label}:  {format_pnl(pnl)}{marker}\n"

            if worst_pnl < 0:
                worst_line = f"\n⚠️ Worst 4-hour window: <b>{worst_chunk}</b>
            else:
                worst_line = f"\n✅ No losing 4-hour windows today."

            send_telegram(
                f"🕐 <b>P&L by 4-hour window</b>\n\n"
                f"{chunk_lines}"
                f"{worst_line}"
            )
except Exception as e:
    print(f"[EOD Chunks] Error: {e}")

positions   = group_by_position(trades)
ind_trades  = [p for p in positions if any(str(oid) in indicator_trade_ids fo
man_trades  = [p for p in positions if not any(str(oid) in indicator_trade_id

if ind_trades and man_trades:
    ind_wins = len([p for p in ind_trades if p["realizedPnl"] > 0])
    man_wins = len([p for p in man_trades if p["realizedPnl"] > 0])
    ind_wr   = ind_wins / len(ind_trades) * 100
```

```
    man_wr   = man_wins / len(man_trades) * 100
    ind_pnl  = sum(p["realizedPnl"] for p in ind_trades)
    man_pnl  = sum(p["realizedPnl"] for p in man_trades)

    diff = ind_wr - man_wr
    if diff > 0:
        verdict = f"📡 Indicator trades outperformed by {diff:.1f}% WR — your
    elif diff < 0:
        verdict = f"🖐 Manual trades outperformed by {abs(diff):.1f}% WR — yo
    else:
        verdict = "🤝 Indicator and manual trades matched on win rate today."

    breakdown = (
        f"📊 <b>Indicator vs Manual</b>\n\n"
        f"📡 Indicator:  {len(ind_trades)} positions  |  {ind_wins}W/{len(ind
        f"🖐 Manual:     {len(man_trades)} positions  |  {man_wins}W/{len(man
        f"{verdict}"
    )
    send_telegram(breakdown)
elif ind_trades and not man_trades:
    ind_wins = len([p for p in ind_trades if p["realizedPnl"] > 0])
    ind_wr   = ind_wins / len(ind_trades) * 100
    send_telegram(f"📡 <b>All trades today were indicator-triggered</b>\n{len

weekly      = load_last_7_days_stats()
yesterday   = weekly[-1] if weekly else None
positives = []

if stats["net_pnl"] > 0:
    positives.append(f"✅ Profitable day — {format_pnl(stats['net_pnl'])} net

if stats["win_rate"] >= 55 and stats["closed_positions"] >= 3:
    positives.append(f"🎯 Win rate {stats['win_rate']:.1f}% — above your 55%

if yesterday and yesterday.get("closed_positions", 0) > 0 and stats["closed_p
    today_r   = stats["net_pnl"] / stats["closed_positions"] if stats["closed
    yest_r    = yesterday["net_pnl"] / yesterday["closed_positions"] if yeste
    if today_r > yest_r:
        positives.append(f"📈 R ratio improved vs yesterday ({format_pnl(toda

if yesterday:
    week_pnl_before = sum(d["net_pnl"] for d in weekly[:-1]) if len(weekly) >
    week_pnl_now    = sum(d["net_pnl"] for d in weekly)
    gap_before      = WEEKLY_PNL_TARGET - week_pnl_before
    gap_now         = WEEKLY_PNL_TARGET - week_pnl_now
    if gap_now < gap_before and gap_now > 0:
```

```
            positives.append(f"📊 Closer to weekly target — {format_pnl(gap_now)}
        elif gap_now <= 0:
            positives.append(f"🎖️ Weekly target already cleared this week")

    if positives:
        pos_msg = "🌟 <b>Today's wins</b>\n\n" + "\n".join(positives)
        send_telegram(pos_msg)

    if rules_broken_today:
        broken_list = "\n".join(
            f"• {r}  ×{c}" if c > 1 else f"• {r}"
            for r, c in rules_broken_today.items()
        )
        total_breaks = sum(rules_broken_today.values())
        send_telegram(
            f"🚨 <b>You broke your own rules today — {total_breaks} time(s).</b>\
            f"{broken_list}\n\n"
            f"This is self-sabotage. You <i>know</i> what works — you wrote the r
            f"Every time you cross these lines you're choosing noise over discipl
            f"Stop it.\n\n"
            f"Tomorrow: stick to the routine. Trust the process. You have the str
            f"you have the edge — just get out of your own way. You can do this.
        )


def send_weekly_summary():
    weekly = load_last_7_days_stats()
    if not weekly:
        return

    total_trades    = sum(d["total_trades"] for d in weekly)
    total_wins      = sum(d["wins"] for d in weekly)
    total_losses    = sum(d["losses"] for d in weekly)
    total_closed    = sum(d["closed_positions"] for d in weekly)
    total_pnl       = sum(d["total_pnl"] for d in weekly)
    total_fees      = sum(d["total_fees"] for d in weekly)
    net_pnl         = sum(d["net_pnl"] for d in weekly)
    win_rate        = (total_wins / total_closed * 100) if total_closed else 0

    best_day  = max(weekly, key=lambda d: d["net_pnl"])
    worst_day = min(weekly, key=lambda d: d["net_pnl"])

    win_holds  = [d["avg_hold_win"]  for d in weekly if d.get("avg_hold_win")  is
    loss_holds = [d["avg_hold_loss"] for d in weekly if d.get("avg_hold_loss") is
    hold_line  = ""
    if win_holds or loss_holds:
```

```
    parts = []
    if win_holds:
        parts.append(f"Winners {sum(win_holds)/len(win_holds):.0f}m")
    if loss_holds:
        parts.append(f"Losers {sum(loss_holds)/len(loss_holds):.0f}m")
    hold_line = f"\nAvg hold time:         ⏱️ {' | '.join(parts)}"

msg = (
    f"📊 <b>Weekly Summary ({weekly[0]['date']} - {weekly[-1]['date']})</b>\n
    f"Total trades:   {total_trades}\n"
    f"Wins / Losses:  {total_wins} / {total_losses}\n"
    f"Win rate:       {win_rate:.1f}%\n\n"
    f"Gross P&L:      {format_pnl(total_pnl)}\n"
    f"Fees paid:      -${total_fees:.2f}\n"
    f"Net P&L:        {format_pnl(net_pnl)}\n\n"
    f"Best day:       {best_day['date']} ({format_pnl(best_day['net_pnl'])})\
    f"Worst day:      {worst_day['date']} ({format_pnl(worst_day['net_pnl'])}
    f"Weekly target:  {format_pnl(WEEKLY_PNL_TARGET)} — "
    f"{'✅ HIT' if net_pnl >= WEEKLY_PNL_TARGET else f'❌ missed by {format_p
    f"Trade limit breached:  "
    + (lambda days: f"{'⚠️ ' if days > 0 else '✅ '}{days}/{len(weekly)} days
        sum(1 for d in weekly if d.get("total_trades", 0) >= DAILY_TRADE_CAP)
    )
    + "\n"
    f"Win streaks (3+):       🔥 {sum(d.get('streaks_3plus', 0) for d in weekl
    + hold_line
)
send_telegram(msg)

worst_chunk       = worst_day.get("worst_chunk_label")
worst_chunk_pnl   = worst_day.get("worst_chunk_pnl")
if worst_day["net_pnl"] < 0:
    chunk_detail = ""
    if worst_chunk and worst_chunk_pnl is not None:
        chunk_detail = f"\nHeaviest loss window: <b>{worst_chunk}</b> ({forma
    send_telegram(
        f"📉 <b>Worst day breakdown — {worst_day['date']}</b>\n\n"
        f"Positions:  {worst_day['closed_positions']}  "
        f"({worst_day['wins']}W / {worst_day['losses']}L  —  {worst_day['win_
        f"Net P&L:    {format_pnl(worst_day['net_pnl'])}\n"
        f"Fees:       -${worst_day['total_fees']:.2f}"
        f"{chunk_detail}"
    )

weekly_rules = {}
for d in weekly:
```

```
            for rule, count in d.get("rules_broken", {}).items():
                weekly_rules[rule] = weekly_rules.get(rule, 0) + count

    if weekly_rules:
        total_breaks = sum(weekly_rules.values())
        broken_list  = "\n".join(
            f"• {r}  ×{c}" if c > 1 else f"• {r}"
            for r, c in sorted(weekly_rules.items(), key=lambda x: -x[1])
        )
        send_telegram(
            f"🚨 <b>Rule breaks this week — {total_breaks} total</b>\n\n"
            f"{broken_list}"
        )
    else:
        send_telegram("✅ <b>No rules broken this week.</b> Discipline held.")


# ─────────────────────────────────────────────
# MIDNIGHT RESET
# ─────────────────────────────────────────────

def check_and_reset(trades: list):
    global early_warned_today, warned_today, stopped_today
    global DAILY_TRADE_CAP, first_loss_tightened_today
    global balance_up_alerted, balance_down_alerted, fee_alerted_today
    global eod_summary_sent_today
    global morning_recap_sent_today
    global last_cooldown_warned_count, last_reset_day, snapshot_balance, yesterda
    global same_side_alerted
    global daily_losses_block, daily_losses_block_reset_at, last_loss_close_time,
    global daily_bias
    global trigger_counts_today, long_trigger_times_today, short_trigger_times_to
    global overtrade_alerted, daily_target_alerted, weekly_target_alerted
    global dayscore_ticked_today
    global daily_loss_warn_alerted, daily_loss_stop_alerted, profit_locked_today,
    global overnight_alerted_1015, overnight_alerted_1045, position_mins_profit,
    global win_streak_alerted_at, daily_streaks_3plus, rules_broken_today

    now_uk = datetime.now(UK_TZ)
    today  = now_uk.date()

    eod_due = now_uk.hour == 23 and now_uk.minute < 10

    if eod_due and not eod_summary_sent_today:
        try:
            balance_end = get_usdt_balance()
```

```
        send_eod_summary(trades, snapshot_balance or 0, balance_end, str(toda
        db_log_daily_balance(today, balance_end)
        if today.weekday() == 4:
            send_weekly_summary()
        eod_summary_sent_today = True
    except Exception as e:
        print(f"[ERROR] EOD summary failed: {e}")

# ── 6am recap — yesterday's full EOD summary, resent fresh ────────
recap_due = now_uk.hour == 6 and now_uk.minute < 10
if recap_due and not morning_recap_sent_today:
    try:
        yesterday_uk_date = today - timedelta(days=1)
        day_start_uk = datetime.combine(yesterday_uk_date, dt_time(0, 0), tzi
        day_end_uk   = datetime.combine(today, dt_time(0, 0), tzinfo=UK_TZ)
        start_ms = int(day_start_uk.astimezone(timezone.utc).timestamp() * 10
        end_ms   = int(day_end_uk.astimezone(timezone.utc).timestamp() * 1000
        yesterday_trades = get_futures_trades(start_ms, end_ms)
        yesterday_balance_end = snapshot_balance or get_usdt_balance()
        send_telegram(f"☕ <b>6am recap — yesterday's full summary:</b>")
        send_eod_summary(yesterday_trades, yesterday_balance_end, yesterday_b
        morning_recap_sent_today = True
    except Exception as e:
        print(f"[ERROR] 6am recap failed: {e}")

if last_reset_day != today:
    print(f"[DAILY RESET] Running — last_reset_day was {last_reset_day}, toda
    try:
        snapshot_balance = get_usdt_balance()
    except Exception as e:
        print(f"[ERROR] Balance snapshot failed: {e}")

    early_warned_today          = False
    warned_today                = False
    stopped_today               = False
    DAILY_TRADE_CAP              = DAILY_TRADE_CAP_DEFAULT
    first_loss_tightened_today  = False
    balance_up_alerted          = False
    balance_down_alerted        = False
    fee_alerted_today           = False
    eod_summary_sent_today      = False
    morning_recap_sent_today    = False
    last_cooldown_warned_count  = 0
    same_side_alerted           = False
    same_side_block.clear()
    revenge_trade_ids_alerted.clear()
```

```
        daily_losses_block          = False
        daily_losses_block_reset_at = time.time()
        last_loss_close_time        = 0.0
        last_tp2_close_time         = 0.0
        elevated_silence_until      = 0.0
        overtrade_alerted           = False
        daily_target_alerted        = False
        weekly_target_alerted       = False
        dayscore_ticked_today       = False
        daily_loss_warn_alerted     = False
        daily_loss_stop_alerted     = False
        profit_locked_today         = 0.0
        profit_lock_last_attempt_at = 0.0
        overnight_alerted_1015      = False
        overnight_alerted_1045      = False
        overnight_auto_breakeven_done = False
        for _c in CLIENTS:
            _c.trades_today       = 0
            _c.wins_today         = 0
            _c.losses_today       = 0
        position_mins_profit        = 0.0
        position_mins_under         = 0.0
        win_streak_alerted_at       = 0
        daily_streaks_3plus         = 0
        rules_broken_today          = {}
        morning_brief_sent_today    = False
        weekly_digest_sent_today    = False
        weekly_okr_sent_today       = False
        monthly_review_sent_today   = False
        bias_question_sent_today    = False
        daily_bias                  = None
        indicator_trade_ids.clear()
        trigger_counts_today.clear()
        long_trigger_times_today.clear()
        short_trigger_times_today.clear()
        last_reset_day              = today
        print(f"[{today}] New day — all flags reset. Snapshot: ${snapshot_balance


def register_telegram_commands() -> None:
    commands = [
        {"command": "long",        "description": "Manually open a long"},
        {"command": "short",       "description": "Manually open a short"},
        {"command": "close",      "description": "Close position — /close or /clo
        {"command": "cut",        "description": "Cut position size by 50%"},
        {"command": "mirroropen", "description": "Bring one client into the open
```

```
        {"command": "sl",         "description": "Move stop loss by % — /sl 0.5"}
        {"command": "tp",         "description": "Move TP1 by % — /tp 0.4"},
        {"command": "tp2",        "description": "Move TP2 by % — /tp2 0.6"},
        {"command": "breakeven",  "description": "Move SL to entry price"},
        {"command": "fill2",      "description": "Fill Entry2 limit at market now
        {"command": "bullish",    "description": "Set HTF bias to bullish"},
        {"command": "bearish",    "description": "Set HTF bias to bearish"},
        {"command": "cancel_entry", "description": "Cancel pending maker entry be
        {"command": "cancel2",    "description": "Cancel Entry2 limit order"},
        {"command": "status",     "description": "Show open position & unrealised
        {"command": "stats",      "description": "Today's full stats"},
        {"command": "patterns",   "description": "Show best performing setups"},
        {"command": "live",           "description": "Are prompts on right now, a
        {"command": "vwinrate",        "description": "Per-trigger win rate (fixe
        {"command": "compound",        "description": "Weekly/4-week compounding
        {"command": "winrate",         "description": "Win rate today/7d/30d vs 6
        {"command": "wsstatus",        "description": "Websocket connection healt
        {"command": "discipline",      "description": "Week/month discipline tren
        {"command": "unmatched",      "description": "Alerts that fired but match
        {"command": "warnings",      "description": "Recent discipline warnings"}
        {"command": "analytics",     "description": "Behavioral pattern analysis"
        {"command": "invalidations", "description": "Invalidation rates per trigg
        {"command": "quality",       "description": "Entry quality: MFE, MAE, tim
        {"command": "compare",    "description": "Manual vs indicator win rate to
        {"command": "signals",    "description": "All configured signal prompts a
        {"command": "guards",     "description": "Every guard: status and live va
        {"command": "blocked",    "description": "Staircase states where a side i
        {"command": "track",      "description": "Performance record since 1 Oct"
        {"command": "config",     "description": "Every setting and any Railway o
        {"command": "help",       "description": "Full command list"},
    ]
    try:
        url  = f"https://api.telegram.org/bot{TELEGRAM_BOT_TOKEN}/setMyCommands"
        resp = requests.post(url, json={"commands": commands}, timeout=10)
        if resp.ok:
            print("[Telegram] Commands registered ✅")
        else:
            print(f"[Telegram] Command registration failed: {resp.text}")
    except Exception as e:
        print(f"[Telegram] Command registration error: {e}")


# ─────────────────────────────────────────────
# DAYSCORE — auto-tick "$60 profit day" when DAILY_PNL_TARGET is hit
# ─────────────────────────────────────────────
# Talks directly to the same PocketBase record DAYSCORE itself reads/writes
```

```
# (dayscore_state, owner="nathan"). Deliberately defensive: any failure here
# is logged and swallowed — a DAYSCORE hiccup must never affect trading.

def dayscore_today_str() -> str:
    # DAYSCORE's own JS tags every day with new Date().toISOString().split('T')[0
    # which is a UTC date — NOT the UK-local date bot.py uses for its own daily
    # reset. Replicated here on purpose so the tick lands on the same calendar
    # day DAYSCORE itself thinks "today" is, even in the narrow window around UK
    # midnight during BST where the two dates can briefly disagree.
    return datetime.now(timezone.utc).strftime("%Y-%m-%d")


def dayscore_tick_profit_day() -> bool:
    """Ticks DAYSCORE's '$60 profit day' criterion for today, once. Returns True
    only if it actually made a change. Safe to call repeatedly — idempotent."""
    if not DAYSCORE_PB_ADMIN_EMAIL or not DAYSCORE_PB_ADMIN_PASSWORD:
        print("[DAYSCORE] Missing PB_ADMIN_EMAIL / PB_ADMIN_PASSWORD env vars — s
        return False
    try:
        auth_res = requests.post(
            f"{DAYSCORE_PB_URL}/api/collections/_superusers/auth-with-password",
            json={"identity": DAYSCORE_PB_ADMIN_EMAIL, "password": DAYSCORE_PB_AD
            timeout=10,
        )
        auth_res.raise_for_status()
        token = auth_res.json()["token"]

        fetch_res = requests.get(
            f"{DAYSCORE_PB_URL}/api/collections/dayscore_state/records",
            params={"filter": f"(owner='{DAYSCORE_OWNER}')", "sort": "-updated",
            headers={"Authorization": token},
            timeout=10,
        )
        fetch_res.raise_for_status()
        items = fetch_res.json().get("items", [])
        if not items:
            print(f"[DAYSCORE] No dayscore_state record found for owner '{DAYSCOR
            return False

        record   = items[0]
        pb_state = record["json"]
        today_str  = dayscore_today_str()
        active_day = pb_state.get("activeDay")

        # Already closed today (manually, or a prior call already ticked and it's
        # been closed)? Nothing to do — the day's done.
```

```
        if any(e.get("date") == today_str for e in pb_state.get("entries", [])):
            return False

        # No active day yet for today, or the stored active day is stale (from a
        # date that never got closed)? Leave that to the app's own recovery flow
        # than guessing — same principle as the 23:30 auto-close script.
        if not active_day or active_day.get("date") != today_str:
            print(f"[DAYSCORE] No active day for {today_str} yet — skipping auto-
            return False

        checked = active_day.get("checkedCriteria", [])
        if DAYSCORE_PROFIT_CRITERIA_ID in checked:
            return False  # already ticked — manually, or by an earlier call toda

        active_day["checkedCriteria"] = checked + [DAYSCORE_PROFIT_CRITERIA_ID]
        pb_state["activeDay"]    = active_day
        pb_state["lastModified"] = datetime.now(timezone.utc).isoformat().replace

        patch_res = requests.patch(
            f"{DAYSCORE_PB_URL}/api/collections/dayscore_state/records/{record['i
            json={"json": pb_state},
            headers={"Authorization": token},
            timeout=10,
        )
        patch_res.raise_for_status()
        print(f"[DAYSCORE] Auto-ticked '$60 profit day' for {today_str}")
        return True
    except Exception as e:
        print(f"[DAYSCORE] Auto-tick failed (non-fatal): {e}")
        return False


def run():
    global early_warned_today, warned_today, stopped_today
    global DAILY_TRADE_CAP, first_loss_tightened_today
    global loss_streak_alerted, balance_up_alerted, balance_down_alerted
    global fee_alerted_today, eod_summary_sent_today
    global snapshot_balance, last_reset_day, last_cooldown_warned_count
    global same_side_alerted, same_side_block
    global daily_losses_block, daily_losses_block_reset_at, last_loss_close_time,
    global revenge_trade_ids_alerted, overtrade_alerted
    global daily_target_alerted, weekly_target_alerted
    global dayscore_ticked_today
    global daily_loss_warn_alerted, daily_loss_stop_alerted
    global win_streak_alerted_at, rules_broken_today
    global morning_brief_sent_today, weekly_digest_sent_today, weekly_okr_sent_to
```

```
global daily_streaks_3plus
global profit_locked_today, profit_lock_last_attempt_at, overnight_alerted_10
global entry2_placed_at
global position_mins_profit, position_mins_under, underwater_ratio_alerted, p
global trade_mfe, trade_mae, trade_entry_time, retrace_protect_triggered
global _last_slow_checks_run

last_trade_count = 0

print("Trade Alert Bot started.")

webhook_thread = threading.Thread(target=start_webhook_server, daemon=True)
webhook_thread.start()
print(f"[Webhook] Server listening on port {WEBHOOK_PORT}")

start_websocket_feed()
start_staircase_alerts()

register_telegram_commands()
init_db()

tg_cmd_thread = threading.Thread(target=poll_telegram_commands, daemon=True)
tg_cmd_thread.start()
if DATABASE_URL:
    threading.Thread(target=track_record_loop, daemon=True).start()
if HL_SDK_AVAILABLE:
    threading.Thread(target=client_watch_loop, daemon=True).start()
if EXECUTION_ENABLED:
    fill_thread = threading.Thread(target=poll_order_fills, daemon=True)
    fill_thread.start()

try:
    snapshot_balance = get_usdt_balance()
except Exception as e:
    print(f"[ERROR] Startup balance: {e}")

last_reset_day = datetime.now(timezone.utc).date()

now_uk = datetime.now(ZoneInfo("Europe/London"))
try:
    if now_uk.hour >= 7 and now_uk.hour < 23:
        send_telegram(
            f"⚠️ <b>Bot restarted mid-session ({now_uk.strftime('%H:%M')} UK)
            f"Order fill tracking has been reset — Entry2, TP1, TP2 and SL fi
            f"from earlier today will not fire alerts.\n\n"
            f"🚨 <b>If you have an open position:</b>\n"
```

```
            f"• Use /status to check your position\n"
            f"• Use /sl to re-set your stop loss immediately\n"
            f"• Use /tp and /tp2 to re-set your take profits\n"
            f"• You are at risk of liquidation until SL is re-set\n\n"
            f"Balance: <b>${snapshot_balance:,.2f} USDT</b>"
            if snapshot_balance else
            f"⚠️ <b>Bot restarted mid-session ({now_uk.strftime('%H:%M')} UK)
            f"🚨 <b>If you have an open position, re-set your SL and TP immed
            f"Order fill tracking has been reset. Check positions on Binance.
        )
    else:
        send_telegram(
            f"✅ <b>Trade Alert Bot is running.</b>\n"
            f"Monitoring Binance Futures (USDⓈ-M) — {SYMBOL}\n"
            f"Starting balance: <b>${snapshot_balance:,.2f} USDT</b>"
            if snapshot_balance else
            "✅ <b>Trade Alert Bot is running.</b>\nMonitoring Binance Future
        )
except Exception as e:
    # A Telegram delivery failure (e.g. this bot/chat pair hasn't started a
    # conversation yet, or the token/chat_id is momentarily wrong) must never
    # take down the whole service — the dashboard and trading logic below
    # still need to come up regardless of whether this one notification sent.
    print(f"[ERROR] Startup Telegram notification failed (non-fatal): {e}")

while True:
    try:
        trades, cached_balance = get_slow_refresh_data()
        orders  = group_by_order(trades)
        count   = len(group_by_position(trades))
        now_utc = datetime.now(timezone.utc)
        now_uk  = datetime.now(UK_TZ)
        now_str = now_utc.strftime("%H:%M UTC")

        print(f"[{now_str}] Orders today: {count} (from {len(trades)} fills)"

        check_and_reset(trades)

        # ── First trade of the day was a loss — tighten daily cap ──
        # Only tightens if FIRST_LOSS_TRADE_CAP is genuinely LOWER
        # than the normal cap — if Nathan has deliberately raised both
        # to the same temporary value (as tonight), this correctly does
        # nothing, instead of re-tightening every restart because a
        # loss earlier today is permanently true for the rest of the day.
        if not first_loss_tightened_today:
            positions_today = group_by_position(trades)
```

```
    if positions_today and positions_today[0]["realizedPnl"] < 0:
        first_loss_tightened_today = True
        if FIRST_LOSS_TRADE_CAP < DAILY_TRADE_CAP_DEFAULT:
            old_threshold = DAILY_TRADE_CAP
            DAILY_TRADE_CAP = FIRST_LOSS_TRADE_CAP
            db_log_warning("first_loss_tighten", f"First trade was a
            send_telegram(
                f"🎯 <b>First trade of the day was a loss</b>\n\n"
                f"Daily trade cap tightened from {old_threshold} to {
                f"Rough start — trade fewer, trade better today."
            )

# ── Trade count alerts ──────────────────────────────────────
if count >= DAILY_TRADE_CAP and not stopped_today:
    send_telegram(
        f"🚨 <b>{count} trades hit — STOP NOW.</b>\n\n"
        f"Daily limit of {DAILY_TRADE_CAP} reached.\n"
        f"🔌 All signal prompts and manual trades are now silenced un
        f"You're done for today. Walk away."
    )
    stopped_today = warned_today = early_warned_today = True
    rule = f"Exceeded daily trade limit ({count} trades vs max {DAILY
    rules_broken_today[rule] = rules_broken_today.get(rule, 0) + 1

elif count >= WARN_THRESHOLD and not warned_today:
    send_telegram(
        f"⚠️ <b>{count} trades taken today</b> — approaching your lim
        f"{DAILY_TRADE_CAP - count} trades left before the hard stop.
    )
    warned_today = early_warned_today = True

elif count >= EARLY_THRESHOLD and not early_warned_today:
    send_telegram(
        f"📊 <b>{count} trades taken today.</b>\n"
        f"Halfway to your warning threshold — stay selective."
    )
    early_warned_today = True

# ── Loss streak alert ───────────────────────────────────────
streak = get_consecutive_losses(trades)
if streak >= CONSECUTIVE_LOSS_ALERT and not loss_streak_alerted:
    db_log_warning("loss_streak", f"{streak} losses in a row")
    send_telegram(
        f"🔴 <b>{streak} losses in a row.</b>\n"
        f"Step back, review the setup — don't chase."
    )
```

```
    loss_streak_alerted = True
elif streak < CONSECUTIVE_LOSS_ALERT and loss_streak_alerted:
    loss_streak_alerted = False

# ── Win streak alert ──────────────────────────────────────
win_streak = get_consecutive_wins(trades)
WIN_STREAK_TRIGGER = 3
if win_streak >= WIN_STREAK_TRIGGER and win_streak > win_streak_alert
    if win_streak == 3:
        daily_streaks_3plus += 1
        msg = (
            f"🔥🎉🚀 <b>{win_streak} wins in a row!</b>\n"
            f"You're locked in. Keep trusting the process — don't get
            f"just keep reading it right. 💪😤"
        )
    elif win_streak == 4:
        msg = (
            f"🔥🔥🎯🏆 <b>{win_streak} wins straight!</b>\n"
            f"This is what discipline looks like. Stay sharp, stay se
        )
    elif win_streak == 5:
        msg = (
            f"🚨 <b>STOP. 5 wins in a row.</b>\n\n"
            f"The odds of winning 5 trades in a row at your win rate
            f"This is a statistical anomaly — not a sign to keep goin
            f"The market will revert. Overconfidence after a streak i
            f"account killers in trading.\n\n"
            f"<b>Walk away. Lock in the day. Come back tomorrow.</b>"
        )
    else:
        msg = (
            f"🚨 <b>{win_streak} in a row — seriously, stop now.</b>\
            f"You are deep in anomaly territory. The longer this goes
            f"the harder the snapback. Protect what you've made today
        )
    send_telegram(msg)
    win_streak_alerted_at = win_streak
elif win_streak == 0:
    win_streak_alerted_at = 0

# ── Fee tracker ───────────────────────────────────────────
stats = build_stats(trades)
fees_today = stats["total_fees"]
print(f"[{now_str}] Fees today: ${fees_today:.2f}")

if fees_today >= FEE_ALERT_THRESHOLD and not fee_alerted_today:
```

```
        db_log_warning("fee_threshold", f"${fees_today:.2f} paid in fees
        send_telegram(
            f"💸 <b>Fees alert — ${fees_today:.2f} paid in fees today.</b
            f"You've crossed the ${FEE_ALERT_THRESHOLD:.0f} threshold. "
            f"Every extra trade makes this worse."
        )
        fee_alerted_today = True
        rule = f"Fee threshold breached (fees vs ${FEE_ALERT_THRESHOLD:.0
        rules_broken_today[rule] = rules_broken_today.get(rule, 0) + 1

    # ── Daily losses block — hard stop after DAILY_LOSS_COUNT_LIMIT loss
    # Restart-proof: checks the DATABASE for whether this alert was
    # already sent today, not just an in-memory flag — a flag resets
    # to False on every Railway restart, but a DB record survives,
    # so this can no longer re-spam the same alert after a crash.
    if stats["losses"] >= DAILY_LOSS_COUNT_LIMIT and not daily_losses_blo
        already_alerted_today = db_was_warning_logged_today("daily_loss_b
        daily_losses_block = True  # block trading regardless of whether
        if not already_alerted_today:
            db_log_warning("daily_loss_block", f"{DAILY_LOSS_COUNT_LIMIT}
            send_telegram(
                f"🚫 <b>{DAILY_LOSS_COUNT_LIMIT} losses today — trading b
                f"You've had {DAILY_LOSS_COUNT_LIMIT} losing trades today
                f"No more trades until midnight. Walk away.\n\n"
                f"Losses today: {stats['losses']}  |  Net P&L: {format_pn
            )
            send_push_notification("Stopped for the day", f"{DAILY_LOSS_C
            rule = f"Hit {DAILY_LOSS_COUNT_LIMIT} daily loss limit"
            rules_broken_today[rule] = rules_broken_today.get(rule, 0) +


    if count > last_trade_count:
        closing_trades = [t for t in trades if float(t.get("realizedPnl",
        if len(closing_trades) >= 1:
            last_close = closing_trades[-1]
            last_pnl   = float(last_close["realizedPnl"])

            if last_pnl < 0:
                last_close_time_ms = int(last_close["time"])
                revenge_window_ms  = REVENGE_WINDOW_MINS * 60 * 1000
                cutoff_ms          = last_close_time_ms + revenge_window_

                revenge_trades = [
                    t for t in trades
                    if int(t["time"]) > last_close_time_ms
                    and int(t["time"]) <= cutoff_ms
```

```
                    and float(t.get("realizedPnl", 0)) == 0
                ]

                for rt in revenge_trades:
                    trade_id = rt.get("orderId")
                    if trade_id and trade_id not in revenge_trade_ids_ale
                        # Restart-proof dedup: also check the DB in case
                        # a restart wiped the in-memory set — embeds the
                        # trade_id in the message so we can match on it.
                        already_alerted = db_was_message_logged_today("re
                        revenge_trade_ids_alerted.add(trade_id)
                        if not already_alerted:
                            db_log_warning("revenge_trade", f"Possible re
                            send_telegram(
                                f"😤 <b>Possible revenge trade detected.<
                                f"You opened a new position within {REVEN
                                f"of closing a losing trade.\n"
                                f"Are you trading the setup or chasing th
                            )

# ── Same side consecutive losses ──────────────────────────
same_side_count, same_side = get_same_side_consecutive_losses(trades)

if same_side_count == 2 and not same_side_alerted:
    direction_key = "long" if same_side == "BUY" else "short"
    same_side_block[direction_key] = time.time() + 2700  # 45 minutes
    if same_side == "BUY":
        msg = (
            f"🛑 <b>2 losing longs in a row — longs blocked for 45 mi
            f"The market is not rewarding longs right now. Before you
            f"• Wait for your long triggers to fully form\n"
            f"• Wait for a key level to be swept and reclaimed\n"
            f"• Go up a timeframe and reassess the trend\n\n"
            f"Don't force it. Let the setup come to you."
        )
    else:
        msg = (
            f"🛑 <b>2 losing shorts in a row — shorts blocked for 45
            f"The downside momentum isn't there. Before you try again
            f"• Wait for momentum to be exhausted\n"
            f"• Go up a timeframe and reassess the trend\n"
            f"• Wait for a higher key level to be swept\n\n"
            f"Don't fight the move. Wait for clear exhaustion."
        )
    send_telegram(msg)
    same_side_alerted = True
```

```
elif same_side_count == 3 and not same_side_alerted:
    direction_key = "long" if same_side == "BUY" else "short"
    same_side_block[direction_key] = time.time() + 5400  # 90 minutes
    side_label_str = "longs" if same_side == "BUY" else "shorts"
    send_telegram(
        f"🚨 <b>3 losing {side_label_str} in a row — blocked for 90 m
        f"This direction is clearly not working today. Stop. Complete
        f"Go up a timeframe. Reassess your bias. Do not override this
    )
    same_side_alerted = True
elif same_side_count < SAME_SIDE_LIMIT and same_side_alerted:
    same_side_alerted = False

# ── Avg gap overtrading warning ──────────────────────────
try:
    day_positions = group_by_position(trades)
    if len(day_positions) >= 3:
        sorted_pos  = sorted(day_positions, key=lambda p: int(p["time
        entry_times = [int(p["time"]) for p in sorted_pos]
        gaps_ms     = [(entry_times[i] - entry_times[i-1]) for i in r
        avg_gap_ms  = sum(gaps_ms) / len(gaps_ms)
        avg_gap_m   = avg_gap_ms / 60000
        if avg_gap_m < OVERTRADE_GAP_MINS and not overtrade_alerted:
            send_telegram(
                f"<b>Overtrading warning</b>\n\n"
                f"Your average gap between trades today is {avg_gap_m
                f"Benchmark is {OVERTRADE_GAP_MINS}m minimum.\n\n"
                f"You may be forcing trades. Slow down."
            )
            overtrade_alerted = True
        elif avg_gap_m >= OVERTRADE_GAP_MINS and overtrade_alerted:
            overtrade_alerted = False
except Exception:
    pass

last_trade_count = count

# ── Cooldown window check ────────────────────────────────
if in_cooldown():
    window_start = now_uk.replace(
        hour=COOLDOWN_START.hour, minute=COOLDOWN_START.minute,
        second=0, microsecond=0)
    window_start_ms  = int(window_start.timestamp() * 1000)
    cooldown_orders  = [o for o in orders if int(o["time"]) >= window
    cooldown_count   = len(cooldown_orders)
```

```
    if cooldown_count > last_cooldown_warned_count:
        send_telegram(
            f"🚫 <b>Cooldown violation — {cooldown_count} trade(s) pl
            f"between {COOLDOWN_START.strftime('%H:%M')}-{COOLDOWN_EN
            f"You set this window as off-limits. Close the chart and
        )
        last_cooldown_warned_count = cooldown_count
        rule = "Traded during cooldown window"
        rules_broken_today[rule] = rules_broken_today.get(rule, 0) +

# ── 7am bias question / 8am morning brief ──────────────────
if BIAS_FILTER_ENABLED and now_uk.hour == 7 and now_uk.minute < 10 an
    send_bias_question()
    bias_question_sent_today = True

if now_uk.hour == 8 and now_uk.minute < 10 and not morning_brief_sent
    send_morning_brief()
    morning_brief_sent_today = True

# ── Monday 8:10am weekly OKR reflection ────────────────────
if now_uk.weekday() == 0 and now_uk.hour == 8 and 10 <= now_uk.minute
    send_weekly_okr()
    weekly_okr_sent_today = True

# ── 1st of month, 8:30am — monthly discipline review ──────
if now_uk.day == 1 and now_uk.hour == 8 and 30 <= now_uk.minute < 40
    send_monthly_discipline_review()
    monthly_review_sent_today = True

# ── Sunday 8pm weekly digest ──────────────────────────────
if now_uk.weekday() == 6 and now_uk.hour == 20 and now_uk.minute < 10
    send_weekly_digest()
    weekly_digest_sent_today = True

# ── Overnight position alerts (10:15 and 10:45pm) ─────────
is_1015 = now_uk.hour == 22 and 15 <= now_uk.minute < 25
is_1045 = now_uk.hour == 22 and 45 <= now_uk.minute < 55
if (is_1015 and not overnight_alerted_1015) or (is_1045 and not overn
    try:
        pos = get_open_position()
        if pos:
            amt       = float(pos["positionAmt"])
            direction = "LONG" if amt > 0 else "SHORT"
            entry     = float(pos.get("entryPrice", 0))
            unreal    = float(pos.get("unRealizedProfit", 0))
            send_telegram(
```

```
                    f"🌙 <b>Overnight position warning</b>\n\n"
                    f"You still have a {direction} open at {'10:15' if is
                    f"Entry:       ${entry:,.2f}\n"
                    f"Unrealised:  {format_pnl(unreal)}\n\n"
                    f"Are you holding this intentionally? "
                    f"Use /close if not."
                )
        except Exception:
            pass
        if is_1015:
            overnight_alerted_1015 = True
        else:
            overnight_alerted_1045 = True

    # ── Overnight auto-breakeven safety net (22:45) ──────────────
    # If a position is still open and in profit at 22:45, move SL to
    # entry automatically — a safety net in case Nathan has fallen
    # asleep and forgotten to manage it. Fires once, only if genuinely
    # in profit (never moves SL to a worse position than it already
    # has, and never touches a losing position).
    is_2245_breakeven = now_uk.hour == 22 and 45 <= now_uk.minute < 55
    if is_2245_breakeven and not overnight_auto_breakeven_done:
        try:
            pos = get_open_position()
            if pos:
                amt    = float(pos["positionAmt"])
                entry  = float(pos.get("entryPrice", 0))
                unreal = float(pos.get("unRealizedProfit", 0))
                current_sl = current_trade_entry.get("sl_price")
                # Only move if genuinely in profit AND the new SL would
                # actually be an improvement on the current one (never
                # loosen an already-better SL, e.g. one already past
                # breakeven from a manual TP1 move).
                already_better_or_equal = (
                    current_sl is not None and entry > 0 and (
                        (amt > 0 and current_sl >= entry) or
                        (amt < 0 and current_sl <= entry)
                    )
                )
                mark_px = float(pos.get("markPrice", 0) or 0)
                profit_pct = 0.0
                if entry > 0 and mark_px > 0:
                    profit_pct = ((mark_px - entry) / entry * 100) if amt
                # A stop exactly on entry while price is sitting on entry
                # just gets clipped for nothing — only act once price has
                # moved a real distance in our favour.
```

```
            if unreal > 0 and profit_pct >= OVERNIGHT_BE_MIN_PROFIT_P
                sym_info_bo = get_symbol_info()
                be_price = round_step(entry, sym_info_bo["price_tick"
                adjust_sl(be_price)
                db_log_warning("overnight_auto_breakeven", f"Auto-mov
                send_telegram(
                    f"😴 <b>Overnight safety net — SL auto-moved to b
                    f"Position was in profit ({format_pnl(unreal)}) a
                    f"breakeven move made — assuming you're asleep.\n
                    f"SL moved to entry (${be_price:,.2f}) so this ca
                )
    except Exception as e:
        print(f"[Overnight Breakeven] Error: {e}")
    overnight_auto_breakeven_done = True

# ── Entry2 auto-cancel after 2 hours ──────────────────────
if entry2_placed_at > 0 and (time.time() - entry2_placed_at) >= ENTRY
    try:
        entry2_id = None
        for oid, meta in list(tracked_orders.items()):
            if meta["label"] == "Entry2":
                entry2_id = int(oid)
                break
        if entry2_id:
            now_ms  = int(time.time() * 1000)
            params  = {"symbol": SYMBOL, "orderId": entry2_id, "times
            params  = sign_request(params)
            headers = {"X-MBX-APIKEY": BINANCE_API_KEY}
            resp    = requests.delete(f"{FUTURES_BASE}/fapi/v1/order"
                                      params=params, headers=headers,
            if resp.status_code == 200:
                tracked_orders.pop(str(entry2_id), None)
                entry2_placed_at = 0.0
                send_telegram(
                    f"⏱️ <b>Entry2 cancelled — 2 hour expiry reached.
                    f"Price never pulled back to the limit. "
                    f"Position continues with original size."
                )
    except Exception as e:
        print(f"[Entry2 Expiry ERROR] {e}")

# ── Software stop-loss check ──────────────────────────────
shared_open_pos = None
if EXECUTION_ENABLED:
    try:
        shared_open_pos = get_open_position()
```

```
    except Exception as e:
        print(f"[Position Fetch] Error: {e}")

if EXECUTION_ENABLED and current_trade_entry.get("sl_price"):
    try:
        sl_pos = shared_open_pos
        if sl_pos:
            mark      = get_mark_price()
            sl_price  = current_trade_entry["sl_price"]
            direction = current_trade_entry.get("direction")
            hit = (direction == "long"  and mark <= sl_price) or \
                  (direction == "short" and mark >= sl_price)
            if hit:
                amt        = float(sl_pos["positionAmt"])
                close_side = "SELL" if amt > 0 else "BUY"
                quantity   = abs(amt)
                place_order(close_side, "MARKET", quantity, reduce_on
                sl_loss = abs(sl_price - mark) * quantity
                time_to_sl = (time.time() - trade_entry_time) / 60 if
                db_update_outcome(pending_trigger_id, "loss", -sl_los
                                  mae=trade_mae, mfe=trade_mfe,
                                  time_to_sl_mins=time_to_sl)
                last_loss_close_time = time.time()
                cancel_open_orders()
                current_trade_entry.clear()
                send_telegram(
                    f"🛑 <b>Stop loss hit — {direction.upper()} close
                    f"SL price:  ${sl_price:,.2f}\n"
                    f"Exit:      ${mark:,.2f}\n\n"
                    f"Loss taken. Assess before the next trade.\n\n"
                    f"{after_trade_summary()}"
                )
                send_push_notification("Stop loss hit", f"{direction.
                # Same reasoning as the other automatic
                # SL/TP-hit closes — trust each client's own
                # real Hyperliquid order first.
                mirror_close_unhealthy()
        else:
            current_trade_entry.clear()
    except Exception as e:
        print(f"[Software SL] Error: {e}")

# ── Profit / breakeven alert + time tracking ──────────────
if EXECUTION_ENABLED:
    try:
        open_pos = shared_open_pos
```

```
if open_pos:
    if position_first_seen_at <= 0:
        position_first_seen_at = time.time()
    check_profit_alert(open_pos)
    unreal = float(open_pos.get("unRealizedProfit", 0))
    poll_mins = POLL_INTERVAL / 60
    if unreal > 0:
        position_mins_profit += poll_mins
    elif unreal < 0:
        position_mins_under  += poll_mins

    # ── Underwater vs profit time ratio warning ──────
    # Fires once per position if time spent underwater
    # reaches 1.5x time spent in profit — a signal the
    # trade is dragging rather than working, even if
    # still open and not yet at SL. Requires the trade
    # to have been open at least UNDERWATER_ALERT_MIN_AGE_MIN
    # before either underwater alert can fire at all,
    # otherwise it triggers on trades that are just
    # naturally early/noisy.
    # Age is taken from the NEWEST reliable timestamp: a
    # position opened outside the bot's own execution path
    # (e.g. in the Binance app) never refreshes
    # trade_entry_time, so it held the previous trade's
    # start and made a brand-new position look hours old.
    _ages = [(time.time() - position_first_seen_at) / 60]
    if trade_entry_time > 0:
        _ages.append((time.time() - trade_entry_time) / 60)
    trade_age_mins = min(_ages)
    if trade_age_mins >= UNDERWATER_ALERT_MIN_AGE_MINS:
        if not underwater_ratio_alerted and position_mins_pro
            if position_mins_under >= position_mins_profit *
                underwater_ratio_alerted = True
                db_log_warning("underwater_ratio", f"Underwat
                send_telegram(
                    f"⏱️ <b>Spending more time underwater tha
                    f"In profit: {position_mins_profit:.0f}m
                    f"This trade has been underwater for {UND
                    f"it's been in profit. Worth reassessing
                )
        elif not underwater_ratio_alerted and position_mins_p
            # Never been in profit at all, and already past t
            # 30-min minimum — worth flagging.
            underwater_ratio_alerted = True
            db_log_warning("underwater_ratio", f"Underwater {
            send_telegram(
```

```
                f"⏱️ <b>This trade has never been in profit –
                f"Worth reassessing rather than waiting it ou
            )

    # ── Track MFE/MAE ──────────────────────────────
    ep   = float(open_pos.get("entryPrice", 0))
    mark = get_mark_price()
    amt  = float(open_pos.get("positionAmt", 0))
    if ep > 0 and mark > 0:
        if amt > 0:   # long
            excursion = (mark - ep) / ep * 100
        else:         # short
            excursion = (ep - mark) / ep * 100
        if excursion > trade_mfe:
            trade_mfe = excursion
        if excursion < trade_mae:
            trade_mae = excursion

        # ── Retracement protection: lock in real profit ──
        # If trade reached RETRACE_PROTECT_PCT profit but has
        # pulled back down to only RETRACE_LOCK_IN_PCT still
        # move SL to lock in that remaining profit — not just
        # breakeven, an actual small win rather than a scratc
        if (trade_mfe >= RETRACE_PROTECT_PCT
                and excursion <= RETRACE_LOCK_IN_PCT
                and not retrace_protect_triggered
                and current_trade_entry.get("sl_price")):
            sym_info_rp = get_symbol_info()
            tick_rp     = sym_info_rp["price_tick"]
            if amt > 0:  # long
                lock_price_rp = round_step(ep * (1 + RETRACE_
            else:        # short
                lock_price_rp = round_step(ep * (1 - RETRACE_
            try:
                adjust_sl(lock_price_rp)
                current_trade_entry["sl_price"] = lock_price_
                retrace_protect_triggered = True
                db_log_warning("retrace_protect", f"SL moved
                send_telegram(
                    f"🛡️ <b>Retracement protection — profit l
                    f"Trade reached +{trade_mfe:.2f}% profit
                    f"{RETRACE_PROTECT_PCT:.2f}% threshold) a
                    f"to {excursion:+.2f}%.\n\n"
                    f"SL moved to ${lock_price_rp:,.2f} — loc
                    f"minimum from here, worst case is a real
                )
```

```
                    except Exception as e:
                        print(f"[Retrace Protect] SL move error: {e}"
            else:
                breakeven_suggested       = False
                breakeven_last_suggested_at = 0.0
                position_mins_profit      = 0.0
                position_mins_under       = 0.0
                underwater_ratio_alerted  = False
                retrace_protect_triggered = False
                position_first_seen_at    = 0.0
        except Exception:
            pass

    if snapshot_balance:
        current_pnl = build_stats(trades)["net_pnl"]

        if current_pnl >= DAILY_PNL_TARGET and not daily_target_alerted:
            send_telegram(
                f"🎯 <b>Daily target hit — {format_pnl(current_pnl)} net
                f"You've cleared your ${DAILY_PNL_TARGET:.0f} daily targe
                f"Lock it in or keep pushing."
            )
            daily_target_alerted = True
            if not dayscore_ticked_today:
                if dayscore_tick_profit_day():
                    dayscore_ticked_today = True
        elif current_pnl < DAILY_PNL_TARGET and daily_target_alerted:
            daily_target_alerted = False

        # ── Profit lock — re-arms every time another full target's
        # worth of NEW profit accumulates, not just once per day ──
        # Cooldown added: if a transfer attempt fails (e.g. Binance
        # API permission issue), don't retry/re-alert every single
        # poll cycle — wait PROFIT_LOCK_RETRY_COOLDOWN_SEC between
        # attempts, whether the last one succeeded or failed.
        unlocked_gain = current_pnl - profit_locked_today
        if (PROFIT_LOCK_ENABLED and unlocked_gain >= DAILY_PNL_TARGET
                and (time.time() - profit_lock_last_attempt_at) >= PROFIT
            profit_lock_last_attempt_at = time.time()
            try:
                available = get_usdt_available_balance()
                transfer_amount = round(min(unlocked_gain, available), 2)
                if transfer_amount <= 0:
                    send_telegram(
                        f"🔒 <b>Profit lock — nothing to move</b>\n\n"
                        f"${unlocked_gain:.2f} unlocked profit available
```

```
                    f"available balance is ${available:.2f}. Skipping
                )
            else:
                result = binance_transfer_futures_to_spot(transfer_am
                profit_locked_today += transfer_amount
                capped_note = (
                    f"\n⚠️ Capped to available balance (${available:.
                    f"the rest will carry forward and lock next time
                    f"or profit grows further."
                    if transfer_amount < unlocked_gain else ""
                )
                send_telegram(
                    f"🔒 <b>Profit locked — ${transfer_amount:.2f} mc
                    f"Daily P&L: {format_pnl(current_pnl)}\n"
                    f"Locked today (total): ${profit_locked_today:.2f
                    f"{capped_note}\n\n"
                    f"Walk away and protect it. 💰"
                )
        except Exception as e:
            send_telegram(
                f"⚠️ <b>Profit lock transfer FAILED</b>\n\n"
                f"${unlocked_gain:.2f} unlocked profit but the automa
                f"transfer to Spot did not go through:\n{e}\n\n"
                f"Move it manually if you want it protected."
            )
            print(f"[Profit Lock] Transfer error: {e}")

    loss_warn_threshold = -(DAILY_LOSS_LIMIT * DAILY_LOSS_WARN_PCT /
    if current_pnl <= -DAILY_LOSS_LIMIT and not daily_loss_stop_alert
        send_telegram(
            f"🚫 <b>DAILY LOSS LIMIT HIT — ${abs(current_pnl):,.2f} d
            f"You have lost ${DAILY_LOSS_LIMIT:.0f} today. That is yo
            f"<b>Stop trading. Now.</b>\n\n"
            f"🔕 All signal prompts and manual trades are now silence
            f"This is not a suggestion — you set this rule yourself.
            f"Close everything, step away, and come back tomorrow. 🛑
        )
        send_push_notification("Stopped for the day", f"Daily loss li
        daily_loss_stop_alerted = True
        stopped_today = True
        db_log_warning("daily_loss_limit", f"${abs(current_pnl):.2f}
        rule = f"Hit ${DAILY_LOSS_LIMIT:.0f} daily loss limit"
        rules_broken_today[rule] = rules_broken_today.get(rule, 0) +

    elif current_pnl <= loss_warn_threshold and not daily_loss_warn_a
        remaining = DAILY_LOSS_LIMIT - abs(current_pnl)
```

```
                send_telegram(
                    f"⚠️ <b>Approaching daily loss limit — ${abs(current_pnl)
                    f"You're {DAILY_LOSS_WARN_PCT:.0f}% of the way to your ${
                    f"${remaining:,.2f} left before the hard stop.\n\n"
                    f"Be very selective. Every trade now carries more weight.
                )
                daily_loss_warn_alerted = True

            if not weekly_target_alerted:
                weekly = load_last_7_days_stats()
                week_pnl = sum(d["net_pnl"] for d in weekly) + current_pnl
                if week_pnl >= WEEKLY_PNL_TARGET:
                    send_telegram(
                        f"🏅 <b>Weekly target hit — ${week_pnl:,.2f} net P&L
                        f"You've cleared your ${WEEKLY_PNL_TARGET:.0f} weekly
                        f"Solid week."
                    )
                    weekly_target_alerted = True

    except requests.exceptions.RequestException as e:
        print(f"[ERROR] Request: {e}")
    except Exception as e:
        print(f"[ERROR] {e}")

    # ── Slower periodic checks (WPR, aggressive volume, virtual ──────
    # trigger resolution) — these don't need the fast SL/TP polling
    # cadence, so they run on the same SLOW_REFRESH_INTERVAL (60s
    # default) as trade history/balance, not every fast tick.
    now_for_slow_checks = time.time()
    if (now_for_slow_checks - _last_slow_checks_run) >= SLOW_REFRESH_INTERVAL
        _last_slow_checks_run = now_for_slow_checks

        try:
            check_virtual_trigger_trades()
        except Exception as e:
            print(f"[ERROR] Virtual trigger check: {e}")

        try:
            check_wpr_short_cross()
        except Exception as e:
            print(f"[ERROR] WPR short cross check: {e}")

        try:
            check_wpr_long_cross()
        except Exception as e:
            print(f"[ERROR] WPR long cross check: {e}")
```

```
        time.sleep(POLL_INTERVAL)


if __name__ == "__main__":
    run()
```